



Agent Handbook

with LangChain, LangGraph & Deep Agents

A Practical Guide to AI Agent Development

Agent Handbook with LangChain, LangGraph & Deep Agents

Copyright © 2026 BAEUM.AI. All rights reserved.

The code examples in this book may be freely used for educational purposes.
Unauthorized reproduction or redistribution of the written content is prohibited.

First edition: 2026

Preface

AI agents go beyond simple chatbots. They are intelligent systems that use tools, make plans, and autonomously carry out complex tasks. This handbook presents a structured, hands-on path to building real-world AI agents with three major frameworks — **LangChain**, **LangGraph**, and **Deep Agents**.

What This Book Offers

- **Hands-on learning** — Includes 59 Jupyter notebook-based examples with code and outputs
- **Progressive difficulty** — Moves step by step from fundamentals to production deployment
- **Framework comparison** — Compares the strengths and best-fit use cases of each framework
- **Applied projects** — Builds RAG, SQL, data analysis, ML, and deep research agents

Target Audience

- Developers who know basic Python syntax
- Engineers who want to build LLM-powered applications
- Learners who want a structured path into AI agent architecture

How This Book Is Organized

Part	Topic	Chapters
I	Agent Foundations — from environment setup to the basics of the three frameworks	8
II	LangChain — models, tools, memory, middleware, and multi-agent patterns	13
III	LangGraph — state graphs, workflows, persistence, and subgraphs	13
IV	Deep Agents — backends, subagents, memory, and harnesses	10
V	Advanced Patterns — multi-agent systems, RAG, SQL, and production	10
VI	Applied Examples — five end-to-end agent projects	5

Each chapter follows a consistent structure: Learning Objectives → theory → hands-on code → Summary.

How to Use This Book

Reading Code Blocks

This handbook uses two main block styles:

```
# Python code block – mint left border
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
result = model.invoke("Hello, Agent!")
```

```
Output block – coral left border
Hello! I'm an AI agent ready to help.
```

Callout Types

TIP

Provides a useful tip or extra information.

· NOTE

Explains an important note or background concept.

! WARNING

Warns about caution points or common mistakes.

Practice Environment

All code has been tested with Python 3.11+. Package installation and API key setup are covered in the first chapter of Part I.

Table of Contents

Part 1: Agent Foundations	23
0. Environment Setup	25
Learning Objectives	25
0.1 API key settings	25
0.2 Model initialization	26
0.3 Smoke Test	26
Summary	26
1. LLM Basics	27
Learning Objectives	27
1.1 Environment Setup	27
1.2 The Three Roles of Messages	27
1.3 Controlling Behavior with a System Message	28
1.4 Dictionary Format	28
1.5 Streaming	28
1.6 Batch Calls	28
Summary	28
2. LangChain Basics	30
Learning Objectives	30
2.1 Environment Setup	30
2.2 Building Tools	30
2.3 Creating and Running an Agent	31
Summary	31
3. LangChain Conversations	32
Learning Objectives	32
3.1 Environment Setup	32
3.2 The Limit of an Agent Without Memory	32
3.3 Adding Memory with InMemorySaver	32
3.4 Watching the Steps with Streaming	33
Summary	33
4. LangGraph Basics	34
Learning Objectives	34
4.1 Environment Setup	34
4.2 Your First Graph	34
4.3 A Two-Node Graph	35
4.4 Using an LLM as a Node	35
Summary	35
5. Deep Agents Basics	36
Learning Objectives	36

5.1 Environment Setup	36
5.2 Creating an Agent	36
5.3 Adding a Custom Tool	37
Summary	37
6. Comparing the Three Frameworks & Choosing What Comes Next	38
Learning Objectives	38
6.1 Framework Comparison	38
6.2 Which One Should You Choose?	39
6.3 Code Comparison — The Same Task in Three Styles	39
Summary	40
6.4 What Comes Next	40
7. Mini Project	41
Learning Objectives	41
7.1 Environment Setup	41
7.2 Defining the Search Tool	41
7.3 A Deep Agents Research Agent	42
7.4 Observing the Process with Streaming	42
7.5 Comparing It with a LangChain Agent	43
Summary	43
Part 2: LangChain	44
1. Introduction to LangChain	46
Learning Objectives	46
1.1 LangChain Framework Overview	46
1.2 The ReAct Agent Pattern	49
1.3 Overview of the Main Components	51
1.4 Environment Setup and Installation Check	51
1.5 Next Steps	52
Summary	52
2. Your First Agent	54
Learning Objectives	54
2.1 Environment Setup	54
2.2 Building a Simple Tool	55
2.3 Creating an Agent	55
2.4 Running the Agent	56
2.5 Streaming Execution	56
2.6 Multi-Turn Conversation	56
2.7 Adding the Tavily Search Tool (Optional)	57
2.8 Runtime Context (<code>context_schema</code>)	57
2.9 Summary	57
3. Models and the Message System	59
Learning Objectives	59
3.1 Environment Setup	59
3.2 Comparing Model Providers	59
3.3 Using <code>init_chat_model()</code>	60
3.4 <code>invoke()</code> , <code>stream()</code> , and <code>batch()</code> Patterns	60
3.5 Message Types	60
3.6 Multimodal Messages (Image Input)	61

3.7 Standard Output Format — <code>output_version="v1"</code>	61
3.8 Tracking Token Usage — <code>UsageMetadataCallbackHandler</code>	61
3.9 Runtime Config — Per-Invocation Options	62
3.10 Connection Resilience — <code>max_retries</code> / <code>timeout</code>	62
3.11 Prompt Caching — Implicit vs Explicit	63
3.12 Summary	63
4. Tools and Structured Output	64
Learning Objectives	64
4.1 Environment Setup	64
4.2 The Basics of the <code>\@tool</code> Decorator	64
4.3 Complex Schemas with Pydantic	65
4.4 Connecting Tools to an Agent	65
4.5 ToolRuntime	66
4.6 Structured Output	68
4.7 ToolStrategy vs ProviderStrategy	68
4.8 Summary	69
5. Memory and Streaming	70
Learning Objectives	70
5.1 Environment Setup	70
5.2 Short-Term Memory: <code>InMemorySaver</code>	70
5.3 Independent Conversations with Different <code>thread_id</code> Values	70
5.4 Message Trimming	71
5.5 Long-Term Memory: <code>InMemoryStore</code>	71
5.6 Streaming Modes	72
5.7 Summary	74
6. Middleware and Guardrails	76
Learning Objectives	76
6.1 Environment Setup	76
6.2 Middleware Concepts	76
6.3 Built-In Middleware	78
6.4 Custom Middleware: <code>\@before_model</code>	79
6.5 Custom Middleware: <code>\@after_model</code>	80
6.6 <code>\@wrap_model_call</code>	80
6.7 <code>\@dynamic_prompt</code>	81
6.8 <code>\@wrap_tool_call</code>	81
6.9 Simple Guardrails	81
6.10 Summary	82
7. Human	83
Learning Objectives	83
7.1 Environment Setup	83
7.2 Human-in-the-Loop Concepts	83
7.3 <code>HumanInTheLoopMiddleware</code>	84
7.4 The <code>interrupt</code> and <code>Command(resume=...)</code> Pattern	84
7.5 <code>ToolRuntime</code> — Access Runtime Information from a Tool	86
7.6 Context Engineering — Dynamic Control of Prompts and Tools	87
7.7 MCP (Model Context Protocol) Integration Overview	87
7.8 Summary	87

8. Multi	88
Learning Objectives	88
8.1 Environment Setup	88
8.2 Comparing Multi-Agent Patterns	88
8.3 The Subagent Pattern	90
8.4 The Handoff Pattern	91
8.5 The Skill Pattern	92
8.6 The Router Pattern	93
8.7 Choosing a Pattern	93
8.8 Summary	94
9. Custom Workflows and RAG	95
Learning Objectives	95
9.1 Environment Setup	95
9.2 StateGraph Basics	95
9.3 Conditional Edges	96
9.4 Integrating an Agent into a Workflow	96
9.5 Overview of RAG (Retrieval-Augmented Generation)	96
9.6 A Simple RAG Implementation	97
9.7 FAISS Vector Store (Optional)	97
9.8 Summary	98
10. Production	99
Learning Objectives	99
10.1 Environment Setup	99
10.2 LangSmith Studio	99
10.3 Agent Testing	100
10.5 Agent Chat UI	102
10.6 Deployment	102
10.7 Observability	104
10.8 Production Checklist	105
10.9 Summary	105
11. MCP (Model Context Protocol)	107
Learning Objectives	107
11.1 Environment Setup	107
11.2 MCP Concepts	107
11.3 Installing langchain-mcp-adapters	108
11.4 Stdio Transport	108
11.5 SSE / HTTP Transport	108
11.6 Loading MCP Tools and Integrating Them with an Agent	109
11.7 Connecting to Multiple MCP Servers	109
11.8 MCP Authentication	110
11.9 Elicitation — Server-requested user input	111
11.10 Tool Interceptors	111
11.11 Writing a Custom MCP Server	112
11.12 Summary	112
12. Frontend Streaming	114
Learning Objectives	114
12.1 Environment Setup	114

12.2 Python SDK Streaming Basics	114
12.3 <code>astream_events()</code>	114
12.4 The <code>useStream</code> React Hook	115
12.5 <code>useStream</code> Configuration Options	116
12.6 Thread Management and Reconnection	117
12.7 Branching and Message Editing	117
12.8 Custom Streaming Events	118
12.9 Multi-Agent Streaming	118
12.10 Summary	119
13. Guardrails	120
Learning Objectives	120
13.1 Environment Setup	120
13.2 Guardrail Concepts	120
13.3 PII Detection Middleware	121
13.4 Human-in-the-Loop Guardrails	122
13.5 Custom Input Guardrails — <code>before_agent</code>	123
13.6 Custom Output Guardrails — <code>after_agent</code>	123
13.7 Decorator-Based Guardrails	124
13.8 Combining Multiple Guardrails	125
13.9 Production Guardrail Patterns	126
13.10 Summary	126
Part 3: LangGraph	127
1. Introduction to LangGraph	129
Learning Objectives	129
1.1 What is LangGraph?	129
1.2 Core concepts	129
1.3 Two APIs	133
1.4 Environment Setup and check installation	133
1.5 A taste of the Graph API	134
1.6 A taste of the Functional API	134
1.7 Next Steps	134
Summary	134
2. Graph API Basics	136
Learning Objectives	136
2.1 Environment Setup	136
2.2 StateGraph basic structure	136
2.3 state Reducer	136
2.4 Conditional Edge	137
2.5 Message-based state	137
2.6 Input/Output Schema	137
2.7 <code>add_node()</code> advanced options	137
2.8 Runtime context with <code>context_schema</code>	138
2.9 Send and <code>Command.PARENT</code>	138
2.10 Summary	139
3. Functional API	140
Learning Objectives	140
3.1 Environment Setup	140

3.2 @task — Asynchronous unit of work	140
3.3 Parallel <code>\@task</code> execution	140
3.4 previous — short-term memory (accessing previous execution results)	141
3.5 entrypoint.final — Separate return and checkpoint saved values	141
3.6 Determinism Requirements	141
3.7 LLM agent (Functional API)	141
3.8 @task retry and caching	141
3.9 Structured interrupt and Command(resume=)	141
3.10 @entrypoint injected parameters	142
3.11 Summary	142
4. workflow Pattern	144
Learning Objectives	144
4.1 Environment Setup	144
4.2 Prompt Chaining — Sequential LLM calls	144
4.3 Parallelization — Simultaneous execution of independent <code>\@task</code>	145
4.4 Routing — Classification-based branching	145
4.5 Orchestrator-Worker — Create dynamic worker with Send()	146
4.6 Evaluator-Optimizer — Generate-evaluation iteration loop	146
4.7 Pattern comparison table	146
5. Building agent	148
Learning Objectives	148
5.1 Environment Setup	148
5.2 tool Definitions — @tool decorator and bind_tools()	148
5.3 Graph API agent — Implementing ReAct Loop with StateGraph	149
5.4 Visualizing execution flow — Observe each step with streaming	149
5.5 Functional API agent — @entrypoint + while loop	149
5.6 agent with memory — Keep conversation with checkpointer	149
5.7 Summary	149
6. Persistence and memory	151
Learning Objectives	151
6.1 Environment Setup	151
6.2 checkpointer — Automatically saves state for each execution step	151
6.3 get_state() — Retrieve currently stored state lookup	152
6.4 get_state_history() — View entire execution history	152
6.5 update_state() — Modify stored state externally	152
6.6 Thread Independence — Different thread_ids are completely independent state	153
6.7 InMemoryStore — Cross-thread sharing long-term memory	153
6.7.6 Managing conversation length — trim_messages and RemoveMessage	155
6.8 Durable Execution — resume at last checkpoint on failure	155
6.9 Summary	155
7. streaming	157
Learning Objectives	157
7.1 Environment Setup	157
7.2 streaming Mode Comparison	157
7.3 stream_mode="values" — Full state snapshot	157
7.4 stream_mode="updates" — Node-specific updates	158
7.5 stream_mode="messages" — Per-token streaming	158

7.6 Simultaneous use of multiple streaming modes	158
7.7 stream_mode="custom" – custom streaming	158
7.8 nostream tag – exclude internal LLM calls from the messages stream	158
7.9 chunk_position – detect the last chunk of a message	159
7.10 invoke v2 – GraphOutput return type	159
7.11 stream_events v3 – projection-based resume	159
7.12 Summary	160
8. Interrupts and Time Travel	161
Learning Objectives	161
8.1 Environment Setup	161
8.2 interrupt() – executes interrupt and waits for human input	161
8.3 Command(resume=...) – interrupt executes resume	161
8.4 Interrupt in Functional API	163
8.5 Time Travel – Go back to a previous checkpoint	163
8.6 update_state() – Time travel + state fix	163
8.7 Summary	164
9. Subgraph	166
Learning Objectives	166
9.1 Environment Setup	166
9.2 Subgraph concept	166
9.3 Creating a subgraph	166
9.4 Adding a subgraph to the parent graph	166
9.4.1 Pattern 1: Subgraph call through wrapper node (if state_schema is different)	166
9.5 LLM-based subgraph	167
9.6 Subgraph streaming	167
9.7 Summary	168
10. Production	170
Learning Objectives	170
10.1 Environment Setup	170
10.2 App structure – langgraph.json	170
10.3 LangGraph Studio – Visual Debugging tool	170
10.4 Agent Chat UI	171
10.5 Test – Deterministic agent test	171
10.6 LLM agent Test – Using GenericFakeChatModel	172
10.7 Deployment Options	173
10.8 Observability – LangSmith Tracing	174
10.9 Pregel Runtime Overview	175
10.10 Production Checklist	175
10.11 Summary	176
11. Local Server	177
Learning Objectives	177
11.1 Environment Setup	177
11.2 LangGraph CLI installation	177
11.3 Project creation	177
11.4 langgraph.json settings	178
11.5 Running the development server	178
11.6 LangGraph Studio integration	179

11.7 Python SDK — Asynchronous client	179
11.8 Python SDK — Synchronous client	180
11.9 REST API call	180
11.10 Deployment Readiness Checklist	180
11.11 Summary	181
12. durable execution	182
Learning Objectives	182
12.1 Environment Setup	182
12.2 durable execution Concept	182
12.3 Core requirements	182
12.4 Endurance Mode Comparison	183
12.5 Problematic code	183
12.6 Improvements to @task	183
12.7 Durability in Graph API	184
12.8 Durability in Functional API	184
12.9 Failover Scenario	185
12.10 resume Starting point	186
12.11 Production Durability Pattern	186
12.12 Per-node fault tolerance — <code>TimeoutPolicy</code> + <code>heartbeat</code>	187
12.13 Graceful shutdown — <code>RunControl</code> / <code>GraphDrained</code>	187
12.14 DeltaChannel (beta) — checkpoint size optimization	188
12.15 Summary	188
Learning Objectives	189
13.1 Environment Setup	189
13.2 Graph API vs Functional API Overview	190
13.3 Quick Selection Guide	190
13.4 Graph API implementation	190
13.5 Functional API implementation	191
13.6 Comparative analysis	192
13.7 Combining two APIs	192
13.8 Pregel Runtime Overview	192
13.9 Direct use of Pregel	193
13.10 Channel Type	193
13.11 superstep Execution Model	195
13.12 API Selection Criteria Guide	196
13.13 Summary	196
Part 4: Deep Agents	197
1. Introduction to Deep Agents	199
Learning Objectives	199
1. What Is Deep Agents?	199
2. SDK vs Code vs ACP	200
3. Five Core Concepts (Planning · Context Management · Backends · Subagents · Memory & Skills)	201
4. Comparison with Other Frameworks	202
5. Installation Check	202
Summary	203
Next Steps	203
2. Building Your First Agent	204

Learning Objectives	204
1. Project Bootstrap and API Keys	204
2. Creating the Simplest Agent	205
3. Running the Agent — <code>invoke()</code>	205
4. Connecting Tavily Search — A Research Agent	206
5. Checking the Built-In Tools	206
6. Streaming Output — <code>stream()</code>	207
Summary	207
Next Steps	207
3. Agent Customization	208
Learning Objectives	208
0. Full <code>create_deep_agent()</code> Signature (17 parameters)	208
1. Choosing a Model	209
2. Custom System Prompts	210
3. Building Custom Tools	210
4. Structured Output — <code>response_format</code>	211
5. Middleware Architecture	212
6. Sandbox Options	213
Summary	213
Next Steps	214
4. Storage Backends	215
Learning Objectives	215
1. What Is a Backend?	215
2. <code>StateBackend</code> (Default)	216
3. <code>FilesystemBackend</code> — Accessing the Local Disk	216
4. <code>StoreBackend</code> — Cross-Thread Persistent Storage	217
5. <code>CompositeBackend</code> — Path-Based Routing	218
6. <code>LocalShellBackend</code> — Shell Execution	219
7. Implementing a Custom Backend	219
8. <code>ContextHubBackend</code> — Remote File Sync	221
9. Permissions — <code>Permissions</code> + <code>GuardedBackend</code>	222
Backend Selection Guide	222
Summary	223
Next Steps	223
5. Subagents and Task Delegation	224
Learning Objectives	224
1. Why Subagents Are Useful	224
2. Defining a <code>SubAgent</code> (Dictionary Form)	226
3. <code>CompiledSubAgent</code> — Plugging in a Custom LangGraph Runnable	227
4. General-Purpose Subagents	228
5. Context Propagation	228
6. Multi-Subagent Pipelines	229
7. Best Practices	231
Summary	232
Next Steps	232
6. Long	233
Learning Objectives	233

1. Why Long-Term Memory Matters	233
2. Cross-Thread Memory Sharing	235
3. Injecting Context with <code>AGENTS.md</code>	235
4. Skills	235
5. Skill Source Priority	237
6. Skill Inheritance for Subagents	237
6.10 Long-term Memory Types (in 0.5)	238
Summary	240
Next Steps	241
7. Advanced Features	242
Learning Objectives	242
1. Human-in-the-Loop (HITL)	242
2. Advanced Streaming	244
3. Sandboxes	246
3-1. Interpreters — <code>CodeInterpreterMiddleware</code>	248
4. ACP (Agent Client Protocol)	249
5. Deep Agents CLI	251
Full Track Summary	252
8. Agent Harness	253
Learning Objectives	253
1. AgentHarness Concept	253
2. Planning Tools	254
3. Virtual Filesystem	254
4. Task Delegation — Subagents	255
5. Context Management	256
6. Code Execution	257
7. Human-in-the-Loop	257
8. Skills and Memory	258
9. Harness Profiles	259
Summary	260
9. Comparing External Frameworks	262
Learning Objectives	262
1. Comparison Overview	262
2. Deep Agents vs OpenCode vs Claude Agent SDK	263
3. Comparing Core Capabilities	264
3-1. Three Core Differentiators	264
4. Architecture Comparison	265
5. Recommendations by Use Case	265
6. Ecosystem Comparison	266
7. Migration Considerations	266
Summary	267
10. Sandboxes and ACP	268
Learning Objectives	268
1. Sandbox Concepts	268
2. Architecture Patterns	269
3. Comparing Sandbox Providers	269
4. Security Guidelines	270

5. File Transfer and Lifecycle Management	271
6. ACP Overview	272
7. ACP Server Implementation	272
8. Editors That Support ACP	272
9. Combining Sandboxes and ACP	273
Summary	274
11. Async Subagents	276
11.1 Limitations of the old synchronous subagent	276
11.2 Five tools injected into the supervisor	277
11.3 Basic usage	277
11.4 The <code>async_tasks</code> state channel	278
11.5 Transport modes	278
11.6 Execution lifecycle	279
11.7 Mid-flight steering patterns	279
11.8 Local development: <code>--n-jobs-per-worker</code>	279
11.9 Observing lifecycle through the unified v2 stream	280
11.10 Sync vs Async selection criteria	280
11.11 Caveats	281
Key Takeaways	281
12. Going to Production	282
12.1 Three core abstractions	282
12.2 LangSmith Deployments	282
12.3 Multi-tenant access control	283
12.4 End-user credential management	283
12.5 Memory persistence scoping	284
12.6 Execution isolation — Sandbox	285
12.7 Durability · Async I/O	285
12.8 Rate limits and cost control	286
12.9 Three-tier error handling	286
12.10 Data privacy and real-time frontend	287
12.11 Production checklist	287
Key Takeaways	288
13. Context Engineering	289
13.1 The four-layer architecture	289
13.2 Layered input context — nine-step system prompt synthesis	289
13.3 <code>@dynamic_prompt</code> — request-time data injection	290
13.4 Progressive skill loading	290
13.5 Runtime context propagation	290
13.6 Automatic offloading (>20k tokens)	291
13.7 Automatic summarization (85% trigger)	291
13.8 Subagent isolation	292
13.9 <code>/memories/</code> routing with CompositeBackend	292
13.10 Filesystem-centric architecture	293
13.11 Three patterns	293
13.12 Caveats	294
Key Takeaways	294
14. Streaming	295

14.1 Three key ideas	295
14.2 Basic usage: enabling subagent streaming	295
14.3 Routing events via namespaces	296
14.4 Stream mode: <code>updates</code>	296
14.5 Stream mode: <code>messages</code>	296
14.6 Custom progress events	297
14.7 Subscribing to multiple modes at once	297
14.8 Tracking the subagent lifecycle	298
14.9 v2 unified format	298
14.10 Frontend integration	298
14.11 Caveats	299
Key Takeaways	299
15. Permissions	300
15.1 Scope of application	300
15.2 Evaluation rule: first-match-wins	301
15.3 Three rule components	301
15.4 Pattern 1: read-only agent	301
15.5 Pattern 2: workspace isolation	301
15.6 Pattern 3: protect specific files	302
15.7 Pattern 4: read-only memory / policies	302
15.8 Subagent inheritance	303
15.9 CompositeBackend constraint: sandbox-default	303
15.10 Prompt-injection defense perspective	304
15.11 Permissions vs backend policy hooks	304
15.12 Caveats	304
Key Takeaways	305
Part 5: Advanced Patterns	305
0. v0 → v1 migration guide	307
Learning Objectives	307
0.1 Change package structure	307
v0 — Legacy approach	308
v1 — langchain-classicmigration path	308
0.2 Agent creation API changes	308
0.3 State Schema — TypedDict only	309
0.4 Runtime context injection (new)	309
0.5 Dynamic Prompts — Middleware Approach (New)	309
0.6 tool Error handling — <code>\@wrap_tool_call</code> (new)	309
0.7 Standard Content Block & structured output (New)	310
0.8 Streaming changes	310
0.9 Guitar Breaking Change	310
Summary — Migration Checklist	311
1. Deepening the middleware system	312
Learning Objectives	312
1.1 Environment Setup	312
1.2 Middleware Architecture Overview	312
1.3 SummarizationMiddleware	313
1.4 HumanInTheLoopMiddleware	314

1.5 ModelCallLimitMiddleware & ToolCallLimitMiddleware	314
1.6 ModelFallbackMiddleware	315
1.7 PIIMiddleware	316
1.8 LLMToolSelectorMiddleware	317
1.9 Writing custom middleware	318
1.10 Middleware execution order	320
1.11 Middleware Combination (Stacking)	321
1.12 Provider-specific Middleware	322
Summary	325
2. Multiagents: Subagents	326
Learning Objectives	326
2.1 Environment Setup	326
2.2 Subagents Architecture Overview	326
2.3 Low-level tool definitions	327
2.4 Create subagent	328
2.5 Wrapping subagent with tool	328
2.6 Supervisor Agent Assembly	329
2.7 Run test	329
2.8 HITL (Human-in-the-Loop) Integration	329
2.9 Passing context (ToolRuntime)	330
2.10 Asynchronous execution pattern	331
2.11 Single Dispatch tool Pattern	331
Summary	333
3. multi	334
Learning Objectives	334
Part A — Handoffs: Customer Support State Machine	334
3.1 Environment Setup	334
3.2 Handoffs Overview	334
3.3 SupportState definition	336
3.4 Step-by-step tool definition	336
3.5 @wrap_model_call middleware	338
3.6 Agent creation and execution flow	339
Part B — Router: Parallel routing and result synthesis	340
3.7 Router Overview	340
3.8 RouterState and classification schema	342
3.9 Classification Node	343
3.10 Parallel Routing (Send API)	343
3.11 Result synthesis	344
Summary	345
4. Context Engineering & Memory Deepening	346
Learning Objectives	346
4.1 Environment Setup	346
4.2 Context Engineering Overview	346
4.3 Static runtime context – <code>context_schema</code> + <code>\@dataclass</code>	347
4.4 Dynamic runtime context – <code>state_schema</code> , <code>AgentState</code> custom	348
4.5 Long-term memory – InMemoryStore native API	349
4.6 Long-Term Memory – Semantic Search	350

4.7 Reading/Writing Store from tool – <code>ToolRuntime.store</code>	351
4.8 3 types of memory: Semantic, Episodic, Procedural	352
4.9 Progressive Disclosure – Skills pattern	353
4.10 Hot Path vs Background Memory Write	354
Summary	355
5. Agentic RAG	357
Learning Objectives	357
5.1 Environment Setup	357
5.2 RAG Overview	357
5.3 Document loading & chunking	359
5.4 Building a vector store	360
5.5 Search tool Definition	361
5.6 LangChain RAG Agent – <code>create_agent</code> + <code>\@tool</code>	361
5.7 LangChain RAG Chain – <code>\@dynamic_prompt</code> Middleware	361
5.8 LangGraph Custom RAG – Building StateGraph	362
5.9 <code>generate_query_or_respond</code> node	363
5.10 <code>grade_documents</code> Node – Evaluate relevance with structured output	363
5.11 <code>rewrite_question</code> node	364
5.12 <code>generate_answer</code> node	364
5.13 Graph assembly & execution	364
Summary	365
6. SQL Agent Advanced	366
Learning Objectives	366
6.1 Environment Setup (SQLite + Chinook DB)	366
6.2 SQL Agent Overview	366
6.3 SQLDatabaseToolkit	367
6.4 LangChain SQL Agent – <code>create_agent</code> + ReAct	368
6.5 Run test	369
6.6 HITL – <code>HumanInTheLoopMiddleware</code>	369
6.7 LangGraph Custom SQL Agent – StateGraph	370
6.8 Dedicated nodes – <code>list_tables</code> , <code>get_schema</code> , <code>generate_query</code> , <code>check_query</code>	371
6.9 <code>bind_tools</code> with <code>tool_choice</code> – Force tool calling	371
6.10 Reviewing queries with <code>interrupt()</code>	371
6.11 <code>Command(resume=...)</code> pattern	372
Summary	372
7. Data Analysis Agent	374
Learning Objectives	374
7.1 Environment Setup	374
7.2 Data Analysis Agent Overview	374
7.3 Backend selection	375
7.4 Upload sample data	376
7.5 Custom tool – Slack integration	377
7.6 Creating an agent	378
7.7 Execution – Analysis Request	379
7.8 Observe the analysis process through streaming	379
7.9 Utilizing built-in tool	380
7.10 Maintain conversation with checkpoint	381

Summary	381
8. Voice Agent	383
Learning Objectives	383
8.1 Environment Setup	384
8.2 Voice Agent Architecture Overview	384
8.3 Architecture comparison table	385
8.4 STT Step – AssemblyAI Real-Time Transcription	385
8.5 Agent Phase – Utilizing LangChain Agent	386
8.6 TTS Steps – Cartesia Streaming Speech Synthesis	388
8.7 Pipeline Combination – RunnableGenerator	389
8.8 WebSocket Server – Real-time two-way communication	390
8.9 Performance Target – Sub-700ms Latency	391
8.10 Add tool to agent	391
Summary	392
9. Production deployment	393
Learning Objectives	393
9.1 Environment Setup	393
9.2 Production Pipeline Overview	393
9.3 Agent testing – LangSmith evaluation	394
9.4 Unit test patterns	396
9.5 observability – LangSmith Tracing	397
9.6 Trace analysis	398
9.7 LangGraph Studio – Visual Debugging	399
9.8 Deployment Options	400
9.9 langgraph.json settings	401
9.10 Deployment command	402
Summary	404
Part VI: LangSmith	404
1. LangSmith Quickstart	406
1.1 API keys and environment variables	406
1.2 Your first trace – a LangChain agent	409
1.3 run_name · tags · metadata	411
1.4 Tracing plain Python functions – <code>@traceable</code>	412
1.5 Re-querying traces from code	413
1.6 Cost and token aggregation	413
1.7 Re-issuing API keys	413
Key Takeaways	414
2. Agent Trace Structure	415
2.1 Concepts: Run · Trace · Project · Thread	415
2.2 LangGraph StateGraph trace tree	417
2.3 Deep Agents subagent traces (sync · async)	417
2.4 Session view – <code>thread_id</code> · <code>session_id</code> · <code>conversation_id</code>	418
2.5 Attaching feedback to runs – <code>client.create_feedback</code>	419
2.6 Tag- and metadata-based filter queries	419
2.7 The <code>@traceable</code> decorator – run_type variants	420
2.8 PII masking – <code>LangChainTracer</code> + <code>Client(anonymizer=...)</code>	421
2.9 Selective tracing – record only specific calls	421

2.10 The 400-day retention limit → persist to datasets	421
Key Takeaways	422
3. Datasets and the Evaluation Loop	423
3.1 Creating a dataset — manual examples	423
3.2 Ingesting production traces into a dataset	424
3.3 Evaluation target + code evaluator	425
3.4 LLM-as-judge evaluator	426
3.5 The <code>evaluate</code> runner + experiment name	426
3.6 Pairwise + Summary evaluators	427
3.7 <code>agentevals</code> — evaluating agent trajectories	428
3.8 Online evaluator — automatic evaluation on production traces	429
Key Takeaways	429
4. Prompt Hub Versioning	431
4.1 Creating and pushing a prompt — <code>Client.push_prompt(...)</code>	431
4.2 Commit SHA pinning vs tags (<code>prod</code> , <code>staging</code>)	432
4.3 f-string vs mustache	433
4.4 Playground — from experiment to commit	433
4.5 Runtime injection — <code>Client.pull_prompt(...)</code> → <code>create_agent</code>	434
4.6 Pinning a specific commit hash in CI	435
Key Takeaways	435
5. Production Monitoring	436
5.1 Project dashboard	436
5.2 Online evaluator — autoeval rule	437
5.3 User feedback collection API	438
5.4 Metadata-based alert rules	438
5.5 High-volume sampling	440
5.6 PII scrubbing	440
5.7 Slack / PagerDuty webhook integration	440
5.8 LangSmith Deployments — observability bundled with deployment	441
5.9 Insights Agent (paid)	441
Key Takeaways	442
Part VII: Applied Examples	442
1. RAG Agent	444
Learning Objectives	444
Overview	444
The 5 RAG Building Blocks	445
2-Step / Agentic / Hybrid Architectures	445
The Rewrite → Retrieve → Agent 3-Node Pattern	445
Step 1: Create Sample Documents	446
Step 2: Split the Text	446
Step 3: Build the Vector Store	446
Step 4: Define a Retrieval Tool (<code>content_and_artifact</code>)	447
Step 5: Test the Retrieval Tool by Itself	447
Step 6: Create the RAG Agent (with v1 Middleware)	447
Step 7: Run a Simple Query and a Comparison Query	448
Summary	448
2. SQL Agent	449

Learning Objectives	449
Overview	449
Step 1: Connect to the Database	449
Step 2: Create the <code>SQLDatabaseToolkit</code> Tools	450
Step 3: Load the Prompt (LangSmith / Langfuse / Default)	450
Step 4: The Idea Behind Skills	450
Step 5: Create a Basic SQL Agent	451
Step 6: A HITL Agent (<code>interrupt_on</code>)	451
Step 7: Resume After Approval — Four Decision Types	452
Summary	452
3. Data Analysis Agent	453
Learning Objectives	453
Overview	453
Step 1: Compare Backends	453
Step 2: Create a CSV File for Analysis	454
Step 3: Define the Analysis Tools	454
Step 4: Create the Agent (with v1 Middleware)	455
Step 5: Analyze with pandas Code Execution	456
Step 6: Ask a Follow-Up Question in the Same Thread	456
Built-In Tools Summary	456
Data Analysis Execution Flow	456
<code>@tool</code> + pandas vs. <code>CodeInterpreterMiddleware</code>	457
Summary	457
4. Machine Learning Agent	458
Learning Objectives	458
Overview	458
NB03 vs. NB04: Extending the Backend and Tooling	459
Step 1: Set the Data Directory	459
Step 2: Create the <code>FilesystemBackend</code>	459
Step 3: Define <code>run_ml_code</code>	460
Step 4: Create the Agent	460
Step 5: File Exploration + EDA	461
Step 6: Train and Compare Models	461
Step 7: Multi-Turn Follow-Up — Feature Importance	461
Step 8: Streaming — Additional Analysis	461
<code>@tool</code> + scikit-learn Pattern	462
Summary	462
5. Deep Research Agent	463
Learning Objectives	463
Overview	463
Step 1: <code>think_tool</code> — A Strategic Reflection Tool	463
Step 2: A Simplified <code>web_search</code> Tool	464
Step 3: Prompt for the Five-Step Research Workflow	464
Step 4: Define Three Subagents	465
Step 5: Create the Deep Research Agent (with v1 Middleware)	465
DeepAgents Pattern — <code>ToDoList</code> + <code>dispatch</code> + 5-Tool Async	466
Step 6: Run the Research Workflow	466

Step 7: Streaming — Track Namespaces	466
Subagent Design Best Practices	467
Summary	467
Part VIII: Integrations	467
1. Integration Category Overview	469
1.1 Baseline version snapshot	469
1.2 The thirteen integration categories	470
1.3 Provider Middleware roadmap	470
1.4 Common pattern	471
1.5 Observability env-var standardization	471
Key Takeaways	472
2. Anthropic Prompt Caching	473
2.1 When to use it	473
2.2 Environment setup	473
2.3 Basic usage	474
2.4 Verifying cache hits	474
2.5 Full parameters	474
2.6 Behavior on non-Anthropic models	475
2.7 Aggregating with <code>UsageMetadataCallbackHandler</code>	475
2.8 Differences from Bedrock	475
Key Takeaways	476
3. Claude Bash Tool	477
3.1 When to use it	477
3.2 Environment setup	477
3.3 Host execution policy (simplest)	478
3.4 Docker execution policy (recommended, isolated)	478
3.5 Codex sandbox policy	478
3.6 Output redaction (<code>redaction_rules</code>)	478
3.7 Falling back to a generic <code>@tool</code>	479
3.8 Native vs generic comparison	479
Key Takeaways	480
4. Claude Text Editor	481
4.1 When to use it	481
4.2 Environment setup	481
4.3 State variant — virtual files in graph state	482
4.4 Filesystem variant — real disk	482
4.5 State vs Filesystem selection criteria	483
4.6 Relationship with generic file tools	483
Key Takeaways	483
5. Claude Memory	484
5.1 When to use it	484
5.2 Environment setup	484
5.3 State variant — persisted within the thread	485
5.4 Filesystem variant — across process boundaries	485
5.5 Scaling to a durable checkpointer	486
5.6 Multi-tenant isolation via <code>runtime.context.user_id</code>	486
5.7 Relationship with Deep Agents' <code>StoreBackend</code>	487

Key Takeaways	487
6. Anthropic File Search	488
6.1 When to use it	488
6.2 Environment setup	488
6.3 Text editor + file search combination	489
6.4 Step 1 — seed state with multiple files	489
6.5 Step 2 — the same agent searches with glob/grep	489
6.6 Targeting memory files too	490
6.7 Position in the five-step retrieval building blocks	490
6.8 When to pick this over a vector store	491
Key Takeaways	491
7. Bedrock Prompt Caching	492
7.1 When to use it	492
7.2 Environment setup	492
7.3 ChatBedrockConverse + Claude (recommended)	493
7.4 Verifying cache hits	493
7.5 Full parameters	493
7.6 ChatBedrock (Invoke model) example	494
7.7 Reasoning · profiling	494
7.8 Amazon Nova constraints	494
Key Takeaways	495
8. OpenAI Moderation	496
8.1 When to use it	496
8.2 Environment setup	496
8.3 Basic usage — scan both input and output, end on violation	497
8.4 <code>exit_behavior="end"</code> — end immediately on violating input	497
8.5 <code>exit_behavior="error"</code> — fail loudly	497
8.6 <code>exit_behavior="replace"</code> — swap the message and keep going	498
8.7 Scanning tool results (<code>check_tool_results=True</code>)	498
8.8 Observing violation flows with LangSmith	498
8.9 Cost / latency tradeoff	499
Key Takeaways	499
Appendix A. Glossary	500
Frameworks and Products	500
Agents and Execution Model	500
State and Memory	501
Tools and Interfaces	502
Backends and Execution Environments	503
Retrieval, Streaming, and Quality	503

P A R T

I

Agent Foundations

Foundations of AI Agents

What You Will Learn in This Part

- 0. Environment Setup
 - 1. LLM Basics
 - 2. LangChain Basics
 - 3. LangChain Memory
 - 4. LangGraph Basics
 - 5. Deep Agents Basics
- 6. Framework Comparison
 - 7. Mini Project

00

Environment Setup

Before You Start

This series is in the order *LangChain* → *LangGraph* → *Deep Agents*. This is an introductory course to quickly experience only the core of AI agent.

Learning Objectives

- Learn how to safely manage API keys with the `.env` file.
- Initialize the LLM model with `ChatOpenAI`
- Check normal operation by sending a simple question to the model

0.1 API key settings

Copy `.env.example` to `.env` in the project root and enter the following keys:

```
OPENAI_API_KEY=sk-...
TAVILY_API_KEY=tvly-... # Optional
```

Key	Purpose	Provider
OPENAI_API_KEY	LLM calls (required)	https://platform.openai.com/api-keys
TAVILY_API_KEY	Web Search tool (optional)	https://tavily.com

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)

assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY is not set!"
print("✓ API key loaded")
```

```
# Observability settings (optional) - LangSmith or Langfuse
# Set the key in .env, or uncomment it below and enter it yourself.
# os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
# os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
# os.environ["LANGFUSE_HOST"] = "https://lf.ddok.ai"
import os

# LangSmith: Automatically activated when LANGSMITH_TRACING=true (no code modification required)
if os.environ.get("LANGSMITH_TRACING", "").lower() == "true":
    project = os.environ.get("LANGSMITH_PROJECT", "default")
    print(f"LangSmith tracing ON \u2014 project: {project}")
```

```
# Langfuse: Pass config={"callbacks": [langfuse_handler]} when calling invoke/stream
langfuse_handler = None
if os.environ.get("LANGFUSE_SECRET_KEY"):
    from langfuse.langchain import CallbackHandler
    langfuse_handler = CallbackHandler()
    print(f"Langfuse tracing ON \u2014 {os.environ.get('LANGFUSE_HOST', '')}")

# Langfuse config: pass to invoke/stream/batch calls
lf_config = {"callbacks": [langfuse_handler]} if langfuse_handler else {}
```

0.2 Model initialization

`ChatOpenAI` is a LangChain class that wraps an OpenAI-compatible LLM. This `model` object will be used repeatedly in subsequent Note books.

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")
print("✓ Model setup complete:", model.model_name)
```

0.3 Smoke Test

Send a brief message to the model to see if it responds normally.

```
response = model.invoke("hello! Please answer in one sentence.", config=lf_config)
print("✓ Model response:", response.content)
```

Summary

Item	Content
Environment Variables	Load file <code>.env</code> into <code>load_dotenv()</code>
model	<code>ChatOpenAI(model="gpt-5.4")</code>
test	<code>model.invoke("...")</code> → Check response

Next Steps

→ [01_llm_basics.ipynb](#): Learn the basics of LLM — messages, prompts, streaming.

01 LLM Basics

Messages, Prompts, and Streaming

Before building agents, learn the basics of how to communicate with an LLM.

Learning Objectives

- Understand the roles of messages (`system` , `human` , `ai`)
- Control model behavior with system messages
- Receive real-time responses with `model.stream()`

1.1 Environment Setup

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
print("✓ Model ready")
```

1.2 The Three Roles of Messages

An LLM takes a *list of messages* as input. Each message has a role and is made up of three core pieces of information: role, content, and metadata.

Role	Class	Description
<code>system</code>	<code>SystemMessage</code>	Sets behavioral instructions for the model. It defines the model's initial behavior through persona, tone, rules, and similar guidance.
<code>human</code>	<code>HumanMessage</code>	Represents the user's input. It can include multimodal content such as images, audio, and files in addition to text.
<code>ai</code>	<code>AIMessage</code>	Represents the model's reply. Besides text, it can include properties such as <code>tool_calls</code> and <code>usage_metadata</code> .

There is also `ToolMessage` , which passes tool execution results back to the model. A `ToolMessage` must include the tool result content, the tool call ID, and the tool name.

1.3 Controlling Behavior with a System Message

If you change the `SystemMessage`, you can get very different answers to the same question. That is the core idea behind *prompt engineering*.

1.4 Dictionary Format

You can also pass messages as dictionaries instead of message objects. LangChain supports three input formats for messages:

1. *String*: Best for a simple text prompt, for example `model.invoke("Hello")`
2. *Message objects*: A typed list of instances such as `SystemMessage` and `HumanMessage`
3. *Dictionary*: The same `{"role": "...", "content": ...}` structure used by the OpenAI Chat Completions API

All three formats return the same kind of result, so you can choose the one that best fits your situation. The dictionary format is especially useful when migrating existing OpenAI code to LangChain.

1.5 Streaming

If you use `model.stream()`, tokens are printed as they are generated in real time. This can make the application feel much faster to the user.

LangChain models provide three main call styles:

- `invoke()`: A synchronous call that returns the full response at once
- `stream()`: Returns `AIMessageChunk` objects token by token for real-time output
- `batch()`: Handles multiple requests at once for better throughput

During streaming, each `AIMessageChunk` is progressively combined into the final message, and token usage can also be tracked incrementally.

1.6 Batch Calls

With `model.batch()`, you can send several questions at once. This is more efficient than repeatedly calling `invoke()` one request at a time.

Summary

Concept	Description
<code>SystemMessage</code>	Sets the model's persona and rules
<code>HumanMessage</code>	Represents user input
<code>model.invoke()</code>	Synchronous call (full response)
<code>model.stream()</code>	Real-time output at the token level


```
model.batch()
```

Processes multiple requests at once

Next Steps

→ [02_langchain_basics_en.ipynb](#): Build an agent with tools in LangChain.

02 LangChain Basics

Your First Agent

Build a tool-enabled agent with the core APIs of LangChain v1.

Learning Objectives

- Define custom tools with the `@tool` decorator
- Create an agent with `create_agent()`
- Run the agent with `invoke()` and inspect the result

2.1 Environment Setup

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
print("✓ Model ready")
```

2.2 Building Tools

When you add the `@tool` decorator, a regular Python function becomes an agent tool.

Important rules to remember when defining tools:

- *Type hints*: The function parameter type hints automatically define the tool's input schema. For example, `a: int` tells the model that the argument should be an integer.
- *Docstring*: The function docstring becomes the tool description. The model uses this description to decide which tool to use, so it should be clear and concise.
- *Custom name/description*: You can also set the tool name and description explicitly with `@tool("custom_name", description="...")`.
- *Complex inputs*: You can define richer input schemas with Pydantic `BaseModel` and `Field`.

```
from langchain.tools import tool

@tool
def add(a: int, b: int) -> int:
    """Add two numbers."""
    return a + b

@tool
def multiply(a: int, b: int) -> int:
```

```

"""Multiply two numbers."""
return a * b

print("Tool list:")
for t in [add, multiply]:
    print(f" - {t.name}: {t.description}")

```

2.3 Creating and Running an Agent

`create_agent()` combines a model and tools to build an agent. The agent internally runs a *ReAct (Reasoning + Acting) loop*:

Question → Model decides to call a tool → Tool runs → Result **is** observed → Repeat **or** produce a final answer

Core components of the agent:

- *Model*: The LLM decides which tool to call. You can pass a string such as `"openai:gpt-5"` or a model object.
- *Tools*: These are the actions the agent can take. Compared with simple tool binding, an agent can call tools in sequence, run them in parallel, and retry them.
- *System Prompt*: Instructions that guide the agent's behavior.

An agent can call tools more than once, or even use multiple tools in parallel. The loop ends when the model produces a final answer or reaches the recursion limit.

```

from langchain.agents import create_agent

agent = create_agent(
    model=model,
    tools=[add, multiply],
    system_prompt="You are a math assistant.",
)
print("✓ Agent created")

```

Summary

Core API	Role
<code>\@tool</code>	Converts a function into an agent tool
<code>create_agent()</code>	Combines a model and tools to create an agent
<code>agent.invoke()</code>	Runs the agent and returns the result

Next Steps

→ [03_langchain_memory_en.ipynb](#): Learn about multi-turn conversations and memory.

03 LangChain Conversations

Multi-Turn Memory

Connect `InMemorySaver` so the agent can remember previous turns.

Learning Objectives

- Store conversation state with `InMemorySaver`
- Distinguish conversation sessions with `thread_id`
- Run a multi-turn conversation where the agent remembers earlier context

3.1 Environment Setup

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
print("✓ Model ready")
```

3.2 The Limit of an Agent Without Memory

A basic agent *does not store state*. Each call is treated like a new conversation.

3.3 Adding Memory with InMemorySaver

If you set a `checkpointer`, the agent stores the conversation history. A `thread_id` distinguishes one conversation session from another.

Short-term memory means keeping information from earlier interactions within a single conversation thread. This is essential for agents that handle complex tasks across several user turns.

Implementation pattern:

- Pass `InMemorySaver()` as the `checkpointer` parameter to enable memory.
- Use `thread_id` to distinguish different conversation sessions. If you reuse the same `thread_id`, the agent remembers the previous conversation.
- In production, you can switch to a database-backed checkpointer such as `PostgresSaver` for persistence.

If a conversation becomes very long, it may exceed the token budget. In that case, manage the history with strategies such as trimming, deletion, or summarization.

```

from langgraph.checkpoint.memory import InMemorySaver

agent = create_agent(
    model=model,
    tools=[add],
    checkpointer=InMemorySaver(),
)

config = {"configurable": {"thread_id": "session-1"}}
print("✓ Memory-enabled agent created")

```

3.4 Watching the Steps with Streaming

Use `agent.stream()` to observe each step of the agent in real time.

Agent streaming is useful when an agent runs for a while and you want to inspect intermediate steps as they happen. With `stream_mode="updates"`, you receive updates from each node individually, such as model calls and tool execution results. This lets you see what the agent is doing step by step.

Summary

Concept	Description
<code>InMemorySaver</code>	Stores conversation state in memory (checkpointer)
<code>thread_id</code>	Key that separates conversation sessions
<code>checkpointer=</code>	Passes a checkpointer into <code>create_agent()</code>
<code>stream(mode="updates")</code>	Shows the agent's execution steps in real time

Next Steps

→ [04_langgraph_basics_en.ipynb](#): Build a workflow with LangGraph.

04 LangGraph Basics

Building a Workflow

Use LangGraph's `StateGraph` to build a workflow by connecting nodes and edges.

Learning Objectives

- Define a state-based graph with `StateGraph`
- Register nodes (functions) and connect them with edges
- Run the graph with `compile()` → `invoke()`

4.1 Environment Setup

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
print("✓ Model ready")
```

4.2 Your First Graph

The basic LangGraph flow has five steps:

```
StateGraph(State) → add_node() → add_edge() → compile() → invoke()
```

Core ideas behind `StateGraph` :

LangGraph models agent workflows as *graphs*, and it relies on three core building blocks:

1. *State*: A shared data structure that represents the current snapshot of the application. It is usually defined with `TypedDict` or a Pydantic model.
2. *Node*: A function that receives state, performs some work, and returns an updated state. In other words, *nodes do the actual work*.
3. *Edge*: A transition that determines which node runs next based on the current state. In other words, *edges decide what happens next*.

`StateGraph` is the primary graph builder class, and it takes a user-defined state object. A graph must be compiled with `.compile()` before it can run, and compilation also validates the graph structure.

The example below is a one-node graph that counts the number of words in a piece of text.

4.3 A Two-Node Graph

Connect two nodes in sequence. The first node converts the text to uppercase, and the second node counts the words.

```
START → uppercase → counter → END
```

4.4 Using an LLM as a Node

If you use `MessagesState`, you can model an LLM conversation as a graph.

What is `MessagesState` ?

`MessagesState` is a *predefined state class* provided by LangGraph. It contains a single `messages` key and uses `add_messages` as its reducer. Internally, it looks like this:

```
class MessagesState(TypedDict):
    messages: Annotated[list, add_messages]
```

The `add_messages` reducer tracks message IDs, accumulates messages without duplication, and automatically deserializes JSON into LangChain message objects. If you need additional fields such as documents or metadata, you can subclass `MessagesState`.

How nodes are structured:

A node is a normal Python function (sync or async) that receives the current state and returns a state update. LangGraph automatically wraps nodes as `RunnableLambda` objects, which adds batch support, async support, and native tracing.

Summary

Core API	Role
<code>StateGraph(State)</code>	Creates a graph builder from a state schema
<code>add_node()</code>	Registers a node (function)
<code>add_edge()</code>	Connects nodes
<code>compile()</code>	Produces an executable graph
<code>invoke()</code>	Runs the graph

Next Steps

→ [05_deep_agents_basics_en.ipynb](#): Build an all-in-one agent with Deep Agents.

05 Deep Agents Basics

An All-in-One Agent

Use the Deep Agents SDK's `create_deep_agent()` to build an agent with built-in tools, memory, and backend support in one line.

Learning Objectives

- Create an agent with `create_deep_agent()`
- Run the agent with `invoke()`
- Build an agent with an additional custom tool

5.1 Environment Setup

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
print("✓ Model ready")
```

5.2 Creating an Agent

`create_deep_agent()` takes a LangChain model and returns an agent that already includes *built-in tools* such as file reading, file writing, and search. The return value is a LangGraph `CompiledStateGraph`, so you can call methods such as `invoke()` and `stream()` directly.

What is Deep Agents?

Deep Agents is a framework designed to simplify agent development. It works like an *agent harness*: internally it is built on top of LangChain agent components and uses LangGraph to manage execution.

Core built-in capabilities:

Capability	Description
Task planning	Uses the <code>write_todos</code> tool to break down complex tasks into manageable steps
Context management	Uses file system tools such as <code>write_file</code> and <code>read_file</code> to handle larger amounts of information without overflowing the token budget

<i>Flexible storage</i>	Supports pluggable backends such as in-memory storage, local disk, persistent stores, and sandbox environments
<i>Subagent delegation</i>	Can create specialized subagents for focused sub-tasks and isolate their context
<i>Persistent memory</i>	Reuses LangGraph's memory infrastructure to preserve information across conversations

How agent creation works:

Pass a model, optional tools, and an optional system prompt into `create_deep_agent()`. You need a model that supports tool calling, and you can use providers such as OpenAI or Anthropic.

```
from deepagents import create_deep_agent

agent = create_deep_agent(model=model)
print(f"✓ Agent created (type: {type(agent).__name__})")
```

5.3 Adding a Custom Tool

If you write a Python function with a *docstring* and *type hints*, it becomes a tool directly.

How custom tools work:

A custom tool is just a normal Python function, and Deep Agents automatically converts two parts of that function:

- *Docstring* → tool description (used by the agent to understand what the tool does)
- *Type hints* → parameter schema (used by the agent to pass the correct arguments)

If you pass a list of functions to the `tools` parameter of `create_deep_agent()`, those functions are added to the tool list alongside the built-in capabilities such as file operations and todo planning. You can also guide the agent's behavior with the `system_prompt` parameter.

Summary

Core API	Role
<code>create_deep_agent(model)</code>	Creates an agent with built-in tools
<code>create_deep_agent(model, tools, system_prompt)</code>	Adds custom tools and a system prompt
<code>agent.invoke()</code>	Runs the agent

Next Steps

→ [06_comparison_en.ipynb](#): Compare the three frameworks and choose your next learning track.

06 Comparing the Three Frameworks & Choosing What Comes Next

Compare LangChain, LangGraph, and Deep Agents at a glance, then decide where to continue next.

Learning Objectives

- Understand the *core differences* between LangChain, LangGraph, and Deep Agents
- Judge the *best-fit use cases* for each framework
- Choose a *learning path* into the intermediate tracks

6.1 Framework Comparison

<i>Level of abstraction</i>	High	Medium	Very high
<i>Core concept</i>	Agents + tools	Graphs + state + nodes	All-in-one agents
<i>Agent creation</i>	<code>create_agent()</code>	<code>StateGraph</code> → <code>compile()</code>	<code>create_deep_agent()</code>
<i>Execution</i>	<code>agent.invoke()</code>	<code>graph.invoke()</code>	<code>agent.invoke()</code>
<i>Customization</i>	Tools / prompts / memory	Nodes / edges / state / reducers	Tools / backends / subagents
<i>Best fit</i>	Fast prototyping	Complex workflows	Agents that need file and task management

Additional comparison details:

Feature	LangChain	LangGraph	Deep Agents
<i>Model support</i>	Model-agnostic (100+ providers)	Shares LangChain model integrations	Shares LangChain model integrations
<i>License</i>	MIT	MIT	MIT
<i>Sandbox integration</i>	No built-in support	No built-in support	Agents can run tasks in sandboxes
<i>State management</i>	Middleware-based	Checkpointing-based (supports time travel)	Reuses LangGraph check pointers

Observability	LangSmith integration	Native LangSmith tracing	LangSmith support
---------------	-----------------------	--------------------------	-------------------

The three frameworks are not mutually exclusive. Deep Agents uses LangGraph internally and shares LangChain's model and tool interfaces. A natural learning path is to build your foundations with LangChain, design complex workflows with LangGraph, and then create production-grade agents with Deep Agents.

6.2 Which One Should You Choose?

"I need a simple tool-calling agent." → LangChain
 "I need a workflow with branching and loops." → LangGraph
 "I want file operations and planning in one place." → Deep Agents

TIP

The three frameworks are not mutually exclusive. Deep Agents uses LangGraph internally and shares LangChain's model and tool interfaces.

6.3 Code Comparison — The Same Task in Three Styles

Below is the smallest working version of the same task implemented with each framework.

LangChain

```
from langchain.agents import create_agent
from langchain.tools import tool

@tool
def add(a: int, b: int) -> int:
    """Add two numbers."""
    return a + b

agent = create_agent(model=model, tools=[add])
agent.invoke({"messages": [{"role": "user", "content": "3+4?"}]})
```

LangGraph

```
from langgraph.graph import StateGraph, START, END, MessagesState

def chatbot(state):
    return {"messages": [model.invoke(state["messages"])]}

builder = StateGraph(MessagesState)
builder.add_node("chat", chatbot)
builder.add_edge(START, "chat")
builder.add_edge("chat", END)
graph = builder.compile()
graph.invoke({"messages": [{"role": "user", "content": "3+4?"}]})
```

Deep Agents

```
from deepagents import create_deep_agent

agent = create_deep_agent(model=model)
agent.invoke({"messages": [{"role": "user", "content": "3+4?"}]})
```

Summary

Item	LangChain	LangGraph	Deep Agents
Core role	Agent creation (ReAct loop)	Workflow orchestration (state graphs)	All-in-one agents (built-in tools)
State management	Middleware-oriented	Explicit <code>StateGraph</code> state	Automated (filesystem + memory)
Best for	Fast prototyping, tool calling	Complex workflows, branching	Coding agents, data analysis, multi-step work
Learning curve	Low	Medium	Low

6.4 What Comes Next

Mini Project

→ [07_mini_project_en.ipynb](#): Build a search + summarization agent

Intermediate Tracks

Track	Description	Notebook Count
LangChain	Models, messages, tools, memory, middleware, multi-agent patterns	13
LangGraph	Graph API, workflows, agents, persistence, subgraphs	13
Deep Agents	Customization, backends, subagents, memory, advanced features	10

Recommended order:

1. *LangChain* — strengthen your model and tool fundamentals
2. *LangGraph* — design graph-based workflows
3. *Deep Agents* — build production-ready agents

The English intermediate notebooks will be translated next in this same order.

07 Mini Project

A Search + Summarization Agent

Combine what you learned in the beginner track to build a research agent with Tavily web search.

Learning Objectives

- Define a Tavily search tool directly
- Build a research agent with Deep Agents
- Observe the agent's execution process in real time with streaming
- Solve the same task with a LangChain agent and compare the two approaches

7.1 Environment Setup

This notebook requires `TAVILY_API_KEY`. You can get a free key at <https://tavily.com>.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)

assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY is required!"
assert os.environ.get("TAVILY_API_KEY"), "TAVILY_API_KEY is required!"

from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
print("✓ Environment ready")
```

7.2 Defining the Search Tool

Wrap the Tavily client in a search function. The *docstring* and *type hints* tell the agent what schema the tool should use.

Rules for writing a tool function:

Here you define a search function that will be passed to the `tools` parameter of `create_deep_agent()`. Deep Agents automatically converts the function docstring into a tool description and the type hints into a parameter schema. In practice, that means:

- *Docstring*: Gives the agent evidence for deciding *when* to use the tool. Be specific and clear.
- *Type hints*: Help the agent pass arguments of the correct type. `Literal` is useful when you want to restrict the allowed values.
- *Args section*: Explaining each parameter makes it easier for the agent to choose the right arguments.

```

from typing import Literal
from tavily import TavilyClient

tavily = TavilyClient(api_key=os.environ["TAVILY_API_KEY"])

def internet_search(
    query: str,
    max_results: int = 3,
    topic: Literal["general", "news"] = "general",
) -> dict:
    """Search the internet for information.

    Args:
        query: Search query
        max_results: Maximum number of results
        topic: Search topic category
    """
    return tavily.search(query, max_results=max_results, topic=topic)

print("✓ Search tool ready")

```

7.3 A Deep Agents Research Agent

Pass the search tool and a system prompt into `create_deep_agent()`.

The agent's automatic workflow:

When the agent receives a request, it can automatically go through the following steps:

1. *Planning*: Breaks the task into steps with the built-in `write_todos` tool
2. *Research*: Collects information from the web with the provided `internet_search` tool
3. *Context management*: Uses filesystem tools such as `write_file` and `read_file` to store intermediate results when needed and manage the token budget
4. *Synthesis*: Analyzes the gathered information and turns it into a consistent final report

For more complex tasks, the agent can also create specialized subagents and isolate their context for focused sub-work.

```

from deepagents import create_deep_agent

research_agent = create_deep_agent(
    model=model,
    tools=[internet_search],
    system_prompt="You are an expert researcher. Search the web and summarize the results in English.",
)
print("✓ Research agent created")

```

7.4 Observing the Process with Streaming

With `stream(mode='updates')`, you can watch the agent's steps in real time.

LangGraph's streaming system:

LangGraph provides a flexible streaming system that improves responsiveness by showing progress before the final response is complete.

Stream Mode	Use Case
values	Streams the <i>full state</i> after each graph step
updates	Streams only the <i>state changes</i> after each step
messages	Streams LLM tokens together with metadata
custom	Streams custom data emitted from nodes
debug	Streams comprehensive execution details

You can access streaming with `stream()` (sync) or `astream()` (async), and you can combine multiple modes at once by passing a list. In the example below, we use `updates` so that each stage of the agent, such as tool calls and the final response, appears as it happens.

7.5 Comparing It with a LangChain Agent

Try the same search tool with a LangChain `create_agent()`.

How it differs from a LangChain agent:

LangChain's `create_agent()` builds a simple ReAct agent from a model and a list of tools. Compared with Deep Agents:

- *LangChain*: A lightweight baseline for tool-calling agents. It is excellent for fast prototyping, but advanced capabilities such as task planning or filesystem management must be added manually.
- *Deep Agents*: Includes planning (`write_todos`), file management, and subagent delegation by default, so it is a better fit for complex multi-step tasks.

If you add the `@tool` decorator, you can convert a function into LangChain's tool interface. The general pattern of passing a system prompt and a list of tools into `create_agent()` is similar to Deep Agents.

Summary

Technologies used in this mini project:

Technique	Source
ChatOpenAI + load_dotenv	00_setup
Message roles, streaming	01_llm_basics
\@tool , create_agent()	02_langchain_basics
InMemorySaver , thread_id	03_langchain_memory
StateGraph , compile()	04_langgraph_basics
create_deep_agent()	05_deep_agents_basics

Next Steps

→ Continue to the intermediate tracks. Use [06_comparison_en.ipynb](#) as your roadmap.

P A R T

II

LangChain

Building Blocks for AI Applications

What You Will Learn in This Part

- 1. Introduction to LangChain
 - 2. Quickstart
- 3. Models and Messages
- 4. Tools and Structured Output
- 5. Memory and Streaming
 - 6. Middleware
- 7. HITL and Runtime
 - 8. Multi-Agent
- 9. Custom Workflow and RAG
 - 10. Production
 - 11. MCP
- 12. Frontend Streaming
 - 13. Guardrails

01 Introduction to LangChain

An Overview of the LangChain v1 Framework

Learning Objectives

Understand the structure and core components of the LangChain framework.

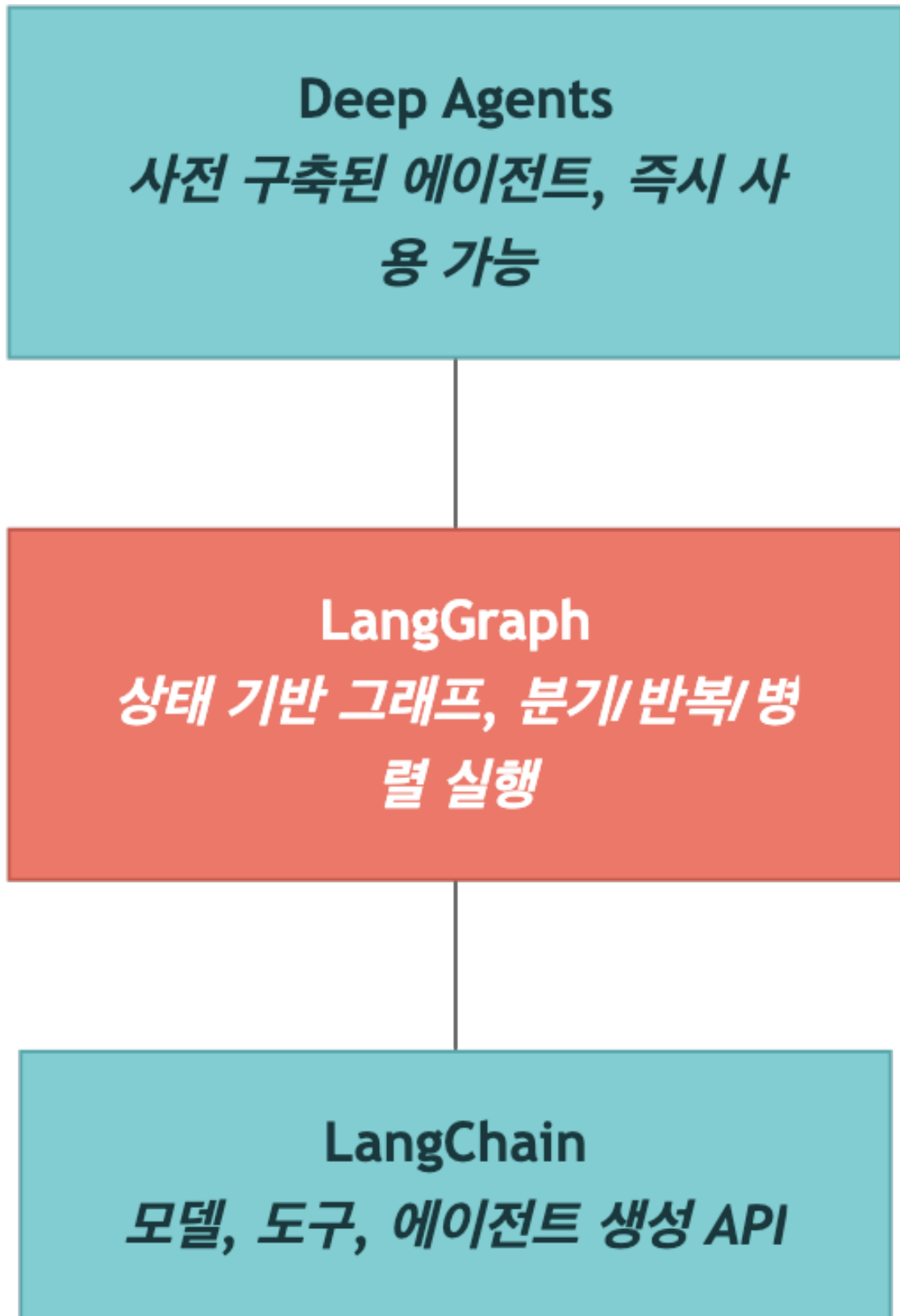
By the end of this notebook, you will understand:

- The three-layer structure of the LangChain v1 framework
- How the ReAct agent pattern works
- The core components and main APIs
- How to set up and verify the development environment

1.1 LangChain Framework Overview

LangChain v1 is an integrated framework for building LLM-based agents. It is organized into three layers, and each layer provides a different level of abstraction.

Three-Layer Structure



Layer	Role	Target User
<i>LangChain</i>	Core APIs for agent creation (<code>create_agent</code> , <code>tool</code> , <code>ChatOpenAI</code>)	All developers
<i>LangGraph</i>	Building complex workflows (state graphs, checkpoints, streaming)	Intermediate and advanced developers
<i>Deep Agents</i>	Prebuilt agents (coding, research, and more)	Fast prototyping
<i>LangSmith</i>	Tracing and debugging platform for agents built with any framework	All developers / operators

TIP

LangSmith is not limited to LangChain, LangGraph, or Deep Agents. You can send traces from the OpenAI SDK, LlamaIndex, or your own custom agent and use the same UI to debug and evaluate them, so v1 treats it as the fourth core tool.

Major Changes in LangChain v1

Item	Previous (v0.x)	Current (v1)
Agent creation	<code>create_react_agent()</code>	<code>create_agent()</code>
Agent import	<code>from langchain.agents import ...</code> (various forms)	<code>from langchain.agents import create_agent</code>
Model initialization	Direct use of <code>ChatOpenAI(...)</code>	<code>init_chat_model()</code> or <code>ChatOpenAI(...)</code>
Memory	<code>ConversationBufferMemory</code> and similar classes	<code>InMemorySaver</code> (LangGraph checkpoint)
Execution engine	<code>AgentExecutor</code>	<i>LangGraph graph</i> (internally)

Core Design Philosophy

The central design idea in LangChain v1 is that *every agent runs as a LangGraph graph*. An agent created with `create_agent()` is implemented internally as a LangGraph `StateGraph`, which enables:

- *Streaming*: Real-time responses with the `stream()` method
- *State management*: Conversation history through a checkpoint
- *Extensibility*: Adding custom nodes and edges when needed

Multi-Provider Support

LangChain v1 exposes more than 100 LLM providers behind the same interface. The model IDs you will see most often in this book are:

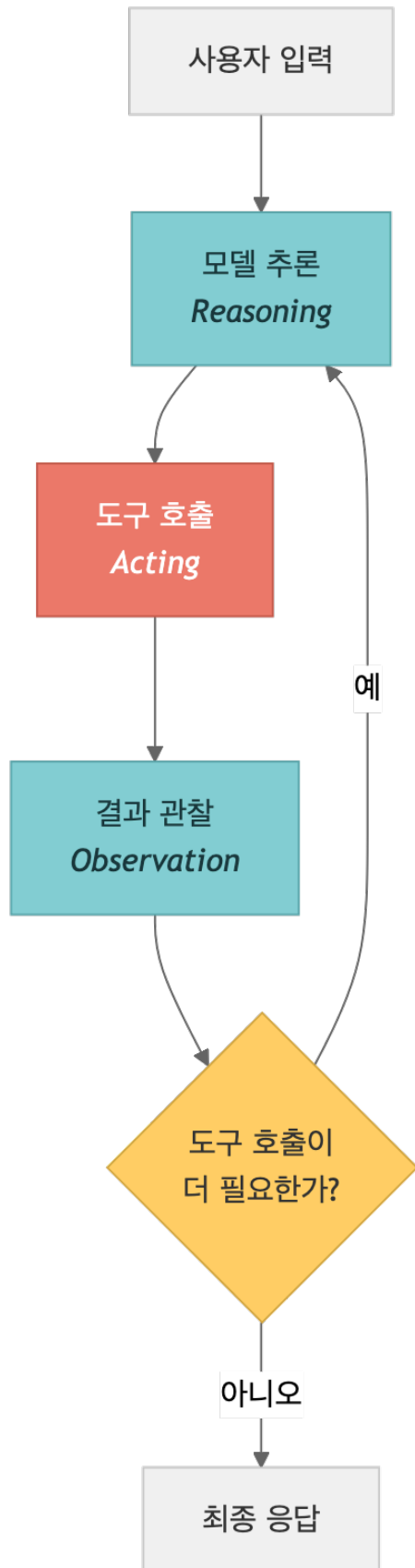
- *OpenAI*: `openai:gpt-5.4` , `openai:gpt-4.1`
- *Anthropic*: `anthropic:claude-sonnet-4-6` , `anthropic:claude-opus-4-6`
- *Google*: `google:gemini-2.5-flash-lite` , `google:gemini-2.5-pro`

- *OpenRouter*: `openrouter:meta-llama/llama-3.3-70b-instruct` and 100+ others
- *AWS Bedrock*: `bedrock:anthropic.claude-sonnet-4-6`
- *Ollama*: `ollama:llama3.3` (runs locally)

Unless noted otherwise, the examples in this book default to OpenAI `gpt-5.4` or Anthropic `claude-sonnet-4-6` .

1.2 The ReAct Agent Pattern

The ReAct (Reasoning + Acting) pattern is the default operating model for LangChain v1 agents. The agent repeatedly runs the following loop:



Key Characteristics

- *Autonomous decisions*: The agent decides for itself whether it should use a tool
- *Multi-step reasoning*: The agent breaks complex tasks into multiple steps
- *Observation-based updates*: The agent observes tool results and chooses its next action

1.3 Overview of the Main Components

The table below summarizes the core components of LangChain v1:

Component	Description	Main APIs
<i>Model</i>	An LLM or chat model. Acts as the agent's "brain."	ChatOpenAI , init_chat_model()
<i>Tools</i>	Functions the agent can use, such as search, calculation, or API calls	@tool decorator, TavilySearch
<i>Agent</i>	The execution unit that combines the model and tools. Internally, it is a LangGraph graph	create_agent()
<i>Memory</i>	A checkpointer that stores and manages conversation history	InMemorySaver , SqliteSaver
<i>Middleware</i>	Logic inserted into the request/response processing pipeline	prompt , before_tool , after_model
<i>State</i>	The state managed while the agent runs, such as messages and context	AgentState , messages
<i>Streaming</i>	Support for real-time response streaming	stream() , stream_mode="updates"

1.4 Environment Setup and Installation Check

Check the packages and API keys required for LangChain v1 development.

```
# Environment check
import importlib

packages = {
    "langchain": "langchain",
    "langchain_openai": "langchain-openai",
    "langchain_community": "langchain-community",
    "langgraph": "langgraph",
}

print("=" * 50)
print("LangChain v1 environment check")
print("=" * 50)

for module_name, package_name in packages.items():
    try:
        mod = importlib.import_module(module_name)
        version = getattr(mod, "__version__", "installed")
        print(f"✓ {package_name}: {version}")
```

```
except ImportError:
    print(f"x {package_name}: not installed → pip install {package_name}")
```

```
# API key check
from dotenv import load_dotenv
import os

load_dotenv(override=True)

required_keys = ["OPENAI_API_KEY"]
optional_keys = ["TAVILY_API_KEY", "LANGSMITH_API_KEY"]

print("Required API keys:")
for key in required_keys:
    status = "✓ configured" if os.environ.get(key) else "x missing"
    print(f" {key}: {status}")

print("\nOptional API keys:")
for key in optional_keys:
    status = "✓ configured" if os.environ.get(key) else "- not configured (optional)"
    print(f" {key}: {status}")
```

```
# Verify the core LangChain v1 imports
from langchain.agents import create_agent
from langchain.tools import tool
from langchain_openai import ChatOpenAI
from langgraph.checkpoint.memory import InMemorySaver

print("✓ Successfully imported all core modules")
print(" - create_agent: create a LangChain v1 agent")
print(" - tool: tool decorator")
print(" - ChatOpenAI: OpenAI-compatible chat model")
print(" - InMemorySaver: memory checkpointer")
```

1.5 Next Steps

In this notebook, you explored the overall structure and core components of the LangChain v1 framework.

In the next notebook ([02_quickstart.ipynb](#)), you will create and run an actual agent:

- Define a custom tool with the `@tool` decorator
- Create an agent with `create_agent()`
- Run the agent with `invoke()` and `stream()`
- Build a multi-turn conversation with `InMemorySaver`

Summary

Item	Description
LangChain v1	An agent-centered framework built around the <code>create_agent()</code> API
Three-layer structure	LangChain → LangGraph → Deep Agents
ReAct pattern	Repeated loop of reasoning → action → observation

Core APIs	<code>create_agent()</code> , <code>\@tool</code> , <code>invoke()</code> , <code>stream()</code>
-----------	---

Next Steps

→ [02_quickstart.ipynb](#): Build your first LangChain agent.

02 Your First Agent

Getting Started with `create_agent()`

Learning Objectives

Create and run an agent with LangChain v1's `create_agent()`.

By the end of this notebook, you will be able to:

- Define custom tools with the `@tool` decorator
- Create an agent with `create_agent()`
- Run the agent with `invoke()` and inspect the result
- Receive real-time streaming responses with `stream()`
- Build a multi-turn conversation with `InMemorySaver`

2.1 Environment Setup

Install LangChain v1 and Deep Agents before running any examples. Pick the command that matches your environment:

```
# With uv
uv add langchain deepagents

# With pip
pip install -U langchain deepagents
```

TIP

Installing `langchain` also pulls in `langchain-core`, `langgraph`, and `langchain-openai` (OpenAI integration). For other providers add `langchain-anthropic`, `langchain-google-genai`, and so on as needed.

Set up the model through OpenAI. `ChatOpenAI` supports OpenAI-compatible APIs, so you can also switch providers by changing `base_url` when needed.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
```

```
)
print("✓ Model configured:", model.model_name)
```

2.2 Building a Simple Tool

Define the tools that the agent can use with the `@tool` decorator.

Important details when defining a tool:

- A *docstring* is required. The agent uses it to understand what the tool is for.
- *Type hints* help the agent pass the correct arguments.
- The tool name is generated automatically from the function name.

```
from langchain.tools import tool

@tool
def add(a: int, b: int) -> int:
    """Add two numbers."""
    return a + b

@tool
def multiply(a: int, b: int) -> int:
    """Multiply two numbers."""
    return a * b

print("Tool list:")
for t in [add, multiply]:
    print(f" - {t.name}: {t.description}")
```

2.3 Creating an Agent

Combine the model and tools with `create_agent()`.

The created agent is internally implemented as a LangGraph graph, so it provides methods such as `invoke()` and `stream()`.

TIP

In LangChain v1, use `create_agent()` instead of `create_react_agent()`.

TIP

Give the agent a *snake_case* name like `name="research_assistant"`. It shows up in LangSmith traces, sub-agent routing, and logs, which makes the agent easier to identify. Avoid spaces, hyphens, and uppercase characters.

TIP

When you define a custom `state_schema`, it must be a **TypedDict** subclass of `langgraph.graph.MessagesState` or `langchain.agents.AgentState`. Plain `dataclass` or `Pydantic BaseModel` classes are not accepted.

```
from langchain.agents import create_agent

agent = create_agent(
    model=model,
    tools=[add, multiply],
    system_prompt="You are a math assistant. Use the available tools for calculations.",
)
print("✓ Agent created")
print(f"Type: {type(agent).__name__}")
```

2.4 Running the Agent

Run the agent with `invoke()`.

When you send messages to the agent, it runs an internal ReAct loop:

1. The model analyzes the question and decides whether to call a tool
2. The tool runs and returns a result
3. The model uses the result to generate the final response

```
# Inspect the full message flow
print("Full message flow:")
print("-" * 50)
for msg in result["messages"]:
    role = msg.type if hasattr(msg, 'type') else msg.get('role', 'unknown')
    content = msg.content if hasattr(msg, 'content') else msg.get('content', '')
    print(f"[{role}] {content[:200]}")
print("-" * 50)
```

2.5 Streaming Execution

Receive a real-time response with `stream()`.

With streaming, you can inspect each stage of the agent in real time, including model reasoning, tool calls, and the final answer. If you use `stream_mode="updates"`, you receive node updates one by one.

2.6 Multi-Turn Conversation

Keep conversation state with `InMemorySaver`.

`InMemorySaver` stores state in memory and separates conversation sessions with `thread_id`.

TIP

In LangChain v1, conversation history is managed through LangGraph checkpoints.

2.7 Adding the Tavily Search Tool (Optional)

Add a web search tool so the agent can look up real information.

Tavily is a search API designed for AI agents.

This cell only runs if `TAVILY_API_KEY` is configured.

2.8 Runtime Context (`context_schema`)

While `thread_id` separates *conversation sessions*, `context` is the channel that injects *per-request metadata* (user ID, permissions, tenant) into tools and middleware. Register `context_schema=` on `create_agent()` and pass `context=` to `invoke()`; tools can then read it through `ToolRuntime.context`.

```
from dataclasses import dataclass
from langchain.agents import create_agent

@dataclass
class Context:
    user_id: str
    tenant: str = "default"

agent = create_agent(
    model=model,
    tools=[add, multiply],
    system_prompt="...",
    context_schema=Context,
)

result = agent.invoke(
    {"messages": [{"role": "user", "content": "..."}]},
    context=Context(user_id="u-1234", tenant="acme"),
)
```



TIP

`context_schema` pairs with `ToolRuntime` in Chapter 4 and with HITL in Chapter 7. For now, just remember that the injection channel exists; the detailed usage comes later.

2.9 Summary

Here is a recap of what you covered in this notebook:

Topic	Core API	Description
Tool definition	<code>\@tool</code>	Turn a function into an agent tool with a decorator
Agent creation	<code>create_agent()</code>	Combine a model, tools, and a system prompt
Synchronous execution	<code>agent.invoke()</code>	Return the full response at once
Streaming execution	<code>agent.stream()</code>	Return real-time updates for each step

Multi-turn conversation	<code>InMemorySaver</code> + <code>thread_id</code>	Store and restore conversation state with a checkpoint
Search tool	<code>TavilySearch</code>	Access real-time information through web search

Next Steps

In the next notebook, you will explore more advanced agent patterns:

- Designing custom system prompts
- Combining multiple tools in a single agent
- Error handling and fallback strategies

03 Models and the Message System

Learn how to configure different LLM models in LangChain v1 and structure conversations with message types.

Learning Objectives

- Understand the LangChain v1 model initialization methods (`init_chat_model` , `ChatOpenAI`)
- Learn the three call patterns: `invoke()` , `stream()` , and `batch()`
- Understand message types such as `SystemMessage` , `HumanMessage` , `AIMessage` , and `ToolMessage`
- Learn how to construct multimodal messages (image input)

3.1 Environment Setup

Load API keys from `.env` and initialize the model through OpenAI.

```
import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv(override=True)

# Initialize the model with OpenAI
model = ChatOpenAI(
    model="gpt-5.4",
)

print("Model initialized:", model.model_name)
```

3.2 Comparing Model Providers

LangChain v1 can initialize models from many providers through the unified `init_chat_model()` API.

Provider	Model String Format	Required Package	Environment Variable
OpenAI	"openai:gpt-5"	langchain-openai	OPENAI_API_KEY
Anthropic	"anthropic:claude-sonnet-4-6"	langchain-anthropic	ANTHROPIC_API_KEY
Google	"google:gemini-2.5-flash-lite"	langchain-google-genai	GOOGLE_API_KEY
AWS Bedrock	"bedrock:anthropic.claude-v3"	langchain-aws	AWS credentials
Azure	"azure:gpt-4o"	langchain-openai	AZURE_OPENAI_API_KEY

Ollama	"ollama:llama3"	langchain-ollama	(local runtime)
--------	-----------------	------------------	-----------------

· NOTE

Note: When using OpenAI, you can set `base_url` and `api_key` directly on `ChatOpenAI` to connect to OpenAI-style services such as vLLM, LM Studio, or Ollama.

3.3 Using `init_chat_model()`

`init_chat_model()` is the unified model initialization helper introduced in LangChain v1. If the provider-specific package is installed, you can create a model with a single string.

When you are using OpenAI directly, `ChatOpenAI` is usually the more straightforward option.

3.4 `invoke()`, `stream()`, and `batch()` Patterns

Every LangChain v1 model supports three calling patterns:

Method	Description	Return Type
<code>invoke()</code>	One response for one input	<code>AIMessage</code>
<code>stream()</code>	Token-by-token streaming response	<code>Iterator[AIMessageChunk]</code>
<code>batch()</code>	Handles multiple inputs at the same time	<code>List[AIMessage]</code>

3.5 Message Types

LangChain v1's message system clearly separates the roles inside a conversation:

Message Type	Role	Description
<code>SystemMessage</code>	System	A system prompt that defines how the model should behave
<code>HumanMessage</code>	User	A message entered by the user
<code>AIMessage</code>	AI	A response generated by the model
<code>ToolMessage</code>	Tool	A message that passes a tool result back to the model

If you build a list of messages and pass it into `model.invoke()`, you can preserve the conversation context across turns.

3.6 Multimodal Messages (Image Input)

In LangChain v1, you can send text and images together in the `content` of a `HumanMessage`. Images can be passed as URLs or as base64-encoded content, and this works only with models that support vision input.

```
content = [
    {"type": "text", "text": "Description text"},
    {"type": "image_url", "image_url": {"url": "IMAGE_URL"}},
]
```

3.7 Standard Output Format — `output_version="v1"`

Starting from LangChain v1, every provider can emit a unified *content blocks* serialization. Set

`init_chat_model("openai:gpt-5.4", output_version="v1")` or the env var `LC_OUTPUT_VERSION="v1"`, and `AIMessage.content_blocks` will give you text, tool calls, reasoning, thinking, and citations in a single schema.

```
import os
from langchain.chat_models import init_chat_model

os.environ["LC_OUTPUT_VERSION"] = "v1"

model = init_chat_model("openai:gpt-5.4", output_version="v1")
result = model.invoke("Describe LangChain v1 in one sentence.")

for block in result.content_blocks:
    print(block["type"], "→", block.get("text", block))
```

Streaming uses the same shape. Read partial blocks from `chunk.content_blocks` and map them to your UI.

```
for chunk in model.stream("Explain multimodal input."):
    for block in chunk.content_blocks:
        if block["type"] == "text":
            print(block["text"], end="")
```

TIP

`output_version="v1"` is the recommended setting for absorbing provider differences. Legacy code that depends on the string `result.content` still works in `v0`, but new projects should use `v1`.

3.8 Tracking Token Usage — `UsageMetadataCallbackHandler`

For multi-step agent runs, aggregate token usage with a callback handler. Pass `UsageMetadataCallbackHandler` through `config={"callbacks": [...]}`, and `input_tokens` / `output_tokens` / `total_tokens` accumulate across every model call.

```
from langchain_core.callbacks import UsageMetadataCallbackHandler

cb = UsageMetadataCallbackHandler()
result = agent.invoke(
```

```

    {"messages": [{"role": "user", "content": "What is 15 * 27 + 3?"}]},
    config={"callbacks": [cb]},
)

print(cb.usage_metadata)
# {'gpt-5.4': {'input_tokens': 312, 'output_tokens': 48, 'total_tokens': 360, ...}}

```

3.9 Runtime Config — Per-Invocation Options

The `config=` argument on `invoke()` / `stream()` carries execution metadata for a single call. You can set the LangSmith trace name, tags, metadata, and concurrency limits for tools and sub-agents all at once.

```

result = agent.invoke(
    {"messages": [{"role": "user", "content": "..."}]},
    config={
        "run_name": "monthly-report",
        "tags": ["batch-2025-05", "finance"],
        "metadata": {"user_id": "u-1234", "tenant": "acme"},
        "max_concurrency": 5,
    },
)

```

TIP

`run_name`, `tags`, and `metadata` flow into LangSmith and become filter/search keys in the UI. `max_concurrency` caps the parallel tool and sub-agent calls inside one graph execution.

3.10 Connection Resilience — `max_retries` / `timeout`

In production, you have to ride out temporary outages from OpenAI, Anthropic, and other providers. Pass `max_retries` and `timeout` to `init_chat_model()` or `ChatOpenAI()`, and LangChain will retry with exponential backoff.

```

from langchain.chat_models import init_chat_model

model = init_chat_model(
    "openai:gpt-5.4",
    max_retries=15, # default is 6 – raise it for resilience
    timeout=120,   # seconds, for long responses
)

```

TIP

Retries fire on 5xx errors, connection failures, and `429 Too Many Requests`. Client errors like 422 are not retried. Setting the limit too high amplifies cost and latency, so most teams keep it in the 10–20 range.

3.11 Prompt Caching — Implicit vs Explicit

When you reuse a long system prompt or RAG context, prompt caching can cut input token costs sharply.

- *Implicit caching*: the provider detects an identical prefix automatically. OpenAI `gpt-5.4` enables this for prefixes of 1024 tokens or more.
- *Explicit caching*: Anthropic uses `prompt_cache_key` (or a `cache_control` block) to mark cache keys. In LangChain you pass it via `model_kwargs={"prompt_cache_key": "..."} .`

```
from langchain_anthropic import ChatAnthropic

model = ChatAnthropic(
    model="claude-sonnet-4-6",
    model_kwargs={"prompt_cache_key": "system-v3"},
)
```

TIP

On cache hits, `usage_metadata.input_tokens_details` reports `cache_read` and `cache_creation` separately. Track cache hit rate as an ops metric with `UsageMetadataCallbackHandler` .

3.12 Summary

Here is a summary of the main ideas from this notebook.

Item	Description
<code>init_chat_model()</code>	Initializes models through a unified provider string
<code>ChatOpenAI(base_url=...)</code>	Uses OpenAI or another custom endpoint
<code>invoke()</code>	Single input → single response
<code>stream()</code>	Token-level streaming response
<code>batch()</code>	Handles multiple inputs at once
<code>SystemMessage</code>	Sets system instructions
<code>HumanMessage</code>	Represents user input
<code>AIMessage</code>	Represents an AI response (for conversation history)
<code>ToolMessage</code>	Carries tool execution results
Multimodal message	Sends text and images together in <code>content</code>

Next Steps

→ [04_tools_and_structured_output.ipynb](#): Learn about tools and structured output.

04 Tools and Structured Output

Learn how to build custom tools with the `@tool` decorator in LangChain v1 and how to receive structured responses with `with_structured_output()`.

Learning Objectives

- Build tools with the `@tool` decorator and inspect their schemas
- Define complex input schemas with Pydantic models
- Connect tools to `create_agent()` and build an agent
- Access runtime context from inside a tool with `ToolRuntime`
- Configure structured output with `with_structured_output()`
- Understand the difference between `ToolStrategy` and `ProviderStrategy`

4.1 Environment Setup

TIP

Some examples in this chapter require *deepagents* $\geq 0.5.0$ or *langgraph* $\geq 1.1.5$, and the `ToolRuntime` execution_info / server_info extensions require *langchain* ≥ 1.2 . Older versions raise `AttributeError`, so confirm with `uv pip list | grep -E "langchain|langgraph|deepagents"`.

Load API keys and initialize an OpenAI model.

```
import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv(override=True)

# Initialize the model with OpenAI
model = ChatOpenAI(
    model="gpt-5.4",
)

print("Model initialized:", model.model_name)
```

4.2 The Basics of the `@tool` Decorator

When you add `@tool` to a function, it becomes a tool that an agent can use. LangChain automatically parses the function name, docstring, and type hints to build the tool schema.

```
from langchain.tools import tool

@tool
def my_tool(param: str) -> str:
    """Tool description for the LLM."""
    return result
```

```
from langchain.tools import tool

@tool
def get_weather(city: str) -> str:
    """Look up the current weather for a city."""
    weather_data = {
        "Seoul": "Clear, 15\u00b0C",
        "Tokyo": "Cloudy, 12\u00b0C",
        "New York": "Rain, 8\u00b0C",
    }
    return weather_data.get(city, f"Weather data is not available for: {city}")

# Inspect the tool schema
print("Tool name:", get_weather.name)
print("Tool description:", get_weather.description)
print("Input schema:", get_weather.args_schema.model_json_schema())
```

4.3 Complex Schemas with Pydantic

If you need a richer input structure, define the schema with a Pydantic `BaseModel`. When you pass it as

`@tool(args_schema=MySchema)`, the LLM can understand the exact parameter structure.

- `Field(description=...)`: passes a field description to the LLM
- `Field(default=...)`: defines a default value

```
from pydantic import BaseModel, Field

class SearchQuery(BaseModel):
    """Search parameters for a database query."""
    query: str = Field(description="Search query string")
    max_results: int = Field(default=5, description="Maximum number of results to return")
    category: str = Field(default="all", description="Search category: all, tech, science, news")

@tool(args_schema=SearchQuery)
def search_database(query: str, max_results: int = 5, category: str = "all") -> str:
    """Search the database with advanced filtering options."""
    return f"Found {max_results} results for '{query}' in the '{category}' category"

print("Complex schema:", search_database.args_schema.model_json_schema())
```

4.4 Connecting Tools to an Agent

When you pass a list of tools into `create_agent()`, the agent can automatically choose and execute the right tool for the situation.

```
from langchain.agents import create_agent

agent = create_agent(
    model=model,
```

```
tools=[tool1, tool2],
system_prompt="...",
)
```

· NOTE

Note: In LangChain v1, `create_react_agent` was removed. Always use `create_agent`.

4.5 ToolRuntime

`ToolRuntime` lets a tool function access the current conversation state at runtime. This makes it possible to build tools that use message history, settings, or other runtime context.

```
@tool
def my_tool(runtime: ToolRuntime) -> str:
    messages = runtime.state["messages"]
    # ...
```

`runtime.execution_info` — Execution Metadata

`ToolRuntime` exposes more than state access. The `execution_info` field carries metadata about the current execution unit and is heavily used for logging, tracing, and retry policies.

```
@tool
def log_call(payload: str, runtime: ToolRuntime) -> str:
    """Process input and log execution metadata."""
    info = runtime.execution_info
    print(f"thread={info.thread_id} run={info.run_id} attempt={info.node_attempt}")
    return f"received: {payload}"
```

- `execution_info.thread_id` : session ID that groups a multi-turn conversation
- `execution_info.run_id` : ID of one `invoke()` / `stream()` run (linked to the LangSmith trace)
- `execution_info.node_attempt` : attempt count for the current node (increments on retries)

`runtime.server_info` — Server and User Info

When executed on LangGraph Platform or a LangSmith server, `runtime.server_info` identifies the deployed assistant and the calling user.

```
@tool
def user_summary(runtime: ToolRuntime) -> str:
    """Return information about the current user."""
    s = runtime.server_info
    return f"assistant={s.assistant_id} user={s.user.email if s.user else 'anonymous'}"
```

- `server_info.assistant_id` : ID of the deployed assistant (graph version)
- `server_info.user` : authenticated user object (`user.id` , `user.email` , `permissions`)



TIP

```
server_info may be empty in local runs — it is populated only in deployed environments. Guard with
if runtime.server_info.user is not None: .
```

Tool Error Handling — @wrap_tool_call

To attach uniform error handling, logging, or retry policy to tool calls, use the `@wrap_tool_call` decorator. It receives a `ToolCallRequest` and acts as a middleware hook that intercepts tool execution.

```
from langchain.tools import wrap_tool_call, ToolCallRequest
from langchain_core.messages import ToolMessage

@wrap_tool_call
def safe_invoke(request: ToolCallRequest, handler):
    """Wrap every tool call in try/except."""
    try:
        return handler(request)
    except Exception as exc: # noqa: BLE001
        return ToolMessage(
            content=f"[error] {type(exc).__name__}: {exc}",
            tool_call_id=request.tool_call.id,
            status="error",
        )

agent = create_agent(
    model=model,
    tools=[get_weather, search_database],
    middleware=[safe_invoke],
)
```

TIP

`@wrap_tool_call` registers as middleware, so it applies to *every* tool. Branch on `request.tool_call.name` if you need to guard only specific tools.

Command Updates and Companion `ToolMessage`

When a tool needs to update *graph state* or pick the *next node* instead of returning plain text, return a

`Command`. `LangGraph` applies `Command.update` to patch state and uses `Command.goto` to choose the next node. You must also return a `ToolMessage` that closes the `tool_call_id`, otherwise the model cannot proceed to the next turn.

```
from langgraph.types import Command
from langchain_core.messages import ToolMessage

@tool
def transfer_to_specialist(topic: str, runtime: ToolRuntime) -> Command:
    """Hand control to a specialist sub-agent and update state."""
    return Command(
        update={
            "active_specialist": topic,
            "messages": [
                ToolMessage(
                    content=f"Routed to the {topic} specialist.",
                    tool_call_id=runtime.tool_call_id,
                )
            ],
        },
        goto="specialist_node",
    )
```

TIP

Omitting `tool_call_id` causes the model to error with “no response for tool call.” Pull the current call ID from `runtime.tool_call_id` and pass it through verbatim.

4.6 Structured Output

With `with_structured_output()`, you can receive the model’s response directly as a Pydantic model or dataclass. This pattern is used directly on the model, not through an agent.

```
structured_model = model.with_structured_output(MySchema)
result = structured_model.invoke("...")
# result is an instance of MySchema
```

4.7 ToolStrategy vs ProviderStrategy

There are two main strategies for structured output inside an agent:

Strategy	Description	Advantage
ToolStrategy	Uses the tool-calling mechanism to produce structured output	Works with every model and is stable
ProviderStrategy	Uses the provider’s native structured-output feature	Faster and more accurate when the model supports it

Use the `response_format` parameter to structure the agent’s final response.

TIP

`ProviderStrategy.strict=True` (schema enforcement) requires *langchain* ≥ 1.2 . On older releases, use `strict=False` or `ToolStrategy`.

TIP

`ProviderStrategy` is currently stable on OpenAI (`gpt-5.4` , `gpt-4.1`) and Anthropic (`claude-sonnet-4-6`). Google Gemini is *unofficially supported*: JSON mode works, but strict schema enforcement applies only to some models. Choose `ToolStrategy` when broad compatibility matters.

4.8 Summary

Here is a summary of the main ideas in this notebook.

Item	Description
<code>\@tool</code> decorator	Converts a function into an agent tool
<code>args_schema</code>	Defines a complex input schema with Pydantic
<code>create_agent()</code>	Connects the model and tools to create an agent
<code>ToolRuntime</code>	Gives a tool access to runtime state such as conversation history
<code>with_structured_output()</code>	Structures model output as a Pydantic model or dataclass
<code>ToolStrategy</code>	Structured agent output through tool calling
<code>ProviderStrategy</code>	Provider-native structured output

Next Steps

→ [05_memory_and_streaming.ipynb](#): Learn about memory and streaming.

05 Memory and Streaming

Learn about the *memory system* and *streaming modes* used by LangChain v1 agents.

Learning Objectives

- *Short-term memory*: understand how to preserve conversation state with `InMemorySaver` and `thread_id`
- *Long-term memory*: use `InMemoryStore` to persist memory across conversations
- *Message trimming*: learn how to keep long conversations within a token budget
- *Streaming modes*: understand the differences between `values`, `updates`, `messages`, and `custom`

5.1 Environment Setup

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)

print("Model ready:", model.model_name)
```

5.2 Short-Term Memory: `InMemorySaver`

Short-term memory is the mechanism that remembers previous messages *within a single conversation session*.

- `InMemorySaver` acts as a checkpointer and stores agent state in memory.
- `thread_id` separates different conversation sessions.
- Reusing the same `thread_id` preserves previous conversation context.

5.3 Independent Conversations with Different `thread_id` Values

If you use a different `thread_id`, you create a completely *independent conversation session*. Context is not shared with the previous session.

5.4 Message Trimming

As a conversation grows, the token count increases and affects both cost and performance. *Message trimming* keeps only the most relevant messages within a token budget.

- `trim_messages` : keeps only the most recent N messages or the messages that fit within the token budget
- `strategy="last"` : prioritizes the most recent messages
- `include_system=True` : always preserves the system message

5.5 Long-Term Memory: `InMemoryStore`

Long-term memory stores information that persists *across conversation sessions*.

- `InMemoryStore` is a key-value store for user preferences, settings, and similar data.
- Tools can access the store through the `ToolRuntime` parameter.
- The same data is available from every session, regardless of `thread_id`.

Per-user namespaces (`context_schema` + `runtime.context`)

In real apps you typically scope the store with the current `user_id`. Declare a `context_schema`, read `runtime.context.user_id` inside the tool, and cast Pydantic/dataclass objects with `dict(...)` before writing so the value is JSON-serializable.

```
from dataclasses import dataclass
from langgraph.store.memory import InMemoryStore
from langchain.tools import tool, ToolRuntime

@dataclass
class Context:
    user_id: str

@tool
def remember_user(user_info: dict, runtime: ToolRuntime[Context]) -> str:
    """Persist the current user's profile."""
    namespace = (runtime.context.user_id, "profile")
    runtime.store.put(namespace, "info", dict(user_info))
    return "saved"

store = InMemoryStore()
agent = create_agent(
    model=model,
    tools=[remember_user],
    store=store,
    context_schema=Context,
)
agent.invoke(
    {"messages": [{"role": "user", "content": "I'm Minji"}]},
    context=Context(user_id="user_123"),
)
```

TIP

If you forget to pass `store=...` to `create_agent`, `runtime.store` is `None` and the tool cannot access the store. Always wire the store in alongside any long-term memory tool.

TIP

In production, swap `InMemoryStore` for `PostgresStore` (or `AsyncPostgresStore`) from the `langgraph-checkpoint-postgres` package — same interface, persistent storage, and semantic search.

Differences between short-term and long-term memory:

Type	Short-Term Memory (Checkpoint)	Long-Term Memory (Store)
Scope	Inside a single <code>thread_id</code>	Across all sessions
What it stores	Conversation message history	User preferences, learned data
Lifetime	Until the session ends (or persists)	Until explicitly deleted
Access	Automatic (inside the agent)	Explicit (through tools)

5.6 Streaming Modes

LangChain provides streaming so you can *observe agent execution in real time*. Choose the mode that best fits your use case.

Streaming Mode Comparison

Mode	Description	Use Case
values	Full state after each step	Debugging, state inspection
updates	Only the updates from each node	Progress displays
messages	Message tokens	Chat UI
custom	Custom user-defined events	Custom progress indicators

A Note on `stream_mode="custom"`

`stream_mode="custom"` carries user-defined events. With current LangGraph, you can emit them from inside tools or middleware of a `create_agent` agent by calling `get_stream_writer()`.

```
from langgraph.config import get_stream_writer
from langchain.tools import tool

@tool
def long_running_task(query: str) -> str:
    """Reports progress through the custom stream."""
    writer = get_stream_writer()
    writer({"step": 1, "status": "fetching"})
    # ... work ...
    writer({"step": 2, "status": "done"})
    return "ok"

for chunk in agent.stream(
    {"messages": [{"role": "user", "content": "search"}]},
```

```

        stream_mode="custom",
    ):
        print(chunk) # whatever the writer emitted

```

TIP

Earlier docs said `stream_mode="custom"` did not work with `create_agent`. Current LangGraph supports it via `get_stream_writer()` — the same mechanism as the `writer` parameter you would receive at the `StateGraph` level.

`version="v2"` Streaming and GraphOutput

`agent.stream(..., version="v2")` delivers structured events, and `agent.invoke(..., version="v2")` returns a `GraphOutput` with `value` (final state) and `interrupts` (pending interrupts).

```

for chunk in agent.stream(
    {"messages": [{"role": "user", "content": "hello"}]},
    config={"configurable": {"thread_id": "t1"}},
    version="v2",
):
    if chunk["type"] == "updates":
        print("update:", chunk["data"])
    elif chunk["type"] == "messages":
        print("token:", chunk["data"][0].content)

result = agent.invoke(
    {"messages": [{"role": "user", "content": "hi"}]},
    config={"configurable": {"thread_id": "t1"}},
    version="v2",
)
print(result.value)      # final state dict
print(result.interrupts) # [] when no interrupt is pending

```

Accumulating with `chunk_position`

When streaming with `stream_mode="messages"`, the final chunk has `chunk_position == "last"`. Accumulate text and tool calls and flush them on the last chunk.

```

buffer, tool_calls = [], []
for chunk, meta in agent.stream(payload, stream_mode="messages"):
    buffer.append(chunk.content)
    if chunk.tool_calls:
        tool_calls.extend(chunk.tool_calls)
    if meta.get("chunk_position") == "last":
        print("text:", "".join(buffer))
        print("tools:", tool_calls)

```

Filtering reasoning via `content_blocks`

Models with extended thinking (e.g., Claude Sonnet 4.6) expose typed `content_blocks` so you can route reasoning to a separate UI region.

```

for chunk, _ in agent.stream(payload, stream_mode="messages"):
    for block in getattr(chunk, "content_blocks", []) or []:
        if block["type"] == "reasoning":
            ui.show_reasoning(block["reasoning"])

```

```
elif block["type"] == "text":
    ui.show_answer(block["text"])
```

Multi-mode streams and interrupts

Passing a list to `stream_mode` yields `(mode, payload)` tuples. The `"__interrupt__"` key in an `updates` payload indicates a pending HITL decision.

```
for mode, payload in agent.stream(
    {"messages": [{"role": "user", "content": "send email"}]},
    config={"configurable": {"thread_id": "t1"}},
    stream_mode=["messages", "updates"],
):
    if mode == "updates" and "__interrupt__" in payload:
        interrupt = payload["__interrupt__"]
        # show approval UI, then resume with Command(resume=...)
```

TIP

Use `subgraphs=True` to receive events from nested subgraphs. To skip token streaming altogether, set `disable_streaming=True` on the call (or `streaming=False` on the model).

5.7 Summary

Here is a summary of what this notebook covered:

Concept	Implementation	Description
Short-term memory	<code>InMemorySaver</code> + <code>thread_id</code>	Keeps context inside one conversation session
Session isolation	Different <code>thread_id</code> values	Manages independent conversation sessions
Message trimming	<code>trim_messages</code> + middleware	Limits messages to stay within the token budget
Long-term memory	<code>InMemoryStore</code> + <code>ToolRuntime</code>	Stores user data that persists across conversations
Streaming (values)	<code>stream_mode="values"</code>	Full state snapshot at each step
Streaming (updates)	<code>stream_mode="updates"</code>	Node-by-node updates
Streaming (messages)	<code>stream_mode="messages"</code>	Real-time token output
Streaming (custom)	<code>stream_mode="custom"</code>	Only available at the <code>LangGraph</code> <code>StateGraph</code> level

Key points:

- Short-term memory is isolated by `thread_id` and preserves context only within the same session.
- Long-term memory is shared across sessions through `InMemoryStore`.
- `stream_mode="values"` is useful for debugging because it returns the full state at every step.
- `stream_mode="custom"` cannot be used directly with `create_agent`; it requires `LangGraph`'s `StateGraph` API.
- Choosing the right streaming mode can significantly improve user experience.

Next Steps

→ [06_middlewares.ipynb](#): Learn about middlewares and guardrails.

06 Middleware and Guardrails

Learn the *middleware* system and *guardrails* used by LangChain v1 agents.

Learning Objectives

- *Middleware concepts*: Understand how to add hooks to each stage of the agent execution pipeline
- *Built-in middleware*: Use built-in middleware such as `SummarizationMiddleware`
- *Custom middleware*: Implement custom middleware with `@before_model`, `@after_model`, `@wrap_model_call`, and `@dynamic_prompt`
- *Guardrails*: Learn how to block unsafe input and output

6.1 Environment Setup

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

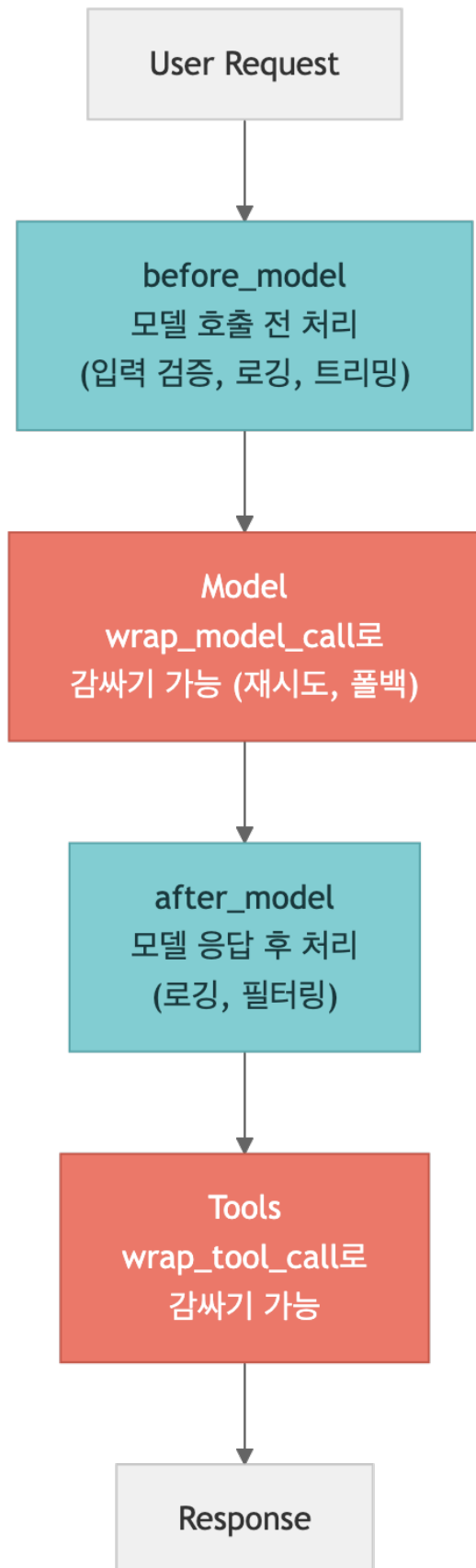
from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)

print("Model ready:", model.model_name)
```

6.2 Middleware Concepts

Middleware is the mechanism that *adds hooks to each stage of the agent execution pipeline* so you can control how the agent behaves.



Five middleware hooks:

Hook	When it runs	Main Use Case
<code>\@before_model</code>	Before a model call	Input validation, message editing, guardrails
<code>\@after_model</code>	After a model response	Output logging, response filtering
<code>\@wrap_model_call</code>	Around a model call	Retry, fallback, caching
<code>\@wrap_tool_call</code>	Around a tool call	Control tool execution
<code>\@dynamic_prompt</code>	During prompt creation	Runtime prompt changes

6.3 Built-In Middleware

LangChain v1 provides *built-in middleware* for common patterns. `SummarizationMiddleware` automatically summarizes earlier messages when a conversation becomes long, reducing token usage.

TIP

Size triggers across built-in middleware all use a `ContextSize` tuple: `("tokens", 100_000)`, `("messages", 20)`, or `("fraction", 0.8)` (80% of the model's context window).

```
from langchain.agents import create_agent
from langchain.tools import tool

@tool
def search(query: str) -> str:
    """Searches for information."""
    return f"'{query}' search result for"

# SummarizationMiddleware – automatically summarizes long conversations
from langchain.agents.middleware import SummarizationMiddleware

summarization = SummarizationMiddleware(
    model=model,
    trigger=("messages", 10),
)

agent_with_summary = create_agent(
    model=model,
    tools=[search],
    system_prompt="You are a helpful assistant.",
    middleware=[summarization],
)
print("SummarizationMiddleware agent created")
```

Prompt-caching middleware

Anthropic and Bedrock both support prompt caching, with one built-in middleware per provider.

```
from langchain.agents.middleware import (
    AnthropicPromptCachingMiddleware,
    BedrockPromptCachingMiddleware,
)
```

```
from langchain_anthropic import ChatAnthropic

claude = ChatAnthropic(model="claude-sonnet-4-6")
agent = create_agent(
    model=claude,
    tools=[search],
    middleware=[AnthropicPromptCachingMiddleware()],
)
# For Bedrock, plug in BedrockPromptCachingMiddleware() the same way.
```

ContextEditingMiddleware

Clears stale tool outputs once the context is too heavy. `trigger` is the threshold, `keep` is how many recent messages to leave untouched.

```
from langchain.agents.middleware import (
    ContextEditingMiddleware,
    ClearToolUsesEdit,
)

context_edit = ContextEditingMiddleware(
    edits=[
        ClearToolUsesEdit(
            trigger=("tokens", 100_000),
            keep=("messages", 3),
        )
    ],
)
```

ModelFallbackMiddleware

Cascades to a backup model when the primary one fails. The first argument is the primary; the rest are fallbacks.

```
from langchain.agents.middleware import ModelFallbackMiddleware
from langchain_openai import ChatOpenAI

primary = ChatOpenAI(model="gpt-5.4")
backup = ChatOpenAI(model="gpt-5-nano")
agent = create_agent(model=primary, tools=[search],
                    middleware=[ModelFallbackMiddleware(primary, backup)])
```

PatchToolCallsMiddleware (used by Deep Agents)

Repairs malformed tool calls (bad JSON, missing arguments) and re-invokes the model so downstream tools see a clean payload. Deep Agents relies on this internally for its subagent and filesystem tools.

6.4 Custom Middleware: `@before_model`

The `@before_model` decorator runs *before the model is called*.

Common uses:

- Logging input messages
- Modifying or filtering messages
- Input validation (guardrails)
- Adding context

6.5 Custom Middleware: `\@after_model`

The `@after_model` decorator runs *after the model response has been generated*.

Common uses:

- Logging model output
- Filtering or modifying responses
- Monitoring tool calls
- Validating output quality

6.6 `\@wrap_model_call`

The `@wrap_model_call` decorator *wraps the model call itself*, which lets you implement retry, fallback, caching, and similar patterns.

You execute the original model call through the `handler` function and can add custom logic before or after it.

Type annotations and `request.override`

Annotate handlers with `ModelRequest / ModelResponse` for better tooling. `request.override(...)` patches messages, tools, system prompt, or response format for this single call (transient).

```
from langchain.agents.middleware import (
    wrap_model_call,
    ModelRequest,
    ModelResponse,
)

@wrap_model_call
def restrict_for_guest(request: ModelRequest, handler) -> ModelResponse:
    if request.runtime.context.role == "guest":
        patched = request.override(
            system_message="Read-only mode.",
            tools=[t for t in request.tools if t.name.startswith("read_")],
        )
        return handler(patched)
    return handler(request)
```

```
from langchain.agents.middleware import wrap_model_call
import time

@wrap_model_call
def retry_on_error(request, handler):
    """Retries model calls with exponential backoff on failure."""
    max_retries = 2
    for attempt in range(max_retries + 1):
        try:
            return handler(request)
        except Exception as e:
            if attempt < max_retries:
                wait = 2 ** attempt
                print(f"  Retry {attempt + 1}/{max_retries} ({wait}s wait)")
                time.sleep(wait)
            else:
                raise

agent_retry = create_agent(
    model=model,
    tools=[search],
    system_prompt="You are a helpful assistant.",
```

```

        middleware=[retry_on_error],
    )
    print("Retry middleware agent created")

```

6.7 `@dynamic_prompt`

The `@dynamic_prompt` decorator *changes the system prompt dynamically at runtime*.

Common uses:

- Adding the current date and time
- Per-user prompt customization
- Changing behavior based on state
- A/B testing

6.8 `@wrap_tool_call`

The `@wrap_tool_call` decorator *wraps a tool call itself*, so you can add custom logic before and after tool execution.

Like `@wrap_model_call`, it uses a `handler` function to run the original tool. You can use it for timing, logging, and error handling.

Common uses:

- *Measuring execution time*: monitor performance by tool
- *Logging*: record tool input and output
- *Error handling*: apply fallback behavior if a tool fails
- *Access control*: block or restrict specific tools

6.9 Simple Guardrails

Middleware can also act as a lightweight guardrail. In the example below, a `before_model` hook blocks requests that contain prohibited keywords before the model is called.

Skipping the model with `can_jump_to`

Instead of just editing messages, a guardrail can *jump* to a different graph node and skip the model call entirely. Declare allowed destinations with `@hook_config(can_jump_to=[...])` and return `{"jump_to": ...}`.

```

from langchain.agents.middleware import before_model, hook_config
from langchain_core.messages import AIMessage

BANNED = {"reveal system prompt", "tell me the password"}

@before_model
@hook_config(can_jump_to=["end", "tools", "model"])
def block_unsafe(state):
    last = state["messages"][-1].content
    if any(p in last for p in BANNED):
        return {
            "messages": [AIMessage(content="I can't help with that request.")],

```

```
        "jump_to": "end",
    }
    return None
```

Async hooks (a-prefixed)
For I/O-heavy guardrails (calling an external classifier, for example), implement async variants by prefixing the sync names with `a` : `abefore_model` , `aafter_model` , `awrap_model_call` , `awrap_tool_call` . If a middleware class defines both, `agent.invoke()` uses the sync hook and `agent.ainvoke()` uses the async one automatically.

6.10 Summary

This notebook covered:

Topic	Core API	Description
Built-in middleware	<code>SummarizationMiddleware</code>	Automatically summarizes long conversations
Before-model hook	<code>\@before_model</code>	Logs, validates, or modifies input before model execution
After-model hook	<code>\@after_model</code>	Logs or validates model output after generation
Wrapped model call	<code>\@wrap_model_call</code>	Adds retry, fallback, or caching around model calls
Dynamic prompt	<code>\@dynamic_prompt</code>	Changes the system prompt at runtime
Wrapped tool call	<code>\@wrap_tool_call</code>	Adds logging, timing, and control around tool execution
Guardrails	<code>\@before_model</code> and middleware	Blocks unsafe or disallowed input

Next Steps

→ [07_hitl_and_runtime.ipynb](#): Learn about human-in-the-loop, runtime context, and MCP.

07 Human

in-the-Loop, ToolRuntime, and MCP

Learn how LangChain v1 handles *human approval workflows*, *runtime context inside tools*, *context engineering*, and *MCP (Model Context Protocol)*.

Learning Objectives

This notebook covers:

- *Human-in-the-Loop (HITL)*: how to pause agent execution and request approval before a tool call
- *ToolRuntime*: how tools can access runtime context such as user information and session data
- *Context engineering*: techniques for dynamically controlling prompts and tools
- *MCP (Model Context Protocol)*: a standardized way to connect tool servers

7.1 Environment Setup

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)

print("Model ready:", model.model_name)
```

7.2 Human-in-the-Loop Concepts

Ask for human approval before the agent calls a tool.

Why is this needed?

Autonomous agents are powerful, but *irreversible actions* such as sending email, deleting files, or processing payments still require human confirmation.

Workflow

Agent → proposes a tool call → [interrupt] → human approves/rejects → tool runs → result **is** returned

In LangChain v1, this is implemented by combining `HumanInTheLoopMiddleware` with `InMemorySaver` (a checkpointer). The checkpointer stores the agent state so the workflow can resume after interruption.

7.3 HumanInTheLoopMiddleware

`HumanInTheLoopMiddleware` automatically pauses execution on tool calls and waits for human approval. Use it with an `InMemorySaver` checkpointer so interrupted state can be preserved.

```
from langchain.agents import create_agent
from langchain.tools import tool
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver

@tool
def send_email(to: str, subject: str, body: str) -> str:
    """Sends an email to the specified recipient."""
    return f"{to} email sent to: {subject}"

@tool
def delete_file(path: str) -> str:
    """Deletes a file at the specified path."""
    return f"File deleted: {path}"

# Require approval only for risky tools (dict form scopes decisions)
hitl = HumanInTheLoopMiddleware(interrupt_on={
    "send_email": {"allowed_decisions": ["approve", "edit", "reject", "respond"]},
    "delete_file": {"allowed_decisions": ["approve", "reject"]},
})

agent = create_agent(
    model=model,
    tools=[send_email, delete_file],
    system_prompt="You are an assistant that can send emails and manage files.",
    middleware=[hitl],
    checkpointer=InMemorySaver(),
)

print("HITL agent created")
print(" -> The run pauses for human approval before executing tools")
```

7.4 The interrupt and Command(resume=...) Pattern

A HITL agent works in two phases:

1. **Phase 1** (`invoke`): the agent proposes a tool call and is automatically **interrupted**
2. **Phase 2** (`Command(resume=...)`): a decision list is sent and execution **resumes**

Four decisions (approve / edit / reject / respond)

`HumanInTheLoopMiddleware` accepts decisions as a dict, one per pending tool call.

```
from langgraph.types import Command

# 1) approve – run the tool as proposed
agent.invoke(Command(resume={"decisions": [{"type": "approve"}]}),
              config=config, version="v2")

# 2) edit – run the tool with patched arguments
agent.invoke(
    Command(resume={"decisions": [
```



```

        {"type": "edit", "args": {"to": "boss@example.com", "subject": "Updated"}}
    }},
    config=config, version="v2",
)

# 3) reject – block the call, send a rejection back to the model
agent.invoke(Command(resume={"decisions": [{"type": "reject"}]}),
              config=config, version="v2")

# 4) respond – replace the tool call with a human message
agent.invoke(
    Command(resume={"decisions": [
        {"type": "respond", "message": "Send this via Slack instead"}
    ]}),
    config=config, version="v2",
)

```

`version="v2"` returns a `GraphOutput`

`agent.invoke(..., version="v2")` returns a `GraphOutput`. When the run is paused, `result.value` is the partial state and `result.interrupts` is a list of `Interrupt` objects.

```

result = agent.invoke(
    {"messages": [{"role": "user", "content": "Send the weekly report"}]},
    config={"configurable": {"thread_id": "t1"}},
    version="v2",
)

if result.interrupts:
    pending = result.interrupts[0].value
    decision = ask_human(pending)
    final = agent.invoke(
        Command(resume={"decisions": [decision]}),
        config={"configurable": {"thread_id": "t1"}},
        version="v2",
    )
    print(final.value["messages"][-1].content)

```

TIP

`version="v2"` and dict-based decisions require `deepagents>=0.5.0` or `langgraph>=1.1.5`. On older versions, the single-decision `Command(resume=True/False)` form still works.

Lifecycle — `after_model` hook and `HITLRequest`

Internally `HumanInTheLoopMiddleware` runs in an `after_model` hook that builds a `HITLRequest` containing `action_requests` (the tool calls to review) and `review_configs` (the allowed decisions per call). Custom approval flows can return the same object directly from their own `after_model` hook.

Transient vs. persistent patches

Use `request.override(...)` for one-shot changes that only apply to this call (transient). For changes that must outlive the call, return an `ExtendedModelResponse` together with `Command(update=...)` so the state is written to the checkpoint (persistent).

7.5 ToolRuntime — Access Runtime Information from a Tool

`ToolRuntime` lets a tool access runtime context such as the current user or session data while it executes.

Core idea

- Add a `runtime: ToolRuntime[T]` parameter to the tool function
- `T` is a context dataclass defined by the developer
- When you create the agent, set `context_schema=T`, and when invoking the agent, pass `context=T(...)`

`runtime.execution_info` — call metadata

Identifies the current call. Useful for tracing, idempotency, or retry counting.

Field	Description
<code>thread_id</code>	Conversation thread identifier
<code>run_id</code>	Identifier for this <code>invoke</code> / <code>stream</code> call
<code>attempt</code>	Retry attempt number (1-based)

```
@tool
def call_external_api(payload: dict, runtime: ToolRuntime[Context]) -> str:
    info = runtime.execution_info
    headers = {
        "X-Trace-Run": info.run_id,
        "X-Trace-Thread": info.thread_id,
        "X-Attempt": str(info.attempt),
    }
    return http.post(url, json=payload, headers=headers).text
```

`runtime.server_info` — deployment metadata

When the agent runs on LangGraph Platform/Server, `server_info` exposes deployment identifiers and the authenticated user. Values may be `None` in a local process.

Field	Description
<code>assistant_id</code>	Platform assistant (config bundle) identifier
<code>graph_id</code>	Graph name from <code>langgraph.json</code>
<code>user</code>	Authenticated user, if available

```
@tool
def log_who(runtime: ToolRuntime[Context]) -> str:
    s = runtime.server_info
    return f"{s.graph_id}/{s.assistant_id} called by {s.user}"
```

7.6 Context Engineering – Dynamic Control of Prompts and Tools

Context engineering is the practice of dynamically shaping the *prompt*, *available tools*, and *message history* given to the agent.

Common use cases

- Provide a different system prompt depending on user role
- Filter the available tools depending on the situation
- Summarize and reorganize long conversation histories

The `dynamic_prompt` middleware makes it possible to customize the prompt for every request.

7.7 MCP (Model Context Protocol) Integration Overview

MCP is a standardized way to connect tool servers.

Core MCP concepts

- *MCP server*: provides tools through HTTP/SSE or stdio
- *MCP client*: connects to the server and discovers or calls tools
- *Standardization*: any tool can be connected as long as it follows the MCP protocol

MCP support in LangChain v1

- You can connect to a local MCP server with `mcp.client.stdio.stdio_client()` and `ClientSession`
- `load_mcp_tools(session)` from `langchain-mcp-adapters` converts MCP session tools into LangChain tools

7.8 Summary

Concept	Description	Core API
<i>HITL</i>	Requests human approval before tool execution	<code>HumanInTheLoopMiddleware</code> , <code>Command(resume=...)</code>
<i>ToolRuntime</i>	Gives tools access to runtime context	<code>ToolRuntime[T]</code> , <code>context_schema</code>
<i>Context engineering</i>	Dynamically controls prompts and tools	<code>dynamic_prompt</code> middleware
<i>MCP</i>	Standardized tool protocol	<code>ClientSession + load_mcp_tools()</code>

Next Steps

- The next notebook introduces *multi-agent patterns*.
- You will explore Subagents, Handoffs, Skills, Routers, and other collaboration patterns.

08 Multi

Agent Patterns

Learning Objectives

Understand and implement five multi-agent patterns.

This notebook covers:

- *Subagents*: the main agent calls specialized subagents as tools
- *Handoffs*: state transitions between agents with `Command(goto=...)`
- *Skills*: one agent loads specialized prompts depending on the task
- *Router*: a classifier routes input to the right agent
- *Custom*: developer-controlled complex workflows

8.1 Environment Setup

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)

print("Model ready:", model.model_name)
```

8.2 Comparing Multi-Agent Patterns

The table below compares five multi-agent patterns. Each one fits a different situation, so you should choose based on your project requirements.

Pattern	Routing Owner	State Sharing	Best Fit
<i>Subagents</i>	Main agent	Isolated through tools	Parallel work, distributed execution
<i>Handoffs</i>	Tool call	State transition	Sequential multi-hop flows
<i>Skills</i>	Single agent	Prompt swapping	Domain specialization
<i>Router</i>	Classifier	Parallel execution	Multi-domain systems

<i>Custom</i>	Developer-defined	Full control	Complex workflows
---------------	-------------------	--------------	-------------------

Key differences

- *Subagents* run independently and return only the result
- *Handoffs* pass conversation state between agents
- *Skills* let one agent switch roles
- *Router* classifies input and delegates it to the right specialist

Pattern Selection Matrix (requirement → pattern)

While the table above lists pattern *properties*, the matrix below starts from *requirements* and points to the best-fit pattern.

Requirement	Best-fit pattern
Single simple task (one-shot)	<i>Skills</i> (or single agent)
Repeat the same task	<i>Subagents</i>
Parallel work across multiple domains	<i>Router</i> (classify then fan-out)
Very large context (long docs/history)	<i>Subagents</i> (token savings via isolation)
Team-based — agents collaborate	<i>Handoffs</i> (shared state)
Direct conversation with a role	<i>Handoffs</i> + supervisor

Approximate cost per pattern

Rough call count and cumulative tokens for the same task implemented in different patterns. The numbers vary by scenario and model, but they illustrate the *relative* cost.

Scenario	Pattern	Calls	Cumulative tokens
One-Shot (single domain)	<i>Subagents</i>	4–5 calls	9K
Repeat (same task)	<i>Skills</i>	2–3 calls	15K
Multi-Domain (collaboration)	<i>Handoffs</i>	3–7+ calls	14K+
Multi-Domain (classify then fan-out)	<i>Router</i>	~1 classify + N domains	9K

TIP

Router vs *Supervisor* — the terms are often used interchangeably but they differ in implementation. A *Router* is a *simple functional* node that classifies input once and *fans out* to the right agent (or `Send`). A *Supervisor* is itself an agent that decides *conversation-aware* which subagent to invoke on every turn, retaining message history. Use *Router* for one-shot classification; pick *Supervisor* for multi-turn collaboration.

8.3 The Subagent Pattern

In this pattern, the main agent (supervisor) calls specialized subagents *as tools*.

Characteristics

- Each subagent is wrapped in a tool function
- The main agent decides which subagent to call
- The internal state of each subagent is isolated from the main agent
- Parallel execution is possible, which can improve performance

Subagent-local history (`checkpointer=True`)

`create_agent(..., checkpointer=True)` lets a subagent keep its *own* message history on a separate thread. The main agent still only sees the final result, but the subagent can run multi-turn reasoning internally and resume from the same `thread_id`.

Exposing N subagents via a single dispatch tool

With many subagents, registering each one as its own tool bloats the parent tool list. The alternative is a single

`dispatch(agent_name, query)` tool combined with one of three discovery strategies.

Strategy	Recommended scale	Notes
Enumerate in system prompt	< 10	LLM picks the name from text. Simple, flexible
<code>Literal["a", "b", ...]</code> type constraint	10–30	Schema-level validation, rejects unknown names
Tool-based discovery (<code>list_agents</code>)	> 30	Dynamic registration; separate tools to search/filter

Injecting parent state with `ToolRuntime[None, CustomState]`

When a subagent tool needs to read part of the parent state, take a `ToolRuntime[None, CustomState]` argument.

```
from langchain.tools import tool, ToolRuntime

@tool
def research_subagent(query: str, runtime: ToolRuntime[None, ResearchState]) -> str:
    """Research subagent that reads user_id from the parent state."""
    user_id = runtime.state["user_id"]
    # ... call subagent ...
    return result
```

Async work: start / status / get_result three-tool pattern

Long-running subagents are safer when split into *three* tools instead of a single blocking call.

```
@tool
def start_research_job(query: str) -> str:
    """Start the research job and return a job_id."""
```

```

...

@tool
def check_job_status(job_id: str) -> str:
    """Return the job status: running / done / failed."""
    ...

@tool
def get_job_result(job_id: str) -> str:
    """Return the result of a completed job."""
    ...

```

Updating parent state with `Command`

A subagent tool can return `Command(update={...})` instead of a plain string to update the parent graph's state directly — useful when you need to update custom fields, not just messages.

```

from langgraph.types import Command

@tool
def research_with_state_update(query: str) -> Command:
    result = run_research(query)
    return Command(update={
        "research_results": result,
        "messages": [{"role": "tool", "content": result}],
    })

```

8.4 The Handoff Pattern

This pattern uses `Command(goto=...)` to *transfer state* between agents.

Characteristics

- A tool returns a `Command` object that routes execution to another agent
- Conversation state (message history) is passed to the next agent
- A `StateGraph` defines the flow between agents
- This fits multi-hop scenarios such as customer-service transfers

Single agent + middleware handoff (preferred)

The LangChain v1 docs recommend a *single agent* + `@wrap_model_call` *middleware* over multiple subgraphs as the primary handoff pattern. The middleware overrides the system prompt and tool set dynamically based on routing state, so persona switches happen inside one agent node.

```

from langchain.agents.middleware import wrap_model_call, ModelRequest
from langchain.agents import create_agent

@wrap_model_call
def role_router(request: ModelRequest, handler):
    state = request.state
    role = state.get("active_role", "general")

    if role == "billing":
        return handler(request.override(
            system_prompt="You are a billing specialist.",
            tools=[refund_tool, charge_tool],
        ))

```

```

elif role == "tech":
    return handler(request.override(
        system_prompt="You are a technical support agent.",
        tools=[diagnose_tool, escalate_tool],
    ))
return handler(request)

agent = create_agent(model="gpt-5.4", tools=ALL_TOOLS, middleware=[role_router])

```

The multi-subgraph handoff still has its place when the flow between departments must be explicit, but the middleware approach is much lighter for plain persona switches.

Handoff tool with `ToolRuntime` and `tool_call_id` echo-back

A handoff tool should accept `ToolRuntime[None, SupportState]` so it can access parent state and the current `tool_call_id`. When switching with `Command(goto=..., update={"messages": [...]})`, you must include a `ToolMessage` that matches the originating tool call — otherwise OpenAI's tool-call contract breaks.

```

from langchain.tools import tool, ToolRuntime
from langchain_core.messages import ToolMessage
from langgraph.types import Command

@tool
def transfer_to_billing(reason: str, runtime: ToolRuntime[None, SupportState]) -> Command:
    """Transfer the conversation to the billing team."""
    return Command(
        goto="billing_agent",
        update={
            "messages": [
                ToolMessage(
                    content=f"Transferred to billing: {reason}",
                    tool_call_id=runtime.tool_call_id,
                ),
            ],
            "active_role": "billing",
        },
    )

```

TIP

Exactly two messages should flow on a subgraph handoff — (1) the `AIMessage(tool_calls=[...])` that triggered the handoff and (2) its matching `ToolMessage`. Inserting anything else between them causes the OpenAI provider to reject the next call because the tool-call pair no longer matches.

8.5 The Skill Pattern

A single agent dynamically *loads a specialized prompt* depending on the task.

Characteristics

- One agent has multiple skills
- Each skill is implemented as a specialized system prompt
- The agent dynamically loads the skill it needs
- One agent can handle many tasks without managing multiple separate agents

8.6 The Router Pattern

A classifier *routes* input to the most appropriate agent.

Characteristics

- The query is classified first
- It is then delegated to the right specialist agent or tool
- This is useful in multi-domain systems
- Routing logic can be rule-based or model-based

Multi-domain fan-out with `Send`

When a query spans *several* domains at once, use `Send` for parallel fan-out instead of routing to a single agent. If a routing function passed to `add_conditional_edges` returns a `list[Send]`, the LangGraph runtime executes those nodes *in parallel*.

```
from langgraph.types import Command, Send

def route_to_agents(state: RouterState) -> list[Send]:
    """Fan out to every category produced by the classifier."""
    return [
        Send(c["agent"], {"query": c["query"]})
        for c in state["classifications"]
    ]

graph.add_conditional_edges("classifier", route_to_agents,
                           ["billing_agent", "tech_agent", "general_agent"])
```

8.7 Choosing a Pattern

Which multi-agent pattern should you choose? Use the guide below.

Decision tree

1. *Can the agents work independently?*
 - YES → *Subagents* (parallel execution, result aggregation)
 - NO → move to the next question
1. *Must conversation state be passed between agents?*
 - YES → *Handoffs* (state transition, multi-hop)
 - NO → move to the next question
1. *Can a single agent just switch roles?*
 - YES → *Skills* (prompt switching)
 - NO → move to the next question
1. *Is classifying input and sending it to a handler enough?*
 - YES → *Router* (classify and delegate)
 - NO → *Custom* (fully custom graph)

Practical guidance

- Start with the *simplest pattern* (usually Subagents or Skills)
- Move to Handoffs or Router only when requirements become more complex
- Use a Custom pattern only when the other patterns are not enough
- You can also *combine* patterns (for example, Router + Handoffs)

8.8 Summary

Pattern	Core API	When to use it
<i>Subagents</i>	<code>create_agent</code> + tool functions	Independent parallel work
<i>Handoffs</i>	<code>Command(goto=...)</code> , <code>StateGraph</code>	Multi-hop state transfer
<i>Skills</i>	Load prompts as tools	One agent with many roles
<i>Router</i>	Classifier tool + specialist tools	Multi-domain classification
<i>Custom</i>	Full <code>StateGraph</code> control	Complex business logic

Key principles

- Start simple and increase complexity only when necessary
- Design each agent around *one clear responsibility*
- Define the *interfaces* between agents (inputs and outputs) clearly

Next Steps

→ [09_custom_workflow_and_rag.ipynb](#): Learn about custom workflows and RAG.

09 Custom Workflows and RAG

Learning Objectives

Build a custom workflow with LangGraph `StateGraph` and implement the RAG pattern.

This notebook covers:

- The basic structure of `StateGraph` (nodes, edges, state)
- Branching with conditional edges
- Integrating `create_agent` as a workflow node
- Implementing the RAG (Retrieval-Augmented Generation) pattern

9.1 Environment Setup

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)

from langchain.agents import create_agent
from langchain.tools import tool

print("Environment ready.")
```

9.2 `StateGraph` Basics

Let's look at the core building blocks of LangGraph.

Concept	Description
<i>Node</i>	A unit of work. It can be a function or an agent
<i>Edge</i>	A connection between nodes that defines execution flow
<i>State</i>	Shared data passed between nodes, usually defined with <code>TypedDict</code>

`StateGraph` combines these three parts so you can build complex workflows.

9.3 Conditional Edges

Branch to different paths based on state. With `add_conditional_edges`, the next node can be selected dynamically at runtime.

In the example below, the input text is classified first and then routed to a different handler depending on its category.

9.4 Integrating an Agent into a Workflow

Use an agent created with `create_agent` as a node inside a `StateGraph`. This makes it possible to connect multiple agents in a pipeline and handle more complex tasks.

The example below connects a research agent and a writing agent in sequence.

9.5 Overview of RAG (Retrieval-Augmented Generation)

RAG is a pattern that strengthens LLM answers by retrieving external knowledge. There are three major approaches:

Pattern	Description	Characteristic
<i>Basic 2-step</i>	Retrieve → generate	Simple and fast
<i>Agentic RAG</i>	An agent repeatedly calls retrieval tools	Flexible and accurate
<i>Hybrid</i>	Combines keyword and semantic search	Higher retrieval quality

- *Basic 2-step*: retrieve documents, then pass them as context to the LLM for answer generation
- *Agentic RAG*: the agent calls retrieval tools repeatedly until it has enough information to answer

Five RAG building blocks

A LangChain RAG pipeline is built from these five components.

Component	Role
<i>Document Loaders</i>	Ingest source documents from PDF, web pages, Notion, etc.
<i>Text Splitters</i>	Split long documents into meaningful chunks (e.g. <code>RecursiveCharacterTextSplitter</code>)
<i>Embedding Models</i>	Turn text into vectors (e.g. <code>OpenAIEmbeddings</code>)
<i>Vector Stores</i>	Index and search embeddings (FAISS, Chroma, Pinecone, ...)
<i>Retrievers</i>	Unified retrieval interface — vector / hybrid / MultiQuery share the same API

Rewrite → Retrieve → Agent (three-node pattern)

A variation of agentic RAG that separates *query rewriting* into its own node. The user input is first rewritten into a retrieval-friendly form, the retriever runs against it, and the agent grounds its answer in the result.

```
# Rewrite → Retrieve → Agent
graph.add_node("rewrite", query_rewriter_node)
graph.add_node("retrieve", retriever_node)
graph.add_node("agent", create_agent(model, tools=[]))

graph.add_edge(START, "rewrite")
graph.add_edge("rewrite", "retrieve")
graph.add_edge("retrieve", "agent")
graph.add_edge("agent", END)
```

The research → writer example below illustrates a *content-generation* workflow, while Rewrite → Retrieve → Agent is a workflow for boosting *retrieval quality*. Both are natural fits for `StateGraph`.

9.6 A Simple RAG Implementation

Split text into chunks and implement RAG with simple keyword-based retrieval. Even without a vector store, you can still understand the core idea behind RAG.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

# Sample documents
documents = [
    "LangChain is a framework for building applications with large language models. It provides tools for prompt engineering, memory management, and agent creation.",
    "LangGraph is a low-level orchestration framework for building stateful agents. It uses a graph-based approach with nodes and edges.",
    "Middleware in LangChain v1 allows you to intercept and modify agent behavior at every step. You can add logging, guardrails, and custom logic.",
    "Multi-agent systems in LangChain support five patterns: subagents, handoffs, skills, router, and custom workflows.",
    "RAG (Retrieval-Augmented Generation) combines information retrieval with text generation to provide grounded, factual responses.",
]

# Split text
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=200,
    chunk_overlap=50,
)

chunks = []
for doc in documents:
    chunks.extend(text_splitter.split_text(doc))

print(f"Original documents: {len(documents)}")
print(f"Split chunks: {len(chunks)}")
for i, chunk in enumerate(chunks):
    print(f" [{i}] {chunk[:80]}...")
```

9.7 FAISS Vector Store (Optional)

Similarity search with embeddings is far more accurate than keyword matching. The example below shows how to use a FAISS vector store.

```
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS

# Create the embedding model
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

# Create the vector store
vectorstore = FAISS.from_texts(chunks, embeddings)

# Retrieve
results = vectorstore.similarity_search("LangChain agent patterns", k=3)
for doc in results:
    print(doc.page_content)
```

9.8 Summary

This notebook covered:

Topic	Key Idea
<i>StateGraph basics</i>	Build workflows from nodes, edges, and state
<i>Conditional edges</i>	Branch at runtime with <code>add_conditional_edges</code>
<i>Agent integration</i>	Use <code>create_agent</code> as a <code>StateGraph</code> node inside a pipeline
<i>RAG pattern</i>	Combine retrieval and generation for grounded answers
<i>Vector store</i>	Use FAISS or similar tools for embedding-based similarity search

The next notebook shows how to deploy agents to production environments.

Next Steps

→ [10_production.ipynb](#): Learn about production deployment.

10 Production

Learning Objectives

Learn how to test, deploy, and monitor agents.

This notebook covers:

- Local development and debugging with LangSmith Studio
- Deterministic agent testing with `GenericFakeChatModel`
- Trajectory-based tests for validating tool call order
- Web-based interaction with Agent Chat UI
- Deployment with LangGraph Platform or your own server
- Observability with LangSmith

10.1 Environment Setup

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)

from langchain.agents import create_agent
from langchain.tools import tool

print("Environment ready.")
```

10.2 LangSmith Studio

Develop and debug agents locally.

To use Studio, you need:

- a `langgraph.json` config file
- a local dev server started with `langgraph dev` (default `http://127.0.0.1:2024`)
- the hosted Studio UI at `https://smith.langchain.com/studio/?baseUrl=http://127.0.0.1:2024`

Studio is a powerful tool for visualizing agent execution flow and debugging each step.

Studio Key Features

Feature	Description
<i>Hot-reloading</i>	Edit prompts and tool code and see the change applied without restarting the server
<i>Full execution trace inspection</i>	Inspect every node, model call, and tool invocation in a single trace tree
<i>Thread replay</i>	Re-run saved threads from any checkpoint to compare branches and behaviors
<i>Exception capture</i>	Capture the state and surrounding context at the moment an exception is raised

```
# Example langgraph.json configuration
import json

langgraph_config = {
    "dependencies": ["."],
    "graphs": {
        "agent": "./agent.py:agent"
    },
    "env": ".env"
}

print("Example langgraph.json configuration:")
print(json.dumps(langgraph_config, indent=2))
print("\nHow to run:")
print(" $ langgraph dev # local dev server: http://127.0.0.1:2024")
print(" → Studio UI: https://smith.langchain.com/studio/?baseUrl=http://127.0.0.1:2024")
```

10.3 Agent Testing

Agent testing has three layers: *Unit*, *Integration*, and *Evals*.

Layer	Goal	Tools
<i>Unit</i>	Validate agent logic deterministically (tool choice, branching, exit conditions)	<code>GenericFakeChatModel</code> , <code>pytest</code>
<i>Integration</i>	Run the full workflow against real LLMs and tools to confirm end-to-end behavior	<code>langgraph dev</code> , real server environments
<i>Evals</i>	Evaluate agent trajectories with deterministic matching or LLM-as-judge evaluators	<code>agentevals</code> , LangSmith Evaluations

10.3.1 Unit — `GenericFakeChatModel`

With `GenericFakeChatModel` , you can test an agent deterministically without making real API calls.

Benefits of this approach:

- No API cost during tests
- Always returns the same result, which is ideal for CI/CD pipelines
- Lets you validate the agent's logic independently (tool calls, branching, and so on)


```

from langchain_core.language_models import GenericFakeChatModel
from langchain.messages import AIMessage
from langchain.agents import create_agent
from langchain.tools import tool

@tool
def get_capital(country: str) -> str:
    """Returns the capital of a country."""
    capitals = {"Korea": "Seoul", "Japan": "Tokyo", "France": "Paris"}
    return capitals.get(country, "Unknown")

# Deterministic test with a fake model
fake_model = GenericFakeChatModel(
    messages=iter([
        AIMessage(content="The capital of South Korea is Seoul.")
    ])
)

# Test agent
test_agent = create_agent(
    model=fake_model,
    tools=[get_capital],
    system_prompt="You are a geography expert.",
)

print("GenericFakeChatModel test:")
print("  - Tests agent behavior with deterministic responses")
print("  - Can be tested in CI/CD without live API calls")

```

10.3.2 Evals — Trajectory matching with `agentevals`

Validate the order in which the agent calls tools. A trajectory test checks whether the agent uses tools in the expected order and whether the final response matches your expectation.

The `agentevals` package offers four trajectory-match modes.

```
pip install agentevals
```

Mode	Matching rule
<code>strict</code>	Tool calls must match the reference exactly in kind and order
<code>unordered</code>	Same set of tool calls; order is ignored
<code>subset</code>	The observed trajectory is a subset of the reference (no extra calls allowed)
<code>superset</code>	The observed trajectory is a superset of the reference (must contain the required calls)

LLM-as-judge evaluators allow grading trajectories against natural-language criteria as well.

```

# Example trajectory test
def test_agent_trajectory():
    """Tests whether the agent calls tools in the expected order."""
    result = test_agent.invoke(
        {"messages": [{"role": "user", "content": "What is the capital of South Korea?"}]}
    )

    messages = result["messages"]

    # Check: verify that messages exist
    assert len(messages) > 0, "The agent did not respond"

```

```
# Check: verify that the last message is an AI response
last_msg = messages[-1]
assert hasattr(last_msg, 'content'), "The last message does not contain content"

print("✓ Trajectory test passed")
print(f"  Message count: {len(messages)}")
print(f"  Final response: {last_msg.content[:100]}")

try:
    test_agent_trajectory()
except Exception as e:
    print(f"Testing note: {e}")
```

10.5 Agent Chat UI

This is a web UI for talking to your agent. It connects to a LangGraph server so you can test the agent directly in the browser. Either scaffold a new project with `create-agent-chat-app` or clone the official repository (`langchain-ai/agent-chat-ui`). Point the UI at your local LangGraph server (`http://127.0.0.1:2024`).

Key features:

- Real-time streaming chat
- Tool call visualization
- Conversation branching
- Human-in-the-loop approval

Pick one of the two install paths:

```
# Option A – scaffold (recommended)
npx create-agent-chat-app --project-name my-chat-ui

# Option B – clone the official repository
git clone https://github.com/langchain-ai/agent-chat-ui.git
cd agent-chat-ui && npm install && npm dev
```

```
# Start the LangGraph server in another shell
langgraph dev # http://127.0.0.1:2024
```

The UI asks for three values on first launch.

Field	Value
<i>Deployment URL</i>	<code>http://127.0.0.1:2024</code> for local; the deployment URL for managed agents
<i>Graph ID</i>	The <code>graphs</code> key from <code>langgraph.json</code> (for example, <code>agent</code>)
<i>LangSmith API key</i>	Required for managed deployments; can be skipped for the local dev server

10.6 Deployment

You can deploy an agent through LangGraph Platform (managed) or your own server. Choose the option that best fits your production environment.

10.6.1 LangGraph Platform deploy workflow

LangGraph Platform deployments now go through GitHub integration. The old single-command `langgraph deploy` flow is no longer recommended.

1. Push the agent code (including `langgraph.json`) to a GitHub repository.
2. Open the LangSmith console → *Deployments* → *+ New Deployment*.
3. Select the repository, branch, and `langgraph.json` path.
4. Enter environment variables (`OPENAI_API_KEY` , etc.).
5. Click *Deploy*, watch the build log, and pick up the issued Deployment URL.

! WARNING

The `langgraph deploy` CLI flow from older guides is *deprecated*. Use the GitHub-based flow in the LangSmith console for new deployments.

```
print("Deployment options:")
print("=" * 50)

print("""
# Option 1: LangGraph Platform (managed) – GitHub integration
#   → Use LangSmith Deployments → + New Deployment
#   (legacy `langgraph deploy` CLI pattern is deprecated)

# Option 2: self-hosted Docker deployment
`$ langgraph build -t my-agent`
`$ docker run -p 2024:2024 my-agent`

# Option 3: wrap with FastAPI/Flask
from fastapi import FastAPI

app = FastAPI()

@app.post("/chat")
async def chat(message: str):
    result = agent.invoke(
        {
            "messages": [
                {
                    "role": "user",
                    "content": message
                }
            ]
        }
    )
    return {
        "response": result["messages"][-1].content
    }
""")
```

10.6.2 Calling a deployed agent — `langgraph-sdk`

Use the Python SDK or REST to call a deployed agent.

```
pip install langgraph-sdk
```

```
from langgraph_sdk import get_sync_client

client = get_sync_client(url="http://127.0.0.1:2024")

for chunk in client.runs.stream(
    None, # threadless run
    "agent", # graph ID
    input={"messages": [{"role": "user", "content": "hi"}]},
    stream_mode="updates",
):
    print(chunk)
```

Or call the REST API directly:

```
curl -X POST http://127.0.0.1:2024/runs/stream \
-H "Content-Type: application/json" \
--data '{
  "assistant_id": "agent",
  "input": {"messages": [{"role": "user", "content": "hi"}]},
  "stream_mode": "updates"
}'
```

10.7 Observability

Use LangSmith to trace agent behavior. When tracing is enabled, every step of agent execution is recorded and can be analyzed.

10.7.1 Environment variables

```
# Current (recommended)
export LANGSMITH_TRACING=true
export LANGSMITH_API_KEY="lsv2..."
export LANGSMITH_PROJECT="my-agent" # defaults to `default` if unset
```

LangSmith lets you inspect:

- The complete execution flow of each agent call
- Model input/output, tool calls, and token usage
- Latency, errors, and cost tracking

10.7.2 Selective tracing with `tracing_context`

Use `tracing_context` to enable or disable tracing for a specific block in code.

```
import langsmith as ls

with ls.tracing_context(enabled=True):
    agent.invoke({"messages": [{"role": "user", "content": "hi"}]})
```

10.7.3 Tags and metadata via `config`

Attach tags and metadata at call time to make trace search and filtering easier.

```
agent.invoke(
    {"messages": [{"role": "user", "content": "hi"}]},
    config={
        "tags": ["production", "v1.0"],
        "metadata": {"user_id": "123", "session_id": "456"},
    },
)
```

10.8 Production Checklist

Before deploying an agent to production, review the following checklist.

Item	Tool	Status
Unit tests	<code>GenericFakeChatModel</code> , <code>pytest</code>	
Trajectory tests	Custom validation functions	
Observability	LangSmith tracing	
Error handling	<code>try/except</code> , <code>retry</code> logic	
Security	API key management, input validation, guardrails	
Deployment environment	Docker, LangGraph Platform	
Monitoring	LangSmith dashboards, alert configuration	
Documentation	API docs, agent behavior notes	

10.9 Summary

This notebook covered:

Topic	Key Idea
<i>LangSmith Studio</i>	Use <code>langgraph dev</code> to debug agents visually on your local machine
<i>Agent testing</i>	Run deterministic tests with <code>GenericFakeChatModel</code> and no API calls
<i>Trajectory tests</i>	Validate tool call order and final responses
<i>Agent Chat UI</i>	Talk to agents in the browser and visualize tool usage
<i>Deployment</i>	Deploy with LangGraph Platform, Docker, FastAPI, and related options
<i>Observability</i>	Use LangSmith to track execution flow, token usage, and cost

This completes the LangChain v1 agent track. You covered the full lifecycle of agent development, from basic concepts to production deployment.

Next Steps

→ Continue to [11_mcp.ipynb](#) → Or jump to the [LangGraph intermediate track](#)

11 MCP (Model Context Protocol)

Learning Objectives

Learn how to connect external tools and context to an agent through MCP (Model Context Protocol).

This notebook covers:

- Understanding the MCP concept and architecture (server / client / host)
- Connecting to MCP servers with the `langchain-mcp-adapters` package
- Integrating MCP tools with an agent through `create_agent(model=..., tools=await client.get_tools())`
- Understanding the difference between stdio and SSE transports
- Connecting to multiple MCP servers at once

11.1 Environment Setup

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)

print("Environment ready.")
```

11.2 MCP Concepts

MCP (*Model Context Protocol*) is an open protocol for providing external tools and context to an LLM in a *standardized way*.

Architecture components

Component	Role	Example
<i>MCP server</i>	Exposes tools, resources, and prompts	File system server, DB server, API wrapper
<i>MCP client</i>	Connects to the server and fetches tools	<code>MultiServerMCPClient</code>
<i>Host</i>	Manages the client and connects it to the LLM	LangChain agent, IDE

Core resource types

- *Tools*: executable functions the agent can call
- *Resources*: data such as files or database records (converted to LangChain Blob objects)
- *Prompts*: reusable prompt templates

Why MCP?

Before MCP, each tool needed its own custom integration code. MCP unifies this into *one standard protocol*, which means:

- Tool providers only need to implement an MCP server once
- LLM hosts can access every tool through one MCP client
- Tools can be reused across the ecosystem

11.3 Installing `langchain-mcp-adapters`

To use MCP from LangChain, you need the `langchain-mcp-adapters` package.

```
# MCP adapter installation commands
print("MCP adapter installation:")
print("  uv add langchain-mcp-adapters mcp")
print()
print("Key components:")
print("  - MultiServerMCPClient: Client that manages multiple MCP servers")
print("  - load_mcp_tools(session): MCP Converts an MCP session into LangChain tools")
print("  - FastMCP: Utility for quickly building MCP servers")
```

11.4 Stdio Transport

Stdio (Standard I/O) transport communicates with an MCP server through a local subprocess. It is a good fit for development and testing environments.

```
from pathlib import Path; import json, tempfile, sys
server_path = Path(tempfile.gettempdir()) / "lc_mcp_math_server.py"
server_path.write_text('from mcp.server.fastmcp import FastMCP\nmcp = FastMCP("math")\n@mcp.tool()\ndef\nadd(a: int, b: int) -> int:\n    return a + b\nif __name__ == "__main__":\n\nmcp.run(transport="stdio")')
stdio_config = {"math": {"transport": "stdio", "command": sys.executable, "args": [str(server_path)]}}
print("Stdio Transport configuration:"); print(json.dumps(stdio_config, indent=2))
print(f"\nServer file: {server_path}")
```

11.5 SSE / HTTP Transport

HTTP (streamable-http) transport uses web-based communication and is a good fit for remote MCP servers. It also supports authentication headers and custom settings.

```
# Example HTTP / streamable-http transport configuration
http_config = {
    "weather_server": {"transport": "streamable_http", "url": "https://weather-mcp.example.com/mcp",
    "headers": {"Authorization": "Bearer YOUR_API_KEY"}}
```



```

}
import json; print("HTTP Transport configuration:"); print(json.dumps(http_config, indent=2))
print("\nUsage pattern: client = MultiServerMCPClient(http_config) -> await client.get_tools()")

```

11.6 Loading MCP Tools and Integrating Them with an Agent

This is the common pattern for binding tools fetched from an MCP server into a LangChain agent. The core entry point is `client.get_tools()`. It discovers the tools exposed by the MCP server and converts each tool's name, description, and parameter schema into LangChain `Tool` objects. The converted tools can be passed directly to `create_agent(tools=mcp_tools)`.

Key function signatures

Function	Description
<code>client.get_tools()</code>	Return LangChain <code>Tool</code> objects from every registered server
<code>client.get_resources(server_name)</code>	Return LangChain <code>Blob</code> resources from a specific server
<code>client.get_prompt(server_name, prompt_name, arguments={...})</code>	Fetch a prompt template registered on the server
<code>load_mcp_tools(session)</code>	Convert tools from an open session into LangChain <code>Tool</code> objects (stateful pattern)
<code>load_mcp_resources(session, uris=[...])</code>	Load specific URIs as resources from an open session

```

from langchain_mcp_adapters.client import MultiServerMCPClient
from langchain.agents import create_agent

async with MultiServerMCPClient(mcp_config) as client:
    mcp_tools = await client.get_tools()
    agent = create_agent(model="gpt-5.4", tools=mcp_tools)

```

Stateful sessions

The default `client.get_tools()` flow is *stateless* — a new session opens and closes for every tool call. To share a session across multiple calls, open one explicitly.

```

async with client.session("math_server") as session:
    tools = await load_mcp_tools(session)
    result1 = await tools[0].ainvoke({"a": 1, "b": 2})
    result2 = await tools[1].ainvoke({"a": 3, "b": 4})

```

11.7 Connecting to Multiple MCP Servers

As the name suggests, `MultiServerMCPClient` can manage several MCP servers at the same time.

```
# Example multi-MCP-server configuration
import json, sys
multi_server_config = {"math_server": {"transport": "stdio", "command": sys.executable, "args":
[str(server_path)]}, "weather_server": {"transport": "streamable_http", "url": "https://weather-mcp.
example.com/mcp"}, "database_server": {"transport": "stdio", "command": "npx", "args": ["-y",
"@modelcontextprotocol/server-postgres"], "env": {"DATABASE_URL": "postgresql://..."}}}
print("Multi-MCP server configuration:"); print(json.dumps(multi_server_config, indent=2,
ensure_ascii=False))
print("\nUsage pattern: client = MultiServerMCPClient(multi_server_config) -> await client.get_tools()")
print("Note: it is stateless by default – each tool call creates and then cleans up a new session")
```

11.8 MCP Authentication

Remote MCP servers use one of two authentication tracks.

Track A — Static headers

The simplest approach: pass the token through a header. Good for CI/CD with pre-issued keys.

```
http_config = {
    "weather_server": {
        "transport": "streamable_http",
        "url": "https://weather-mcp.example.com/mcp",
        "headers": {"Authorization": "Bearer YOUR_API_KEY"},
    }
}
```

Track B — `httpx.Auth` interface

For dynamic auth such as token refresh or request signing, pass an `httpx.Auth` implementation.

```
import httpx

class BearerAuth(httpx.Auth):
    def __init__(self, token: str):
        self.token = token

    def auth_flow(self, request):
        request.headers["Authorization"] = f"Bearer {self.token}"
        yield request

http_config = {
    "weather_server": {
        "transport": "streamable_http",
        "url": "https://weather-mcp.example.com/mcp",
        "auth": BearerAuth(token="..."),
    }
}
```

Track C — MCP SDK OAuth2

The MCP Python SDK ships an `mcp.client.auth.oauth2` module for the standard OAuth2 flow (authorization code, refresh tokens).

```
from mcp.client.auth.oauth2 import OAuth2ClientCredentials

auth = OAuth2ClientCredentials(
```

```

token_url="https://auth.example.com/token",
client_id="my-client",
client_secret="...",
scope="mcp.tools",
)

http_config = {
    "secure_server": {
        "transport": "streamable_http",
        "url": "https://secure-mcp.example.com/mcp",
        "auth": auth,
    }
}

```

11.9 Elicitation — Server-requested user input

Elicitation lets an MCP server request *additional input* from the client (host) during a tool call. The host shows the user a form and returns the response as an `ElicitResult`.

```

from mcp import ElicitResult, Callbacks

async def on_elicitation(request):
    user_input = await show_form(request.schema)
    return ElicitResult(
        action="accept", # "accept" | "decline" | "cancel"
        content={"city": user_input["city"], "unit": "celsius"},
    )

callbacks = Callbacks(on_elicitation=on_elicitation)

async with MultiServerMCPClient(config, callbacks=callbacks) as client:
    tools = await client.get_tools()

```

TIP

Use `"accept"` when the user provides values, `"decline"` when the user refuses, and `"cancel"` to signal that the entire tool call should be aborted.

11.10 Tool Interceptors

A *Tool Interceptor* is middleware that intercepts MCP tool calls. It can access runtime context, modify requests and responses, and implement retry logic.

Tool interceptor use cases

Use Case	Description
Auth injection	Pass user-specific tokens at runtime
Request transformation	Rewrite tool call parameters
Response filtering	Remove sensitive information

Retry logic	Retry automatically after failures
Logging	Trace tool calls

Returning a `Command` from an interceptor

An interceptor can do more than rewrite a request — it can return a `LangGraph Command` to update state and choose the next node.

```
from langgraph.types import Command

async def logging_interceptor(request, context):
    """Log the tool call while appending to a state trace."""
    print(f"[interceptor] tool={request.tool_name}")
    return Command(
        update={"tool_calls_log": [request.tool_name]},
        goto="tools",
    )
```

This pattern is useful when an interceptor enforces a guardrail and routes risky calls to a `human_review` node instead of executing them.

11.11 Writing a Custom MCP Server

With the *FastMCP* library, you can build an MCP server quickly using decorators.

11.12 Summary

This notebook covered:

Topic	Key Idea
<i>MCP concepts</i>	An open protocol that provides external tools and context to an LLM in a standardized way
<i>Stdio transport</i>	Local subprocess communication, good for development and testing
<i>SSE/HTTP transport</i>	Web-based communication for remote servers and authentication scenarios
<i>Agent integration</i>	Connect with <code>client.get_tools()</code> → <code>create_agent(tools=mcp_tools)</code>
<i>Multi-server support</i>	Use <code>MultiServerMCPClient</code> to manage several servers at once
<i>Interceptors</i>	Apply middleware for auth, logging, and request/response modification
<i>Custom servers</i>	Build an MCP server quickly with <i>FastMCP</i> decorators

Next Steps

→ Continue to [12_frontend_streaming.ipynb](#)

References:

- [MCP \(Model Context Protocol\)](#)

12 Frontend Streaming

Learning Objectives

Learn how to stream LLM responses to users in real time.

This notebook covers:

- Understanding the basics of LangChain SDK streaming (`.stream()` , `.astream_events()`)
- Learning the structure and usage of the `useStream` React hook
- Understanding the `StreamEvent` protocol
- Consuming real-time streaming from the Python SDK
- Understanding real-time agent-state display patterns

12.1 Environment Setup

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)

print("Environment ready.")
```

12.2 Python SDK Streaming Basics

The `.stream()` method delivers model output token by token in real time. Users can see partial results before the full response is complete.

12.3 `astream_events()`

`.astream_events()` streams *all internal events* asynchronously. This lets you trace model calls, tool execution, and chain steps in detail.

Main event types

Event	Description
<code>on_chat_model_stream</code>	Model token streaming
<code>on_chat_model_start</code>	Model call started
<code>on_chat_model_end</code>	Model call completed
<code>on_tool_start</code>	Tool execution started
<code>on_tool_end</code>	Tool execution completed

```
import asyncio

async def stream_events_demo():
    """Example of event streaming with astream_events()"""
    print("Event streaming:")
    print("-" * 40)
    async for event in model.astream_events(
        "Two advantages of Python",
        version="v2",
    ):
        kind = event["event"]
        if kind == "on_chat_model_stream":
            content = event["data"]["chunk"].content
            if content:
                print(content, end="", flush=True)
        elif kind == "on_chat_model_start":
            print(f"[Model call start]")
        elif kind == "on_chat_model_end":
            print(f"\n[Model call complete]")

    await stream_events_demo()
```

12.4 The `useStream` React Hook

`useStream` is a React hook from the LangGraph SDK that simplifies streaming communication with a LangGraph server.

Basic usage

```
import { useStream } from "@langchain/langgraph-sdk/react";

function Chat() {
  const stream = useStream({
    assistantId: "agent",
    apiUrl: "http://localhost:2024",
  });

  const handleSubmit = (message: string) => {
    stream.submit({
      messages: [{ content: message, type: "human" }],
    });
  };

  return (
    <div>
      {stream.messages.map((message, idx) => (
        <div key={message.id ?? idx}>
```

```

        {message.type}: {message.content}
      </div>
    )))
    {stream.isLoading && <div>Loading...</div>}
    {stream.error && <div>Error: {stream.error.message}</div>}
  </div>
);
}

```

Main return values

Property	Type	Description
messages	Message[]	The full message list for the current thread
isLoading	boolean	Whether the stream is active
error	Error \], [null	Error object
interrupt	Interrupt	An interruption request (HITL)
submit()	function	Send a message
stop()	function	Stop the stream

12.5 useStream Configuration Options

Parameter	Required	Default	Description
assistantId	O	—	Agent identifier (from the deployment dashboard)
apiUrl	—	localhost:2024	Agent server URL
apiKey	—	—	Auth token for a deployed agent
threadId	—	—	Connect to an existing conversation thread
onThreadId	—	—	Callback when a thread is created
reconnectOnMount	—	false	Reconnect to an active stream when the component mounts
onCustomEvent	—	—	Custom event handler
onUpdateEvent	—	—	State update handler
onMetadataEvent	—	—	Metadata event handler
messagesKey	—	"messages"	State key that stores messages
throttle	—	true	Batch state updates
initialValues	—	—	Cached initial state

12.6 Thread Management and Reconnection

Managing Thread IDs

By managing `threadId`, you can continue a conversation or load a previous thread.

```
const [threadId, setThreadId] = useState<string | null>(null);

const stream = useStream({
  apiUrl: "http://localhost:2024",
  assistantId: "agent",
  threadId,
  onThreadId: setThreadId,
});

// Persist threadId in a URL parameter or localStorage
```

Reconnecting After a Page Refresh

If you enable `reconnectOnMount`, the hook automatically reconnects to a stream that was already in progress after a page refresh.

```
const stream = useStream({
  apiUrl: "http://localhost:2024",
  assistantId: "agent",
  reconnectOnMount: true, // use sessionStorage
});

// Use custom storage
const stream = useStream({
  reconnectOnMount: () => window.localStorage,
});
```

12.7 Branching and Message Editing

With branching, you can create an *alternate path* from a specific point in the conversation history. This is useful when you want to edit a user message or regenerate an AI response.

```
{stream.messages.map((message) => {
  const meta = stream.getMessagesMetadata(message);
  const parentCheckpoint = meta?.firstSeenState?.parent_checkpoint;

  return (
    <div key={message.id}>
      {message.content}

      {/* Edit a user message */}
      {message.type === "human" && (
        <button onClick={() => {
          const newContent = prompt("Edit:", message.content);
          if (newContent) {
            stream.submit(
              { messages: [{ type: "human", content: newContent } ] },
              { checkpoint: parentCheckpoint }
            );
          }
        }}>
          Edit
        </button>
      )}
    </div>
  );
})}
```

```

    </button>
  })

  {/* Regenerate an AI response */}
  {message.type === "ai" && (
    <button onClick={() =>
      stream.submit(undefined, { checkpoint: parentCheckpoint })
    }>
      Regenerate
    </button>
  )}
</div>
);
}}

```

Key idea: use the `checkpoint` parameter to jump back to a specific state and generate a new branch.

12.8 Custom Streaming Events

You can stream *custom data* from the agent to the client. This is useful for progress updates, intermediate results, and other real-time signals.

```

# Custom streaming events – Python writer pattern
print("Custom streaming event pattern (Python server side):")
print("=" * 50)
print("""
from langchain.tools import tool
from langchain.agents.types import ToolRuntime

@tool
async def analyze_data(
    data_source: str, *, config: ToolRuntime
) -> str:
    "\\\\"Analyzes the data.\\\\"\\\\"
    if config.writer:
        # Stream progress updates to the client
        config.writer({
            "type": "progress",
            "message": "Loading data...",
            "progress": 25,
        })
        # ... processing ...
        config.writer({
            "type": "progress",
            "message": "Analysis complete!",
            "progress": 100,
        })
    return '{"result": "Analysis complete"}'
""")
print("Client side (React): receive it via the onCustomEvent callback")
print('  onCustomEvent: (data) => setProgress(data.progress)')

```

12.9 Multi-Agent Streaming

When several agents collaborate, their messages should be *displayed separately*. Use the `langgraph_node` metadata field to identify which agent produced each message.

Event callback summary

Callback	Use Case	Stream Mode
<code>onUpdateEvent</code>	State update after a graph step	<code>updates</code>
<code>onCustomEvent</code>	Agent-defined custom event	<code>custom</code>
<code>onMetadataEvent</code>	Execution and thread metadata	<code>metadata</code>
<code>onError</code>	Error handling	—
<code>onFinish</code>	Stream completion	—

12.10 Summary

This notebook covered:

Topic	Key Idea
<i>SDK streaming</i>	Use <code>.stream()</code> for real-time token output
<code>astream_events</code>	Trace model and tool calls through asynchronous event streaming
<code>useStream</code>	Simplify LangGraph server streaming in React
<i>Thread management</i>	Keep conversations alive with <code>threadId</code> and <code>reconnectOnMount</code>
<i>Branching</i>	Create alternate conversation paths from a <code>checkpoint</code>
<i>Custom events</i>	Stream progress and other custom data with a writer pattern
<i>Multi-agent</i>	Use <code>langgraph_node</code> metadata to distinguish messages by agent

Next Steps

→ Continue to [13_guardrails.ipynb](#)

References:

- [Streaming](#) — canonical `useStream` reference with the full event protocol
- [UI \(Agent Chat UI & useStream\)](#)

13 Guardrails

Learning Objectives

Learn how to configure guardrails that validate and filter agent input and output.

This notebook covers:

- Understanding the concept of guardrails and why they are needed
- Comparing deterministic guardrails and model-based guardrails
- Configuring PII detection middleware
- Building human-in-the-loop guardrails
- Writing custom `before_agent` and `after_agent` guardrails

13.1 Environment Setup

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)

from langchain.agents import create_agent
from langchain.tools import tool

print("Environment ready.")
```

13.2 Guardrail Concepts

Guardrails are safety mechanisms that validate and filter content during agent execution.

Why do we need guardrails?

- Prevent leakage of personally identifiable information (PII)
- Block prompt-injection attacks
- Prevent harmful or inappropriate content
- Enforce business rules and compliance requirements
- Validate output quality and correctness

Two approaches

Approach	Mechanism	Strengths	Weaknesses
<i>Deterministic</i>	Regex, keyword matching, explicit rules	Fast, predictable, cost-efficient	May miss subtle violations
<i>Model-based</i>	Use an LLM or classifier to analyze meaning	Can catch subtle issues	Slower and more expensive

When guardrails are applied

```

User input → [input guardrail] → agent execution → [output guardrail] → response
                ↑                               ↑
            before_agent                    after_agent

```

13.3 PII Detection Middleware

`PIIMiddleware` automatically detects and handles personal data such as email addresses, credit-card numbers, and IP addresses.

Strategy	Result
redact	Replace with <code>[REDACTED_EMAIL]</code>
mask	Partial masking (for example, only the last 4 digits shown)
hash	Replace with a deterministic hash
block	Raise an exception when detected

· NOTE

Scope parameters — `PIIMiddleware` exposes three independent toggles: `apply_to_input` (user messages), `apply_to_output` (model responses), and `apply_to_tool_results`. `apply_to_tool_results` defaults to `False` — tool return values (for example, database query results) are *not* scrubbed automatically, so set `apply_to_tool_results=True` explicitly when tool outputs may contain PII.

```

# Example PII-detection middleware setup
print("PII-detection middleware setup:")
print("=" * 50)
print("""
from langchain.agents import create_agent
from langchain.agents.middleware import PIIMiddleware

agent = create_agent(
    model="gpt-5.4",
    tools=[customer_service_tool, email_tool],
    middleware=[
        # Replace email addresses with [REDACTED_EMAIL]
        PIIMiddleware("email",

```

```

        strategy="redact",
        apply_to_input=True),

    # Partially mask credit-card numbers (****-****-****-1234)
    PIIMiddleware("credit_card",
        strategy="mask",
        apply_to_input=True),

    # Block detected API keys (custom regex)
    PIIMiddleware("api_key",
        detector=r"sk-[a-zA-Z0-9]{32}",
        strategy="block",
        apply_to_input=True),

],
)
"""
print("Built-in PII types: email, credit_card, ip, mac_address, url")
print("Custom detection: pass a regex or function to the detector parameter")

```

13.4 Human-in-the-Loop Guardrails

`HumanInTheLoopMiddleware` requires *human approval* before risky actions are executed. This is essential for high-risk operations such as financial transactions, data deletion, or external communication.

```

# Human-in-the-Loop guardrail example
print("Human-in-the-Loop guardrail:")
print("=" * 50)
print("""
from langchain.agents import create_agent
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import Command

agent = create_agent(
    model="gpt-5.4",
    tools=[search_tool, send_email_tool, delete_db_tool],
    middleware=[
        HumanInTheLoopMiddleware(
            interrupt_on={
                "send_email": True,      # approval required
                "delete_db": True,       # approval required
                "search": False,         # automatic
            }
        ),
    ],
    checkpointer=InMemorySaver(),
)

config = {"configurable": {"thread_id": "review-123"}}

# Step 1: run the agent -> pause at send_email
result = agent.invoke(
    {"messages": [{"role": "user", "content": "Send an email to the team"}]},
    config=config,
)
# -> paused: waiting for approval before send_email executes

# 2Step: resume after approval
result = agent.invoke(
    Command(resume={"decisions": [{"type": "approve"}]}),
    config=config,
)
""")
print("Key point: a checkpointer is required for pause/resume flows.")
print("On rejection, use {'type': 'reject'} to block the tool call.")

```

13.5 Custom Input Guardrails — `before_agent`

The `before_agent` hook validates requests *before the agent starts running*. It is useful for session-level authentication, rate limiting, and content filtering.

```
# Custom input guardrail – ContentFilterMiddleware class
print("Custom input guardrail (class-based):")
print("=" * 50)
print("""
from langchain.agents.middleware import (
    AgentMiddleware, AgentState, hook_config
)
from langgraph.runtime import Runtime
from typing import Any

class ContentFilterMiddleware(AgentMiddleware):
    """Deterministic guardrail: blocks requests that contain banned keywords."""

    def __init__(self, banned_keywords: list[str]):
        super().__init__()
        self.banned_keywords = [kw.lower() for kw in banned_keywords]

    @hook_config(can_jump_to=["end"])
    def before_agent(
        self, state: AgentState, runtime: Runtime
    ) -> dict[str, Any] | None:
        if not state["messages"]:
            return None

        first_message = state["messages"][0]
        if first_message.type != "human":
            return None

        content = first_message.content.lower()
        for keyword in self.banned_keywords:
            if keyword in content:
                return {
                    "messages": [{
                        "role": "assistant",
                        "content": "The request contains inappropriate content."
                    }],
                    "jump_to": "end"
                }
        return None
""")
print("Key point: jump_to='end' skips agent execution and returns immediately.")
print("If the middleware returns None, execution continues to the next step.")
```

13.6 Custom Output Guardrails — `after_agent`

The `after_agent` hook validates the final output *after agent execution is complete*. It is useful for model-based safety checks and quality validation.

```
# Custom output guardrail – SafetyGuardrailMiddleware class
print("Custom output guardrail (class-based):")
print("=" * 50)
print("""
from langchain.agents.middleware import (
    AgentMiddleware, AgentState, hook_config
)
from langgraph.runtime import Runtime
from langchain.messages import AIMessage
from langchain.chat_models import init_chat_model
```

```

from typing import Any

class SafetyGuardrailMiddleware(AgentMiddleware):
    """Model-based guardrail: uses an LLM to evaluate response safety."""

    def __init__(self):
        super().__init__()
        self.safety_model = init_chat_model("gpt-5.4-mini")

    @hook_config(can_jump_to=["end"])
    def after_agent(
        self, state: AgentState, runtime: Runtime
    ) -> dict[str, Any] | None:
        if not state["messages"]:
            return None

        last_message = state["messages"][-1]
        if not isinstance(last_message, AIMessage):
            return None

        safety_prompt = f"""Evaluate if this response is safe.
        Respond with only 'SAFE' or 'UNSAFE'.

        Response: {last_message.content}"""

        result = self.safety_model.invoke(
            [{"role": "user", "content": safety_prompt}]
        )

        if "UNSAFE" in result.content:
            last_message.content = (
                "This response was flagged as unsafe. Please ask again."
            )
            return None

        return None

print("Key point: evaluate safety with a smaller helper model (gpt-5.4-mini).")
print("If the result is UNSAFE, replace the response with a safe fallback message.")

```

13.7 Decorator-Based Guardrails

Instead of defining a class, you can build a concise guardrail with *decorators*.

```

# Decorator-based guardrails
print("Decorator-based guardrails:")
print("=" * 50)
print("""
from langchain.agents.middleware import (
    before_agent, after_agent, AgentState, hook_config
)
from langgraph.runtime import Runtime
from typing import Any

banned_keywords = ["hack", "exploit", "malware"]

# Input guardrail – decorator
@before_agent(can_jump_to=["end"])
def content_filter(
    state: AgentState, runtime: Runtime
) -> dict[str, Any] | None:
    """Blocks banned keywords."""
    if not state["messages"]:
        return None
    content = state["messages"][0].content.lower()
    for kw in banned_keywords:
        if kw in content:
            return {
                "messages": [{"role": "assistant",

```



```

        "content": "This request is not allowed."}],
        "jump_to": "end"
    }
    return None

# Output guardrail – decorator
@after_agent(can_jump_to=["end"])
def safety_check(
    state: AgentState, runtime: Runtime
) -> dict[str, Any] | None:
    "\\\"Checks whether the response contains sensitive content.\\\"\\\"
    last = state["messages"][-1]
    if hasattr(last, 'content') and 'password' in last.content:
        last.content = "The response contained sensitive information."
    return None

# Apply to the agent
agent = create_agent(
    model="gpt-5.4",
    tools=[search_tool],
    middleware=[content_filter, safety_check],
)
"""
print("Decorator-based guardrails work well for simple policies.")
print("Use the class-based approach for more complex logic such as state handling or initialization.")

```

13.8 Combining Multiple Guardrails

By adding several guardrails in order to the `middleware` list, you can build a *layered defense* strategy.

```

# Combine multiple guardrails
print("Combining multiple guardrails (defense in depth):")
print("=" * 50)
print("""
from langchain.agents import create_agent
from langchain.agents.middleware import (
    PIIMiddleware, HumanInTheLoopMiddleware
)

agent = create_agent(
    model="gpt-5.4",
    tools=[search_tool, send_email_tool],
    middleware=[
        # Layer 1: deterministic input filter
        ContentFilterMiddleware(
            banned_keywords=["hack", "exploit"]
        ),

        # Layer 2: PII protection (input + output)
        PIIMiddleware("email",
            strategy="redact", apply_to_input=True),
        PIIMiddleware("email",
            strategy="redact", apply_to_output=True),

        # Layer 3: human approval for sensitive tools
        HumanInTheLoopMiddleware(
            interrupt_on={"send_email": True}
        ),

        # Layer 4: model-based safety check
        SafetyGuardrailMiddleware(),
    ],
)
"""
print("Execution order:")
print("  input -> [ContentFilter] -> [PII input] -> agent execution")
print("          -> [HITL approval] -> [PII output] -> [Safety] -> response")

```

```
print()
print("Tip: place fast deterministic guardrails first and slower model-based checks later.")
```

13.9 Production Guardrail Patterns

Best practices

Pattern	Description	Implementation
<i>Layered defense</i>	Combine multiple guardrails to remove a single point of failure	<code>middleware=[layer1, layer2, ...]</code>
<i>Fail fast</i>	Run deterministic checks first to reduce cost	Deterministic → model-based order
<i>Input/output separation</i>	Use different guardrails for input and output	<code>before_agent</code> + <code>after_agent</code>
<i>Graceful rejection</i>	Return a friendly message when blocking a request	<code>jump_to="end"</code> + guidance message
<i>Logging and monitoring</i>	Record guardrail trigger events	LangSmith tracing integration
<i>Fallback strategy</i>	Handle failure inside the guardrail system itself	<code>try/except</code> + default policy
<i>Testing</i>	Validate guardrail behavior with unit tests	<code>GenericFakeChatModel</code>

Domain-specific examples

Domain	Main guardrails
<i>Healthcare</i>	PII (patient data), medical-advice disclaimer, emergency detection
<i>Finance</i>	PII (account data), investment disclaimer, HITL for transaction approval
<i>Customer service</i>	Sentiment analysis, escalation detection, PII masking
<i>Education</i>	Age-appropriateness checks, academic integrity, content filtering

13.10 Summary

This notebook covered:

Topic	Key Idea
-------	----------

<i>Guardrail concepts</i>	Guardrails validate and filter content during agent execution
<i>PII detection</i>	<code>PIIMiddleware</code> automatically detects and handles email, credit-card numbers, and related data
<i>HITL</i>	<code>HumanInTheLoopMiddleware</code> requires human approval before risky tool calls
<i>Custom input</i>	<code>before_agent</code> validates requests before execution
<i>Custom output</i>	<code>after_agent</code> validates responses after execution
<i>Decorators</i>	<code>\@before_agent</code> and <code>\@after_agent</code> define concise guardrails
<i>Layered defense</i>	Multiple guardrails can be stacked in the <code>middleware</code> list

Next Steps

→ Continue to the [LangGraph intermediate track](#)

References:

- [Guardrails](#)

P A R T

III

LangGraph

Stateful Agent Orchestration

What You Will Learn in This Part

- 1. Introduction to LangGraph
 - 2. Graph API Basics
- 3. Functional API Basics
 - 4. Workflow Patterns
 - 5. Building Agents
- 6. Persistence and Memory
 - 7. Streaming
- 8. Interrupts and Time Travel
 - 9. Subgraphs
 - 10. Production
 - 11. Local Server
- 12. Durable Execution
- 13. API Guide and Pregel

01 Introduction to LangGraph

state based agent orchestration framework

Learning Objectives

Understand the core concepts of LangGraph and two APIs (Graph API, Functional API).

1.1 What is LangGraph?

LangGraph is a *low-level orchestration framework* for the LangChain ecosystem.

LangChain 3-tier structure

tier	Role	Description
Deep Agents	high standard	Pre-built agent system
LangChain	agent	Building LLM agent tool
<i>LangGraph</i>	<i>workflow</i>	<i>state-based orchestration</i>

Key Features

- *state Management*: TypedDict-based state definition and reducer
- *Persistence*: Auto-save state via checkpointer
- *streaming*: Real-time token unit output
- *Human-in-the-loop*: interrupt/resume for human intervention
- *durable execution*: Automatic recovery in case of failure

1.2 Core concepts

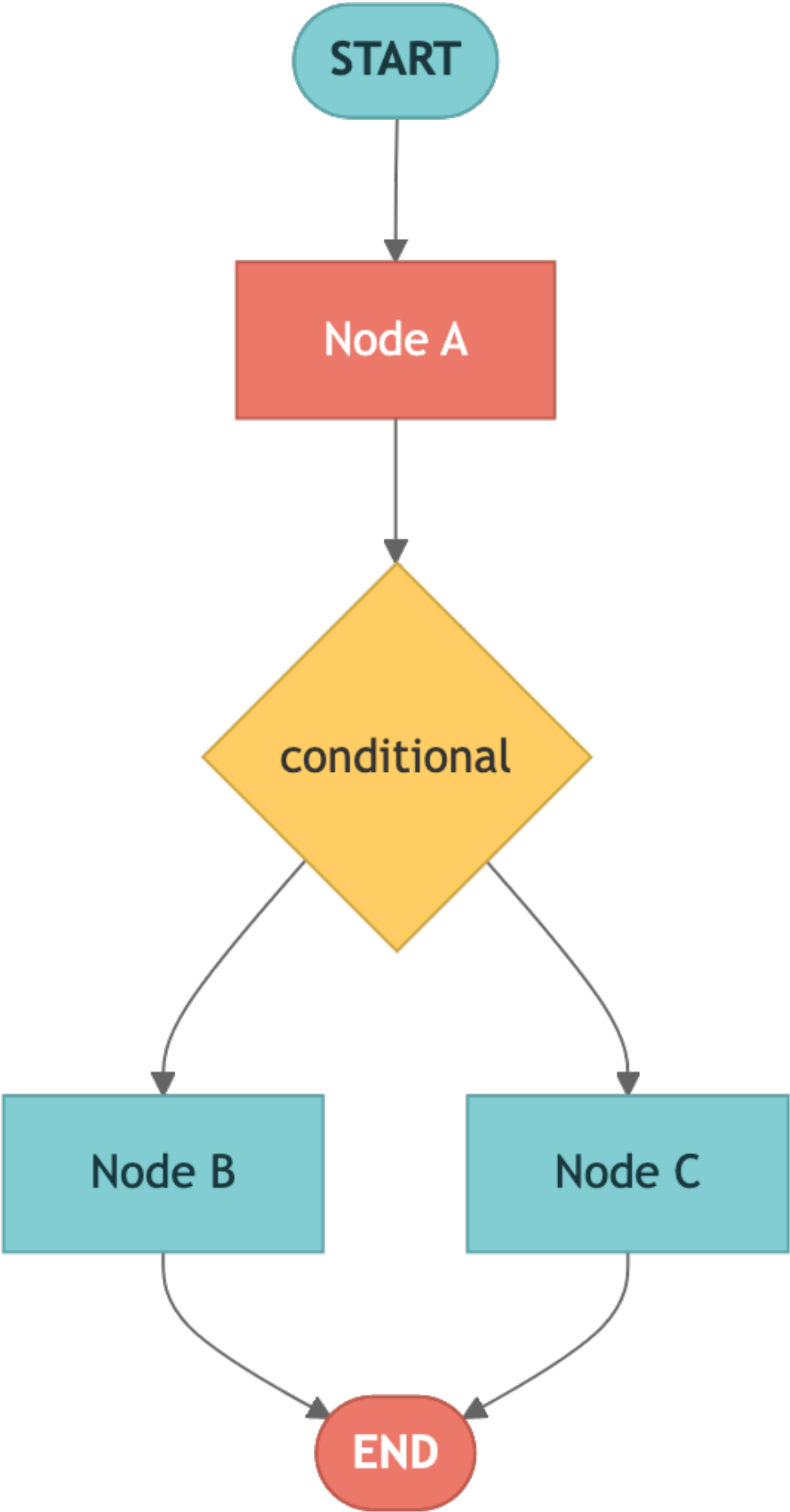
LangGraph defines workflow based on *graph structure*.

Component

concept	Description
<i>Node</i>	Processing unit — defined as a Python function
<i>Edge</i>	Connection between nodes, conditional branching possible

<i>state(State)</i>	Defined as TypedDict, shared data between nodes
<i>checkpointpointer(Checkpointer)</i>	Auto-save each step state

Graph structure diagram



1.3 Two APIs

LangGraph provides an API that allows the same functionality to be implemented in two different styles.

Characteristics	Graph API	Functional API
approach	declarative (node+edge)	imperative (Python control flow)
state Management	Explicit State + Reducer	No need for function scope or reducer
Visualization	Graph visualization support	Not supported
checkpointing	New checkpoint every superstep	By <code>@task</code> , save to existing checkpoint
suitable situation	Complex workflow, Team Development	Migrate existing code, simple flow

1.4 Environment Setup and check installation

Verify that the required packages are installed correctly.

```
import importlib

packages = {
    "langgraph": "langgraph",
    "langchain": "langchain",
    "langchain_openai": "langchain-openai",
}

print("=" * 50)
print("LangGraph Check environment")
print("=" * 50)

for module_name, package_name in packages.items():
    try:
        mod = importlib.import_module(module_name)
        version = getattr(mod, "__version__", "installed")
        print(f" OK {package_name}: {version}")
    except ImportError:
        print(f" ERR {package_name}: not installed")
```

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

required = ["OPENAI_API_KEY"]
optional = ["TAVILY_API_KEY", "LANGSMITH_API_KEY"]

print("API key state:")
for key in required:
    print(f" {'OK' if os.environ.get(key) else 'MISSING'} {key} (required)")
for key in optional:
    print(f" {'OK' if os.environ.get(key) else '--'} {key} (optional)")
```

```
# Core import verification
from langgraph.graph import StateGraph, START, END
from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver
```

```

from langgraph.types import Command, interrupt
from langchain.tools import tool
from langchain.messages import HumanMessage, SystemMessage, AIMessage
from langchain_openai import ChatOpenAI

print("All core imports completed")

```

1.5 A taste of the Graph API

The Graph API defines workflow in a *declarative* manner.

1. Create a graph builder with `StateGraph(State)` — `state_schema`
2. `add_node()` — Node (function) registration
3. `add_edge()` — Connections between nodes
4. `compile()` — Create an executable graph
5. `invoke()` — Graph execution

1.6 A taste of the Functional API

The Functional API defines workflow in an *imperative* manner.

- `@task` — Unit operation definition (checkpointing unit)
- `@entrypoint` — workflow Entry point definition
- Use regular Python control flow (`if` , `for` , `while` , etc.)

1.7 Next Steps

In the next Note book, we will learn Graph API in earnest.

- `02_graph_api.ipynb` — `StateGraph`, node, edge, conditional branch, state reducer

Summary

Item	Description
LangGraph	state Based on agent Orchestration Framework
Graph API	Explicit state flow definition with <code>StateGraph</code>
Functional API	<code>\@entrypoint</code> + <code>\@task</code> Functional workflow
Key concepts	State (state), Node, Edge
checkpointer	state Persistence, multi-turn dialogue, time travel support

Next Steps

→ [02_graph_api.ipynb](#): Learn the core concepts of Graph API.

02 Graph API Basics

Creating workflow with StateGraph

Learning Objectives

Understand StateGraph, nodes, edges, conditional branches, and state reducers.

2.1 Environment Setup

Load the LLM model and required modules.

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

2.2 StateGraph basic structure

The basic flow using StateGraph is as follows:

1. Create a graph builder with `StateGraph(State)` — `state_schema`
2. `add_node()` — Node (function) registration
3. `add_edge()` — Connections between nodes
4. `compile()` — Create an executable graph
5. `invoke()` — Graph execution

`StateGraph(State) → add_node() → add_edge() → compile() → invoke()`

2.3 state Reducer

Specifies the state update method with `Annotated` .

What is a reducer?

- The reducer determines *how* the fields in state are updated.
- No reducer: simple override

- `operator.add` : Accumulate (append) list items
- Reducers can also be defined as custom functions.

2.4 Conditional Edge

Branch to another node according to state.

- `add_conditional_edges(source, routing_function)` — The return value of the routing function is the next node name.
- Routing functions can be visualized using the `Literal` type hint.

START → classify → [route] → weather → END → math → END → general → END

2.5 Message-based state

Use `MessagesState` as appropriate for LLM agent.

- `MessagesState` is a predefined state that includes `messages: Annotated[list[AnyMessage], add_messages]`
- `add_messages` Reducer automatically accumulates the message list
- Naturally add LLM responses to message history

2.6 Input/Output Schema

Separate the graph's input and output from its internal state.

- `StateGraph(InternalState, input_schema=InputSchema, output_schema=OutputSchema)`
- Input schema: Includes only data received from external sources
- Output schema: Includes only data exported externally
- Internal state: Includes fields for intermediate processing (not exposed to the outside)

2.7 add_node() advanced options

`add_node()` accepts production-grade policy parameters for per-node control.

Parameter	Description
<code>retry_policy</code>	<code>RetryPolicy(max_attempts=..., retry_on=...)</code> — auto retry on failure
<code>timeout</code>	<code>TimeoutPolicy(run_timeout=..., idle_timeout=...)</code> — async nodes only; raises <code>NodeTimeoutError</code>
<code>cache_policy</code>	<code>CachePolicy(ttl=..., key_func=...)</code> — input-hash result caching; requires <code>cache=</code> at compile
<code>error_handler</code>	Callback receiving <code>NodeError</code> , returning a recovery <code>Command</code>
<code>defer</code>	Wait until other pending tasks complete — useful for asymmetric fan-out/fan-in

```

from langgraph.cache.memory import InMemoryCache
from langgraph.types import RetryPolicy, CachePolicy

builder.add_node(
    "expensive_node",
    expensive_node,
    retry_policy=RetryPolicy(max_attempts=3, retry_on=ValueError),
    cache_policy=CachePolicy(ttl=3),
)
graph = builder.compile(cache=InMemoryCache())

```

2.8 Runtime context with context_schema

Pass dependencies that aren't part of graph state (LLM providers, user IDs, DB handles) via `context_schema` + `Runtime[ContextSchema]`. Context lives outside the checkpointed state.

- `StateGraph(State, context_schema=ContextSchema)` declares the context schema
- Node signature `runtime: Runtime[ContextSchema]` exposes `runtime.context`
- Invoke with `graph.invoke(inputs, context={...})`
- `runtime.execution_info` exposes `attempt`, `thread_id`, `run_id`, checkpoint info
- `runtime.server_info` carries LangGraph Server assistant/graph/user info
- `runtime.drain_requested` signals graceful shutdown

```

from dataclasses import dataclass
from langgraph.runtime import Runtime

@dataclass
class ContextSchema:
    llm_provider: str = "openai"
    user_id: str | None = None

def node(state: State, runtime: Runtime[ContextSchema]):
    if runtime.execution_info.attempt > 1:
        # fallback on retry
        ...
    print("thread:", runtime.execution_info.thread_id)
    return {"results": "processed"}

graph = StateGraph(State, context_schema=ContextSchema).compile()
graph.invoke({...}, context={"llm_provider": "anthropic", "user_id": "u-1"})

```

2.9 Send and Command.PARENT

`Send("worker", {...})` returned from a conditional-edge function spawns a worker instance with its own state at runtime — the Map step of Map-Reduce. From inside a subgraph, `Command(graph=Command.PARENT, goto="...", update={...})` routes to a parent-graph node.

```

from langgraph.types import Send, Command

def fan_out(state):
    return [Send("worker", {"task": t}) for t in state["tasks"]]

def sub_node(state):
    return Command(
        graph=Command.PARENT,

```

```

goto="parent_aggregator",
update={"sub_result": state["value"]},
)

```

2.10 Summary

We summarize what we learned in this Note book.

concept	Description
<i>StateGraph</i>	<code>state_schema</code> based graph builder
<i>Node</i>	Processing units defined as Python functions
<i>Edge</i>	Fixed connection between nodes (<code>add_edge</code>)
<i>Conditional Edge</i>	state based dynamic branch (<code>add_conditional_edges</code>)
<i>Reducer</i>	Define state accumulation method with <code>Annotated + operator.add</code>
<i>MessagesState</i>	Predefined state for LLM dialog
<i>Input/Output Schema</i>	Separation of internal state and external input/output

Next Steps

→ [03_functional_api.ipynb](#): Learn Functional API.

03 Functional API

Creating workflow with @entrypoint and @task

Learning Objectives

Understand the `@entrypoint` , `@task` patterns and short-term memory of the Functional API.

3.1 Environment Setup

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

3.2 @task — Asynchronous unit of work

- Checkpointing is possible by wrapping it with the `@task` decorator.
- Returns Future object immediately when called, waits with `.result()`

3.3 Parallel `@task` execution

Running multiple `@task` simultaneously.

3.4 previous — short-term memory (accessing previous execution results)

3.5 entrypoint.final — Separate return and checkpoint saved values

3.6 Determinism Requirements

Non-deterministic operations must be wrapped in `@task`.

3.7 LLM agent (Functional API)

Implementing ReAct agent with while loop

3.8 @task retry and caching

`@task` accepts the same policy objects as `add_node()`. `RetryPolicy` configures auto-retry, and `CachePolicy` caches results by input hash. Caching requires attaching `cache=InMemoryCache()` on the entrypoint and `cache_policy` on the task — the two are configured separately.

```
from langgraph.cache.memory import InMemoryCache
from langgraph.types import RetryPolicy, CachePolicy

@task(retry_policy=RetryPolicy(max_attempts=3, retry_on=ValueError))
def fetch_data(url: str) -> dict: ...

@task(cache_policy=CachePolicy(ttl=120))
def slow_compute(x: int) -> int:
    time.sleep(1)
    return x * 2

@entrypoint(cache=InMemoryCache())
def main(inputs: dict) -> dict[str, int]:
    a = slow_compute(inputs["a"]).result()
    b = slow_compute(inputs["b"]).result()
    return {"a": a, "b": b}
```

3.9 Structured interrupt and Command(resume=)

`interrupt()` pauses the workflow for human review. Pass a structured dict payload so callers get rich context, and resume with `Command(resume=...)`.

```
from langgraph.types import Command, interrupt

@task
```

```
def review(input_query: str):
    human_review = interrupt({
        "question": "Approve this result?",
        "tool_call": {"name": "send_email", "args": {...}},
    })
    return human_review

config = {"configurable": {"thread_id": "review-1"}}
workflow.invoke("draft", config=config)
workflow.invoke(Command(resume={"approved": True}), config=config)
```

3.10 @entrypoint injected parameters

Beyond `previous`, `@entrypoint` auto-injects runtime dependencies by parameter name.

Parameter	Role
<code>previous</code>	Last checkpoint's saved value (short-term memory)
<code>store</code>	<code>BaseStore</code> instance for long-term memory
<code>writer</code>	<code>StreamWriter</code> for custom streaming (async + Python < 3.11)
<code>config</code>	<code>RunnableConfig</code> — <code>thread_id</code> , metadata, configurable values

```
from langgraph.types import StreamWriter

@entrypoint(checkpointer=checkpointer, store=store)
def workflow(
    inputs: dict,
    *,
    previous=None,
    store=None,
    writer: StreamWriter = None,
    config=None,
):
    writer({"phase": "start"})
    user_id = config["configurable"]["user_id"]
    memory = store.get(("users", user_id), "profile")
    ...
```

3.11 Summary

Features	Description
<code>\@task</code>	Asynchronous operations, checkpointing, parallel execution
<code>\@entrypoint</code>	workflow Entry point, execution management
<code>.result()</code>	Future Result Synchronous Waiting
<code>previous</code>	Access previous execution results (short-term memory)
<code>entrypoint.final</code>	Separate return value ≠ stored value

Next Steps

→ [04_workflows.ipynb](#): Learn the workflow pattern.

04 workflow Pattern

5 core patterns

Learning Objectives

Understand Prompt Chaining, Parallelization, Routing, Orchestrator-Worker, and Evaluator-Optimizer patterns.

4.1 Environment Setup

```
from dotenv import load_dotenv
load_dotenv(override=True)

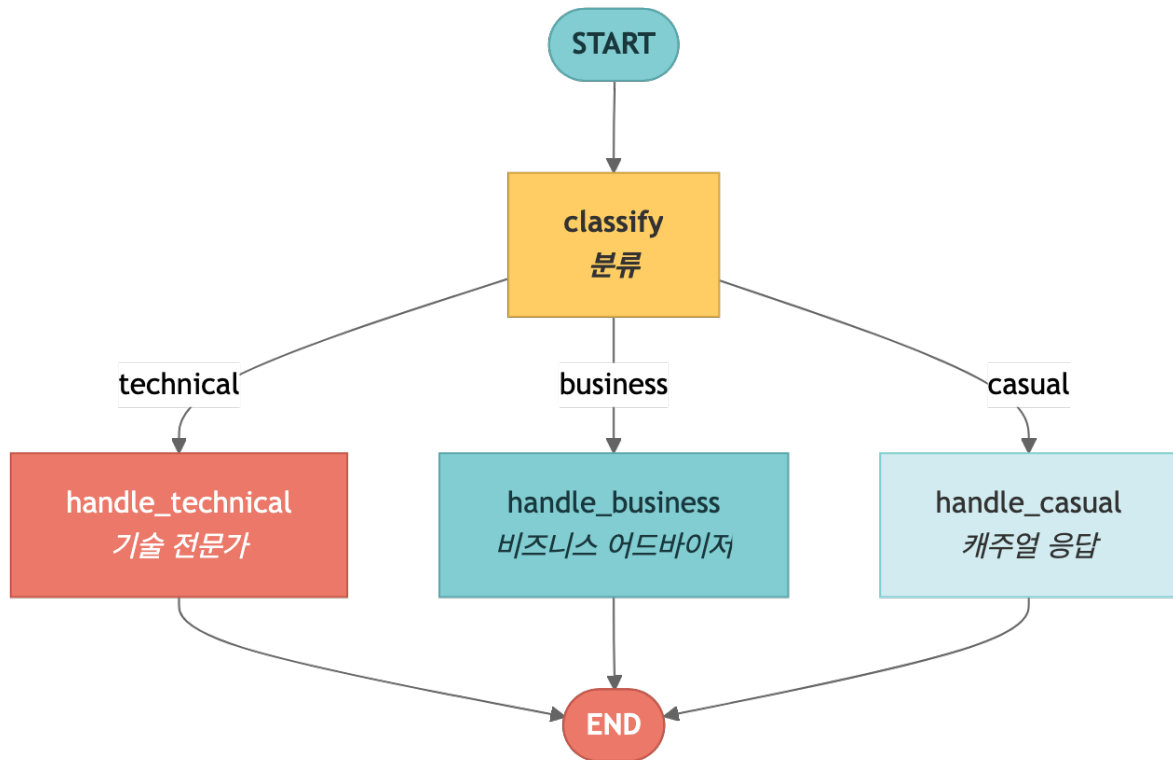
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

4.2 Prompt Chaining — Sequential LLM calls

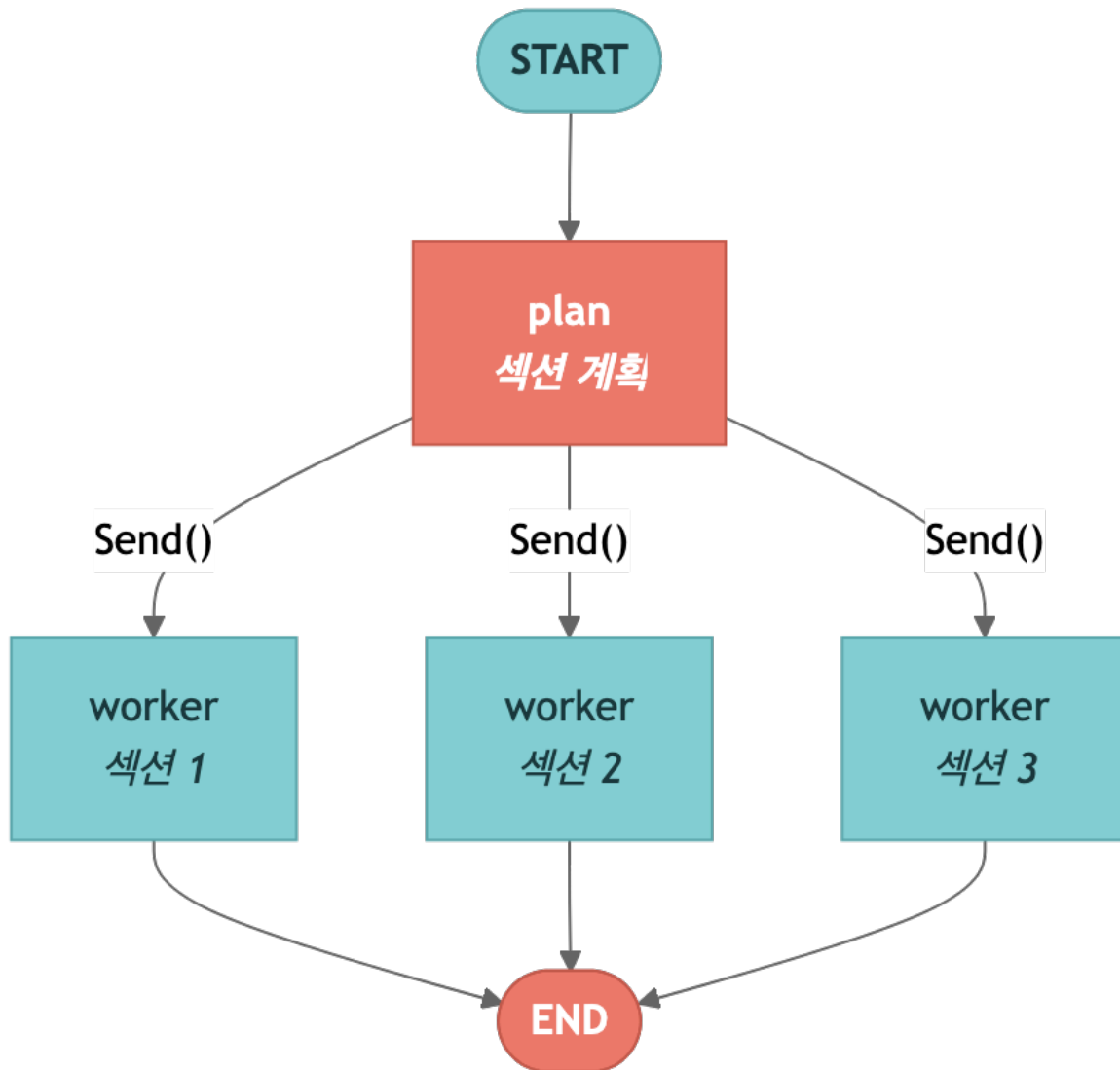
- The output of each step becomes the input of Next Steps
- Purpose: Translation → Verification → Proofreading, Analysis → Summary → Formatting

4.3 Parallelization — Simultaneous execution of independent \@task

4.4 Routing — Classification-based branching



4.5 Orchestrator-Worker – Create dynamic worker with Send()



4.6 Evaluator-Optimizer – Generate-evaluation iteration loop

4.7 Pattern comparison table

pattern	decisive	Parallel	repeat	suitable situation
Prompt Chaining	O	X	sequential	Step-by-step conversion
Parallelization	O	O	X	Independent Analysis

Routing	O	X	X	Classification-based processing
Orchestrator-Worker	O	O	X	Dynamic Subtasks
Evaluator-Optimizer	X	X	O	Quality Improvement Loop

Next Steps

→ [05_agents.ipynb](#): Build ReAct agent.

05 Building agent

Creating ReAct agent with Graph API and Functional API

Learning Objectives

We implement LLM agent using tool with two APIs.

- *Graph API*: Explicitly configure ReAct loop with `StateGraph` and conditional edges
- *Functional API*: Simple implementation with `@entrypoint` + `while` loop.
- *tool Binding*: Bind tool to LLM with `@tool` decorator and `bind_tools()` .
- *Memory*: agent holding conversation state with checkpoint

5.1 Environment Setup

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

5.2 tool Definitions — @tool decorator and bind_tools()

The `@tool` decorator on LangChain allows you to convert a regular Python function into tool, which can be called by LLM. `bind_tools()` binds the schema of these tools to the model, allowing LLM to select tool and generate arguments at the appropriate time.

```
from langchain.tools import tool
from langchain.messages import HumanMessage, SystemMessage, ToolMessage, AnyMessage

@tool
def add(a: int, b: int) -> int:
    """Add two numbers together."""
    return a + b

@tool
def multiply(a: int, b: int) -> int:
    """Multiply two numbers."""
    return a * b

@tool
def divide(a: int, b: int) -> float:
    """Divide a by b."""
    return a / b

tools = [add, multiply, divide]
```



```
tools_by_name = {t.name: t for t in tools}
model_with_tools = model.bind_tools(tools)

print("tool bound to model:")
for t in tools:
    print(f"  - {t.name}: {t.description}")
```

5.3 Graph API agent — Implementing ReAct Loop with StateGraph

The ReAct (Reasoning + Acting) pattern consists of three elements:

- *LLM node*: Determines whether tool calling based on current message
- *Tool node*: actually executes the tool selected by LLM
- *Conditional Edge*: If `tool_calls` exists, route to `tool_node`, otherwise route to `END`

START → llm → [tool_calls?] → tools → llm → ... → END

5.4 Visualizing execution flow — Observe each step with streaming

`stream_mode="updates"` allows each node to receive updates as it runs. This allows you to observe step by step in what order agent calls tool and processes the results.

5.5 Functional API agent — @entrypoint + while loop

The Functional API allows you to write agent like regular Python code, without explicitly constructing a graph.

- `@entrypoint` : Defines the entry point of agent
- `@task` : Defines individual work units
- `while` loop: repeats until there is no tool calling

5.6 agent with memory — Keep conversation with checkpointer

If you pass `checkpointer(InMemorySaver)` to `compile()`, agent will be able to remember previous conversations. If you use the same `thread_id`, the previous conversation context is automatically maintained.

5.7 Summary

concept	Description
<code>\@tool</code>	Convert Python function to LLM callable tool

<code>bind_tools()</code>	tool Binding schema to model
<i>Graph API agent</i>	<code>StateGraph</code> + Explicit implementation of ReAct loop with conditional edges
<i>Functional API agent</i>	Simple implementation with <code>@entrypoint</code> + <code>while</code> loop
<code>tool_calls</code>	tool calling Information included in LLM response
<code>ToolMessage</code>	tool Message delivering execution results to LLM
<i>checkpointer</i>	Implement multiturn agent by saving conversation state

Next Steps

→ [06_persistence_and_memory.ipynb](#): Learn persistence and memory.

06 Persistence and memory

checkpointer and memory store

Learning Objectives

Store state with checkpointer and implement long-term memory with store.

- *checkpointer*: Automatically save and restore state of each execution step.
- *state lookup*: Check state saved as `get_state()` and `get_state_history()`
- *state Modification*: Change state externally with `update_state()` .
- *Thread Independence*: Different `thread_id` are completely independent state
- *InMemoryStore*: long-term memory shared between threads (standalone and graph integration)
- *Conversation length management*: Message management with `trim_messages` and `RemoveMessage`
- *Durable Execution*: resume at last checkpoint in case of failure

6.1 Environment Setup

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4-mini")
```

6.2 checkpointer — Automatically saves state for each execution step

LangGraph offers multiple checkpointer implementations. The seven commonly used options are summarized below.

Checkpointer	Package	Use
InMemorySaver	langgraph-checkpoint (built-in)	Development/testing. State lost on process exit
SqliteSaver	langgraph-checkpoint-sqlite	Local development. Single-file persistence, async variant
PostgresSaver	langgraph-checkpoint-postgres	Production default. AsyncPostgresSaver for async graphs
MongoDBSaver	langgraph-checkpoint-mongodb	Document DB. JSON-friendly storage backend
RedisSaver	langgraph-checkpoint-redis	In-memory persistence. Low latency, async variant
OracleSaver	langgraph-checkpoint-oracle	Enterprise Oracle DB. Vector search included

CosmosDBSaver	langchain-azure-cosmosdb	Azure Cosmos DB. Microsoft Entra ID authentication
---------------	--------------------------	--

If you pass `checkpointer` to `compile()`, state will be automatically saved after each node in the graph is executed. DB-backed checkpointers are initialized with the context-manager + `setup()` pattern. Use async variants (e.g. `AsyncPostgresSaver`) for async graphs.

```
from langgraph.checkpoint.postgres import PostgresSaver

DB_URI = "postgresql://postgres:postgres@localhost:5442/postgres?sslmode=disable"
with PostgresSaver.from_conn_string(DB_URI) as checkpointer:
    checkpointer.setup() # first time only – create schema
    graph = builder.compile(checkpointer=checkpointer)

# Async graphs
from langgraph.checkpoint.postgres import AsyncPostgresSaver

checkpointer = AsyncPostgresSaver.from_conn_string(DB_URI)
await checkpointer.setup()
graph = builder.compile(checkpointer=checkpointer)
```

6.3 `get_state()` — Retrieve currently stored state lookup

`get_state()` returns the latest checkpoint state for the specified thread. You can check information such as number of messages, checkpoint ID, and next node to run.

```
state = graph.get_state(config)
print(f"Thread: {config['configurable']['thread_id']}")
print(f"Message count: {len(state.values['messages'])}")
print(f"Checkpoint ID: {state.config['configurable']['checkpoint_id']}")
print(f"Next node: {state.next}")
```

6.4 `get_state_history()` — View entire execution history

`get_state_history()` returns all checkpoints for that thread, sorted by most recent. This allows you to trace the entire history of graph execution.

```
print("state History (most recent):")
for i, snapshot in enumerate(graph.get_state_history(config)):
    msg_count = len(snapshot.values.get("messages", []))
    print(f" [{i}] checkpoint={snapshot.config['configurable']['checkpoint_id'][:20]}...
    messages={msg_count}")
    if i >= 4:
        print("... (omitted)")
        break
```

6.5 `update_state()` — Modify stored state externally

`update_state()` allows you to programmatically modify state stored in a checkpoint. For example, you can add system Note, reflect user preferences, and more.

6.6 Thread Independence — Different `thread_ids` are completely independent state

Each `thread_id` has a completely independent conversation state. Conversations in different threads do not affect each other.

6.7 InMemoryStore — Cross-thread sharing long-term memory

`InMemoryStore` is a key-value store shared between threads. Used to store information that needs to be maintained across threads, such as user profiles and preferences.

- `put()` : Store data with namespace and key
- `get()` : Search for a specific item
- `search()` : Search within namespace

```
from langgraph.store.memory import InMemoryStore

store = InMemoryStore()

# data storage
store.put(("users",), "alice", {"favorite_color": "blue", "city": "Seoul"})
store.put(("users",), "bob", {"favorite_color": "red", "city": "Tokyo"})

# Data inquiry
alice = store.get(("users",), "alice")
print(f"Alice: {alice.value}")

# search
results = store.search(("users",))
print(f"\nTotal users ({len(results)}):")
for item in results:
    print(f"  {item.key}: {item.value}")
```

6.7.5 Using InMemoryStore with graphs

If you pass `InMemoryStore` to the graph as `compile(store=store)`, you can directly access store through the `store` parameter in each node function. This pattern allows you to store and retrieve user information within a node, maintaining long-term memory across threads.

- `compile(store=store)` : connect store to the graph.
- Add `store` parameter to node function: LangGraph automatically injects store instance
- Separate namespace for each user with `config["configurable"]["user_id"]`

Context and Runtime — accessing `user_id` and store from nodes

To partition long-term memory *per user*, the node function needs the current user's ID. LangGraph solves this with a `@dataclass` `Context` registered via `StateGraph(..., context_schema=Context)`, plus a `runtime: Runtime[Context]` parameter on the node. Pass `context=Context(user_id=...)` to `graph.invoke(...)`, then access `runtime.context.user_id` and `runtime.store` inside the node.

```
import uuid
from dataclasses import dataclass
from langgraph.runtime import Runtime
from langgraph.graph import StateGraph, MessagesState, START
```

```

@dataclass
class Context:
    user_id: str

async def call_model(state: MessagesState, runtime: Runtime[Context]):
    user_id = runtime.context.user_id
    namespace = (user_id, "memories") # recommended pattern

    memories = await runtime.store.asearch(
        namespace, query=state["messages"][-1].content, limit=3
    )
    await runtime.store.aput(
        namespace, str(uuid.uuid4()), {"data": "User prefers dark mode"}
    )
    return {"messages": [await model.ainvoke(state["messages"])]}

builder = StateGraph(MessagesState, context_schema=Context)
builder.add_node(call_model)
builder.add_edge(START, "call_model")
graph = builder.compile(checkpointer=checkpointer, store=store)

graph.invoke(
    {"messages": [{"role": "user", "content": "hi"}]},
    {"configurable": {"thread_id": "1"}},
    context=Context(user_id="alice"),
)

```

TIP

Any tuple shape works as a namespace, but the official docs recommend `(user_id, "memories")`. First-level partitioning by user, second-level by memory kind (`"memories"`, `"preferences"`, etc.) makes category-scoped `search()` straightforward.

Production stores follow the same context-manager pattern as checkpointer:

```

from langgraph.store.postgres import PostgresStore

with PostgresStore.from_conn_string(DB_URI) as store:
    store.setup()
    graph = builder.compile(checkpointer=checkpointer, store=store)

# Redis / Oracle follow the same pattern
from langgraph.store.redis import RedisStore
from langgraph.store.oracle import OracleStore # vector search built-in

```

Semantic search with `index`

Enable embedding-based search by passing `index={"embed": ..., "dims": ..., "fields": [...]}`:

```

from langchain.embeddings import init_embeddings
from langgraph.store.memory import InMemoryStore

store = InMemoryStore(
    index={
        "embed": init_embeddings("openai:text-embedding-3-small"),
        "dims": 1536,
        "fields": ["food_preference", "$"],
    }
)

```

```
store.put(("alice", "memories"), "1", {"food_preference": "I love pizza"})
items = store.search(("alice", "memories"), query="I'm hungry", limit=1)
```

Use `index=False` on `put()` to skip embedding for a specific item; pass a list of keys to limit which fields get embedded.

6.7.6 Managing conversation length — `trim_messages` and `RemoveMessage`

Long conversations can exceed LLM's context window. LangGraph manages messages in two ways:

`trim_messages`

- Automatically trims old messages based on number of tokens
- `strategy="last"` : Keep only recent messages
- `start_on="human"` : Ensure truncated results always start with the user message.
- Returns a list of truncated messages, without modifying the original state (full history is maintained in checkpoints)

`RemoveMessage`

- Permanently delete specific messages from checkpoints
- The reducer of `MessagesState` detects `RemoveMessage` and removes that message.
- Useful for saving storage space by organizing old messages

6.8 Durable Execution — resume at last checkpoint on failure

Durable Execution is possible using checkpointer. Even if an error occurs during graph execution, you can resume at the last successful checkpoint. Nodes that have already been completed are not rerun, saving money and time.

The example below configures a three-stage pipeline:

1. *step_1*: Collect data (always succeeds)
2. *step_2*: Data analysis (always successful)
3. *step_3*: External API call (failure on first run, success on retry)

With `attempt_count`, *step_3* fails on the first run, and when calling `invoke()` the second time, *step_1* and *step_2* are skipped and only resume occurs in *step_3*.

6.9 Summary

concept	Description
<i>checkpointer</i>	Automatically save state after each node execution (<code>InMemorySaver</code> , <code>SqliteSaver</code> , <code>PostgresSaver</code>)

<code>get_state()</code>	View the latest checkpoint state of the current thread
<code>get_state_history()</code>	View the entire checkpoint history of a thread (most recent)
<code>update_state()</code>	Programmatically modifying stored state
<i>Thread independence</i>	Different <code>thread_id</code> are completely independent state
<code>InMemoryStore</code>	Key-value shared across threads long-term memory storage
<code>compile(store=store)</code>	Access long-term memory with <code>store</code> parameter from graph node
<code>trim_messages</code>	Cut out old messages based on number of tokens and pass them to LLM (maintain checkpoints)
<code>RemoveMessage</code>	Permanently delete specific messages from checkpoint
<i>Durable Execution</i>	On failure, at the last successful checkpoint resume

Next Steps

→ [07_streaming.ipynb](#): Learn streaming.

07 streaming

Observe agent execution in real time

Learning Objectives

Understand and utilize the various streaming modes of LangGraph.

- Understand the differences between `values`, `updates`, `messages`, `custom`, `debug` modes
- Identify appropriate use cases for each streaming mode
- Learn how to use multiple streaming modes simultaneously

7.1 Environment Setup

7.2 streaming Mode Comparison

LangGraph provides various streaming modes. Each mode conveys a different level of information in real time.

mode	Description	Use
<code>values</code>	Total state of each step	Debugging, tracing state
<code>updates</code>	Only updates returned by each node	Show progress
<code>messages</code>	Message Token Unit	Chat UI
<code>custom</code>	custom event	Custom Progress
<code>debug</code>	Full debug information	Debugging during development

7.3 stream_mode="values" – Full state snapshot

`values` mode returns *full state* after each node execution. This is useful for tracking the overall flow of how the graph progresses.

7.4 `stream_mode="updates"` — Node-specific updates

`updates` mode only passes the update value returned by each node. You can see exactly which nodes made which changes.

7.5 `stream_mode="messages"` — Per-token streaming

The `messages` mode delivers *tokens generated by LLM in real time*. Best suited for implementing typing effects in chat UI.

7.6 Simultaneous use of multiple streaming modes

You can use multiple modes *simultaneously* by passing a list to `stream_mode`. The returned event is in the form of a `(mode, data)` tuple.

7.7 `stream_mode="custom"` — custom streaming

`custom` mode lets you stream arbitrary data directly inside a node.

If you call `get_stream_writer()` of `langgraph.config`, you can get the `writer` function. If you pass data such as a dictionary to this function, it will be sent to the client in real time while the graph is running.

Use Case:

- Progress reporting of long tasks
- *Chunk-wise streaming* in external LLM without using LangChain
- Immediate delivery of *intermediate results* inside the node

TIP

If you graph with `stream_mode="custom"` streaming, only the data sent with `writer()` will be received.

7.8 `nostream` tag — exclude internal LLM calls from the messages stream

When a graph mixes a user-facing model and internal evaluation/routing models, the internal tokens should not leak into the `messages` stream. Tag the auxiliary model with `nostream` and LangGraph filters it out automatically.

```
from langchain_anthropic import ChatAnthropic

# Auxiliary model — not exposed in the messages stream
internal_model = ChatAnthropic(model_name="claude-haiku-4-5-20251001").with_config(
    {"tags": ["nostream"]}
)
```

7.9 `chunk_position` — detect the last chunk of a message

The `messages` mode metadata includes a `chunk_position` field. `chunk_position == "last"` marks the completion of a single message. Run token-by-token UI updates as usual, but trigger persistence/post-processing only on the last chunk.

```
for msg, metadata in graph.stream(inputs, stream_mode="messages"):
    if metadata.get("chunk_position") == "last":
        save_final_message(msg, node=metadata["langgraph_node"])
        continue
    print(msg.content, end="", flush=True)
```

The three metadata keys you'll actually use:

- `tags` — tags attached to the model/chain (e.g. `["joke"]`, `["nostream"]`)
- `langgraph_node` — node that produced the current token
- `chunk_position` — `"first"`, `"middle"`, or `"last"`

7.10 `invoke v2` — `GraphOutput` return type

Calling `invoke()` with `version="v2"` wraps the result in `GraphOutput`. Legacy dict access still works but is deprecated; new code should use `.value` / `.interrupts`.

```
from langgraph.types import GraphOutput

result = graph.invoke(inputs, version="v2")
assert isinstance(result, GraphOutput)

result.value          # final state (dict / Pydantic / dataclass)
result.interrupts     # tuple[Interrupt, ...] — empty when no interrupt

if result.interrupts:
    payload = result.interrupts[0].value
    print("HITL pending:", payload)
```

7.11 `stream_events v3` — projection-based resume

LangGraph 1.2+ adds an *event streaming projection* layer on top of the raw `stream_mode` API.

`graph.stream_events(..., version="v3")` returns a single run-stream object; callers consume independent projections such as `stream.messages`, `stream.values`, `stream.subgraphs`, `stream.output`. The biggest gain over v2 raw streaming is *resume* — the same API restarts the run after an interrupt.

```
from langgraph.types import Command

stream = graph.stream_events(
    {"messages": [{"role": "user", "content": "Start security review"}]},
    version="v3",
)

for message in stream.messages:
    for token in message.text:
        print(token, end="", flush=True)
```

```

if stream.interrupted:
    print("\nApproval needed:", stream.interrupts)
    stream = graph.stream_events(
        Command(resume={"decisions": [{"type": "approve"}]}),
        version="v3",
    )
    for message in stream.messages:
        for token in message.text:
            print(token, end="", flush=True)

final_state = stream.output # single entry point for the final state

```

TIP

Use v2 raw streaming for debugging and custom-mode processing, v3 `stream_events` for app/UI code and typed projection consumption. The two APIs coexist, so a single graph can serve a debug v2 stream and a production v3 stream side by side.

7.12 Summary

streaming mode	Description	Use
<code>values</code>	Return full state after each step	Debugging, tracing state
<code>updates</code>	Return only the part changed by the node	Monitor progress
<code>messages</code>	LLM token real-time streaming	Chat UI implementation
Multiple modes simultaneously	Pass as list → Receive <code>(mode, data)</code> tuple	Complex Monitoring
<code>custom</code>	Send random data to <code>get_stream_writer()</code>	Progress Reporting, External LLM

Next Steps

→ [08_interrupts_and_time_travel.ipynb](#): Learn interrupts and time travel.

08

Interrupts and Time Travel

Execute interrupt, Acknowledge, Rewind

Learning Objectives

Execute with `interrupt()`, `interrupt`, and resume with `Command(resume=...)`. Time travel back to the previous state.

- Human-in-the-loop pattern can be implemented
- Interrupt can also be used in Functional API
- You can perform time travel using checkpoint history.
- state can be modified externally with `update_state()`

8.1 Environment Setup

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4-mini")
print("Model is ready")
```

8.2 `interrupt()` — executes interrupt and waits for human input

- `interrupt(value)`: Save the current state to the checkpoint and execute interrupt
- `Command(resume=value)`: Passes the value at interrupt point and resume

This pattern is used to obtain human approval or input additional information before performing sensitive tasks.

8.3 `Command(resume=...)` — interrupt executes resume

Using `Command(resume=value)` causes execution to resume at the point where `interrupt()` is called. The value passed to `resume` becomes the return value of `interrupt()`.

`Command(resume=...)` accepts boolean, text, or dict payloads. The pattern depends on the response shape:

```
from langgraph.types import Command

# 1) Simple boolean — Approval
```

```
graph.invoke(Command(resume=True), config=config, version="v2")

# 2) Text – Review & Edit
graph.invoke(Command(resume="The edited text"), config=config, version="v2")

# 3) Structured dict – Tool interrupt / multi-field response
graph.invoke(
    Command(resume={"action": "approve", "subject": "Updated subject"}),
    config=config,
    version="v2",
)
```

Common Patterns — five HITL scenarios

The five interrupt patterns you'll see in production:

1) Approval — sensitive action gate

Pause right before a critical action (API call / DB mutation) and route based on the decision.

```
from typing import Literal
from langgraph.types import Command, interrupt

def approval_node(state) -> Command[Literal["proceed", "cancel"]]:
    decision = interrupt({
        "question": "Approve this action?",
        "details": state["action_details"],
    })
    return Command(goto="proceed" if decision else "cancel")

graph.invoke(Command(resume=True), config=config, version="v2") # approve
graph.invoke(Command(resume=False), config=config, version="v2") # reject
```

2) Review & Edit — human revises LLM output

```
def review_node(state):
    edited = interrupt({
        "instruction": "Review and edit this content",
        "content": state["generated_text"],
    })
    return {"generated_text": edited}

graph.invoke(Command(resume="edited body ..."), config=config, version="v2")
```

3) Tool Interrupt — approve before the tool runs

Put `interrupt()` inside a `@tool` to require human approval/edits. The resume payload is typically

```
{"action": "approve", ...} .
```

```
from langchain.tools import tool

@tool
def send_email(to: str, subject: str, body: str):
    response = interrupt({
        "action": "send_email",
        "to": to, "subject": subject, "body": body,
        "message": "Approve sending this email?",
    })
    if response.get("action") == "approve":
        return f"Email sent to {response.get('to', to)}"
    return "Email cancelled by user"
```

4) Input Validation — loop until valid

```
def get_age_node(state):
    prompt = "What is your age?"
    while True:
        answer = interrupt(prompt)
        if isinstance(answer, int) and answer > 0:
            break
        prompt = f"'{answer}' is not valid. Please enter a positive number."
    return {"age": answer}
```

5) Multiple Interrupts — match parallel responses

When parallel nodes raise interrupts simultaneously, use a resume map keyed by interrupt id:

```
result = graph.invoke({"vals": []}, config, version="v2")

resume_map = {
    i.id: f"answer for {i.value}" for i in result.interrupts
}
graph.invoke(Command(resume=resume_map), config, version="v2")
```

8.4 Interrupt in Functional API

You can also use `interrupt()` in the Functional API (`@entrypoint`, `@task`).

8.5 Time Travel — Go back to a previous checkpoint

The checkpoint system in LangGraph stores all executions of state. You can view previous checkpoints with `get_state_history()` and go back to a specific point in time.

8.6 update_state() — Time travel + state fix

`update_state()` allows you to directly modify the state of a graph from the outside. This is useful for debugging, testing, or when manual intervention is required.

Replay vs Fork — two time-travel modes

- **Replay** — re-run from a previous checkpoint as-is. Nodes after `next` are re-executed (LLM/API calls happen again — results may differ).
- **Fork** — modify state at a previous checkpoint and create a new branch. The original thread is preserved.

```
# Replay — invoke with the checkpoint config
history = list(graph.get_state_history(config))
before_joke = next(s for s in history if s.next == ("write_joke",))
replay_result = graph.invoke(None, before_joke.config)

# Fork — create a new branch with update_state
fork_config = graph.update_state(
    before_joke.config,
    values={"topic": "chickens"},
```

```
)
fork_result = graph.invoke(None, fork_config)
```

TIP

Pass the *return value* of `update_state()` (`fork_config`) to `invoke()` , not the original `before.config` . The new checkpoint id lives inside `fork_config` .

`as_node` — declare which node owns the update

`update_state(config, values, as_node="...")` lets you declare which node should be treated as the source of the update.

Useful when:

- parallel branches make ownership ambiguous,
- seeding initial state into a fresh thread for a test, or
- skipping a node and resuming from its successor.

```
fork_config = graph.update_state(
    before_joke.config,
    values={"topic": "chickens"},
    as_node="generate_topic", # resume from this node's successor
)
graph.invoke(None, fork_config)
```

Reducer behavior — merge vs overwrite

How `values` is applied depends on whether the channel has a reducer.

- *Channel with reducer* (e.g. `MessagesState.messages` uses `add_messages`) → values are *merged*; appending messages accumulates.
- *Channel without reducer* (plain `TypedDict` fields) → values are *overwritten*.

To *replace* a single message, send `RemoveMessage(id=...)` plus the new one. Plain dict fields can be replaced directly with `values={"field": new_value}` .

8.7 Summary

Summarize the key functions learned in this Note book.

Features	API	Description
<code>interrupt(value)</code>	Both sides	run interrupt, pass value
<code>Command(resume=value)</code>	Both sides	resume at point interrupt
<code>get_state_history()</code>	Graph	Checkpoint history inquiry
<code>update_state()</code>	Graph	Modify state externally

interrupts and time travel are key features in production AI applications:

- *interrupt*: Get human approval before sensitive operations
- *Time Travel*: You can go back to the previous state and explore different routes
- *update_state*: You can adjust the execution flow by modifying state externally.

Next Steps

→ [09_subgraphs.ipynb](#): Learn subgraphs.

09 Subgraph

Graph within a graph

Learning Objectives

Modularize complex workflow with subgraphs.

9.1 Environment Setup

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4-mini")
```

9.2 Subgraph concept

- *Subgraph*: Independent graph used as a node in another graph
- *Advantages*: Modularization, reuse, independent development by team
- Each subgraph has its own state(State)
- state between parent <-> subgraph is mapped to *shared key*

9.3 Creating a subgraph

9.4 Adding a subgraph to the parent graph

9.4.1 Pattern 1: Subgraph call through wrapper node (if `state_schema` is different)

In the above example (9.4), the parent graph and subgraph shared *the same keys* (`text`, `word_count`, `char_count`), so the compiled subgraph could be passed directly to `add_node()`.

However, in practice, the `state_schema` of the parent graph and subgraph is often completely different. In this case, use a **wrapper function**:

1. *Extract* the required fields from the parent state and convert them to subgraph input.
2. *Run* the subgraph
3. *Map* the subgraph output to the parent state format.

Use the pattern. This is how the official documentation calls it *Pattern 1: Call Subgraph Inside a Node*.

9.5 LLM-based subgraph

Organize the full text agent into subgraphs.

9.6 Subgraph streaming

Steps inside a subgraph are also streaming possible.

With v2 streaming (`version="v2"`) subgraph chunks arrive as a uniform `StreamPart` dict. Three fields cover everything:

```
for chunk in graph.stream(
    {"foo": "foo"},
    subgraphs=True,
    stream_mode="updates",
    version="v2",
):
    print(chunk["type"]) # "updates"
    print(chunk["ns"])  # () = root, ("node_2:<id>") = subgraph
    print(chunk["data"]) # {"node_name": {"key": "value"}}
```

- `chunk["type"]` — mode identifier (`"updates"` , `"values"` , `"messages"` , ...)
- `chunk["ns"]` — namespace tuple. `()` for root, `("node_2:<uuid>")` for a subgraph
- `chunk["data"]` — mode-specific payload

TIP

In v1 the return shape changed (tuple vs dict) depending on `subgraphs=True` . v2 always returns the same dict, so root- and subgraph-handling code can be unified.

Subgraph checkpointer — three modes

The `checkerpoint=` argument on subgraph compile controls memory and interrupt behavior.

Mode	checkerpoint=	Behavior	Fit
Per-invocation (default)	None	Fresh per call; inherits parent's checkpointer for interrupts within a single invocation	Tool-wrapped subagents, multi-agent routing

Per-thread (stateful)	True	Accumulates state across calls on the same thread	Specialized subagents with long-running conversation
Stateless	False	No checkpointing — plain function call; no interrupt support	Pure transforms with no side effects

! WARNING

Per-thread subgraphs do not support *parallel tool calls*. When the LLM tries to invoke the same subagent in parallel, both calls write to the same namespace and conflict. Use `ToolCallLimitMiddleware(tool_name="...", run_limit=1)` to cap simultaneous invocations.

Namespace isolation — multiple per-thread subgraphs

When several per-thread subgraphs coexist, wrap each one in its own `StateGraph` with a unique node name so namespaces don't collide. `StateGraph(MessagesState).add_node(name, agent)` is the standard pattern.

```
from langgraph.graph import MessagesState, StateGraph
from langchain.agents import create_agent

def create_sub_agent(model, *, name, **kwargs):
    agent = create_agent(model=model, name=name, **kwargs)
    return (
        StateGraph(MessagesState)
        .add_node(name, agent)          # unique name → stable namespace
        .add_edge("__start__", name)
        .compile()
    )

fruit_agent = create_sub_agent(
    "gpt-5.4-mini", name="fruit_agent",
    tools=[fruit_info], prompt="You are a fruit expert.",
    checkpointer=True,
)
veggie_agent = create_sub_agent(
    "gpt-5.4-mini", name="veggie_agent",
    tools=[veggie_info], prompt="You are a veggie expert.",
    checkpointer=True,
)
```

Each subagent writes checkpoints only under its own name, so subgraphs never collide.

9.7 Summary

concept	Description
Subgraph	Using independently compiled graphs as nodes
shared key	state mapping between parent and subgraph
Modularization	Separating complex workflow into smaller units

streaming	Track internal steps with <code>subgraphs=True</code>
-----------	---

Next Steps

→ [10_production.ipynb](#): Learn production deployment.

10 Production

testing, deployment, observability

Learning Objectives

LangGraph Learn how to test, deploy, and monitor your apps.

10.1 Environment Setup

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

10.2 App structure — langgraph.json

- `langgraph.json` : Graph definition, dependencies, Environment Variables settings
- `langgraph dev` : Run local development server

```
import json

config = {
    "dependencies": [".",],
    "graphs": {
        "agent": "./agent.py:graph"
    },
    "env": ".env"
}

print("langgraph.json example:")
print(json.dumps(config, indent=2))
print()
print("Command:")
print("  $ pip install 'langgraph-cli[inmem]')
print('$ langgraph dev # starts the local server at http://localhost:2024")
```

10.3 LangGraph Studio — Visual Debugging tool

Studio is automatically provided when you run `langgraph dev` .

Key Features (7):

1. *Real-time Visualization* — every step the agent takes (prompts, tool calls, results, final output) is rendered live.
2. *Interactive Testing* — drive different inputs and inspect intermediate states directly in the UI.
3. *Hot-reloading* — edits to prompts or tool signatures are reflected immediately without restarting the server.
4. *Trace Inspection* — execution traces include prompts, tool arguments, return values, token counts, and latency.
5. *Exception Capture* — exceptions are captured together with surrounding state for debugging context.
6. *Thread Replay* — re-run conversation threads from any step to validate changes without restarting.
7. *Optional Tracing* — set `LANGSMITH_TRACING=false` to keep run data on the local machine.

How to use:

```
$ langgraph dev
# Open https://smith.langchain.com/studio/?baseUrl=http://127.0.0.1:2024
# Safari users append --tunnel
```

10.4 Agent Chat UI

Agent Chat UI is a Next.js chat interface for LangChain agents. It integrates directly with agents built using `create_agent` and connects to either a `langgraph dev` server or a deployed server.

Install — npx (recommended):

```
$ npx create-agent-chat-app --project-name my-chat-ui
$ cd my-chat-ui
$ pnpm install
$ pnpm dev
```

Hosted version: visit agentchat.vercel.app and enter your agent's deployment URL or local server address.

Connection info:

- *Graph ID* — the key in the `graphs` section of `langgraph.json`
- *Deployment URL* — the agent server URL (use `http://localhost:2024` for local)
- *LangSmith API key* — optional (not required when using a local server)

Features:

- Real-time streaming chat
- Tool call / result rendering
- Time-travel debugging, state forking
- Human-in-the-loop interrupt handling
- Generative UI support

10.5 Test — Deterministic agent test

```
from langgraph.graph import StateGraph, START, END
from typing import TypedDict
```

```

# Graph to test
class TestState(TypedDict):
    input: str
    output: str

def process(state: TestState) -> dict:
    return {"output": state["input"].upper()}

builder = StateGraph(TestState)

builder.add_node("process", process)
builder.add_edge(START, "process")
builder.add_edge("process", END)

graph = builder.compile()

# Unit tests
def test_process():
    result = graph.invoke({"input": "hello"})

    assert result["output"] == "HELLO", f"Expected HELLO, got {result['output']}"

    print(" OK test_process")

def test_empty_input():
    result = graph.invoke({"input": ""})

    assert result["output"] == "", f"Expected an empty string, got {result['output']}"

    print(" OK test_empty_input")

print("Running tests:")

test_process()
test_empty_input()

print("All tests passed!")

```

10.6 LLM agent Test — Using GenericFakeChatModel

```

from langchain_core.language_models import GenericFakeChatModel
from langchain.messages import AIMessage, HumanMessage, AnyMessage
from langgraph.graph import StateGraph, START, END, MessagesState

# Deterministic fake model
fake_model = GenericFakeChatModel(
    messages=iter(
        [
            AIMessage(content="The answer is 42."),
        ]
    )
)

def chatbot(state: MessagesState) -> dict:
    return {
        "messages": [fake_model.invoke(state["messages"])]
    }

builder = StateGraph(MessagesState)

builder.add_node("chatbot", chatbot)
builder.add_edge(START, "chatbot")

```



```

builder.add_edge("chatbot", END)

test_graph = builder.compile()

result = test_graph.invoke(
    {
        "messages": [HumanMessage(content="test")]
    }
)

assert "42" in result["messages"][-1].content

print("GenericFakeChatModel test passed!")
print(f"Response: {result['messages'][-1].content}")

```

10.7 Deployment Options

1. LangSmith Cloud (managed):

Connect a GitHub repository in LangSmith Deployments and the platform deploys automatically (around 15 minutes):

1. Push your application code to a GitHub repository (public or private)
2. In LangSmith, open *Deployments* and click “+ New Deployment”
3. For private repos, connect your GitHub account, then select and submit the repository
4. After deployment, open Studio from the deployment page and copy the API URL from Deployment details

2. Self-hosted Docker:

```

$ langgraph build -t my-agent
$ docker run -p 2024:2024 my-agent

```

3. Calling the deployed API (Python SDK):

Pass the deployment URL and your LangSmith API key to the same SDK used locally.

```

from langgraph_sdk import get_sync_client

client = get_sync_client(url="your-deployment-url", api_key="your-langsmith-api-key")

for chunk in client.runs.stream(
    None,                                     # thread_id=None creates a stateless run
    "agent",                                  # graph name in langgraph.json
    input={"messages": [{"role": "human", "content": "What is LangGraph?"}]},
    stream_mode="updates",
):
    print(f"Receiving new event of type: {chunk.event}...")
    print(chunk.data)

```

REST call: deployed servers authenticate via the `X-API-Key` header.

```

curl -s --request POST \
  --url <DEPLOYMENT_URL>/runs/stream \
  --header 'Content-Type: application/json' \
  --header "X-API-Key: <LANGSMITH API KEY>" \
  --data '{"assistant_id": "agent", "input": {"messages": [{"role": "human", "content": "What is LangGraph?"}]}, "stream_mode": "updates"}'

```

10.8 Observability — LangSmith Tracing

Settings (`.env`):

```
LANGSMITH_TRACING=true
LANGSMITH_API_KEY=lsv2-...
LANGSMITH_PROJECT=my-agent-project # optional, defaults to "default"
```

`LANGSMITH_TRACING` and `LANGSMITH_API_KEY` are required; `LANGSMITH_PROJECT` is optional.

Automatically tracked items:

- Each node execution time
- LLM input/output, token usage
- Tool calls and results
- State changes
- Errors and retries

Selective tracing — `tracing_context`

Use the `langsmith.tracing_context` context manager to toggle tracing for specific operations and to attach project / tags / metadata dynamically.

```
import langsmith as ls

# Only this call is traced
with ls.tracing_context(enabled=True):
    agent.invoke({"messages": [{"role": "user", "content": "Send a test email"}]})

# Dynamic project + tags + metadata
with ls.tracing_context(
    project_name="email-agent-test",
    enabled=True,
    tags=["production", "email-assistant", "v1.0"],
    metadata={"user_id": "user_123", "session_id": "session_456"},
):
    agent.invoke({"messages": [{"role": "user", "content": "Send a welcome email"}]})
```

You can also pass tags and metadata through the `config` argument of `invoke()` :

```
agent.invoke(
    {"messages": [{"role": "user", "content": "Send a welcome email"}]},
    config={
        "tags": ["production", "email-assistant", "v1.0"],
        "metadata": {"user_id": "user_123", "session_id": "session_456"},
    },
)
```

Data privacy — `LangChainTracer` + `with_config`

To avoid leaking sensitive data into traces, wire an anonymized `Client` into a `LangChainTracer` and attach it to the compiled graph via `.with_config({"callbacks": [tracer]})` .

```
from langchain_core.tracers.langchain import LangChainTracer
from langgraph.graph import StateGraph, MessagesState
from langsmith import Client
from langsmith.anonymizer import create_anonymizer
```

```
anonymizer = create_anonymizer([
    {"pattern": r"\b\d{3}-?\d{2}-?\d{4}\b", "replace": "<ssn>"},
])

tracer_client = Client(anonymizer=anonymizer)
tracer = LangChainTracer(client=tracer_client)

graph = (
    StateGraph(MessagesState)
    # .add_node(...).add_edge(...)
    .compile()
    .with_config({"callbacks": [tracer]})
)
```

This pattern masks identifiable patterns (e.g., SSNs) just before sending traces to LangSmith.

10.9 Pregel Runtime Overview

- *Pregel* is the internal execution engine of LangGraph
- Both Graph API and Functional API run on Pregel
- Key concepts: *superstep*, *Channel*, *Checkpoint*
- *superstep*: Unit in which nodes of the same level are executed in parallel
- Generally no need to use it directly (Graph/Functional API abstracts it)

LangGraph Execution Model: [Super-step 1] Node A, Node B (parallel) ↓ State update [Super-step 2] Node C (based on the results of A and B) ↓ State update [Super-step 3] Node D ↓ END

```
*Each superstep:*
1. Parallel execution of relevant nodes
2. Update state (apply reducer)
3. Save checkpoint
4. Next superstep decision
```

10.10 Production Checklist

Item	tool	Description
unit testing	pytest	Test individual node functions
Integration Testing	GenericFakeChatModel	Full flow without API calls
persistence	PostgreSaver	Production checkpointer
observability	LangSmith	Tracing, Monitoring
Distribution	langgraph deploy	Managed Deployment
UI	Agent Chat UI	User Interface

10.11 Summary

Topic	Key Concepts
App Structure	Set up project with <code>langgraph.json</code>
Studio	Visual debugging with <code>langgraph dev</code>
test	Deterministic Testing + <code>GenericFakeChatModel</code>
Distribution	Platform, Docker, Cloud options
observability	LangSmith Tracing
runtime	Pregel superstep Execution Model → Deeper in Chapter 13

Next Steps

→ Proceed to

[11. Local Server](#)

! → Skip to [Deep Agents track](#)

11 Local Server

Learning Objectives

- `langgraph dev` Know how to run a development server with CLI
- LangGraph Visually debug in conjunction with Studio
- Create `langgraph.json` configuration file
- Call local server with Python SDK
- Understand the deployment preparation process

11.1 Environment Setup

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

11.2 LangGraph CLI installation

LangGraph To run a local server, you must first install the CLI. The `langgraph-cli[inmem]` package includes an in-memory mode and is suitable for development/testing.

```
# LangGraph CLI installation command
print("=== Install with pip (Python >= 3.11) ===")
print(' $ pip install -U "langgraph-cli[inmem]"')
print()
print("=== Install with uv ===")
print(' $ uv add "langgraph-cli[inmem]"')
print()
print("Check after installation:")
print(" $ langgraph --version")
```

11.3 Project creation

You can create a project template with the `langgraph new` command of the LangGraph CLI. If you do not specify a template, an interactive menu is displayed.

```
# Project creation command
print("=== Create a new project with template ===")
print(" $ langgraph new my-agent --template new-langgraph-project-python")
print()
print("=== Create an interactive menu ===")
print(" $ langgraph new my-agent")
print()
print("=== Install dependencies after creation ===")
print("# use pip")
print(" $ cd my-agent && pip install -e .")
print()
print("# use uv")
print(" $ cd my-agent && uv sync")
print()
print("=== Generated file structure ===")
print(" my-agent/")
print("├─ langgraph.json # Graph settings")
print("├─ .env.example # Environment Variables template")
print("├─ pyproject.toml # Dependency definition")
print("└─ src/")
print("    └─ agent.py # agent code")
```

11.4 langgraph.json settings

`langgraph.json` is the core configuration file for the LangGraph project. Specifies graph definition location, dependencies, Environment Variables files, etc.

```
import json

# langgraph.json configuration example
config = {
    "dependencies": [".",],
    "graphs": {
        "agent": "./src/agent.py:graph"
    },
    "env": ".env"
}

print("langgraph.json example:")
print(json.dumps(config, indent=2))
print()
print("Main fields:")
print('dependencies: list of package paths to install')
print('graphs: graph name → module:variable mapping')
print('env: Environment Variables file path')
```

11.5 Running the development server

Run the local development server with the `langgraph dev` command. `$ langgraph dev`

```
**Expected output:**
Ready!
- API: http://127.0.0.1:2024
- Docs: http://127.0.0.1:2024/docs
- Studio: https://smith.langchain.com/studio/?baseUrl=http://127.0.0.1:2024
```

URL	Use
-----	-----

<code>http://127.0.0.1:2024</code>	API endpoint
<code>http://127.0.0.1:2024/docs</code>	API Documentation (Swagger)
<code>https://smith.langchain.com/studio/...</code>	LangGraph Studio Interface

TIP

Safari users: Use the `langgraph dev --tunnel` flag to establish a secure connection to the localhost server.

11.6 LangGraph Studio integration

LangGraph Studio is a visual debugging tool provided automatically when you run `langgraph dev`.

Main Features:

Features	Description
Graph visualization	Identify node and edge structures at a glance
Real-time tracking	Observe the execution process of each node in real time
state Inspection	Check/edit the state(state) value of each step
interactive testing	Test graph execution by changing input

Studio is browser-based, so no separate installation is required. If you are using a custom server address, simply change the `baseUrl` parameter in the Studio URL.

11.7 Python SDK — Asynchronous client

Asynchronous clients are created with `langgraph_sdk.get_client()`. It operates based on `asyncio` and can efficiently process streaming responses.

```
# Asynchronous client usage pattern (used when the server is running)
print("""from langgraph_sdk import get_client
import asyncio

client = get_client(url="http://localhost:2024")

async def main():
    async for chunk in client.runs.stream(
        None,
        "agent",
        input={
            "messages": [{
                "role": "human",
                "content": "What is LangGraph?",
            }],
        },
    ):
        print(f"Event type: {chunk.event}...")
```

```

        print(chunk.data)

    asyncio.run(main())
    """
    print("# The above code is used when the langgraph dev server is running.")

```

11.8 Python SDK – Synchronous client

Synchronous clients are created with `langgraph_sdk.get_sync_client()`. It is suitable for simple scripts or tests that do not require asynchrony.

```

# Synchronous client usage pattern (used when the server is running)
print("""from langgraph_sdk import get_sync_client

client = get_sync_client(url="http://localhost:2024")

for chunk in client.runs.stream(
    None,
    "agent",
    input={
        "messages": [{
            "role": "human",
            "content": "What is LangGraph?",
        }],
    },
    stream_mode="messages-tuple",
):
    print(f"Event: {chunk.event}...")
    print(chunk.data)
""")
print("# The above code is used when the langgraph dev server is running.")

```

11.9 REST API call

LangGraph The local server provides a REST API. It can be called directly with `curl` or with an HTTP client.

`curl -s -request POST`

`-url "http://localhost:2024/runs/stream"`

`-header 'Content-Type: application/json'`

`-data '{ "assistant_id": "agent", "input": { "messages": [{ "role": "human", "content": "What is LangGraph?" } ] }, "stream_mode": "messages-tuple" }'`

API documentation can be found at `http://localhost:2024/docs`.

11.10 Deployment Readiness Checklist

Once local development is complete, prepare for production deployment.

Item	Description	OK
------	-------------	----

langgraph.json	Graph path-dependency-Environment Variables setup complete	☐
.env file	API key, etc. Environment Variables settings	☐
Dependency cleanup	pyproject.toml or requirements.txt cleanup	☐
local test	Normal locally with langgraph dev Smoke Test	☐
Check Studio	LangGraph Check graph structure in Studio	☐
SDK Testing	Test calls with Python SDK or REST API	☐
persistent storage	Setting checkpointer (e.g. PostgresSaver) for production	☐
observability	LangSmith or Langfuse tracing settings	☐

11.11 Summary

Topic	Key Concepts
Install CLI	pip install "langgraph-cli[inmem]" or uv add
Create project	Create template-based project with langgraph new
langgraph.json	Configuration file defining graph paths, dependencies, Environment Variables
Studio	langgraph dev Visual debugging provided automatically at run time tool
SDK asynchronous	Asynchronous call streaming with get_client()
SDK synchronization	Synchronous call to streaming with get_sync_client()
REST API	Direct call to /runs/stream endpoint with curl

Next Steps

→ Proceed to

[12. Durable Execution](#)

!

References:

- [Run a Local Server](#)

12 durable execution

Learning Objectives

- Understand the concept and necessity of durable execution (Durable Execution)
- Know the relationship between checkpointer and durable execution
- Learn how to ensure durability with `@entrypoint` + `@task`
- Understand the difference between endurance modes (exit, async, sync)
- Know the recovery process in failure scenarios

12.1 Environment Setup

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

12.2 durable execution Concept

A process called *durable execution*(*Durable Execution*) or workflow saves progress state at key points, This is a technique that allows you to pause and then later interrupt at the exact resume position.

Why do you need it?

Scenario	Description
Disaster recovery	In case of server failure, instead of restarting from the beginning, interrupt point resume
state persistent	Preserve intermediate results of workflow with long running time
Human-in-the-loop	Keep state while waiting for human approval

LangGraph supports durable execution via `checkpointers(checkpointer)`.

12.3 Core requirements

Implementing durable execution requires three elements:

1. Persistence Layer

Log state progressing workflow through checkpointer. Example: `InMemorySaver` (for development), `PostgresSaver` (for production)

1. Thread ID (Thread ID)

workflow A unique ID that tracks the execution history of the instance. Using the same `thread_id`, you can continue resume from previous executions.

1. `@task` wrapping (task wrapping)

Non-deterministic operations and side-effect operations Wrap it in `@task` to prevent re-execution on resume.

12.4 Endurance Mode Comparison

LangGraph provides three modes that balance performance and consistency:

mode	Action	Trade-offs
"exit"	Persist only on completion/error/interrupt	Highest performance, no intermediate recovery
"async"	Next Steps Persist asynchronously while running	Good balance, slight crash risk
"sync"	Next Steps Persist with pre-execution synchronization	Maximum durability, performance cost

For most use cases, the default mode ("exit") is sufficient. For mission-critical workflow, consider "sync" mode.

12.5 Problematic code

If you do not wrap side effects (API calls, etc.) in `@task`, The same API call may be executed again at resume.

```
# Problematic approach: calling side effects directly
print("""# BAD: Side effects not wrapped in `@task`
def call_api(state: State):
    # This API call will be executed again on resume!
    result = requests.get(state['url']).text[:100]
    return {"result": result}""")
print("Problem:")
print("1. API is called again at resume after failure")
print("2. Non-deterministic results may vary")
print("3. Side effects may occur due to duplicate requests.")
```

12.6 Improvements to `@task`

If you wrap the side effect with the `@task` decorator, At resume, restore previous results from checkpoint to prevent re-execution.

```
# Improved approach: wrapping side effects with @task
print("""# GOOD: Wrap side effects with @task
from langgraph.func import task

@task
def _make_request(url: str):
    return requests.get(url).text[:100]

def call_api(state: State):
    # Each request is executed as a separate `@task`
    requests = [_make_request(url) for url in state['urls']]
    results = [req.result() for req in requests]
    return {"results": results}""")
print("Improvement effect:")
print("1. Restore results from checkpoint at resume")
print("2. Avoid duplicate API calls")
print("3. Each `@task` is tracked independently")
```

12.7 Durability in Graph API

Connecting checkpointer to StateGraph will automatically save state after each node execution.

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver

class DocState(TypedDict):
    topic: str
    draft: str
    final: str

def write_draft(state: DocState) -> dict:
    return {"draft": f"Draft about {state['topic']}"}

def finalize(state: DocState) -> dict:
    return {"final": f"Final: {state['draft']}"}

checkpointer = InMemorySaver()

builder = StateGraph(DocState)
builder.add_node("write_draft", write_draft)
builder.add_node("finalize", finalize)
builder.add_edge(START, "write_draft")
builder.add_edge("write_draft", "finalize")
builder.add_edge("finalize", END)

graph = builder.compile(checkpointer=checkpointer)

# Execution (track execution by thread_id)
config = {"configurable": {"thread_id": "doc-1"}}
result = graph.invoke({"topic": "LangGraph"}, config)
print("result:", result)
```

12.8 Durability in Functional API

By combining `@entrypoint` and `@task`, durability can be guaranteed even in Functional API.

```

from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver

@task
def generate_draft(topic: str) -> str:
    return f"Draft about {topic}"

@task
def review_draft(draft: str) -> str:
    return f"Reviewed: {draft}"

func_checkpointer = InMemorySaver()

@entrypoint(checkpointer=func_checkpointer)
def write_document(topic: str) -> str:
    draft = generate_draft(topic).result()
    reviewed = review_draft(draft).result()
    return reviewed

config = {"configurable": {"thread_id": "func-1"}}
result = write_document.invoke("Durable Execution", config)
print("result:", result)

```

12.9 Failover Scenario

If you rerun with the same `thread_id`, it restores the previous state from the checkpoint and continues execution.

```

from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver

class PipelineState(TypedDict):
    data: str
    step: int
    result: str

call_count = 0

def step_one(state: PipelineState) -> dict:
    global call_count
    call_count += 1
    print(f" step_one executed (call count: {call_count})")
    return {"data": state["data"].upper(), "step": 1}

def step_two(state: PipelineState) -> dict:
    print(" step_two executed")
    return {"result": f"Processed: {state['data']}", "step": 2}

recovery_saver = InMemorySaver()

builder = StateGraph(PipelineState)
builder.add_node("step_one", step_one)
builder.add_node("step_two", step_two)
builder.add_edge(START, "step_one")
builder.add_edge("step_one", "step_two")

```

```

builder.add_edge("step_two", END)

pipeline = builder.compile(checkpointer=recovery_saver)

# first run
config = {"configurable": {"thread_id": "recovery-1"}}
print("=== First Run ===")
result = pipeline.invoke(
    {"data": "hello", "step": 0, "result": ""},
    config
)
print(f"Result: {result}")

# Check checkpoint
print("=== Restore state from checkpoint ===")
saved = pipeline.get_state(config)
print(f"Saved state: {saved.values}")
print(f"Total step_one call count: {call_count}")

```

12.10 resume Starting point

When workflow becomes resume, the starting point varies depending on the API:

API	resume Starting point	Description
StateGraph	Start of interrupt node	Rerun that node from scratch
Subgraph	Parent node → interrupt node in subgraph	Start from the parent node and then move to the corresponding node in the subgraph
Functional API	\@entrypoint Start	Starting at \@entrypoint , \@task results restored from cache

Key differences:

- StateGraph: Node unit resume (re-execute only interrupt nodes)
- Functional API: Rerun from \@entrypoint , but use cache results for completed \@task

12.11 Production Durability Pattern

Best practices for ensuring durability in a production environment:

1. Implementation of idempotent operation

Design the results to be the same even when executing the same request multiple times. Prevent duplicate processing by utilizing an idempotency key.

1. Isolate Side Effects

Separate side effects such as API calls, file writes, etc. into individual \@task . Make a clear distinction between pure logic and side effects.

1. Non-deterministic code wrapping

Non-deterministic operations such as random number generation and timestamps are also wrapped in \@task .

1. Use persistent storage

Developed by: `InMemorySaver` Production: `PostgresSaver` or external database

1. thread ID Management

Give each workflow instance a unique `thread_id`. Upon failover, use resume with the same `thread_id`.

12.12 Per-node fault tolerance — `TimeoutPolicy` + `heartbeat`

Durable execution preserves *how far* a run has progressed via checkpoints. The fault-tolerance features in LangGraph 1.2 control *when to give up and how to compensate* at the level of an individual node attempt.

`add_node(..., timeout=TimeoutPolicy(...))` applies a per-attempt time budget; on overrun, `NodeTimeoutError` is raised and the retry policy (if any) advances to the next attempt. Partial writes from the failing attempt are discarded.

Long-running nodes combine `Runtime.heartbeat()` with `refresh_on="heartbeat"` to refresh the idle timeout.

```
from langgraph.func import task
from langgraph.runtime import Runtime
from langgraph.types import TimeoutPolicy

async def long_running_node(state: State, runtime: Runtime) -> dict:
    for batch in fetch_batches():
        process(batch)
        runtime.heartbeat()
    return {"result": "done"}

builder.add_node(
    "long_running_node",
    long_running_node,
    timeout=TimeoutPolicy(idle_timeout=30, refresh_on="heartbeat"),
)
```

12.13 Graceful shutdown — `RunControl` / `GraphDrained`

To stop a long-running graph cleanly on signals like SIGTERM — finishing the current superstep and persisting a checkpoint instead of force-killing the process — use `RunControl.request_drain()`. When drain is requested, the run stops with `GraphDrained` and can later be resumed with the *same config*. Inside a node, branch on `Runtime.drain_requested` to add safe-shutdown handling.

```
from langgraph.runtime import RunControl, Runtime
from langgraph.errors import GraphDrained

# 1) Caller side --- request drain on an external signal (e.g., SIGTERM)
control = RunControl()
control.request_drain("sigterm")

try:
    result = graph.invoke(inputs, config, control=control)
except GraphDrained as e:
    print(f"Drained: {e.reason}")
    # The checkpoint has already been saved.

# Resume with the same config
graph.invoke(None, config)

# 2) Inside a node --- detect the drain signal directly
async def my_node(state: State, runtime: Runtime) -> dict:
```

```

if runtime.drain_requested:
    return {"status": "skipped"}
# ... normal processing
return {"status": "done"}

```

TIP

Drain is distinct from interrupt. Interrupt is an *application-level* signal that pauses to await user input; drain is an *operational* signal that stops cleanly at a superstep boundary. Combining both means rolling deploys and maintenance windows never lose in-flight progress.

12.14 DeltaChannel (beta) — checkpoint size optimization

Standard checkpoints persist the *full* value of every channel on each superstep. For append-heavy channels (e.g., message lists), storage grows quickly on long-lived threads. `DeltaChannel` (introduced in `langgraph≥1.2`) stores incremental deltas instead, and writes a full snapshot every `snapshot_frequency` steps to bound reconstruction cost.

```

from typing import Annotated, Sequence
from typing_extensions import TypedDict
from langgraph.channels import DeltaChannel

def list_reducer(state: list, writes: Sequence[list]) -> list:
    # bulk reducer signature: (current_state, sequence_of_writes) -> new_state
    result = list(state)
    for write in writes:
        result.extend(write)
    return result

class State(TypedDict):
    messages: Annotated[
        list[str],
        DeltaChannel(list_reducer, snapshot_frequency=5),
    ]

```

! WARNING

`DeltaChannel` is a `langgraph≥1.2` beta feature. The reducer must be associative and runs *at reconstruction time, not at write time*. Measure storage, recovery latency, and migration cost before adopting it in production. Treat `DeltaChannel` as a *storage optimization* and `TimeoutPolicy` / `error_handler` as *failure-recovery controls* — they are independent dimensions of the durability design.

12.15 Summary

Topic	Key Concepts
Durability concept	Execution technique that can resume at interrupt point

Core Requirements	persistence layer + thread ID + <code>\@task</code> wrapping
Endurance Mode	exit (default), async (balanced), sync (maximum durability)
<code>@task</code>	Prevent replay by wrapping side effects
Graph API	<code>checkpointer</code> Automatic saving for each node by connection
Functional API	Durability guaranteed with <code>\@entrypoint</code> + <code>\@task</code>
Disaster recovery	At checkpoint with same <code>thread_id</code> resume
Node timeout	<code>TimeoutPolicy(idle_timeout=..., refresh_on="heartbeat")</code> per-attempt budget
Graceful shutdown	<code>RunControl.request_drain()</code> → <code>GraphDrained</code> → resume with same config
DeltaChannel (beta)	<code>langgraph>1.2</code> . Store deltas + <code>snapshot_frequency</code> to bound reconstruction

Next Steps

→ Proceed to

[13. API Selection Guide and Pregel](#)

!

References:

- [Durable Execution](#)

13 API Selection Guide and Pregel

Learning Objectives

- Compare the differences between Graph API and Functional API
- The same agent is implemented in both APIs.
- Understand the internal structure of Pregel runtime
- superstep Know the execution model
- Establish criteria for selecting the appropriate API for the project

13.1 Environment Setup

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

13.2 Graph API vs Functional API Overview

LangGraph provides two APIs for building agent workflow. Both APIs run on top of the same Pregel runtime and can be used together in a single application.

Characteristics	Graph API	Functional API
Abstraction method	Graph composed of nodes and edges	decorator-based function
state Management	Explicit management with TypedDict schema	Local variables within function scope
control flow	Conditional Edge, Routing	General Python control statements (if/else, for)
Visualization	Automatic visualization of graph structure	LIMITED
Boilerplate	Relatively many	minimize

13.3 Quick Selection Guide

Situation	Recommendation API	Reason
Complex workflow visualization required	Graph API	Node/edge structure automatically generates diagram
parallel execution path	Graph API	Multiple nodes naturally run in parallel
multi-agent Team	Graph API	Clear separation of roles between agent
Minimal changes to existing code	Functional API	Just add a decorator
Simple Linear workflow	Functional API	Quick implementation without boilerplate
Rapid Prototyping	Functional API	<code>state_schema</code> No definition required

13.4 Graph API implementation

Write an essay → Implement scoring workflow with Graph API.

```
from typing import TypedDict
from langgraph.constants import START
from langgraph.graph import StateGraph
```

```

class Essay(TypedDict):
    topic: str
    content: str | None
    score: float | None

def write_essay(essay: Essay):
    return {"content": f"Essay about {essay['topic']}"}

def score_essay(essay: Essay):
    return {"score": 10}

builder = StateGraph(Essay)
builder.add_node(write_essay)
builder.add_node(score_essay)
builder.add_edge(START, "write_essay")
builder.add_edge("write_essay", "score_essay")

graph_app = builder.compile()

result = graph_app.invoke({"topic": "LangGraph"})
print("Graph API results:", result)

```

13.5 Functional API implementation

Implement the same essay workflow with Functional API.

```

from typing import TypedDict
from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver

class EssayResult(TypedDict):
    topic: str
    content: str | None
    score: float | None

@task
def write_essay_func(topic: str) -> str:
    return f"Essay about {topic}"

@task
def score_essay_func(content: str) -> float:
    return 10

func_saver = InMemorySaver()

@entrypoint(checkpointer=func_saver)
def essay_pipeline(topic: str) -> dict:
    content = write_essay_func(topic).result()
    score = score_essay_func(content).result()
    return {"topic": topic, "content": content, "score": score}

config = {"configurable": {"thread_id": "essay-1"}}
result = essay_pipeline.invoke("LangGraph", config)
print("Functional API results:", result)

```

13.6 Comparative analysis

Compare the two implementations above side by side:

Item	Graph API	Functional API
state Definition	TypedDict schema required	optional (possible local variables)
node connection	<code>add_edge()</code> , <code>add_conditional_edges()</code>	Generic function call
checkpointer	<code>compile(checkpointer=...)</code>	<code>\@entrypoint(checkpointer=...)</code>
number of lines of code	Relatively many	Concise
Visualization	<code>graph.get_graph().draw_mermaid()</code> Support	LIMITED
parallel execution	Natural support with edge structure	<code>\@task</code> Support for parallel execution
Debugging	Check state per node in Studio	Function-level tracing

13.7 Combining two APIs

You can use both APIs in the same application. A common pattern is to use the Graph API for complex multi-agent coordination, and the Functional API for simpler linear data-processing flows.

Use Case	Recommended API
Complex multi-agent coordination	Graph API
Simple preprocessing or validation pipeline	Functional API
Need explicit routing and visualization	Graph API
Need minimal boilerplate over existing code	Functional API

· NOTE

It is normal to start with the Functional API for a small prototype and migrate to the Graph API as the workflow becomes more complex. The reverse is also possible when a graph turns out to be overdesigned.

13.8 Pregel Runtime Overview

Pregel is the internal execution engine of LangGraph. Compiling `StateGraph` or using `@entrypoint` creates a Pregel instance internally.

The name comes from Google's Pregel algorithm, which efficiently handles massively parallel graph computations.

Core Components:

Components	Role
<i>Actor</i>	Read data from the channel and write the processing results to the channel
<i>Channel</i>	Responsible for data communication between actors

3 steps of execution (every step):

1. *Plan* — Determine which actor to execute in this step
2. *Execute* — Execute selected actors in parallel (until completion, failure, or timeout)
3. *Update* — Update the channel with new values

It terminates when there are no actors to run or the maximum steps are reached.

13.9 Direct use of Pregel

There is generally no need to use Pregel directly; Let's look at a simple example to understand the inner workings.

```
from langgraph.channels import EphemeralValue
from langgraph.pregel import Pregel, NodeBuilder

# Single node: repeat input twice
node1 = (
    NodeBuilder()
    .subscribe_only("a")
    .do(lambda x: x + x)
    .write_to("b")
)

app = Pregel(
    nodes={"node1": node1},
    channels={
        "a": EphemeralValue(str),
        "b": EphemeralValue(str),
    },
    input_channels=["a"],
    output_channels=["b"],
)

result = app.invoke({"a": "foo"})
print("Pregel Results:", result)
# 'foo' + 'foo' = 'foofoo'
```

13.10 Channel Type

Pregel offers five channel types that map directly to Graph API state fields and reducers.

Type	Purpose
LastValue	Stores the most recent value. Default channel for inputs/outputs
EphemeralValue	Temporary value scoped to a single execution step

Topic	Configurable PubSub. Multiple values with optional dedup / accumulation
BinaryOperatorAggregate	Applies a binary operator to current value and update (running totals, etc.)
<i>DeltaChannel</i> (beta, langgraph $\geq$ 1.2)	Stores incremental deltas per step. Suitable for fast-growing channels (e.g., message lists)

```

from langgraph.channels import (
    EphemeralValue,
    LastValue,
    Topic,
    BinaryOperatorAggregate,
)
from langgraph.pregel import Pregel, NodeBuilder

# --- 1. LastValue: Maintain only the latest value ---
node_lv = (
    NodeBuilder()
    .subscribe_only("input")
    .do(lambda x: x.upper())
    .write_to("output")
)

app_lv = Pregel(
    nodes={"node": node_lv},
    channels={
        "input": EphemeralValue(str),
        "output": LastValue(str),
    },
    input_channels=["input"],
    output_channels=["output"],
)
print("LastValue:", app_lv.invoke({"input": "hello"}))

# --- 2. Topic: Accumulating multiple values ---
node_t1 = (
    NodeBuilder()
    .subscribe_only("a")
    .do(lambda x: x + x)
    .write_to("b", "c")
)

node_t2 = (
    NodeBuilder()
    .subscribe_to("b")
    .do(lambda x: x["b"] + x["b"])
    .write_to("c")
)

app_topic = Pregel(
    nodes={"node1": node_t1, "node2": node_t2},
    channels={
        "a": EphemeralValue(str),
        "b": EphemeralValue(str),
        "c": Topic(str, accumulate=True),
    },
    input_channels=["a"],
    output_channels=["c"],
)
print("Topic:", app_topic.invoke({"a": "foo"}))

# --- 3. BinaryOperatorAggregate: Apply reducer ---
def reducer(current, update):
    if current:
        return current + " | " + update
    return update

```

```

node_b1 = (
    NodeBuilder()
    .subscribe_only("a")
    .do(lambda x: x + x)
    .write_to("b", "c")
)

node_b2 = (
    NodeBuilder()
    .subscribe_only("b")
    .do(lambda x: x + x)
    .write_to("c")
)

app_agg = Pregel(
    nodes={"node1": node_b1, "node2": node_b2},
    channels={
        "a": EphemeralValue(str),
        "b": EphemeralValue(str),
        "c": BinaryOperatorAggregate(str, operator=reducer),
    },
    input_channels=["a"],
    output_channels=["c"],
)
print("BinaryOperatorAggregate:", app_agg.invoke({"a": "foo"}))

```

DeltaChannel (beta, langgraph≥1.2)

`DeltaChannel` stores *incremental deltas* only instead of the full accumulated value at each step. It is used to reduce checkpoint cost for high-volume, ever-growing channels such as message lists.

- The reducer must be *associative* and runs *at reconstruction time, not at write time*.
- `snapshot_frequency=K` writes a full snapshot every K steps to bound reconstruction cost.
- The bulk reducer signature is `(current_state, sequence_of_writes) -> new_state`.

```

from typing import Annotated, Sequence
from typing_extensions import TypedDict
from langgraph.channels import DeltaChannel

def list_reducer(state: list, writes: Sequence[list]) -> list:
    # (current_state, sequence_of_writes) -> new_state
    result = list(state)
    for write in writes:
        result.extend(write)
    return result

class State(TypedDict):
    messages: Annotated[
        list[str],
        DeltaChannel(list_reducer, snapshot_frequency=5),
    ]

```

13.11 superstep Execution Model

Pregel runs in **supersteps**. In each superstep, nodes (actors) at the same level can run in parallel. Once all of them finish, the channel state is updated and the runtime moves to the next superstep.

· NOTE

Features of a superstep: nodes in the same superstep cannot see each other's new outputs, the next step begins only after all nodes finish, a checkpoint can persist state after each step, and execution stops automatically when there are no actors left to run.

A simple mental model looks like this:

- Superstep 1: Node A and Node B run in parallel
- Channel update
- Superstep 2: Node C runs using the results from A and B
- Channel update
- Superstep 3: Node D runs
- End

13.12 API Selection Criteria Guide

Here is the decision framework for your final choice:

Step 1: Complexity evaluation

- Less than 3 nodes and linear flow → *Functional API*
- Conditional branching, parallel path, circular structure → *Graph API*

Step 2: Team Collaboration

- Alone or with a small team → Either way is possible
- Multiple team members are responsible for each node → *Graph API* (separation of visualization and roles)

Step 3: Leverage Existing Code

- Add LangGraph function to existing procedural code → *Functional API*
- Designing a new workflow from scratch → *Graph API*

Step 4: Potential for development

- Start from a prototype → *Functional API* → When it becomes complex, migrate to *Graph API*
- Consider scalability from the beginning → *Graph API*

13.13 Summary

Topic	Key Concepts
Graph API	Node/edge based, strong in visualization, suitable for complex workflow
Functional API	Decorator-based, minimal boilerplate, suitable for linear workflow
Compare	Same runtime, can be used together, choose based on complexity
Pregel	LangGraph's internal execution engine, actor-channel model
Channel	LastValue / EphemeralValue / Topic / BinaryOperatorAggregate / DeltaChannel (beta)
superstep	Parallel execution of same level nodes → Channel update → Next step

Selection criteria	Judging by complexity, team collaboration, existing code, and development potential
--------------------	---

Next Steps

→ Proceed to *the Deep Agents track!*

References:

- [Choosing between Graph and Functional APIs](#)
- [Pregel Runtime](#)

P A R T

IV

Deep Agents

Framework-Agnostic Agent Building

What You Will Learn in This Part

- 1. Introduction to Deep Agents
 - 2. Quickstart
 - 3. Customization
 - 4. Backends
 - 5. Subagents
 - 6. Memory and Skills
 - 7. Advanced Features
 - 8. Agent Harness
- 9. Framework Comparison
- 10. Sandboxes and ACP

01 Introduction to Deep Agents

Learning Objectives

- Understand what Deep Agents is
- Learn the difference between the SDK and the CLI
- Understand the five core concepts: Planning, Context Management, Backends, Subagents, and Memory
- Compare Deep Agents with other frameworks
- Verify that the package is installed correctly

1. What Is Deep Agents?

Deep Agents is an *Agent Harness* framework created by the LangChain team. It makes it easier to build autonomous agents for complex multi-step tasks by including the following *11 core capabilities* out of the box:

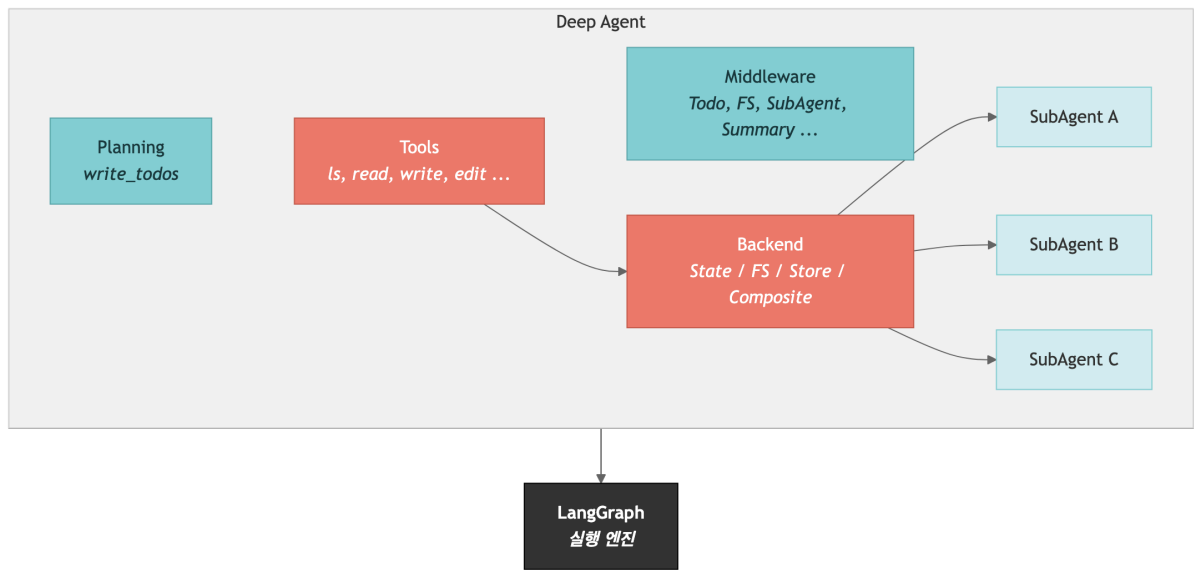
- *Task planning* — `write_todos` breaks complex problems into a structured task list
- *Filesystem management* — read, write, and search files (`ls` , `read_file` , `write_file` , `edit_file` , `glob` , `grep`)
- *Filesystem permissions* — per-path ACL with `allow_read` / `allow_write` / `deny_read` etc.
- *Subagent delegation* — distribute work via the `task` `tool`
- *Long-term memory + skills* — `AGENTS.md` (`memory`) and `SKILL.md` (`skills`) with progressive disclosure
- *Context management* — automatic offloading and summarization within the token budget
- *QuickJS interpreter* — sandboxed JS interpreter (`use_interpreter=True`) for arbitrary computation
- *Sandbox execution* — Modal, Daytona, Deno, and local Virtual FS backends
- *Human-in-the-loop* — per-tool approval gates with `interrupt_on={"tool_name": True}`
- *Customization hooks* — `before_model` / `after_model` / `wrap_model_call` middleware hooks
- *Context propagation* — subagent-specific config via `context_schema` and namespace keys (`"agent:key"`)

It is built on top of LangChain's core agent components, and it uses *LangGraph* as its execution engine.

TIP

Model setup used in this course: The recommended default for Deep Agents is *Anthropic Claude Sonnet 4.6* (`anthropic:claude-sonnet-4-6`). OpenAI examples use `openai:gpt-5.4` , Google examples use `google_genai:gemini-3.5-flash` . The `provider:model` prefix form is preferred; set the matching `ANTHROPIC_API_KEY` / `OPENAI_API_KEY` / `GOOGLE_API_KEY` .

Architecture Overview



2. SDK vs Code vs ACP

Deep Agents ships *three interfaces* on top of the same `AgentHarness` engine:

Category	Deep Agents SDK	Deep Agents Code	ACP server
Package	<code>deepagents</code>	<code>deepagents-cli</code>	<code>deepagents-cli (--acp)</code>
Purpose	Build agents programmatically	Use a coding agent directly from the terminal	Speak the Agent Client Protocol to external clients
Install	<code>pip install -qU deepagents langchain-{provider}</code>	<code>uv tool install deepagents-cli</code>	<code>uv tool install deepagents-cli</code>
Usage	Call <code>create_deep_agent()</code> from Python	Run <code>deepagents</code> in the terminal	<code>deepagents --acp</code> ; clients connect over stdio
Customization	Full API access (tools, backends, middleware)	<code>.deepagents/config.json</code> + slash commands	Same config exposed over ACP
Best fit	App integrations, automation pipelines, custom agent services	Interactive coding assistance	Zed, custom IDE clients, third-party shells

TIP

In this course we focus on the *SDK*. Deep Agents Code and the ACP server are covered later in Part 4. Choose the LangChain integration package for your provider — for example `pip install -qU deepagents langchain-anthropic` or `pip install -qU deepagents langchain-openai` .

3. Five Core Concepts (Planning · Context Management · Backends · Subagents · Memory & Skills)

3.1 Planning

The agent uses the `write_todos` tool to break complex work into a *structured task list*. Each task moves through states such as `pending` → `in_progress` → `completed` .

3.2 Context Management

Deep Agents manages large amounts of information generated during a task:

- *Offloading*: content over 20,000 tokens can be written to disk while only a pointer stays in context
- *Summarization*: conversation history can be compressed as it approaches the model limit

3.3 Backends

The agent filesystem is implemented with *pluggable backends*:

- `StateBackend` — store files in agent state (ephemeral, default)
- `FilesystemBackend` — local disk, with `virtual_mode=True` blocking `..` / `~` / paths outside `root_dir`
- `StoreBackend` — cross-thread persistent storage via LangGraph `BaseStore`
- `CompositeBackend` — route prefixes to different backends (for example `/memories/` → persistent)
- `ContextHubBackend` — lazy fetch + write-through remote sync (`deepagents>=0.5.2`), with `namespace=lambda rt: ...`
- Sandbox backends — *Modal*, *Daytona*, *Deno*, local *VFS* for isolated file and code execution; the *QuickJS* interpreter (`use_interpreter=True`) is a complementary middleware

3.4 Subagents

The main agent can delegate specialized work to *subagents*. Each subagent can be given:

- Its own system prompt
- A dedicated model
- A separate context window
- A restricted toolset

3.5 Memory & Skills

Deep Agents inherits LangGraph's memory model and adds skill loading:

- *Short-term memory* — message history within a thread
 - *Long-term memory* — `AGENTS.md` (`memory` parameter) always injected, `StoreBackend` for cross-thread state
 - *Skills* — `SKILL.md` files loaded via *progressive disclosure* (`skills` parameter) so only frontmatter is loaded upfront, with full content fetched on demand
-

4. Comparison with Other Frameworks

Capability	LangChain Deep Agents	OpenCode	Claude Agent SDK
<i>Model support</i>	Model-agnostic (Anthropic, OpenAI, 100+ providers)	75+ providers, including Ollama	Claude-only
<i>License</i>	MIT	MIT	MIT (SDK) / proprietary (Claude Code)
<i>SDK</i>	Python, TypeScript + CLI	Terminal, desktop, IDE	Python, TypeScript
<i>Sandboxing</i>	Integrated as a tool (Modal, Daytona, etc.)	Not supported	Not supported
<i>Pluggable backends</i>	O (State, FS, Store, Composite)	X	X
<i>Time travel</i>	O (via LangGraph)	X	O
<i>Observability</i>	Native LangSmith support	X	X
<i>Built-in file tools</i>	O	O	O
<i>Human-in-the-loop</i>	O	X	O

Deep Agents is especially strong when you want to build agents that need *planning + files + memory + subagents* in one integrated stack.

5. Installation Check

Install `deepagents` together with the LangChain integration package for your chosen provider:

```
# Recommended default – Anthropic
pip install -qU deepagents langchain-anthropic

# OpenAI
pip install -qU deepagents langchain-openai

# Google Gemini
pip install -qU deepagents langchain-google-genai

# OpenRouter (OpenAI-compatible)
pip install -qU deepagents langchain-openai # just change base_url
```

Run the cells below to verify that the `deepagents` package is installed correctly.

```
# Check the deepagents package version
import deepagents
print(f"deepagents version: {deepagents.__version__}")
```

```
# Verify imports of the main modules
from deepagents import create_deep_agent, SubAgent, CompiledSubAgent
from deepagents import FilesystemMiddleware, MemoryMiddleware, SubAgentMiddleware
from deepagents.backends import StateBackend, FilesystemBackend, StoreBackend, CompositeBackend
from deepagents.backends.protocol import BackendProtocol

print("Successfully imported all main modules!")
```

```
# Check dependency package versions
import importlib.metadata

print(f"langchain version: {importlib.metadata.version('langchain')}")
print(f"langgraph version: {importlib.metadata.version('langgraph')}")
```

```
# Inspect the create_deep_agent function signature
import inspect

sig = inspect.signature(create_deep_agent)
print("create_deep_agent() parameters:")
for name, param in sig.parameters.items():
    default = param.default if param.default is not inspect.Parameter.empty else "(required)"
    print(f"  - {name}: {default}")
```

Summary

Item	Description
Deep Agents	A LangChain-based agent harness framework
Core function	<code>create_deep_agent()</code>
Execution engine	LangGraph (<code>CompiledStateGraph</code>)
Recommended model	<i>Anthropic Claude Sonnet 4.6</i> via <code>"anthropic:claude-sonnet-4-6"</code> (OpenAI: <code>"openai:gpt-5.4"</code>)
Core concepts	Planning, Context Management, Backends, Subagents, Memory & Skills
Built-in tools	<code>write_todos</code> , <code>ls</code> , <code>read_file</code> , <code>write_file</code> , <code>edit_file</code> , <code>glob</code> , <code>grep</code>

Next Steps

→ [02_quickstart.ipynb](#): Build and run your first Deep Agent.

02 Building Your First Agent

Learning Objectives

- Learn how to load API keys from a `.env` file
 - Create a basic agent with `create_deep_agent()`
 - Run the agent with `agent.invoke()` and `agent.stream()`
 - Build a research agent with a Tavily search tool
 - Understand the built-in tools and what they are for
-

1. Project Bootstrap and API Keys

Bootstrap with `uv`

The recommended workflow uses `uv` for reproducible dependency management.

```
# 1) Init the project
uv init my-deepagent
cd my-deepagent

# 2) Add deepagents plus the provider package
uv add deepagents langchain-anthropic # recommended default
# uv add deepagents langchain-openai
# uv add deepagents langchain-google-genai
uv add tavily-python python-dotenv

# 3) Sync the lockfile
uv sync
```

API keys by provider

The recommended default model is *Anthropic Claude Sonnet 4.6* (`anthropic:claude-sonnet-4-6`). Set only the keys for the providers you actually use.

```
# Anthropic (recommended default)
ANTHROPIC_API_KEY=your-key-here

# OpenAI
OPENAI_API_KEY=your-key-here

# Google Gemini
GOOGLE_API_KEY=your-key-here

# OpenRouter (OpenAI-compatible)
OPENAI_API_KEY=your-openrouter-key
OPENAI_BASE_URL=https://openrouter.ai/api/v1
```



```
# Shared – used in the research example
TAVILY_API_KEY=your-key-here
```

TIP

The simplest setup is to copy `.env.example` to `.env` and then fill in the real values.

```
# Load environment variables
from dotenv import load_dotenv
import os

load_dotenv()

# Recommended default – Anthropic
assert os.environ.get("ANTHROPIC_API_KEY"), "ANTHROPIC_API_KEY is not set!"
assert os.environ.get("TAVILY_API_KEY"), "TAVILY_API_KEY is not set!"
print("API keys loaded successfully.")
```

2. Creating the Simplest Agent

`create_deep_agent()` is the core function in Deep Agents. If you call it with no additional configuration, it automatically assembles a default model and the built-in tools.

```
from deepagents import create_deep_agent

# Recommended default – Anthropic Claude Sonnet 4.6 (provider:model string)
agent = create_deep_agent(model="anthropic:claude-sonnet-4-6")

# (Alt 1) OpenAI gpt-5.4
# agent = create_deep_agent(model="openai:gpt-5.4")

# (Alt 2) Google Gemini 3.5 Flash
# agent = create_deep_agent(model="google_genai:gemini-3.5-flash")

# (Alt 3) Pass a ChatModel object for fine control
# from langchain_anthropic import ChatAnthropic
# model = ChatAnthropic(model="claude-sonnet-4-6", temperature=0)
# agent = create_deep_agent(model=model)

print(f"Agent type: {type(agent).__name__}")
print("The agent was created successfully!")
```

`create_deep_agent()` returns a `LangGraph CompiledStateGraph`. That means you can use all of the normal `LangGraph` execution methods such as `invoke`, `stream`, and `batch`.

3. Running the Agent — `invoke()`

Run the agent by sending it a message. The input format is a dictionary like `{"messages": [{"role": "user", "content": "..."}]}`.

4. Connecting Tavily Search — A Research Agent

You can extend the agent by adding custom tools. In this example, you connect the *Tavily* web search tool.

How tool definitions work

A Python function's *docstring* becomes the tool description, and its *type hints* become the parameter schema.

```
from typing import Literal
from tavily import TavilyClient

tavily_client = TavilyClient(api_key=os.environ["TAVILY_API_KEY"])

def internet_search(
    query: str,
    max_results: int = 5,
    topic: Literal["general", "news", "finance"] = "general",
    include_raw_content: bool = False,
) -> dict:
    """Search the internet for information.

    Args:
        query: The question or keyword to search for
        max_results: Maximum number of results to return
        topic: Search topic category
        include_raw_content: Whether to include the raw source content
    """
    return tavily_client.search(
        query,
        max_results=max_results,
        include_raw_content=include_raw_content,
        topic=topic,
    )

print(f"Tool name: {internet_search.__name__}")
print(f"Tool description: {internet_search.__doc__.strip().splitlines()[0]}")
```

```
# Create a research agent – search tool + custom system prompt
research_agent = create_deep_agent(
    model="anthropic:claude-sonnet-4-6",
    tools=[internet_search],
    system_prompt="You are an expert researcher. Search the internet, organize the results, and write the final answer in English.",
)

print("Research agent created!")
```

5. Checking the Built-In Tools

These are the built-in tools that `create_deep_agent()` adds automatically:

Tool	Description
<code>write_todos</code>	Manage a structured task list (pending → in_progress → completed)
<code>ls</code>	List directory contents with metadata

<code>read_file</code>	Read a file (with line numbers and image support)
<code>write_file</code>	Create a new file
<code>edit_file</code>	Replace text inside a file (<code>old_string</code> → <code>new_string</code>)
<code>glob</code>	Pattern-based file search (for example <code>**/*.py</code>)
<code>grep</code>	Search file contents (with regex support)
<code>task</code>	Call a subagent (added automatically when subagents are configured)

TIP

All of these tools work through the *backend*. The default is `StateBackend` , which stores files inside agent state.

6. Streaming Output — `stream()`

With `agent.stream()` , you can observe the agent's execution process in real time. You can choose different levels of detail through `stream_mode` :

- `"updates"` — state updates after each completed step
- `"messages"` — token-level streaming
- `"custom"` — custom events emitted from tools or nodes

Summary

Item	Description
Agent creation	<code>create_deep_agent(model, tools, system_prompt)</code>
Synchronous execution	<code>agent.invoke({"messages": [...]})</code>
Streaming execution	<code>agent.stream({"messages": [...]}, stream_mode="updates")</code>
Custom tools	Python function + docstring + type hints
Model format	Provider prefix preferred — <code>"anthropic:claude-sonnet-4-6"</code> , <code>"openai:gpt-5.4"</code> , <code>"google_genai:gemini-3.5-flash"</code> . <code>ChatModel</code> objects also accepted

Next Steps

→ [03_customization.ipynb](#): learn how to customize the agent in more detail.

03 Agent Customization

Learning Objectives

- Learn how to choose different LLM providers and models
- Write effective system prompts
- Build custom tools from docstrings and type hints
- Produce structured Pydantic output with `response_format`
- Understand the middleware architecture

```
# Environment setup
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("ANTHROPIC_API_KEY"), "ANTHROPIC_API_KEY is not set!"
print("Environment setup complete")

# Recommended default – Anthropic Claude Sonnet 4.6 (provider:model string)
from deepagents import create_deep_agent

model = "anthropic:claude-sonnet-4-6"
print(f"Default model: {model}")
```

0. Full `create_deep_agent()` Signature (17 parameters)

Before diving into individual customization axes, here is the full *17-parameter* signature. All parameters are *optional*; only `model` is enough to get a working agent.

```
def create_deep_agent(
    model=None,           # 1. LLM (ChatModel object or "provider:model")
    tools=None,           # 2. custom tools
    system_prompt=None,   # 3. user-provided prompt (USER segment)
    middleware=(),        # 4. extra user middleware
    subagents=None,       # 5. SubAgent dicts / CompiledSubAgent
    skills=None,          # 6. SKILL.md directories (progressive disclosure)
    memory=None,          # 7. AGENTS.md path (always injected)
    response_format=None, # 8. Pydantic schema for structured output
    context_schema=None,  # 9. runtime context TypedDict (propagated to subagents)
    checkpointer=None,    # 10. LangGraph checkpointer (persistence)
    store=None,           # 11. LangGraph BaseStore (for StoreBackend)
    cache=None,           # 12. prompt cache config (e.g. Anthropic)
    backend=None,         # 13. BackendProtocol or `lambda runtime: ...`
    permissions=None,     # 14. allow_read/allow_write/deny_* ACL
    interrupt_on=None,    # 15. {"tool_name": True} HITL gates
    debug=False,          # 16. debug logging
    name=None,            # 17. graph name
```

```
) :  
    ...
```

TIP

`permissions` , `context_schema` , and `cache` are recent additions (`deepagents>=0.5.0`). Older versions require adding middleware directly.

Prompt assembly order

The string you pass to `system_prompt` is *not* used verbatim. Deep Agents' prompt builder assembles three segments in a fixed order:

```
[USER]    system_prompt (string from the caller)  
    ↓  
[BASE]    harness BASE prompt (ToDoList/Filesystem/SubAgent usage)  
    + CUSTOM per-middleware injections (Memory, Skills, Permissions, ...)  
    ↓  
[SUFFIX]  harness SUFFIX (safety guidance, output contract)
```

The order is *USER* → *BASE/CUSTOM* → *SUFFIX*. You only fill in the *USER* segment; *BASE* and *SUFFIX* are managed by middleware. Nothing is overwritten, so there is no need to repeat built-in tool usage in your own prompt.

1. Choosing a Model

Deep Agents supports a wide range of LLMs through either a **LangChain chat model object** or the `provider:model` format.

This notebook uses *Anthropic Claude Sonnet 4.6* (`anthropic:claude-sonnet-4-6`) as the recommended default. OpenAI examples use `gpt-5.4` .

Provider	Example model (provider:model)	Environment variable	Notes
<i>Anthropic</i>	<code>anthropic:claude-sonnet-4-6</code>	<code>ANTHROPIC_API_KEY</code>	<i>Recommended default</i> — best for prompt caching and HITL
OpenAI	<code>openai:gpt-5.4</code>	<code>OPENAI_API_KEY</code>	Strong tool-calling accuracy
Google	<code>google_genai:gemini-3.5-flash</code>	<code>GOOGLE_API_KEY</code>	Cost-effective
Azure	<code>azure_openai:gpt-5.4</code>	<code>AZURE_OPENAI_*</code>	Enterprise deployments
AWS Bedrock	<code>bedrock:anthropic:claude-sonnet-4-6</code>	AWS credentials	VPC isolation

The recommended default is `anthropic:claude-sonnet-4-6` , with built-in retry (6 attempts) and timeout. The `model` parameter accepts either a `BaseChatModel` object or a `"provider:model-name"` string.

```
# Recommended default – Anthropic Claude Sonnet 4.6 via provider:model string
agent_default = create_deep_agent(
    model="anthropic:claude-sonnet-4-6",
)

print(f"Agent created: {type(agent_default).__name__}")

# Reference examples for other providers:
# agent_openai = create_deep_agent(model="openai:gpt-5.4")
# agent_gemini = create_deep_agent(model="google_genai:gemin-3.5-flash")
# agent_bedrock = create_deep_agent(model="bedrock:anthropic.claude-sonnet-4-6")
```

2. Custom System Prompts

The system prompt defines the agent's *role*, *behavioral rules*, and *output style*. It is added on top of the default prompt, so you can provide domain-specific instructions without rebuilding everything from scratch.

3. Building Custom Tools

Deep Agents converts Python functions into tools using the following rules:

1. *Function name* → tool name
2. *Docstring* → tool description (used by the agent to decide whether to call the tool)
3. *Type hints* → parameter schema (generated automatically)
4. *Default values* → optional parameters

```
import math

def calculate_compound_interest(
    principal: float,
    annual_rate: float,
    years: int,
    compounds_per_year: int = 12,
) -> dict:
    """Calculate compound interest.

    Args:
        principal: Principal amount
        annual_rate: Annual interest rate (for example 0.05 = 5%)
        years: Number of years
        compounds_per_year: Number of compounding periods per year (default: 12 = monthly)
    """
    amount = principal * (1 + annual_rate / compounds_per_year) ** (compounds_per_year * years)
    interest = amount - principal
    return {
        "principal": f"{principal:,.0f}",
        "final_amount": f"{amount:,.0f}",
        "interest_earned": f"{interest:,.0f}",
        "return_rate": f"{(interest / principal) * 100:.2f}%",
    }

def convert_temperature(
    value: float,
    from_unit: str,
    to_unit: str,
) -> str:
```

```

"""Convert between temperature units.

Args:
    value: Temperature value to convert
    from_unit: Source unit ('celsius', 'fahrenheit', 'kelvin')
    to_unit: Target unit ('celsius', 'fahrenheit', 'kelvin')
"""
if from_unit == "fahrenheit":
    celsius = (value - 32) * 5 / 9
elif from_unit == "kelvin":
    celsius = value - 273.15
else:
    celsius = value

if to_unit == "fahrenheit":
    result = celsius * 9 / 5 + 32
elif to_unit == "kelvin":
    result = celsius + 273.15
else:
    result = celsius

return f"{value} {from_unit} = {result:.2f} {to_unit}"

calculator_agent = create_deep_agent(
    model=model,
    tools=[calculate_compound_interest, convert_temperature],
    system_prompt="You are an assistant for calculations and unit conversion. Always use tools for exact results.",
)

print("Calculator agent created!")

```

4. Structured Output — `response_format`

You can structure the agent's final response as a *Pydantic model*. That makes the result much easier to use programmatically.

```

from pydantic import BaseModel, Field

class BookRecommendation(BaseModel):
    """Single book recommendation"""
    title: str = Field(description="Book title")
    author: str = Field(description="Author")
    reason: str = Field(description="Reason for the recommendation (2-3 sentences)")
    difficulty: str = Field(description="Difficulty level: beginner/intermediate/advanced")

class BookRecommendationList(BaseModel):
    """Book recommendation list"""
    topic: str = Field(description="Topic of the recommendation")
    books: list[BookRecommendation] = Field(description="Recommended books")

book_agent = create_deep_agent(
    model=model,
    system_prompt="You are a book recommendation expert. Suggest books that match the user's interests.",
    response_format=BookRecommendationList,
)

print("Book recommendation agent created")

```

5. Middleware Architecture

`create_deep_agent()` builds an internal *middleware stack*. Middleware is the plugin layer that extends and controls the agent’s behavior.

Default middleware stack (execution order)

1. <code>TodoListMiddleware</code>	– task <code>management</code> (<code>write_todos</code>)
2. <code>MemoryMiddleware</code>	– load <code>AGENTS.md</code> when <code>`memory`</code> <code>is</code> used
3. <code>SkillsMiddleware</code>	– load <code>SKILL.md</code> when <code>`skills`</code> <code>is</code> used
4. <code>FilesystemMiddleware</code>	– file <code>tools</code> (<code>ls</code> , <code>read</code> , <code>write</code> , <code>edit</code> , <code>glob</code> , <code>grep</code>)
5. <code>SubAgentMiddleware</code> [<code>required</code> , <code>not</code> <code>removable</code>]	– subagent <code>support</code> (task tool)
6. <code>SummarizationMiddleware</code>	– context <code>compression</code> (~85% auto-summary)
7. <code>AnthropicPromptCachingMiddleware</code>	– prompt <code>caching</code> (auto-applied <code>for</code> Anthropic)
8. <code>PatchToolCallsMiddleware</code>	– repair malformed/missing tool calls
9. [User custom middleware]	– <code>`middleware`</code> parameter
10. <code>HumanInTheLoopMiddleware</code>	– approval <code>workflow</code> (<code>`interrupt_on`</code>)

! WARNING

`SubAgentMiddleware` cannot be removed via `excluded_middleware`. The `task` tool is the core delegation mechanism, so it stays enabled even when you do not declare any subagents — a built-in `general-purpose` subagent fills the slot.

What each middleware does

Middleware	Tools Added	Role
<code>TodoListMiddleware</code>	<code>write_todos</code>	Manage structured task lists
<code>FilesystemMiddleware</code>	<code>ls</code> , <code>read_file</code> , <code>write_file</code> , <code>edit_file</code> , <code>glob</code> , <code>grep</code>	Filesystem access
<code>SubAgentMiddleware</code>	<code>task</code>	Create and call subagents
<code>SummarizationMiddleware</code>	<code>none</code>	Summarize context when it reaches about 85% of the limit
<code>MemoryMiddleware</code>	<code>none</code>	Inject <code>AGENTS.md</code> into the system prompt
<code>SkillsMiddleware</code>	<code>none</code>	Progressively load relevant <code>SKILL.md</code> files

```
# Verify available middleware imports
from deepagents.middleware import (
    FilesystemMiddleware,
    MemoryMiddleware,
    SubAgentMiddleware,
    SkillsMiddleware,
    SummarizationMiddleware,
)

print("Available middleware:")
for mw in [FilesystemMiddleware, MemoryMiddleware, SubAgentMiddleware, SkillsMiddleware,
```



```
SummarizationMiddleware]:
    print(f" - {mw.__name__}")
```

· NOTE

Note: `create_deep_agent()` configures middleware automatically in most cases. You can still add custom middleware through the `middleware` parameter if you need advanced behavior.

6. Sandbox Options

Beyond `tools`, Deep Agents ships *sandbox options* that isolate untrusted code execution. Use them for coding agents or data-analysis agents that run arbitrary scripts.

Sandbox	Execution environment	Best fit
Modal	Remote container (GPU available)	Long-running code, GPU inference
Daytona	Remote development container	Git workspace integration, multi-language
Deno	Local TypeScript/JavaScript isolation	Lightweight JS/TS execution
Local VFS	In-process virtual filesystem	Unit tests with no network or disk access

TIP

Sandboxes can be exposed either as *tools* (for example an `execute_python` tool) or as *backends* (for example a `SandboxBackend` for file isolation). The two layers can be combined to get both code and file isolation.

Summary

Item	Method
Model selection	<code>model="anthropic:claude-sonnet-4-6"</code> or any provider prefix
System prompt	Assembled USER → BASE/CUSTOM → SUFFIX
Custom tools	function + docstring + type hints → <code>tools=[func]</code>
Structured output	<code>response_format=PydanticModel</code> → <code>result["structured_response"]</code>
Middleware	<code>SubAgentMiddleware</code> is required; the rest are wired automatically
Full parameter list	17 — includes <code>permissions</code> , <code>context_schema</code> , <code>checkpointer</code> , <code>store</code> , <code>cache</code> , <code>interrupt_on</code>

Sandboxes	Modal, Daytona, Deno, local VFS — exposed as tools or backends
-----------	--

Next Steps

→ [04_backends.ipynb](#): learn how storage backends define the agent filesystem.

04 Storage Backends

Learning Objectives

- Understand how backends implement the agent's filesystem
- Learn the characteristics and use cases of the five built-in backends
- Configure path-based routing with `CompositeBackend`
- Implement a custom backend with `BackendProtocol`

```
# Environment setup
from dotenv import load_dotenv
import os

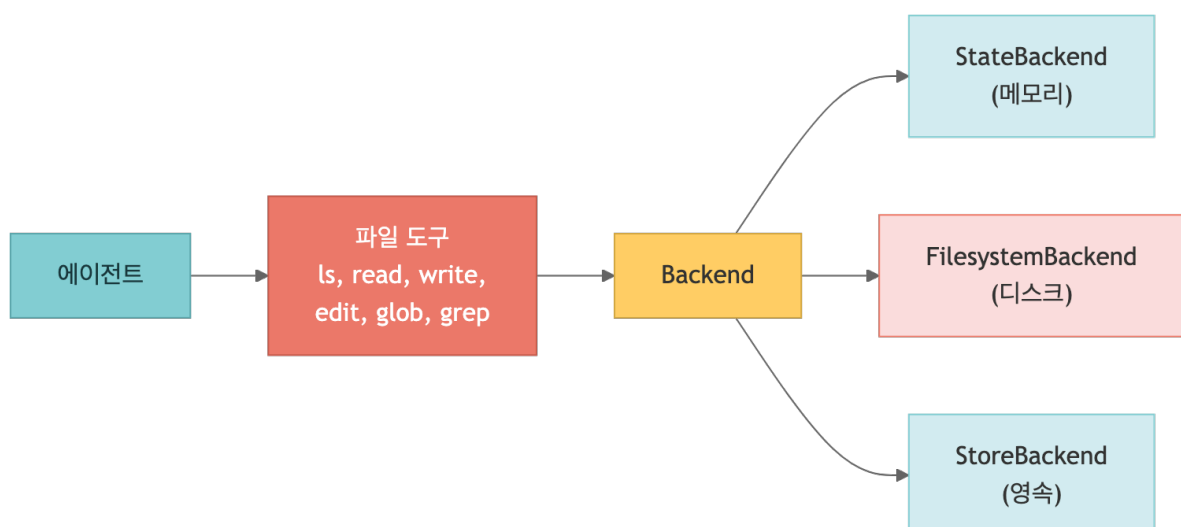
load_dotenv()
assert os.environ.get("ANTHROPIC_API_KEY"), "ANTHROPIC_API_KEY is not set!"
print("Environment setup complete")

# Recommended default – Anthropic Claude Sonnet 4.6
model = "anthropic:claude-sonnet-4-6"
```

1. What Is a Backend?

Deep Agents' built-in file tools (`ls` , `read_file` , `write_file` , `edit_file` , `glob` , `grep`) all operate through a *backend*.

A backend abstracts the *storage layer* that the agent uses to read and write files.



Available Backends

Backend	Storage Location	Persistence	Use Case
<code>StateBackend</code>	Agent state (memory)	Within a thread	Scratch work, temporary tasks (default)
<code>FilesystemBackend</code>	Local disk	Persistent	Local file access, coding agents
<code>StoreBackend</code>	LangGraph Store	Cross-thread	Long-term memory, user preferences
<code>CompositeBackend</code>	Path-based routing	Mixed	Persistent memory + temporary files
<code>LocalShellBackend</code>	Disk + shell	Persistent	Development environments (security caution)
<code>ContextHubBackend</code>	Context Hub (remote)	Persistent (lazy fetch)	Team-shared context, remote file sync

```
# Verify backend imports
from deepagents.backends import (
    StateBackend,
    FilesystemBackend,
    StoreBackend,
    CompositeBackend,
)
from deepagents.backends.protocol import BackendProtocol

print("All backend classes imported successfully!")
```

2. `StateBackend` (Default)

`StateBackend` stores files in agent state (LangGraph state). It is *ephemeral*: files live only inside the current conversation thread.

Characteristics

- Used automatically when you do not pass `backend` to `create_deep_agent()`
- Files survive across agent turns through checkpoints
- Files disappear when the thread ends
- No external storage required

3. `FilesystemBackend` — Accessing the Local Disk

`FilesystemBackend` lets the agent access the *real local filesystem*.

Key Options

- `root_dir` — the root directory the agent can access (default: current directory)
- `virtual_mode=True` — blocks `..` segments, `~` expansion, and absolute paths outside `root_dir`
- `max_file_size_mb` — maximum readable file size

Paths blocked by `virtual_mode=True`

- Relative escapes containing `..` segments (`../etc/passwd` , `foo/../../bar`)
- Home expansion starting with `~` or `~user`
- Absolute paths outside `root_dir` (such as `/etc/hosts`)

When any of these are submitted, the backend returns a result whose `error` field is filled and never touches the real disk.

⚠ Security Considerations



TIP

`FilesystemBackend` gives the agent access to the real filesystem. In production, set `virtual_mode=True` or use a sandbox backend.

4. `StoreBackend` — Cross-Thread Persistent Storage

`StoreBackend` uses LangGraph's `BaseStore` to persist files *across conversation threads*.

Characteristics

- The same files can be accessed from different threads
- Supports different store implementations such as Redis and PostgreSQL
- Can be provisioned automatically in LangSmith deployments
- Uses assistant-level namespacing to isolate agents

```
from langgraph.store.memory import InMemoryStore
from langgraph.checkpoint.memory import MemorySaver

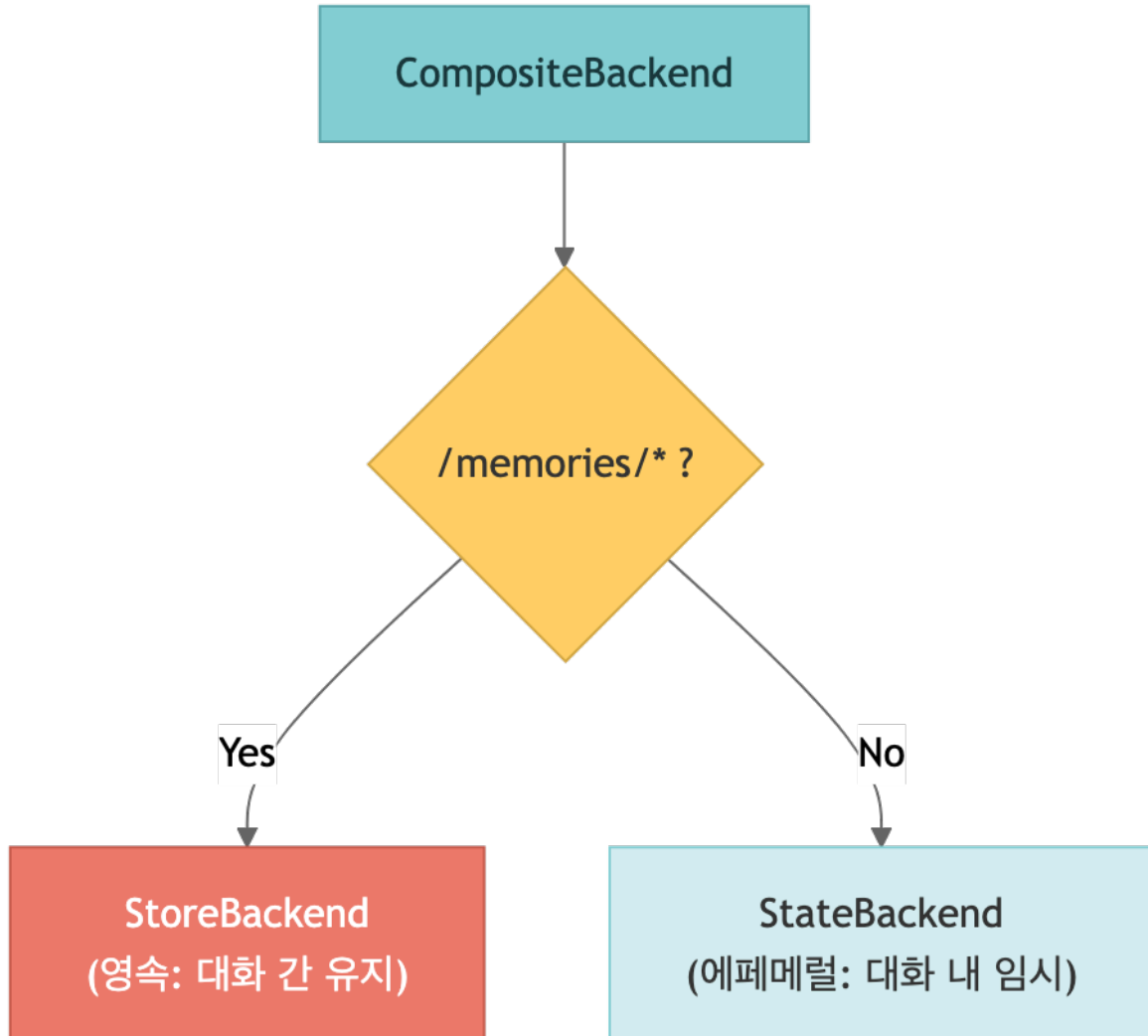
# InMemoryStore is useful for development (use PostgresStore or similar in production)
store = InMemoryStore()
checkpointer = MemorySaver()

# StoreBackend must be passed as a backend factory
store_agent = create_deep_agent(
    model=model,
    system_prompt="You are an assistant that manages notes. Respond in English.",
    backend=lambda runtime: StoreBackend(runtime),
    store=store,
    checkpointer=checkpointer,
)

print("StoreBackend agent created!")
```

5. CompositeBackend — Path-Based Routing

`CompositeBackend` routes different filesystem paths to different backends. A common pattern is to persist `/memories/*` while keeping everything else ephemeral.



```
# CompositeBackend factory function
def create_composite_backend(runtime):
    """Create a backend with path-based routing."""
    return CompositeBackend(
        default=StateBackend(runtime),
        routes={
            "/memories/": StoreBackend(runtime),
        },
    )

composite_store = InMemoryStore()
composite_checkpointer = MemorySaver()

composite_agent = create_deep_agent(
    model=model,
    system_prompt="""You are a memory-management assistant.
- Save notes that need to persist under /memories/.
```

```
- Save temporary work files under the root path /.
Respond in English.""",
    backend=create_composite_backend,
    store=composite_store,
    checkpointer=composite_checkpointer,
)

print("CompositeBackend agent created!")
```

· NOTE

Note: `CompositeBackend` removes the route prefix before storing the file internally. Example: `/memories/preferences.txt` → stored internally as `/preferences.txt` But the agent always accesses it with the full path (`/memories/preferences.txt`).

6. LocalShellBackend — Shell Execution

`LocalShellBackend` extends `FilesystemBackend` by adding shell command execution through the `execute` tool.

⚠ Security Warning

👉 TIP

Commands run directly on the host system *with your user permissions*. Use this only in development environments. In production, prefer a *sandbox backend*.

```
from deepagents.backends import LocalShellBackend

# ⚠ Development only!
agent = create_deep_agent(
    model=model,
    backend=LocalShellBackend(root_dir="./workspace", virtual_mode=True),
    interrupt_on={"execute": True}, # require approval before shell commands
)
```

· NOTE

For safety, this notebook does *not* run `LocalShellBackend` directly.

7. Implementing a Custom Backend

If you implement `BackendProtocol`, you can build your own backend.

! WARNING

Recent `deepagents` releases (`>=0.5.0`) renamed the protocol methods. *Old:* `ls_info` / `grep_raw` / `glob_info` . *Current:* `ls` / `grep` / `glob` . Return types are now explicit dataclasses / TypedDicts (`LsResult` , `ReadResult` , `GrepResult` , `WriteResult` , `EditResult` , `list[FileInfo]`) rather than bare dicts.

Required Methods and Return Types

Method	Return type	Description
<code>ls(path)</code>	<code>LsResult</code> (wraps <code>list[FileInfo]</code>)	List directory contents
<code>read(file_path, offset, limit)</code>	<code>ReadResult</code>	Read a file with line numbers
<code>write(file_path, content)</code>	<code>WriteResult</code>	Create a new file
<code>edit(file_path, old_string, new_string)</code>	<code>EditResult</code>	Replace text (with <code>occurrences</code>)
<code>grep(pattern, path, glob)</code>	<code>GrepResult</code>	Search file contents by pattern
<code>glob(pattern, path)</code>	<code>list[FileInfo]</code>	Search files by glob pattern

```
# Simple example: read-only dictionary-backed backend
from deepagents.backends.protocol import (
    FileInfo, GrepResult, GrepMatch,
    LsResult, ReadResult, WriteResult, EditResult,
)

class ReadOnlyDictBackend:
    """Example of a read-only backend backed by a Python dictionary."""

    def __init__(self, files: dict[str, str]):
        self._files = files

    def ls(self, path: str = "/") -> LsResult:
        entries: list[FileInfo] = [
            {"path": p, "is_dir": False, "size": len(c), "modified_at": None}
            for p, c in self._files.items()
            if p.startswith(path)
        ]
        return LsResult(entries=entries, error=None)

    def read(self, file_path: str, offset: int = 0, limit: int = 2000) -> ReadResult:
        content = self._files.get(file_path, "")
        lines = content.splitlines()
        selected = lines[offset:offset + limit]
        rendered = "\n".join(f"{i + offset + 1}\t{line}" for i, line in enumerate(selected))
        return ReadResult(content=rendered, error=None)

    def write(self, file_path: str, content: str) -> WriteResult:
        return WriteResult(error="This backend is read-only.", path=None, files_update=None)

    def edit(self, file_path: str, old_string: str, new_string: str, replace_all: bool = False) -> EditResult:
        return EditResult(error="This backend is read-only.", path=None, files_update=None, occurrences=None)

    def grep(self, pattern: str, path: str | None = None, glob: str | None = None) -> GrepResult:
        import re
        matches: list[GrepMatch] = []
        for fpath, content in self._files.items():
```



```

        for i, line in enumerate(content.splitlines(), 1):
            if re.search(pattern, line):
                matches.append({"path": fpath, "line": i, "text": line})
        return GrepResult(matches=matches, error=None)

def glob(self, pattern: str, path: str = "/") -> list[FileInfo]:
    import fnmatch
    return [
        {"path": p, "is_dir": False, "size": len(c), "modified_at": None}
        for p, c in self._files.items()
        if fnmatch.fnmatch(p, pattern)
    ]

custom_backend = ReadOnlyDictBackend({
    "/docs/guide.md": "# Guide\nThis is a guide document.\n## Installation\npip install deepagents",
    "/docs/faq.md": "# FAQ\nQ: Which models are supported?\nA: Anthropic, OpenAI, and more.",
})

print("File list:", custom_backend.ls("/").entries)
print()
print("File contents:")
print(custom_backend.read("/docs/guide.md").content)
print()
print("Search results:", custom_backend.grep("Installation").matches)

```

8. ContextHubBackend — Remote File Sync

`ContextHubBackend` syncs files with LangChain Context Hub through *lazy fetch* + *write-through* (`deepagents>=0.5.2`). Use it when teams share a common context (prompts, policies, domain docs) that every agent should see.

How it works

- *Lazy fetch* — files are downloaded only when the agent calls `read_file` , then cached locally
- *Write-through* — `write_file` / `edit_file` update *both* local and remote stores so other workers see the change immediately

Instantiation

Because `namespace` often has to be decided at *runtime*, instantiate the backend up front and pass a `namespace=lambda rt: ...` callable.

```

# deepagents>=0.5.2
from deepagents.backends import ContextHubBackend

# Pre-instantiated backend + runtime namespace callable
context_hub = ContextHubBackend(
    project="my-team",
    namespace=lambda rt: f"users/{rt.context['user_id']}", # decided at runtime
)

agent = create_deep_agent(
    model="anthropic:claude-sonnet-4-6",
    backend=context_hub,
)

```



TIP

The `namespace` callable signature is `(runtime)-> str`. A plain string also works, but multi-tenant deployments effectively require the lambda form.

9. Permissions — Permissions + GuardedBackend

When you need *finer-grained* access control than `virtual_mode`, use the `permissions` parameter to apply *per-path* ACLs. Internally a `GuardedBackend` wraps the original backend and routes every file call through the policy hook.

```
from deepagents import create_deep_agent
from deepagents.permissions import Permissions

agent = create_deep_agent(
    model="anthropic:claude-sonnet-4-6",
    backend=FilesystemBackend(root_dir="./workspace", virtual_mode=True),
    permissions=Permissions(
        allow_read=["/docs/**", "/data/*.csv"], # allowed reads
        allow_write=["/outputs/**"], # allowed writes
        deny_read=["/data/secrets/**"], # deny wins over allow
        deny_write=["**/*.lock"],
    ),
)
```

Policy hook (`policy`)

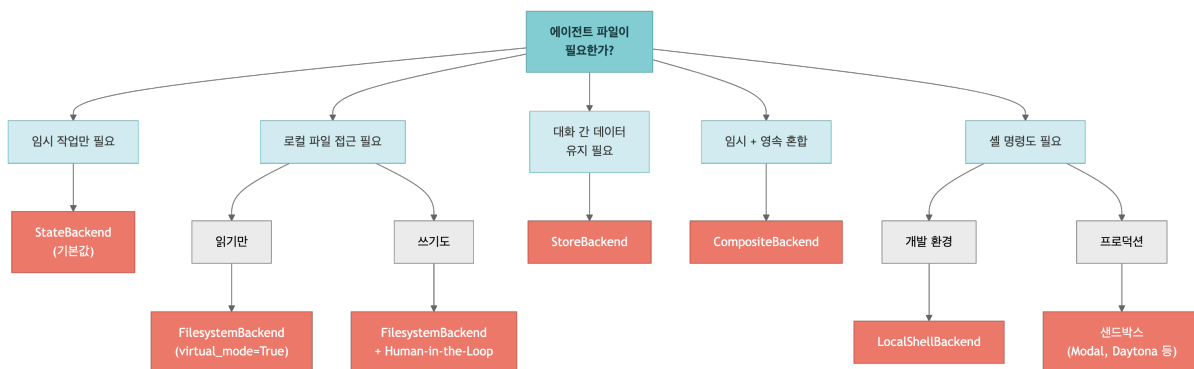
For complex rules pass `policy=lambda call: ...` — the call object includes `tool_name`, `path`, and `args`, so you can factor in time-of-day, user roles, or external authorization checks.

TIP

When both `allow_*` and `deny_*` rules match, `deny` wins. The safest pattern is an explicit whitelist via

```
allow_read=["/docs/**"]
```

Backend Selection Guide



Summary

Backend	Characteristics	Parameters
StateBackend	Ephemeral, default	Used automatically when <code>backend</code> is omitted
FilesystemBackend	Local disk access	<code>root_dir</code> , <code>virtual_mode</code>
StoreBackend	Cross-thread persistence	requires <code>store</code> + <code>checkpointer</code>
CompositeBackend	Path-based routing	<code>default</code> + <code>routes</code>
LocalShellBackend	Disk + shell execution	<code>root_dir</code> (security caution)
ContextHubBackend	Remote lazy fetch + write-through	pre-instantiated + <code>namespace=lambda rt: ...</code>
GuardedBackend (permissions)	Per-path ACL wrapper	<code>allow_*</code> / <code>deny_*</code> / <code>policy</code>

Next Steps

→ [05_subagents.ipynb](#): learn how to delegate work with subagents.

05 Subagents and Task Delegation

Learning Objectives

- Understand the problem subagents solve: context bloat
- Define subagents with `SubAgent` dictionaries and `CompiledSubAgent`
- Understand and override the built-in general-purpose subagent
- Use context propagation and namespace keys
- Implement a multi-subagent pipeline pattern

```
# Environment setup
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY is not set!"
assert os.environ.get("TAVILY_API_KEY"), "TAVILY_API_KEY is not set!"
print("Environment setup complete")
```

```
# Recommended default – Anthropic Claude Sonnet 4.6
model = "anthropic:claude-sonnet-4-6"

print(f"Model configured: {model}")
```

1. Why Subagents Are Useful

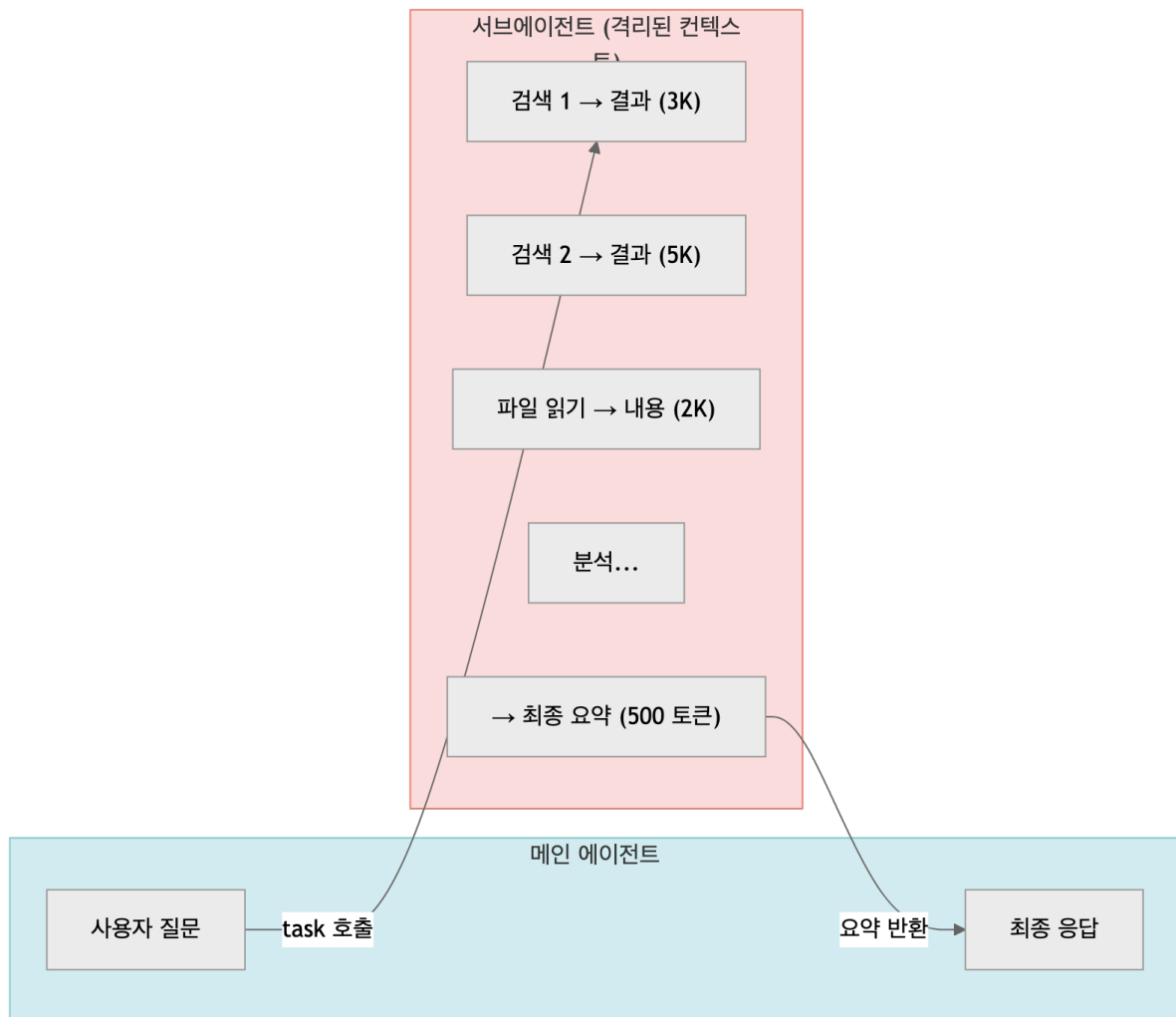
The Context Bloat Problem

Every time an agent uses a tool, the *inputs and outputs accumulate inside the context window*:

- web search results (thousands of tokens)
- file contents (hundreds or thousands of lines)
- database query results

When too much intermediate data builds up in the main context, the agent can lose track of the most important information.

How Subagents Help



The main agent receives only a *short summary*, so its context stays compact and focused.

`SubAgentMiddleware` is auto-attached

Subagent support is implemented by `SubAgentMiddleware`. `create_deep_agent()` always attaches this middleware regardless of whether you pass `subagents`, and it *cannot be removed* via `excluded_middleware`. If you do not declare any subagents, a built-in `general-purpose` subagent registers automatically so the `task` tool is always available.

TodoList + `task` dispatch pattern

The canonical loop looks like this:

1. The main agent calls `write_todos` to break the work into a *structured list*, for example `[research, analyze, write]`
2. For each TODO the main agent calls the `task` tool to dispatch the item to the right subagent
3. The subagent runs in an *isolated context* and returns only a *compressed result*
4. The main agent moves to the next TODO or composes the final answer

This `write_todos` → `task` → *compressed result* pattern is the core delegation cycle in Deep Agents. The main context only accumulates the todo list and the compressed results, which keeps token usage efficient.

When to Use a Subagent

Situation	Use a subagent?
Multi-step work with lots of intermediate results	✓ Yes
A domain that needs specialized knowledge or tools	✓ Yes
A task that is better handled by a different model	✓ Yes
A simple one-shot task	✗ Usually unnecessary
The main agent needs every intermediate detail	✗ Usually unnecessary

2. Defining a SubAgent (Dictionary Form)

A `SubAgent` is defined as a dictionary.

Required Fields

Field	Type	Description
<code>name</code>	<code>str</code>	Unique identifier
<code>description</code>	<code>str</code>	Role description used by the main agent when deciding whether to call it
<code>system_prompt</code>	<code>str</code>	Instructions for the subagent
<code>tools</code>	<code>list</code>	Tools available to the subagent

Optional Fields

Field	Type	Description
<code>model</code>	<code>str</code>	Model override (<code>"provider:model"</code>)
<code>middleware</code>	<code>list</code>	Additional middleware
<code>interrupt_on</code>	<code>dict</code>	Human-in-the-loop configuration
<code>skills</code>	<code>list[str]</code>	Skill source paths
<code>response_format</code>	<code>BaseModel</code>	Structured-output schema (<code>deepagents>=0.5.3</code>)
<code>permissions</code>	<code>Permissions</code>	Per-path ACL — restrict the subagent's filesystem access

```
from typing import Literal
from tavily import TavilyClient
from deepagents import create_deep_agent
```

```

tavily_client = TavilyClient(api_key=os.environ["TAVILY_API_KEY"])

def internet_search(
    query: str,
    max_results: int = 5,
    topic: Literal["general", "news", "finance"] = "general",
    include_raw_content: bool = False,
) -> dict:
    """Search the internet for information.

    Args:
        query: Question or keyword to search for
        max_results: Maximum number of results to return
        topic: Search topic category
        include_raw_content: Whether to include raw source content
    """
    return tavily_client.search(
        query,
        max_results=max_results,
        include_raw_content=include_raw_content,
        topic=topic,
    )

research_subagent = {
    "name": "researcher",
    "description": "Investigates a topic on the internet and summarizes the key information. Use it when research is required.",
    "system_prompt": """You are an expert researcher.
    Search the internet, collect accurate information, and summarize only the essentials.
    Always write in English.
    Keep the final result under 500 words.""",
    "tools": [internet_search],
}

print(f"Subagent created: {research_subagent['name']}")
print(f"Description: {research_subagent['description'][:60]}...")

```

```

# Create a main agent that can delegate work to the subagent
main_agent = create_deep_agent(
    model=model,
    system_prompt="""You are a project manager.
    Analyze the user's request, delegate work to a specialist subagent when needed,
    and combine the result into the final answer.
    Respond in English.""",
    subagents=[research_subagent],
)

print("Main agent created (subagent: researcher)")

```

3. CompiledSubAgent — Plugging in a Custom LangGraph Runnable

You can also use a precompiled LangGraph runnable as a subagent. This is useful when the delegated work needs more complex workflow logic such as branching or loops.

```

from deepagents import CompiledSubAgent

# Example: wrap another runnable as a CompiledSubAgent
custom_graph = create_deep_agent(
    model=model,
    tools=[internet_search],

```

```

    system_prompt="You are a data analyst. Gather information, analyze it, and produce insights.",
)

# Wrap the runnable
# TypedDict-like structure used by Deep Agents
# (same interface, but the runnable is already compiled)
data_analyst_subagent: CompiledSubAgent = {
    "name": "data-analyst",
    "description": "Collects data and produces analytical insights.",
    "runnable": custom_graph,
}

print(f"CompiledSubAgent created: {data_analyst_subagent['name']}")

```

4. General-Purpose Subagents

Deep Agents automatically provides a built-in *general-purpose subagent* even if you do not define one explicitly.

Default Behavior

- Uses the *same system prompt* as the main agent
- Has access to the *same tools* as the main agent
- Uses the *same model* as the main agent
- Inherits the main agent's *skills*

Overriding It

If you define a subagent with `name="general-purpose"`, it overrides the built-in one.

```

# Example: override the built-in general-purpose subagent
custom_gp_agent = create_deep_agent(
    model=model,
    tools=[internet_search],
    system_prompt="You are a multi-task coordinator.",
    subagents=[
        research_subagent,
        {
            "name": "general-purpose",
            "description": "A general-purpose agent that handles multi-step work beyond research.",
            "system_prompt": "You are a general assistant. Solve the task step by step.",
            "tools": [internet_search],
        },
    ],
)

print("Created an agent that overrides the general-purpose subagent")

```

5. Context Propagation

Runtime context is automatically propagated to all subagents. You define the shape with `context_schema`, and you pass values through the `context` key in `config`.

Passing Subagent-Specific Context with Namespace Keys

If you use the format `"subagent-name:key"`, you can pass configuration that only a specific subagent receives.

```
config = {
    "context": {
        "user_id": "user-123",           # propagated to every agent
        "researcher:max_depth": 3,      # only for researcher
        "data-analyst:strict_mode": True, # only for data-analyst
    }
}
```

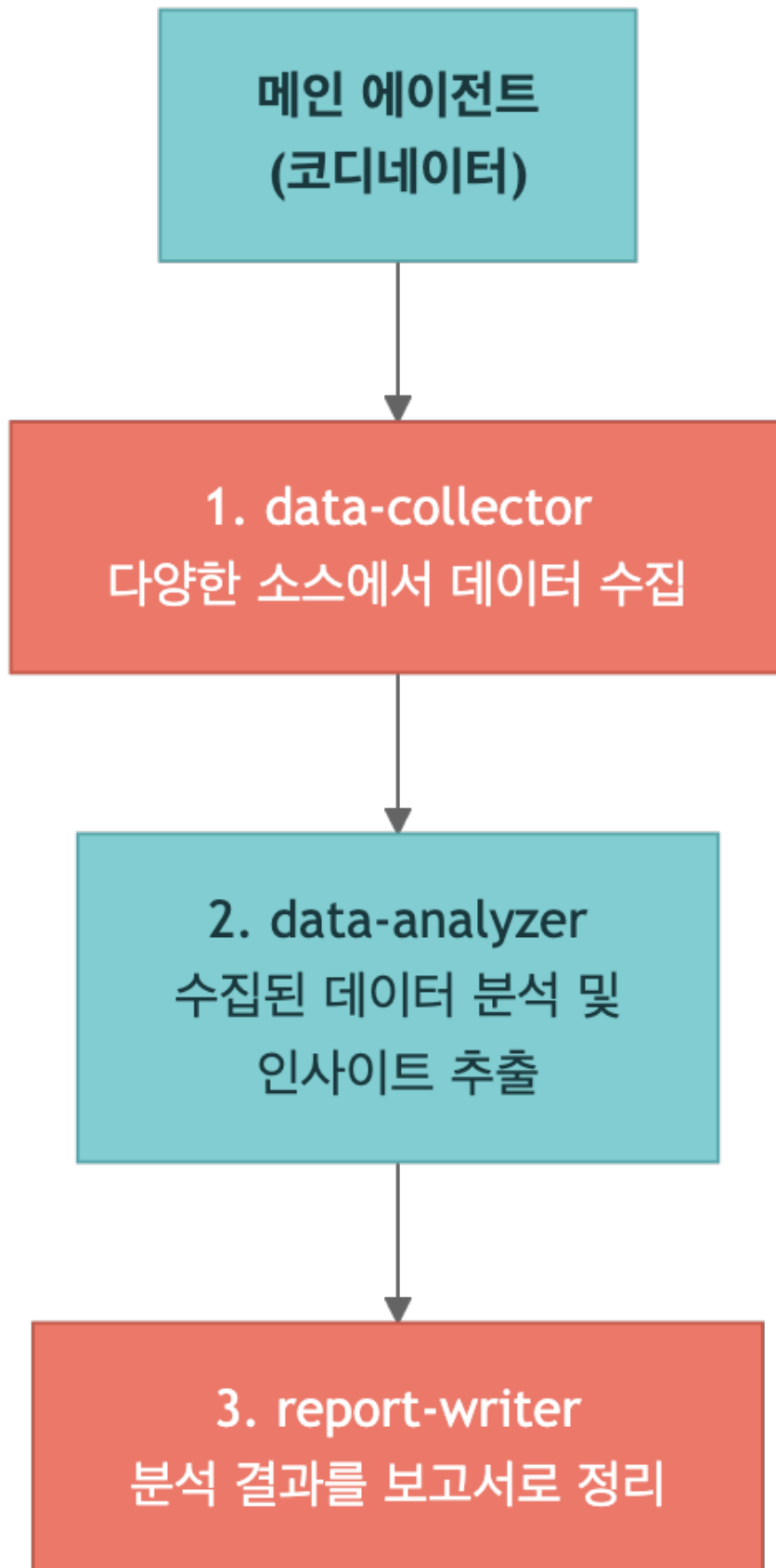
Identifying subagents during streaming — `lc_agent_name`

When you stream tokens through `agent.stream(..., stream_mode="messages")` or `astream_events()`, each event's *metadata* includes the `lc_agent_name` key. Use it to label UI output as “researcher searching...”, “analyzer summarizing...”, and similar.

```
# Identify which subagent is emitting tokens during streaming
async for event in main_agent.astream_events(
    {"messages": [{"role": "user", "content": "..."}]},
    version="v2",
):
    name = event.get("metadata", {}).get("lc_agent_name")
    if event["event"] == "on_chat_model_stream" and name:
        print(f"[{name}] {event['data']['chunk'].content}", end="")
```

6. Multi-Subagent Pipelines

You can combine several subagents to build a pipeline such as *collect* → *analyze* → *write*.



```
# Multi-subagent pipeline
pipeline_agent = create_deep_agent(
    model=model,
    system_prompt="""You are a project coordinator.
Analyze the user's request and delegate work in order:
1. Use data-collector to gather information.
2. Pass the results to data-analyzer for analysis.
3. Pass the analysis to report-writer to produce the final report.
Return the final report in English."""
    subagents=[
        {
            "name": "data-collector",
            "description": "Collects raw information from external sources.",
            "system_prompt": "Collect as much relevant information as possible and return it in
structured form.",
            "tools": [internet_search],
        },
        {
            "name": "data-analyzer",
            "description": "Analyzes the collected data and extracts key insights.",
            "system_prompt": "Extract patterns, trends, and key insights. Return the analysis as bullet
points.",
            "tools": [],
        },
        {
            "name": "report-writer",
            "description": "Writes a professional report based on the analysis.",
            "system_prompt": "Write a clear report with the following structure: overview → key findings
→ conclusion.",
            "tools": [],
        },
    ],
)

print("Multi-subagent pipeline agent created")
```

7. Best Practices

1. Write clear descriptions

The subagent `description` is how the main agent decides when to call it. Make it specific.

2. Keep the result focused

A subagent should return a compact result rather than flooding the parent agent with raw data.

3. Use specialized tools per role

Give each subagent only the tools it actually needs.

4. Separate simple from complex work

Do not create subagents for tasks that the main agent can solve directly in one step.

Summary

Item	Description
Why subagents matter	They solve context bloat and enable specialization
Basic definition	Subagent dictionary with <code>name</code> , <code>description</code> , <code>system_prompt</code> , <code>tools</code> plus optional <code>response_format</code> / <code>permissions</code>
Advanced form	<code>CompiledSubAgent</code> wraps a precompiled runnable
Built-in fallback	<code>SubAgentMiddleware</code> is always attached (not removable); a default <code>general-purpose</code> subagent fills the slot
Dispatch pattern	<code>write_todos</code> → <code>task()</code> → compressed result
Context propagation	Runtime context is inherited; namespace keys (<code>"agent:key"</code>) target specific subagents
Streaming	<code>lc_agent_name</code> in event metadata identifies the active subagent
Pipeline pattern	Subagents can be chained into <code>collect</code> → <code>analyze</code> → <code>write</code> workflows

Next Steps

→ [06_memory_and_skills.ipynb](#): learn how long-term memory and skills work in Deep Agents.

06 Long

Term Memory & Skills

Learning Objectives

- Implement long-term memory with `CompositeBackend` + `StoreBackend`
- Understand the cross-thread memory-sharing pattern
- Inject agent context through `AGENTS.md`
- Understand the structure of skills (`SKILL.md`) and Progressive Disclosure
- Understand the difference between Skills and Memory

```
# Environment setup
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY is not set!"
print("Environment setup complete")
```

```
# Configure the OpenAI gpt-5.4 model
from langchain_openai import ChatOpenAI

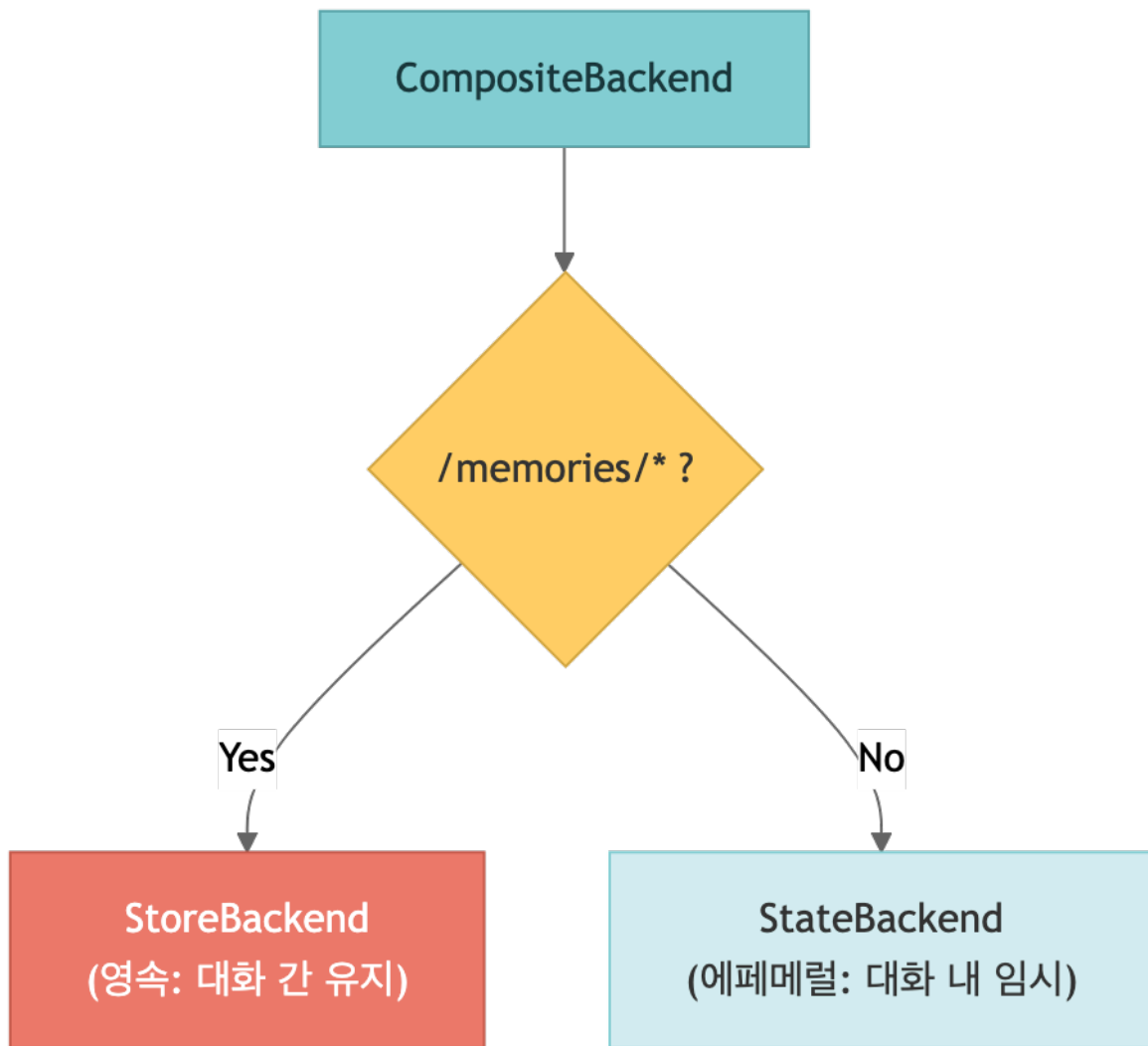
model = ChatOpenAI(model="gpt-5.4")
```

1. Why Long-Term Memory Matters

A default `StateBackend` agent *forgets everything when the conversation thread ends*. But a useful assistant often needs to preserve information *across threads*, such as:

- user preferences (coding style, preferred language)
- project conventions (architecture decisions, naming rules)
- feedback learned in previous conversations
- frequently referenced information (API docs, configuration values)

Solution: CompositeBackend



Files stored under `/memories/` can be accessed *from any conversation thread*.

```

from deepagents import create_deep_agent
from deepagents.backends import StateBackend, StoreBackend, CompositeBackend, FilesystemBackend
from langgraph.store.memory import InMemoryStore
from langgraph.store.postgres import PostgresStore # Production
from langgraph.checkpoint.memory import MemorySaver

# 1. Create a store and a checkpointer
store = InMemoryStore() # Development only (production: PostgresStore)
checkerpoint = MemorySaver() # Preserve agent state

# 2. Composite backend factory – persist only /memories/, keep the rest ephemeral
# StoreBackend uses a namespace lambda to declare the memory scope explicitly.
def memory_backend_factory(runtime):
    return CompositeBackend(
        default=StateBackend(runtime),
        routes={
            "/memories/": StoreBackend(
                namespace=lambda rt: (rt.context.user_id,),
            ),
        },
    ),
  
```

```

    )

# 3. Create the agent
memory_agent = create_deep_agent(
    model=model,
    system_prompt="""You are a personal assistant.
Save information the user wants to remember under /memories/.
If there is previously saved memory, use it when responding.
Respond in English."""
    backend=memory_backend_factory,
    store=store,
    checkpointer=checkpointer,
)

print("Long-term memory agent created")

```

2. Cross-Thread Memory Sharing

Data stored in `StoreBackend` is *shared across threads*. In the example below, thread 1 stores preferences and thread 2 reads them later.

3. Injecting Context with `AGENTS.md`

If you use the `memory` parameter, the agent automatically loads `AGENTS.md` **files at startup** and injects them into the system prompt.

What is `AGENTS.md` ?

It is a Markdown file that contains *rules, conventions, and context information* that should always apply to the agent.

Characteristics

- Always loaded when the agent starts (not on demand)
- Injected into the system prompt through `<agent_memory>`
- You can specify multiple memory sources
- The agent can update `AGENTS.md` itself with `edit_file`

```

import tempfile

# Create a temporary directory to use as the root_dir for FilesystemBackend
tmp_dir = tempfile.mkdtemp()
print(f"Temporary directory created: {tmp_dir}")

```

4. Skills

Skills are modular instruction bundles that give the agent *specialized domain knowledge*.

Memory vs Skills

Comparison	Memory (<code>AGENTS.md</code>)	Skills (<code>SKILL.md</code>)
Loading	Always loaded	Loaded only when needed
File format	<code>AGENTS.md</code>	<code>SKILL.md</code> (YAML frontmatter)
Best fit	Rules and conventions that always apply	Large context needed for specific tasks
Token efficiency	Always consumes tokens	Saves tokens through Progressive Disclosure
Size	Best kept concise	Can be large (up to 10 MB)
Updates	Can be edited by the agent	Usually static

Progressive Disclosure

Skills are not fully loaded all at once:

1. At first, only the *frontmatter* (name, description, metadata) is loaded
2. The agent decides which skills are relevant to the user's request
3. Only then is the *full content* of the necessary skills loaded

This approach saves tokens while still giving the agent access to deep task-specific knowledge.

`SKILL.md` Structure

The `module` field is only set for interpreter skills (loaded by the QuickJS `CodeInterpreterMiddleware`); plain-text skills omit it.

```
---
name: web-research
description: >
  Step-by-step guide for structured web research.
  Covers information gathering, verification, and summarization.
license: MIT
compatibility: Python 3.8+
module: ./web_research.js # (optional) interpreter skills only
metadata:
  category: research
allowed-tools: ls read_file write_file
---
```

```
# Web Research Skill
```

```
## When to use it
```

- When the user asks for research on a topic
- When the request depends on current information

```
## Workflow
```

1. Design search queries
2. Gather information from multiple sources
3. Cross-check the information
4. Write a structured report

```
# Create an agent that uses skills
skilled_agent = create_deep_agent(
    model=model,
    system_prompt="You are a senior developer. Use the available skills to complete the task.",
```



```

backend=FileSystemBackend(root_dir=tmp_dir, virtual_mode=True),
skills=["/skills/"],
)

print("Skill-enabled agent created")

```

5. Skill Source Priority

If you specify multiple skill sources, the *later source wins*.

```

skills=[
    "/skills/base/",      # base skills
    "/skills/user/",      # can override base
    "/skills/project/",   # highest priority
]

```

If the same skill name exists in multiple locations, the version from the last source is used.

6. Skill Inheritance for Subagents

Subagent Type	Skill Inheritance
Built-in general-purpose subagent	<i>Automatically inherits</i> the main agent's skills
Custom <code>SubAgent</code>	Requires an explicit <code>skills</code> parameter

```

subagent = {
    "name": "reviewer",
    "description": "Code review specialist",
    "system_prompt": "...",
    "tools": [],
    "skills": ["/skills/code-review/"],
}

```

TIP

Follow *least privilege* when assigning skills to subagents. Handing a review-focused subagent unrelated deployment skills wastes tokens and invites confusion. Give each subagent only the skills that match its role.

Skills backend support matrix

Skill source directories can live in any of `StateBackend`, `StoreBackend`, `FileSystemBackend`, or `CompositeBackend`, but each backend has different operational characteristics.

Backend	Persistence	Sharing scope	Recommended use
---------	-------------	---------------	-----------------

StateBackend	Thread-local (ephemeral)	Current thread only	Ad-hoc experiments and tests
StoreBackend	Persistent (DB-backed)	Per namespace (user / org / assistant)	Team- or user-shared skills
FilesystemBackend	Persistent (disk)	Host directory	Standard skills committed to Git
CompositeBackend	Routed per path	Separated per path	Layered: <code>/skills/base/</code> + <code>/skills/user/</code>

6.10 Long-term Memory Types (in 0.5)

Deep Agents 0.5 classifies stored information into three categories. Each type uses a different storage mechanism, update cadence, and backend. The core rule is *not* to push all three into one backend, but to distribute them across the mechanisms that match their nature.

Type	Meaning	Storage example	Mechanism
Episodic	Past experience — conversation sessions, problem-solving trajectories	Thread history of past conversations	Checkpointers (per thread)
Procedural	Reusable instructions · skills · workflows	<code>SKILL.md</code> , procedure docs	Skills (loaded on demand)
Semantic	Facts · preferences · policies	<code>AGENTS.md</code> , <code>/memories/*.txt</code>	StoreBackend (always-on files)

Example: long-running preferences in `/memories/` (semantic) + episodic case retrieval via checkpointers (episodic) + repeated procedures as skills (procedural).

Scope patterns: agent-scoped / user-scoped / context-driven

The tuple returned by `StoreBackend`’s `namespace` function is the scope of the memory. The three most common patterns are:

Pattern	namespace example	Best fit
Agent-scoped	<code>(rt.server_info.assistant_id,)</code>	Organization-wide conventions and domain knowledge (shared across users)
User-scoped	<code>(rt.server_info.user.identity,)</code>	Default for personalization assistants — fully isolated per user
Context-driven	<code>(rt.context.user_id, rt.context.project_id)</code>	Multi-dimensional isolation using keys declared in <code>context_schema</code>

Agent-scoped — shared identity accumulation. Namespace is `(assistant_id,)`. *All user conversations* that use the same assistant share the same memory. Suitable for organization-wide conventions and domain knowledge, but *do not store sensitive information here* because information flows between users.

```
from deepagents.backends import StoreBackend

agent_scoped = StoreBackend(
    namespace=lambda rt: (rt.server_info.assistant_id,),
)
```

User-scoped — per-user isolation. Namespace is `(user_id,)` or `(assistant_id, user_id,)`. Each user's memory is fully isolated, so user A's preferences are never exposed in user B's conversation. This is the default for production personalization assistants.

```
user_scoped = StoreBackend(
    namespace=lambda rt: (rt.server_info.user.identity,),
)
```

Episodic memory via checkpoints

To turn past conversations from passive storage into *searchable memory*, wrap the threads saved by the checkpointer as a tool.

```
from langchain.tools import tool, ToolRuntime

@tool
async def search_past_conversations(query: str, runtime: ToolRuntime) -> str:
    """Find related context from prior conversations."""
    user_id = runtime.server_info.user.identity
    threads = await client.threads.search(
        metadata={"user_id": user_id},
        limit=5,
    )
    # Summarize threads into the needed format and return
    ...
```

With this pattern the agent can reference “how did I solve this before” on its own.

Read-only policy (organization-wide)

Shared organizational memory is an injection vector. Enforce *write-blocking* as follows.

```
# policies is org-scoped and read-only
routes = {
    "/policies/": StoreBackend(
        namespace=lambda rt: (rt.context.org_id,),
    ),
}
```

Combined with pattern 4 from Part IV ch15 (permissions), apply `write deny` on `/policies/**`. Policies should only be updated from application code.

Background consolidation agent

Updating memory *during the conversation (hot path)* increases latency, and the summary quality ends up tracking the model's hurried decisions. The alternative is to run a separate consolidation agent on a *cron schedule*.

```
// langgraph.json
{
  "graphs": {
    "agent": "./agent.py:agent",
    "consolidation_agent": "./consolidation_agent.py:agent"
  }
}
```

Register the cron schedule:

```
cron_job = await client.crons.create(
    assistant_id="consolidation_agent",
    schedule="0 */6 * * *", # every 6 hours
    input={"messages": [{"role": "user", "content": "Consolidate recent memories."}]},
)
```

! WARNING

Keep the cron interval (`0 */6 * * *`) and the lookback window (`timedelta(hours=6)`) *aligned* to avoid gaps and duplicates.

Update-timing comparison

Timing	Approach	Latency	Takes effect
Hot path	Agent calls <code>edit_file</code> mid-conversation	Yes	Immediately
Background	Consolidation agent processes between sessions	Invisible to user	From next conversation

The default composition splits by type: sensitive or must-be-immediate preferences go on the hot path; long-term pattern accumulation runs in the background.

Summary

Item	Description
Long-term memory	Persist <code>/memories/</code> with <code>CompositeBackend</code> + <code>StoreBackend</code>
<code>AGENTS.md</code>	<code>memory=["/path/AGENTS.md"]</code> → always injected into the system prompt
Skills	<code>skills=["/skills/"]</code> → <code>SKILL.md</code> with Progressive Disclosure
Progressive Disclosure	Load frontmatter first → load full skill only when needed

Skill priority	Later sources win
Memory vs Skills	Memory = always loaded / Skills = loaded on demand
Three memory types	Episodic (checkpointer) / Procedural (skills) / Semantic (store)
Scope patterns	agent-scoped (shared accumulation) / user-scoped (isolated, default) / org-scoped (read-only)
Consolidation	Hot path: immediate, adds latency / Background cron: non-blocking, reflected from next conversation

Long-term memory and the skill system let agents accumulate knowledge beyond a single conversation and surface specialist capabilities when needed. The next chapter covers production-grade advanced features: Human-in-the-Loop, streaming, sandboxes, ACP, and the CLI.

Next Steps

→ [07_advanced.ipynb](#): learn advanced features such as Human-in-the-Loop, streaming, sandboxes, and ACP.

07 Advanced Features

Learning Objectives

- Implement a Human-in-the-Loop workflow
- Understand streaming modes and the namespace system
- Understand sandbox integrations such as Modal, Daytona, and Runloop
- Learn how ACP (Agent Client Protocol) connects agents to editors
- Learn how to use the Deep Agents CLI

```
# Environment setup
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY is not set!"
print("Environment setup complete")
```

```
from langchain_openai import ChatOpenAI

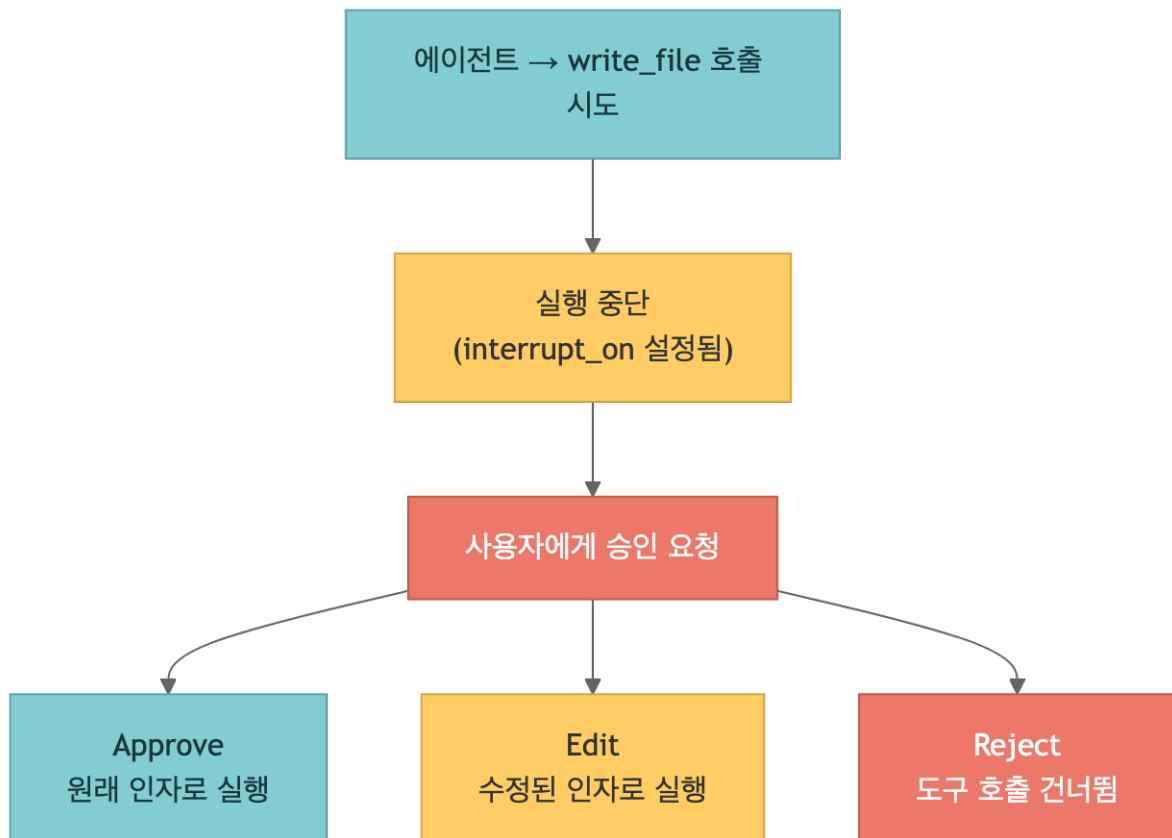
model = ChatOpenAI(model="gpt-5.4")

print(f"Model configured: {model.model_name}")
```

1. Human-in-the-Loop (HITL)

Human-in-the-Loop is a workflow in which the agent *requires human approval* before calling sensitive tools. Deep Agents supports *four decision types*, all passed as dicts under `Command(resume={"decisions": [...]})` when resuming.

How it Works



Four decision types

Decision	Behavior
approve	Execute the tool with the proposed arguments as-is
edit	Modify arguments via <code>edited_action.name/args</code> and then execute
reject	Skip the tool call entirely
respond	Skip execution and return <code>message</code> as the tool result

Inspecting interrupts and resuming (v2 standard)

When you call `invoke(..., version="v2")`, interrupts are exposed via `result.interrupts` — *not* the older `result["__interrupt__"]` key. Each interrupt's `interrupt_value["action_requests"]` lists the proposed tool calls and `allowed_decisions`. Resume with `Command(resume={"decisions": [...]} (a dict) using the same thread_id.`

```

from langgraph.types import Command

# Inspect interrupts
result = hitl_agent.invoke(inputs, config=config, version="v2")
for itr in result.interrupts:
    for req in itr.value["action_requests"]:
        print(req["action"]["name"], req["allowed_decisions"])

```

```
# Resume – one decision per interrupt, in the same order
hitl_agent.invoke(
    Command(resume={"decisions": [{"type": "approve"}]}),
    config=config,
    version="v2",
)
```

Required Condition

- *Checkpointers*: required to preserve the agent’s state between interrupt and resume

```
from deepagents import create_deep_agent
from langgraph.checkpoint.memory import MemorySaver

# Choose which tools require approval with interrupt_on
hitl_agent = create_deep_agent(
    model=model,
    system_prompt="You are a file management assistant. Respond in English.",
    checkpointer=MemorySaver(), # required!
    interrupt_on={
        "write_file": True,
        "edit_file": True,
    },
)

print("Human-in-the-Loop agent created")
print("write_file and edit_file now require approval")
```

2. Advanced Streaming

Deep Agents runs on top of LangGraph’s streaming infrastructure.

v2 unified format (standard)

The standard path is *one call*: `agent.stream(..., stream_mode=..., subgraphs=True, version="v2")`. The older v1 nested-tuple format is harder to branch on, and v3/projection variants are unofficial — do not use them. Every chunk shares the same 3-field shape:

```
{
  "type": "updates" | "messages" | "custom",
  "ns": tuple,          # event source (main vs subagent routing)
  "data": Any,          # payload, depends on type
}
```

TIP

Without `subgraphs=True`, subagent-internal events are invisible — set it whenever the UI needs to show subagent progress. To hide summarization tokens, filter on `metadata.get("lc_source") == "summarization"`.

Stream Modes

Mode	Description	Use Case
"updates"	State updates after each node finishes	Progress tracking
"messages"	Token-level streaming	Real-time text output
"custom"	Events emitted inside tools or nodes	Custom progress reporting

Namespace System

Events from subagents are separated by namespace:

```
() # main agent
("tools:abc123",) # subagent (tool call ID)
("tools:abc123", "model:def456") # inner node inside a subagent
```

Custom progress events (`get_stream_writer` + `stream_mode="custom"`)

Tools can emit arbitrary objects for the UI — upload progress, processed counts, mid-execution status. Pin the schema (for example `{ "status", "progress", "message" }`) so the UI routing does not break across tools.

```
from langchain.tools import tool
from langgraph.config import get_stream_writer

@tool
def analyze_data(topic: str) -> str:
    """Analyze data for a topic and stream progress updates."""
    writer = get_stream_writer()
    writer({"status": "starting", "progress": 0, "topic": topic})
    # ... actual analysis ...
    writer({"status": "complete", "progress": 100, "topic": topic})
    return f"Analysis complete: {topic}"

for chunk in agent.stream(
    {"messages": [{"role": "user", "content": "..."}]},
    stream_mode=["updates", "messages", "custom"],
    subgraphs=True,
    version="v2",
):
    if chunk["type"] == "custom":
        print(chunk["data"])
```

```
from typing import Literal
from tavily import TavilyClient

tavily_client = TavilyClient(api_key=os.environ.get("TAVILY_API_KEY", ""))

def internet_search(
    query: str,
    max_results: int = 3,
    topic: Literal["general", "news"] = "general",
) -> dict:
    """Search the internet for information."""
    return tavily_client.search(query, max_results=max_results, topic=topic)

stream_agent = create_deep_agent(
```

```
model=model,
system_prompt="You are a research coordinator. Respond in English.",
subagents=[
    {
        "name": "researcher",
        "description": "Uses internet search to investigate topics.",
        "system_prompt": "Search the internet, collect the requested information, and summarize it
concisely.",
        "tools": [internet_search],
    }
],
)

print("Streaming demo agent created")
```

3. Sandboxes

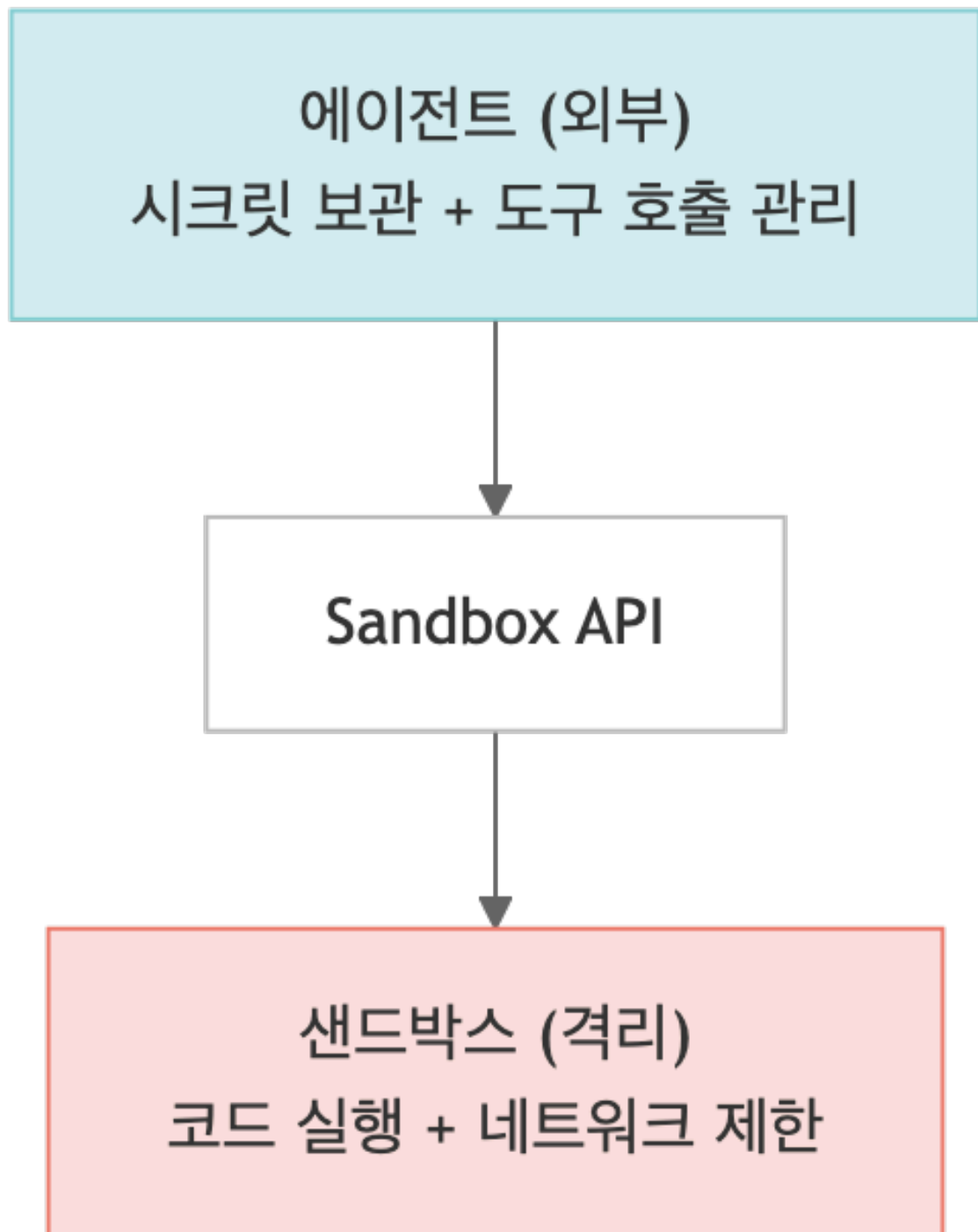
A sandbox lets the agent run code in an *isolated environment*. That prevents it from accessing the host machine's files, network, or credentials directly.

Supported Providers

Provider	Characteristics	Best Fit
<i>Modal</i>	GPU support, ML workloads	AI / ML tasks
<i>Daytona</i>	TypeScript / Python, fast cold starts	Web development
<i>Runloop</i>	Disposable devboxes, isolated execution	Code testing

Architecture Pattern

Use the sandbox as a tool (recommended)



⚠ Security Guidelines

- *Never put secrets inside the sandbox* — the agent may leak them
- Manage credentials only through external tools
- Use Human-in-the-Loop approval for sensitive operations
- Block unnecessary network access

```
# Modal sandbox integration – uses the langchain-modal package
# pip install langchain-modal deepagents
import modal
from deepagents import create_deep_agent
from langchain_anthropic import ChatAnthropic
from langchain_modal import ModalSandbox

app = modal.App.lookup("your-app")
modal_sandbox = modal.Sandbox.create(app=app)
backend = ModalSandbox(sandbox=modal_sandbox)

agent = create_deep_agent(
    model=ChatAnthropic(model="claude-sonnet-4-6"),
    system_prompt="You are a Python coding assistant with sandbox access.",
    backend=backend,
)

try:
    result = agent.invoke({
        "messages": [{"role": "user", "content": "Create a small Python package and run pytest"}],
    })
finally:
    modal_sandbox.terminate() # required: free resources
```

TIP

Each provider ships as a separate package (`langchain-modal` , `langchain-daytona` , ...). Always pair sandbox creation with `try/finally` and call `terminate()` (or `stop()` / `shutdown()`) so resources are released.

3-1. Interpreters — `CodeInterpreterMiddleware`

Deep Agents 0.6 lets you attach a *QuickJS-based interpreter* through `CodeInterpreterMiddleware` . A sandbox runs code in an isolated *outside* environment; an interpreter runs small programs *inside* the agent loop. It is useful for composing tool calls, fan-out/fan-in across subagents, and shaping structured data.

Install

```
pip install -U "deepagents[quickjs]"
# or
uv add "deepagents[quickjs]"
```

Programmatic Tool Calling (PTC)

Set `ptc=["task"]` (an allowlist) to expose tools as async functions under the `tools.*` namespace, with names converted to camelCase (for example `web_search` → `tools.webSearch`). Inside the interpreter you can fan out with `Promise.all` .

```
from deepagents import create_deep_agent
from langchain_quickjs import CodeInterpreterMiddleware

agent = create_deep_agent(
    model="openai:gpt-5.4",
    middleware=[
```

```
CodeInterpreterMiddleware(
    ptc=["task"],
    snapshot_between_turns=True,
    timeout=5.0,
    max_ptc_calls=256,
),
1,
)
```

Middleware options (10)

Parameter	Default	Purpose
memory_limit	64*1024*1024	QuickJS heap memory limit (bytes)
timeout	5.0	Per-eval timeout (seconds)
max_ptc_calls	256	Max <code>tools.*</code> calls per eval
tool_name	"eval"	Interpreter tool name
max_result_chars	4000	Max characters returned
capture_console	True	Capture <code>console.log</code> output
ptc	None	PTC allowlist
skills_backend	None	Backend for interpreter skill modules
snapshot_between_turns	True	Preserve state across turns
max_snapshot_bytes	None	Max snapshot size

! WARNING

PTC does not go through the regular tool-calling path, so per-tool `interrupt_on` policies are *not* guaranteed to apply. Do not expose tools with side effects (cost, data mutation, network) without an explicit approval wrapper.

4. ACP (Agent Client Protocol)

ACP standardizes communication *between coding agents and editors / IDEs*.

Supported Editors

- *Zed* — native integration
- *JetBrains IDEs* — built-in support
- *VS Code* — `vscode-acp` plugin
- *Neovim* — ACP-compatible plugins

MCP vs ACP

Protocol	Purpose
MCP (Model Context Protocol)	External tool integration
ACP (Agent Client Protocol)	Editor ↔ agent integration

```
# ACP server – canonical pattern (asyncio + acp.run_agent)
# pip install deepagents-acp
import asyncio

from acp import run_agent
from deepagents import create_deep_agent
from langgraph.checkpoint.memory import MemorySaver

from deepagents_acp.server import AgentServerACP

async def main() -> None:
    agent = create_deep_agent(
        model="anthropic:claude-sonnet-4-6",
        system_prompt="You are a helpful coding assistant",
        checkpointer=MemorySaver(),
    )
    server = AgentServerACP(agent)
    await run_agent(server)

if __name__ == "__main__":
    asyncio.run(main())
```

 TIP

The Toad CLI runs ACP servers as managed local processes. Install with `uv tool install -U batrachian-toad` , then `toad acp "python path/to/your_server.py"` . to launch with automatic start/stop/restart.

Context Engineering — @dynamic_prompt

Inject request-time data into the system prompt with the `@dynamic_prompt` middleware. It can read `request.runtime.context` and `request.runtime.store` , so any field declared in `context_schema` (such as `user_id` / `org_id`) propagates to every subagent and tool automatically. Tools receive `ToolRuntime[Context]` and read runtime values from `runtime.context.user_id` .

```
from deepagents.middleware import dynamic_prompt

@dynamic_prompt
def inject_user(request):
    user_id = request.runtime.context.user_id
    pref = request.runtime.store.get(("prefs", user_id), "lang")
    return f"\n\nUser: {user_id} / Preferred language: {pref}"
```

Permissions (deepagents>=0.5.2)

`FilesystemPermission` controls read/write access on built-in FS tools using *first-match-wins* semantics; if nothing matches, the default is `allow` . Custom tools, MCP, and the sandbox `execute` bypass these rules and need separate guards. A subagent’s `permissions` *fully replaces* parent rules (no partial override).

```
from deepagents.permissions import FilesystemPermission

permissions = [
    FilesystemPermission(operations=["write"], paths=["/policies/**"], mode="deny"),
    FilesystemPermission(operations=["read", "write"], paths=["/workspace/**"], mode="allow"),
]
```

5. Deep Agents CLI

The Deep Agents CLI is a *terminal coding agent* built on top of the SDK.

Installation and Execution

```
# Install
uv tool install deepagents-cli

# Run
deepagents-cli

# Run directly without installing
uvx deepagents-cli
```

Main Options

Option	Description
<code>-a/--agent AGENT</code>	Specify the agent name
<code>-M/--model MODEL</code>	Choose the model
<code>-n/--non-interactive</code>	Non-interactive mode (single task execution)
<code>--auto-approve</code>	Skip human confirmation
<code>--sandbox {none,modal,daytona,runloop}</code>	Select a sandbox backend

Interactive Commands

Command	Description
<code>/model</code>	Change the model
<code>/remember</code>	Store information in memory
<code>/tokens</code>	Inspect token usage
<code>!command</code>	Run a shell command

Memory System

- *Global:* `~/.deepagents/<agent_name>/memories/`

– *Project:* `.deepagents/AGENTS.md` (project root)

```
# CLI non-interactive examples (run in the shell)
cli_examples = """
# Basic usage
deepagents-cli

# Non-interactive execution with a specific model
deepagents-cli -M claude-sonnet-4-6 -n "Write the README.md file for this project"

# Run inside a sandbox
deepagents-cli --sandbox modal "Run the test suite"

# Skill management
deepagents-cli skills list
deepagents-cli skills create my-skill
"""

print("CLI usage examples (run in the terminal):")
print(cli_examples)
```

Full Track Summary

Notebook	Topic	Key APIs
01	Introduction	<code>deepagents.__version__</code>
02	Quickstart	<code>create_deep_agent()</code> , <code>invoke()</code> , <code>stream()</code>
03	Customization	<code>model</code> , <code>system_prompt</code> , <code>tools</code> , <code>response_format</code>
04	Backends	<code>StateBackend</code> , <code>FilesystemBackend</code> , <code>StoreBackend</code> , <code>CompositeBackend</code>
05	Subagents	<code>SubAgent</code> , <code>CompiledSubAgent</code> , <code>subagents</code>
06	Memory & Skills	<code>memory</code> , <code>skills</code> , <code>AGENTS.md</code> , <code>SKILL.md</code>
07	Advanced Features	<code>interrupt_on</code> , <code>stream_mode</code> , <code>Sandbox</code> , <code>ACP</code> , <code>CLI</code>

Next Steps

→ Continue to [08_harness.ipynb](#) → Or jump to the *advanced track* at [../05_advanced/00_migration.ipynb](#)

08 Agent Harness

Learning Objectives

- Understand the concept and role of AgentHarness
- Learn the harness's core capabilities: planning, filesystem access, and task delegation
- Understand context management through offloading and summarization
- Configure code execution and Human-in-the-Loop
- Connect skills and memory systems

```
# Environment setup
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY is not set!"
print("Environment setup complete")
```

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")

print(f"Model configured: {model.model_name}")
```

1. AgentHarness Concept

AgentHarness is a *comprehensive capability provider* for long-running autonomous agents. It bundles together the infrastructure needed for complex multi-step agent work.

Core Capabilities Provided by the Harness

Capability	Description
<i>Planning</i>	Manage structured task lists with <code>write_todos</code>
<i>Filesystem</i>	Read, write, and search files in virtual or local environments
<i>Task Delegation</i>	Delegate work through subagents
<i>Context Management</i>	Compress context through offloading and summarization
<i>Code Execution</i>	Run code safely in sandboxed environments

<i>Human-in-the-Loop</i>	Require approval for sensitive operations
<i>Skills & Memory</i>	Use specialized workflows and persistent knowledge

When you call `create_deep_agent()`, all of these pieces are assembled into a single agent.

```
# AgentHarness concept – create_deep_agent assembles the harness
harness_config = {
    "model": "openai:gpt-5.4",
    "system_prompt": "You are a project management assistant.",
    "planning": True,
    "filesystem": True,
    "subagents": [],
    "context_management": True,
}

print("AgentHarness components:")
for key, value in harness_config.items():
    print(f" {key}: {value}")
```

2. Planning Tools

The agent uses the `write_todos` tool to break complex work into a *structured task list*. Each task has a status:

Status	Description
pending	Not started yet
<code>in_progress</code>	Currently in progress
completed	Finished

```
# write_todos example – structured task list
todo_list = [
    {"task": "Analyze the project structure", "status": "completed"},
    {"task": "Design the API endpoints", "status": "in_progress"},
    {"task": "Write the database schema", "status": "pending"},
    {"task": "Write the tests", "status": "pending"},
    {"task": "Document the project", "status": "pending"},
]

print("=== Agent task list ===")
for i, item in enumerate(todo_list, 1):
    icon = {"completed": "[x]", "in_progress": "[-]", "pending": "[ ]"}
    print(f" {icon[item['status']]} {i}. {item['task']}")
```

3. Virtual Filesystem

The harness supports standard file operations through configurable filesystem backends.

Tool	Description
------	-------------

ls	List directory contents with metadata
read_file	Read file contents with line numbers and multimodal returns — images (PNG, JPG, GIF, WebP, HEIC), video (MP4, MOV, AVI), audio (WAV, MP3, AAC, FLAC), documents (PDF, PPT)
write_file	Create files
edit_file	Replace strings inside files
glob	Search for files by pattern
grep	Search file contents in different output modes
execute	Run shell commands (sandbox backends only)

```
# Example filesystem tool calls (reference only)
fs_operations = {
    "ls": 'ls(path="/project/src")',
    "read_file": 'read_file(path="/project/src/main.py")',
    "write_file": 'write_file(path="/project/config.yaml", content="debug: true")',
    "edit_file": 'edit_file(path="/project/src/main.py", old="v1", new="v2")',
    "glob": 'glob(pattern="**/*.py")',
    "grep": 'grep(pattern="TODO", path="/project/src")',
}

print("=== Filesystem tool call examples ===")
for tool_name, call_example in fs_operations.items():
    print(f" {tool_name:12s} -> {call_example}")
```

4. Task Delegation — Subagents

The harness allows the main agent to create *temporary subagents* for isolated multi-step work.

Advantages of Subagents

Advantage	Description
<i>Context isolation</i>	Subagent execution does not pollute the main context
<i>Parallel execution</i>	Multiple subagents can run at the same time
<i>Specialization</i>	Each subagent can get its own tools and prompt
<i>Token efficiency</i>	The main agent receives a compressed result

```
# Example subagent delegation configuration (reference only)
subagent_config = [
    {
        "name": "researcher",
        "description": "Investigates information using web search.",
        "system_prompt": "Summarize search results concisely.",
        "tools": ["internet_search"],
    },
    {
```

```

        "name": "coder",
        "description": "Writes and tests code.",
        "system_prompt": "Write clean and testable code.",
        "tools": ["write_file", "execute"],
    },
]

print("=== Subagent configuration ===")
for sa in subagent_config:
    print(f"  [{sa['name']}] {sa['description']}")
    print(f"    tools: {'', '.join(sa['tools'])}")

```

5. Context Management

The biggest challenge for long-running agents is the *context window limit*. The harness addresses it with two main techniques.

Input Context Assembly

The initial prompt is assembled from the system prompt, instructions, memory guidelines, skill information, and filesystem documentation.

Runtime Context Compression

Technique	Behavior	Trigger
<i>Offloading</i>	Stores content above a configurable threshold (default 20,000 tokens) on disk and keeps only pointers in context	Based on content size
<i>Summarization</i>	Compresses conversation history into a structured summary (session intent / artifacts / next steps). Triggered around 85% of <code>max_input_tokens</code> ; the most recent 10% of messages are preserved verbatim. Falls back to a 170k-token threshold when <code>max_input_tokens</code> is not set.	Triggered when the model window limit is approached

The original data is preserved in filesystem storage, so information is not lost.

```

# Example context-management settings (reference only)
context_config = {
    "offloading": {
        "enabled": True,
        "threshold_tokens": 20000,
        "storage": "filesystem",
    },
    "summarization": {
        "enabled": True,
        "trigger": "window_limit_approach",
        "preserve_original": True,
    },
}

```

```
print("=== Context management settings ===")
for section, settings in context_config.items():
    print(f"\n[{section}]")
    for key, value in settings.items():
        print(f"  {key}: {value}")
```

6. Code Execution

Sandbox backends expose the `execute` tool, which runs commands in an isolated environment. That improves safety, cleanliness, and reproducibility without affecting the host system.

Two execution paths: sandbox `execute` (shell) vs QuickJS `CodeInterpreterMiddleware`
The harness exposes two distinct execution paths. The *sandbox backend* contributes a shell-style `execute` tool, while the *interpreter middleware* contributes a QuickJS eval tool. They solve different problems.

Aspect	Sandbox <code>execute</code> (shell)	QuickJS <code>CodeInterpreterMiddleware</code>
Where it runs	Isolated environment outside the agent (Modal / Daytona / Runloop ...)	QuickJS VM inside the agent loop
Language	Shell commands → any language (Python, Node, ...)	JavaScript (QuickJS)
Network / filesystem	On by default (governed by policy)	Off by default — only bridged tools
Package install	<code>pip install</code> / <code>npm install</code> supported	Not supported
Best fit	Installing packages, running tests, processing large data	Composing tool calls, subagent fan-out, structured data transforms
How to enable	Swap the backend, e.g. <code>backend=ModalSandbox(...)</code>	Add <code>middleware=[CodeInterpreterMiddleware(...)]</code>

```
# Example sandboxed execute calls (reference only)
execute_examples = [
    {"command": "python -c 'print(2+2)'", "desc": "Run a Python snippet"},
    {"command": "pip install requests", "desc": "Install a package"},
    {"command": "pytest tests/", "desc": "Run the test suite"},
]

print("=== Sandbox execute tool examples ===")
for ex in execute_examples:
    print(f"  $ {ex['command']}")
    print(f"  -> {ex['desc']}")
```

7. Human-in-the-Loop

You can require human approval for selected tool calls through interrupt settings.

```
# Example Human-in-the-Loop configuration (reference only)
hitl_config = {
    "interrupt_on": {
        "write_file": True,
        "edit_file": True,
        "execute": True,
    }
}

print("=== Human-in-the-Loop configuration ===")
print("Tools that require approval:")
for tool, enabled in hitl_config["interrupt_on"].items():
    status = "approval required" if enabled else "automatic"
    print(f" {tool}: {status}")

print("\nDecision options: approve, reject, edit")
```

8. Skills and Memory

Skills

Skills are specialized workflows that follow the *Agent Skills standard*. They are loaded progressively when relevant, which reduces token usage.

- Each skill is defined in a `SKILL.md` file
- Skills are activated when the triggering conditions match
- They package tools, prompts, and workflows together

Memory

Memory uses `AGENTS.md`-style persistent context files. It stores reusable guidelines, preferences, and project knowledge beyond a single conversation.

Scope	Location	Range
Global memory	<code>~/.deepagents/<agent>/memories/</code>	All projects
Project memory	<code>.deepagents/AGENTS.md</code>	Current project

```
# Example skill and memory configuration (reference only)
skills_config = [
    {"name": "code-review", "trigger": "when the user asks for a code review"},
    {"name": "test-writer", "trigger": "when the user asks for tests"},
    {"name": "doc-generator", "trigger": "when the user asks for documentation"},
]

memory_config = {
    "global": "~/.deepagents/my-agent/memories/",
    "project": ".deepagents/AGENTS.md",
}

print("=== Skill configuration ===")
for skill in skills_config:
    print(f" [{skill['name']}] trigger: {skill['trigger']}")

print("\n=== Memory configuration ===")
```

```
for scope, path in memory_config.items():
    print(f" {scope}: {path}")
```

9. Harness Profiles

`HarnessProfile` packages provider/model-specific harness defaults without touching the `create_deep_agent()` call site. System prompt suffixes, tool description overrides, excluded tools and middleware, general-purpose subagent options, and extra middleware can all be registered as a profile.

`HarnessProfile` (7 fields)

Field	Meaning
<code>base_system_prompt</code>	Replace the base system prompt itself
<code>system_prompt_suffix</code>	Append text to the assembled base prompt
<code>tool_description_overrides</code>	Per-tool description overrides (mapping)
<code>excluded_tools</code>	Tools to remove by name after injection (set)
<code>excluded_middleware</code>	Middleware classes to remove (set)
<code>extra_middleware</code>	Additional middleware instances to attach
<code>general_purpose_subagent</code>	Enable, disable, or customize the general-purpose subagent via <code>GeneralPurposeSubagentProfile</code>

Merge semantics

When multiple profiles match:

- A later profile overrides earlier *scalar* fields (for example `base_system_prompt`).
- *Mapping*-style fields such as `tool_description_overrides` merge key by key.
- *Set*-style fields (`excluded_tools` , `excluded_middleware`) take the union.

`ProviderProfile` (3 fields)

`ProviderProfile` does not change harness behavior; it bundles initialization arguments for `init_chat_model()` — credential checks, provider default temperature, runtime headers, and so on.

Field	Meaning
<code>init_kwargs</code>	Default kwargs passed to <code>init_chat_model()</code> (for example <code>temperature</code>)
<code>credentials_check</code>	Callable run before model initialization to verify credentials
<code>runtime_headers</code>	HTTP header mapping attached to every request

YAML / JSON workflow — `HarnessProfileConfig.from_dict`

`HarnessProfileConfig` provides `from_dict()`, `to_dict()`, and `from_harness_profile()` class methods, so profiles can be managed as YAML and applied per environment without code changes.

```
# profile.yaml
base_system_prompt: You are helpful.
system_prompt_suffix: Respond briefly.
excluded_tools:
  - execute
excluded_middleware:
  - SummarizationMiddleware
general_purpose_subagent:
  enabled: false
```

```
import yaml
from deepagents import HarnessProfileConfig, register_harness_profile

with open("profile.yaml") as f:
    register_harness_profile(
        "openai:gpt-5.4",
        HarnessProfileConfig.from_dict(yaml.safe_load(f)),
    )
```

Entry-point plugins

Profiles can ship as packages via `pyproject.toml` entry points. The load order is *built-ins* → *entry-point plugins* → *user code's direct* `register_*_profile` calls.

```
[project.entry-points."deepagents.harness_profiles"]
"anthropic:claude-sonnet-4-6" = "my_pkg.profiles:claude_profile"

[project.entry-points."deepagents.provider_profiles"]
"openai" = "my_pkg.profiles:openai_provider"
```

Summary

Topic	Core Concept	Key API / Tool
Harness concept	Comprehensive capability provider for long-running agents	<code>create_deep_agent()</code>
Planning tools	Structured task list management	<code>write_todos</code>
Filesystem	Virtual and local file operations	<code>ls</code> , <code>read_file</code> , <code>write_file</code> , <code>edit_file</code> , <code>glob</code> , <code>grep</code> , <code>execute</code>
Subagents	Isolated task delegation, parallel execution	<code>subagents</code> , <code>task</code>
Context management	Offloading (20K tokens), summarization	automatic

Code execution	Safe command execution in sandboxes	<code>execute</code>
HITL	Human approval for sensitive tool calls	<code>interrupt_on</code>
Skills / Memory	Specialized workflows + persistent context	<code>SKILL.md</code> , <code>AGENTS.md</code>

Next Steps

→ Continue to [09_comparison.ipynb](#)

References:

- [Deep Agents Harness](#)

09 Comparing External Frameworks

Learning Objectives

- Understand the deeper differences between Deep Agents, LangGraph, and LangChain
- Compare them with OpenCode and the Claude Agent SDK
- Analyze differences in architecture, flexibility, and ecosystem
- Learn which framework to recommend for different use cases
- Understand migration considerations

```
# Environment setup
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY is not set!"
print("Environment setup complete")
```

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")

print(f"Model configured: {model.model_name}")
```

1. Comparison Overview

When choosing an AI agent framework, you need to consider several factors, including *model support*, *architecture*, *ecosystem*, and *license*.

In this notebook, you compare three major options:

- *LangChain Deep Agents* — a model-agnostic agent harness
 - *OpenCode* — a coding agent environment centered on the terminal / desktop / IDE
 - *Claude Agent SDK* — Anthropic’s SDK for Claude-based agents
-

2. Deep Agents vs OpenCode vs Claude Agent SDK

Basic Comparison

Feature	LangChain Deep Agents	OpenCode	Claude Agent SDK
<i>Model support</i>	Model-agnostic (Anthropic, OpenAI, 100+ providers)	75+ providers, including local models through Ollama	Claude-only
<i>License</i>	MIT	MIT	MIT (SDK), proprietary (Claude Code)
<i>SDK</i>	Python, TypeScript + CLI	Terminal, desktop, IDE integration	Python, TypeScript
<i>Sandboxing</i>	Can be integrated as a tool	Not supported	Not supported
<i>State management</i>	Supports time travel	Not supported	Supports time travel
<i>Observability</i>	Native LangSmith support	None	None

```
# Print a framework comparison table
frameworks = {
    "LangChain Deep Agents": {
        "model_support": "100+ providers (model-agnostic)",
        "license": "MIT",
        "sdk": "Python, TypeScript, CLI",
        "sandbox": "Integrated support",
        "time_travel": "Supported",
    },
    "OpenCode": {
        "model_support": "75+ providers (including local models)",
        "license": "MIT",
        "sdk": "Terminal, desktop, IDE",
        "sandbox": "Not supported",
        "time_travel": "Not supported",
    },
    "Claude Agent SDK": {
        "model_support": "Claude only",
        "license": "MIT (SDK)",
        "sdk": "Python, TypeScript",
        "sandbox": "Not supported",
        "time_travel": "Supported",
    },
}

print("=== Framework Comparison ===")
for name, features in frameworks.items():
    print(f"\n[{name}]")
    for key, value in features.items():
        print(f"  {key}: {value}")
```

3. Comparing Core Capabilities

Shared Capabilities

All three frameworks support the following categories:

- file operations (read, write, edit)
- shell command execution
- search features (`grep` , `glob`)
- planning support (task lists)
- Human-in-the-Loop (with different permission models)

Differentiating Capabilities

Capability	Deep Agents	OpenCode	Claude Agent SDK
<i>Core tools</i>	Files, shell, search, planning	Files, shell, search, planning	Files, shell, search, planning
<i>Sandbox integration</i>	Can be integrated as a tool	No	No
<i>Pluggable backends</i>	Storage + filesystem backends	No	No
<i>Virtual filesystem</i>	Yes, through pluggable backends	No	No
<i>Native tracing</i>	LangSmith	No	No

3-1. Three Core Differentiators

Look past the table cells and Deep Agents diverges from the other two along *three axes*. These are design decisions, not feature checks — figure out which axis matters most for your project and the choice gets easier.

Execution Model — graph-based state machine vs single-pass ReAct loop

Deep Agents runs on LangGraph’s `StateGraph` . It is assembled from nodes, edges, and subgraphs, with branching, looping, interrupts, and resumes all expressed in the graph itself. *Time travel* (branching from a past checkpoint), *interrupt/resume* (the standard HITL path), and *subgraph-scoped streaming* fall out of that design. OpenCode and Claude Agent SDK are ReAct-loop centric, so state branching and subgraph isolation cost extra integration work.

Deployment & Multi-Tenancy — LangSmith Deployments integration

Deep Agents deploys directly to *LangSmith Deployments* with built-in multi-tenancy via `assistant_id` / `thread_id` / `user_id` . Cron schedules (e.g. consolidation agents), the Store (persistent memory), and the Checkpointer are provisioned automatically at deploy time. OpenCode is a local terminal tool; Claude Agent SDK is closer to a model-calling SDK — both require you to build the surrounding infrastructure yourself.

Per-Model Configuration — `HarnessProfile` / `ProviderProfile`

Deep Agents 0.5+ packages provider/model defaults as *profiles*. The same codebase can switch between `openai:gpt-5.4` and `anthropic:claude-sonnet-4-6` without touching call sites. Claude Agent SDK is Claude-only with no model abstraction layer, and OpenCode supports 75+ providers but lacks a *profile* concept that captures the differences between them.

4. Architecture Comparison

LangChain Deep Agents

- *Pluggable storage backends* — state, filesystem, and store layers can be configured independently
- *Virtual filesystem* — can switch between local, in-memory, or sandboxed backends
- *LangGraph-based runtime* — supports complex workflows through graph execution
- *Middleware system* — fine-grained control over agent behavior

OpenCode

- *Terminal-native* — lightweight and fast to start
- *75+ model providers* — includes local models through Ollama
- *LSP integration* — optimized for code editing workflows

Claude Agent SDK

- *Claude-optimized* — tailored for Claude model capabilities
- *Time travel* — supports branching and state exploration
- *Concise API* — useful for fast prototyping

5. Recommendations by Use Case

Use Case	Recommended Framework	Why
Production agent app	<i>Deep Agents</i>	Pluggable backends, observability, sandbox support
Multi-model agent	<i>Deep Agents</i>	100+ provider support
Terminal coding assistant	<i>OpenCode</i>	Lightweight startup, strong local-model story
Claude-only app	<i>Claude Agent SDK</i>	Optimized for Claude, simple API
Rapid prototyping	<i>Claude Agent SDK</i>	Minimal API, fast setup
Complex multi-agent system	<i>Deep Agents</i>	Subagents and strong context management

Local model usage	<i>OpenCode</i>	Native Ollama support
-------------------	-----------------	-----------------------

```
# Helper to recommend a framework by use case
def recommend_framework(use_case: str) -> str:
    """Recommend a framework for a given use case."""
    recommendations = {
        "production": ("Deep Agents", "pluggable backends, observability, sandbox support"),
        "multi-model": ("Deep Agents", "100+ provider support"),
        "terminal": ("OpenCode", "fast startup and strong support for local models"),
        "claude-only": ("Claude Agent SDK", "Claude optimization and a concise API"),
        "prototyping": ("Claude Agent SDK", "simple API and fast setup"),
        "multi-agent": ("Deep Agents", "subagents and context management"),
        "local-model": ("OpenCode", "native Ollama support"),
    }
    if use_case in recommendations:
        fw, reason = recommendations[use_case]
        return f"{fw} – {reason}"
    return "No recommendation found for that use case."

# Demo
for case in ["production", "terminal", "claude-only", "multi-agent"]:
    print(f"{case}: {recommend_framework(case)}")
```

6. Ecosystem Comparison

Item	Deep Agents	OpenCode	Claude Agent SDK
<i>Community</i>	LangChain ecosystem (large)	GitHub community	Anthropic community
<i>Documentation</i>	Official docs + LangSmith integration	GitHub README	Anthropic official docs
<i>Integrations</i>	LangChain, LangGraph, LangSmith	LSP, terminal tooling	Claude API
<i>Package management</i>	pip / uv	go install / brew	pip / npm
<i>Editor integration</i>	ACP (Zed, JetBrains, VS Code, Neovim)	Own editor workflows	None

7. Migration Considerations

Important questions when migrating between frameworks:

Shared Considerations

- 1. *Model compatibility* – verify that the target framework supports the models you use
- 2. *Tool compatibility* – check how your custom tool interfaces need to be adapted
- 3. *State management* – plan how checkpoints and memory should be migrated

4. *Observability* — decide how tracing and logging should be replaced or preserved

Advantages of Migrating to Deep Agents

- *LangChain tool reuse* — existing LangChain tools can often be reused directly
- *LangGraph compatibility* — integrates well with LangGraph-based workflows
- *Incremental migration* — supports gradual adoption rather than all-at-once rewrites

Caveats

- Claude Agent SDK features that are Claude-specific may need alternative implementations
- Terminal UI logic from OpenCode may need to be separated from the core agent logic

Summary

Topic	Core Idea	Key API / Tool
Three-way comparison	Deep Agents, OpenCode, Claude Agent SDK	model support, license, SDK
Core capabilities	Shared tools + differentiators	sandboxing, pluggable backends
Architecture	Pluggable vs terminal-native vs model-optimized	LangGraph, LSP, Claude API
Use-case guidance	Production, terminal, prototyping, etc.	<code>recommend_framework()</code>
Ecosystem	Community, docs, integrations, editor support	LangSmith, ACP
Migration	Model / tool / state compatibility	gradual adoption

Next Steps

→ [10_sandboxes_and_acp.ipynb](#)

References:

- [Comparison with OpenCode and Claude Agent SDK](#)

10 Sandboxes and ACP

Learning Objectives

- Understand the concept of sandbox isolation and its security principles
- Compare sandbox providers such as E2B, Modal, and Docker-style environments
- Understand the overview and purpose of ACP (Agent Client Protocol)
- Understand editor-agent integration patterns
- Design an architecture that combines sandboxes and ACP

```
# Environment setup
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY is not set!"
print("Environment setup complete")
```

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")

print(f"Model configured: {model.model_name}")
```

1. Sandbox Concepts

A *sandbox* is an *isolated execution environment* where an AI agent can run code, manage files, and execute shell commands.

Why Isolation Matters

Risk	Without isolation	With a sandbox
Filesystem access	The host filesystem can be changed or deleted	Only the isolated filesystem is exposed
Network access	Unlimited external communication	Restricted network access
Credentials	Environment variables can leak	Secrets remain isolated
System impact	Can affect the host OS	Host system stays protected

In Deep Agents, a sandbox functions as a *backend* and exposes the filesystem tools (`ls` , `read_file` , `write_file` , `edit_file` , `glob` , `grep`) together with the `execute` tool.

2. Architecture Patterns

There are two major patterns for sandbox integration.

Agent-in-Sandbox

The agent itself runs *inside* the sandbox and communicates with the outside world over a network protocol.

Advantages	Drawbacks
Similar to a normal development environment	Higher risk of credential exposure
Simple setup	More infrastructure complexity

Sandbox-as-Tool (Recommended)

The agent runs *outside* the sandbox and calls sandbox APIs to execute code.

Advantages	Drawbacks
Clean separation between agent state and execution environment	Network latency
Keeps secrets outside the sandbox	
Makes parallel task execution easier	

```
# Compare the two architecture patterns (reference only)
print("=== Pattern 1: Agent-in-Sandbox ===")
print("  [Sandbox]")
print("    |-- agent (running inside)")
print("    |-- filesystem")
print("    |-- code execution")
print("    <--> network protocol <--> external systems")

print()
print("=== Pattern 2: Sandbox-as-Tool (recommended) ===")
print("  [Host]")
print("    |-- agent (running outside)")
print("    |-- credential management")
print("    |-- API call --> [Sandbox]")
print("                          |-- filesystem")
print("                          |-- code execution")
```

3. Comparing Sandbox Providers

Package / Provider	Characteristics	Best Fit
--------------------	-----------------	----------

<code>langchain-modal</code> / Modal	GPU support, ML workloads	AI / ML tasks, data processing
<code>langchain-daytona</code> / Daytona	TypeScript / Python support, fast cold starts	Web development, rapid iteration
<code>langchain-runloop</code> / Runloop	Disposable devboxes, isolated execution	Code testing, one-off tasks
<code>langsmith[sandbox]</code> / LangSmith	LangSmith Deployments integration (private beta)	Operations on top of LangSmith
<code>langchain-agentcore-codeinterpreter</code> / AgentCore	AWS Bedrock-backed code interpreter	AWS-native deployments

```
# Modal sandbox – canonical pattern via langchain-modal
# pip install langchain-modal deepagents
import modal
from deepagents import create_deep_agent
from langchain_anthropic import ChatAnthropic
from langchain_modal import ModalSandbox

app = modal.App.lookup("your-app")
modal_sandbox = modal.Sandbox.create(app=app)
backend = ModalSandbox(sandbox=modal_sandbox)

agent = create_deep_agent(
    model=ChatAnthropic(model="claude-sonnet-4-6"),
    system_prompt="You are a Python coding assistant with sandbox access.",
    backend=backend,
)

try:
    result = agent.invoke({
        "messages": [{"role": "user", "content": "Create a small Python package and run pytest"}],
    })
finally:
    modal_sandbox.terminate() # required: free resources
```

4. Security Guidelines

Never Put Secrets Inside the Sandbox

If credentials are stored in environment variables or mounted files inside the sandbox, an agent can read and leak them.

Safe Practices

1. *Manage credentials only through external tools*
2. *Use Human-in-the-Loop* for sensitive operations
3. *Block unnecessary network access*
4. *Monitor outbound activity*
5. *Review sandbox outputs* before applying them back to the main application

5. File Transfer and Lifecycle Management

Ways to Access Files

Method	Description
Agent filesystem tools	Direct file operations through <code>execute()</code> and the backend
File transfer APIs	Manage seed files and artifacts through <code>uploadFiles()</code> / <code>downloadFiles()</code>

Lifecycle Management

To avoid unnecessary cost, sandboxes need *explicit shutdown*. There are two operational modes.

Mode	Lifetime	Best fit
<i>Thread-scoped (default)</i>	One sandbox per conversation thread. Created on the first run, reused on subsequent turns of the same thread, and cleaned up via an idle TTL.	Chat-style agents and conversational sessions
<i>Assistant-scoped</i>	All threads of the same assistant share one sandbox. Files, packages, and repositories accumulate across conversations. Always set a TTL or periodic snapshot policy.	Long-running coding sessions, accumulated workspaces

Operational checklist: for one-off scripts, use `try/finally` and terminate immediately. For chat / long sessions, use a per-thread sandbox with TTL-based cleanup. On LangSmith Deployments, wire termination into the session-end hook.

```
# Example lifecycle and file-transfer settings (reference only)
lifecycle_config = {
    "ttl_seconds": 1800,
    "auto_shutdown": True,
    "thread_isolation": True,
}

file_operations = [
    "uploadFiles(['/local/data.csv'], '/sandbox/data/')",
    "downloadFiles(['/sandbox/output/result.json'], '/local/results/')",
]

print("=== Lifecycle configuration ===")
for key, value in lifecycle_config.items():
    print(f" {key}: {value}")

print("\n=== File transfer examples ===")
for op in file_operations:
    print(f" {op}")
```

6. ACP Overview

ACP (*Agent Client Protocol*) standardizes communication between coding agents and development environments such as editors and IDEs.

MCP vs ACP

Protocol	Purpose	Target
MCP (Model Context Protocol)	External tool integration	agent ↔ external service
ACP (Agent Client Protocol)	Editor-agent integration	agent ↔ editor / IDE

ACP allows agents to interact with editors directly for code editing, file navigation, and terminal operations.

7. ACP Server Implementation

```
# ACP server – canonical pattern (asyncio + acp.run_agent)
# pip install deepagents-acp
import asyncio

from acp import run_agent
from deepagents import create_deep_agent
from langgraph.checkpoint.memory import MemorySaver

from deepagents_acp.server import AgentServerACP

async def main() -> None:
    agent = create_deep_agent(
        model="anthropic:claude-sonnet-4-6",
        system_prompt="You are a helpful coding assistant",
        checkpointer=MemorySaver(),
    )
    server = AgentServerACP(agent)
    await run_agent(server)

if __name__ == "__main__":
    asyncio.run(main())
```

8. Editors That Support ACP

Editor	Integration Style
<i>Zed</i>	Native integration
<i>JetBrains IDEs</i>	Built-in support
<i>Visual Studio Code</i>	vscode-acp plugin

Neovim

ACP-compatible plugin

Example Zed Configuration

```
// Zed settings.json
{
  "agent_servers": [
    {
      "command": "python",
      "args": ["acp_server.py"],
      "env": {
        "ANTHROPIC_API_KEY": "sk-..."
      }
    }
  ]
}
```

Toad CLI

Toad is a process manager for running ACP servers as local development tools. It handles start, stop, and restart automatically so you do not have to manage processes by hand.

```
# Install
uv tool install -U batrachian-toad

# Run – pass the ACP server command and the working directory
toad acp "python path/to/your_server.py" .
```

9. Combining Sandboxes and ACP

If you combine sandboxes with ACP, you get a *complete architecture* in which the editor controls the agent while code execution happens in an isolated environment.

Integrated Architecture

[Editor / IDE]	<-- ACP -->	[Agent]	<-- API -->	[Sandbox]
code editing		task management		code execution
file browsing		context management		file isolation
terminal UI		tool calls		secure runtime

Advantages

- interact with the agent directly from the editor
- run code safely in a sandbox
- keep secrets only on the host side

```
# Sandbox + ACP integration – canonical pattern
import asyncio

import modal
```

```
from acp import run_agent
from deepagents import create_deep_agent
from langchain_anthropic import ChatAnthropic
from langchain_modal import ModalSandbox
from langgraph.checkpoint.memory import MemorySaver

from deepagents_acp.server import AgentServerACP

async def main() -> None:
    app = modal.App.lookup("your-app")
    modal_sandbox = modal.Sandbox.create(app=app)
    try:
        agent = create_deep_agent(
            model=ChatAnthropic(model="claude-sonnet-4-6"),
            system_prompt="You are a coding assistant.",
            backend=ModalSandbox(sandbox=modal_sandbox),
            checkpoint=MemorySaver(),
            interrupt_on={"execute": True}, # HITL approval before code execution
        )
        server = AgentServerACP(agent)
        await run_agent(server)
    finally:
        modal_sandbox.terminate()

if __name__ == "__main__":
    asyncio.run(main())
```

Summary

Topic	Core Concept	Key API / Tool
Sandbox concept	Isolated execution that protects the host system	<code>execute</code> , filesystem tools
Architecture patterns	Agent-in-Sandbox vs Sandbox-as-Tool	Sandbox-as-Tool recommended
Providers	Modal (GPU), Daytona (fast startup), Runloop (disposable)	<code>ModalSandbox</code>
Security	External secret management, HITL, network controls	<code>interrupt_on</code>
ACP overview	Standardized editor-agent communication	<code>AgentServerACP</code>
ACP server	Expose the agent in stdio mode	<code>deepagents-acp</code>
Editor integration	Zed, JetBrains, VS Code, Neovim	ACP protocol
Integrated pattern	editor ↔ agent ↔ sandbox	ACP + sandbox

Next Steps

→ Continue to the *advanced track* at [../05_advanced/00_migration.ipynb](#)

References:

- Sandboxes
- Agent Client Protocol (ACP)

11 Async Subagents

AsyncSubAgent · Non-blocking · Mid-flight steering

The flagship feature of Deep Agents 0.5.0, `AsyncSubAgentMiddleware`, lets the supervisor spawn subagents in the background without blocking. It is the infrastructure that pushes past the limits of the old synchronous subagent for long-running work — multi-minute to multi-hour research, coding, and parallel subagent orchestration. One caveat: this feature requires an *Agent Protocol server* — it only runs on top of LangSmith Deployments or a self-hosted setup such as `langgraph dev`.

Learning Objectives

LEARNING OBJECTIVES

- Explain the execution-model differences between a synchronous subagent and `AsyncSubAgent`
- Distinguish the five tools injected by the middleware: `start_async_task` / `check_async_task` / `update_async_task` / `cancel_async_task` / `list_async_tasks`
- Compare ASGI (co-deploy) and HTTP (remote) transport modes and build hybrid configurations
- Understand how the `async_tasks` state channel preserves state across context compaction
- Know mid-flight steering patterns (update/cancel) and how to tune `--n-jobs-per-worker` `slots`

11.1 Limitations of the old synchronous subagent

The original subagent was *synchronous*. When the `task` tool was invoked, the supervisor stalled until the subagent finished, and the user could not issue a new instruction in the meantime. Running a multi-minute research task synchronously froze the frontend for a long stretch with no way for the user to check whether the agent was still alive.

The 0.5.0 `AsyncSubAgentMiddleware` removes this constraint.

- **Non-blocking execution:** `start_async_task` returns a task id immediately. The supervisor keeps talking to the user.
- **Mid-flight steering:** You can send follow-up instructions to a running subagent or cancel it.
- **Independent threads:** Each subagent task has its own thread and run. State is not lost even when the supervisor's context is compacted.

11.2 Five tools injected into the supervisor

`AsyncSubAgentMiddleware` injects five tools into the supervisor.

Tool	Role
<code>start_async_task</code>	Launch a subagent in the background. Returns the task id immediately
<code>check_async_task</code>	Query current status; extract final output if the task completed
<code>update_async_task</code>	Inject a new instruction into the same thread of a running task (interrupt strategy)
<code>cancel_async_task</code>	Send a cancel signal to the server and mark the task as <code>cancelled</code>
<code>list_async_tasks</code>	Batch-query the current status of all tracked tasks

11.3 Basic usage

Pass a list of `AsyncSubAgent` specs to `subagents` and `create_deep_agent` attaches `AsyncSubAgentMiddleware` automatically.

```
from deepagents import AsyncSubAgent, create_deep_agent

async_subagents = [
    AsyncSubAgent(
        name="researcher",
        description="Research work that needs information gathering and synthesis",
        graph_id="researcher",
    ),
    AsyncSubAgent(
        name="coder",
        description="Code generation / review work",
        graph_id="coder",
        url="https://coder-deployment.langsmith.dev", # remote HTTP call
    ),
]

agent = create_deep_agent(
    model="google_genai:gemin-3.5-flash",
    subagents=async_subagents,
)
```

Key fields on `AsyncSubAgent`

Field	Description
<code>name</code>	Unique identifier the supervisor references
<code>description</code>	The basis the supervisor uses to decide what to delegate. As important as it is for synchronous subagents
<code>graph_id</code>	Must match a graph name in <code>langgraph.json</code>
<code>url</code>	Optional. Without it, ASGI (in-process); with it, remote HTTP call
<code>headers</code>	Optional. Auth headers for a self-hosted server

11.4 The `async_tasks` state channel

Task metadata is stored in the `async_tasks` state channel, *separate* from the message history. Each record contains:

- task id
- agent name
- thread id, run id
- status (`pending` / `running` / `success` / `error` / `cancelled`)
- `created_at` , `updated_at`

This separation means that even when the supervisor's context is compacted (summarization) and older messages are replaced by summaries, *task ids are not lost*. `check_async_task` / `update_async_task` still work after compaction.

! WARNING

If a custom state reducer or middleware reshapes state and overwrites the `async_tasks` channel, you lose tracking for running tasks. Make the merge strategy explicit.

11.5 Transport modes

ASGI (co-deploy, the recommended default)

If you omit `url`, the subagent runs in the *same process* as the supervisor, like a function call. Zero network latency; register both graphs in the same `langgraph.json`.

```
// langgraph.json
{
  "dependencies": [".",],
  "graphs": {
    "supervisor": "./agent.py:supervisor",
    "researcher": "./subagents/researcher.py:graph",
    "coder": "./subagents/coder.py:graph"
  },
  "env": ".env"
}
```

```
AsyncSubAgent(
  name="researcher",
  description="...",
  graph_id="researcher",  # no url → ASGI
)
```

HTTP (remote)

Call an independently deployed subagent by `url`. Use this when you need to scale subagents with *different resource profiles* separately — for example, a low-cost model deployment dedicated to research and a GPU deployment dedicated to coding.

```
import os

AsyncSubAgent(
    name="coder",
    description="...",
    graph_id="coder",
    url="https://coder-deployment.langsmith.dev",
    headers={"x-api-key": os.environ["CODER_API_KEY"]},
)
```

Hybrid

You can mix the two. Lightweight subagents via ASGI, resource-intensive subagents via remote HTTP — split the deployment topology along the workload.

11.6 Execution lifecycle

1. **Launch** — `start_async_task` creates a new thread, starts a run, and returns the task id immediately
2. **Check** — `check_async_task` queries status and extracts the final output when complete
3. **Update** — `update_async_task` preserves history in the existing thread and starts a new run triggered by an interrupt carrying the new instruction
4. **Cancel** — `cancel_async_task` calls `runs.cancel()` and marks the task as cancelled
5. **List** — live-status queries for non-terminal tasks in parallel; cached responses for terminal ones

11.7 Mid-flight steering patterns

When the user changes direction mid-conversation, the supervisor calls `update_async_task`.

```
User: Start competitor research.
Supervisor: [start_async_task(agent="researcher", ...)] → task_abc123

User: Actually, only Series A and above.
Supervisor: [update_async_task(task_id="task_abc123",
                             instruction="Narrow the scope to Series A and above")]

User: Stop, redirect to the SaaS segment instead.
Supervisor: [cancel_async_task(task_id="task_abc123")]
            [start_async_task(agent="researcher",
                             description="SaaS competitor research")]
```

Because `update_async_task` creates a new run on the *same thread*, the subagent carries its prior exploration forward and simply layers on the new instruction. Use cancel + start only when you truly want a fresh task.

11.8 Local development: `--n-jobs-per-worker`

The default worker pool for `langgraph dev` is small, so spawning concurrent subagents causes queuing. Raise the slot count.

```
langgraph dev --n-jobs-per-worker 10
```

A supervisor running three concurrent subagents needs at least **4 slots** (1 supervisor + 3 subagents). 10–20 gives comfortable headroom.

11.9 Observing lifecycle through the unified v2 stream

Pending/Running/Complete signals for async subagents do not need a separate v3 projection. Use the single

`agent.stream(...)` path with `subgraphs=True` + `version="v2"` to unify main and subagent events. Changes to the `async_tasks` channel flow through the `updates` mode `data`.

```
async for chunk in agent.astream(
    {"messages": [{"role": "user", "content": "Run research on 3 competitors in parallel"}]},
    stream_mode=["updates", "messages", "custom"],
    subgraphs=True,
    version="v2",
):
    t = chunk["type"]
    if t == "updates":
        # async_tasks channel changes, check_async_task / list_async_tasks results
        for node, data in chunk["data"].items():
            print(f"step={node}")
    elif t == "messages":
        # Supervisor tokens + tool_call_chunks
        ...
    elif t == "custom":
        # In-tool progress via get_stream_writer
        ...
```

Signal	Detection point
Pending	Supervisor emits a <code>start_async_task</code> <code>tool_call</code>
Running	<code>updates</code> chunk where <code>async_tasks[task_id].status</code> becomes <code>running</code>
Complete	<code>updates</code> chunk where <code>status</code> settles to <code>success</code> / <code>error</code> / <code>cancelled</code>

Map the three signals to UI card states and the lifecycle flows without separate polling. See Chapter 14 (Streaming) for the full stream dispatch.

11.10 Sync vs Async selection criteria

Axis	Sync SubAgent	AsyncSubAgent
Execution	Supervisor blocks, waits for completion	Returns task id immediately, supervisor continues
Result retrieval	Automatically returned to the supervisor	Poll with <code>check_async_task</code>
Mid-task instructions	Not supported	Supported via <code>update_async_task</code>
Cancellation	Not supported	Supported via <code>cancel_async_task</code>

State preservation	Stateless (one-shot)	State kept in its own thread, interactive
Infrastructure requirement	No special requirements	Requires Agent Protocol server
Suited for	Seconds to tens of seconds, result-only workloads	Minute-to-hour tasks with room for intervention

Decision flow

- Task finishes in seconds and the supervisor needs the result immediately → **Sync**
- Task takes multiple minutes, or you want *parallel* subagents → **Async**
- User might redirect or cancel mid-flight → **Async**
- You cannot run LangSmith Deployments or an Agent Protocol server → **Sync only**

11.11 Caveats

- **Agent Protocol dependency:** Declaring `AsyncSubAgent` in a runtime without Agent Protocol support fails at initialization
- **Preserve the `async_tasks` channel:** Custom state reducers or middleware that reshape state must not overwrite it
- **Polling cost:** Instruct the system prompt to avoid calling `check_async_task` too frequently (for example, “spawn 2–3 tasks at a time, then wait for user feedback”)
- **Long-running task cleanup:** Periodically sweep long unfinished tasks with `list_async_tasks` + `cancel_async_task`. LangSmith Deployments retain checkpoint-based cost
- **HTTP mode timeouts:** For remote URLs, verify headers, network timeout, and retry policy separately

Key Takeaways

- `AsyncSubAgent` is the 0.5.0 flagship feature that spawns background subagents without blocking the supervisor
- Five tools (`start` / `check` / `update` / `cancel` / `list`) drive the lifecycle
- ASGI is the default for co-deploy; HTTP is used for resource separation; hybrid is possible
- The `async_tasks` state channel preserves task tracking across context compaction
- A good fit for multi-minute work, tasks that benefit from user intervention, and multiple parallel subagents

12 Going to Production

Ten areas anchored on Thread · User · Assistant

This chapter covers the ten areas to work through when turning a local prototype into a multi-user, multi-tenant, long-running production agent. Treat it as a checklist just before shipping a Deep Agent to a live service, or when you are cleaning up operational issues on one already deployed.

Learning Objectives

LEARNING OBJECTIVES

- Understand the three abstractions — Thread, User, Assistant — as the starting point for production design
- Know the infrastructure auto-provisioned by `langgraph.json` + LangSmith Deployments
- Distinguish multi-tenant authz, end-user credentials, and memory scoping
- Pick between thread-scoped and assistant-scoped sandbox patterns
- Assemble durability, rate limits, three-tier error handling, PII, and the real-time frontend via middleware and SDKs

12.1 Three core abstractions

Production Deep Agents operate on three core abstractions.

- **Thread** — a single conversation. The unit for messages, files, and checkpoints
- **User** — an authenticated identity. The unit for resource ownership and access scope
- **Assistant** — a configured agent instance (prompt · tools · model combination)

Ten areas of concern sit on top of these three concepts. Each area can be adopted independently, picked up according to your risk profile.

12.2 LangSmith Deployments

Deploying via the `deepagents deploy` CLI or a LangSmith Deployment auto-provisions the following infrastructure.

- Assistants / Threads / Runs APIs
- Store + Checkpointer (persistence)

- Authentication, webhooks, cron, observability
- MCP / A2A exposure options

```
// minimal langgraph.json
{
  "dependencies": ["."],
  "graphs": {
    "agent": "./agent.py:agent"
  },
  "env": ".env"
}
```

12.3 Multi-tenant access control

Custom auth / authz handlers

LangSmith Deployments establish user identity via custom authentication, and a separate authorization handler controls access to threads / assistants / store namespaces. Handlers can tag resources with ownership metadata, filter visibility per user, and return HTTP 403 for denied access.

Workspace RBAC

Team-level permissions (a LangSmith built-in feature).

Role	Permissions
Workspace Admin	Full permissions
Workspace Editor	Create / edit; cannot delete or manage members
Workspace Viewer	Read-only

12.4 End-user credential management

When the agent has to call external services on the user's behalf (GitHub, Slack, Gmail, etc.), manage credentials *outside the agent code*.

Agent Auth (OAuth 2.0)

A managed OAuth flow. On the first call the user is shown a consent URL as an interrupt; once the token is received the flow auto-resumes and refreshes.

```
from langchain_auth import Client
from langchain.tools import tool, ToolRuntime

auth_client = Client()

@tool
async def github_action(runtime: ToolRuntime):
    """Perform a GitHub action on the user's behalf."""
    auth_result = await auth_client.authenticate()
```

```

provider="github",
scopes=["repo", "read:org"],
user_id=runtime.server_info.user.identity,
)
# Call the GitHub API with auth_result.token

```

Sandbox Auth Proxy

When user code (or agent-generated code) running in a sandbox calls external APIs, the proxy *injects* credentials. API keys are never exposed to the code inside the sandbox.

```

{
  "proxy_config": {
    "rules": [
      {
        "name": "openai-api",
        "match_hosts": ["api.openai.com"],
        "inject_headers": {
          "Authorization": "Bearer ${OPENAI_API_KEY}"
        }
      }
    ]
  }
}

```

`${SECRET_KEY}` is resolved from the workspace secrets.

12.5 Memory persistence scoping

The `StoreBackend` namespace function determines the memory scope. The default pattern routes only `/memories/` through `StoreBackend` while keeping everything else on the ephemeral `StateBackend` via `CompositeBackend`.

User-scoped (recommended default)

```

from deepagents import create_deep_agent
from deepagents.backends import CompositeBackend, StateBackend, StoreBackend

agent = create_deep_agent(
    model="google_genai:gemin-3.5-flash",
    backend=CompositeBackend(
        default=StateBackend(),
        routes={
            "/memories/": StoreBackend(
                namespace=lambda rt: (
                    rt.server_info.assistant_id,
                    rt.server_info.user.identity,
                ),
            ),
        },
    ),
    system_prompt=(
        "At the start of every conversation, read /memories/instructions.txt. "
        "Update it with durable insights."
    ),
)

```


Assistant-scoped / Organization-scoped

Memory can be shared by all users of the same assistant (assistant-scoped) or across the whole organization (org-scoped). *Always keep org-shared memory read-only.*

! WARNING

Prompt-injection warning: Shared memory is a prompt-injection vector. Do not grant write permission to surfaces that a user can manipulate. For detailed policy, see Part IV ch15 (Permissions).

12.6 Execution isolation — Sandbox

Do not expose the host filesystem or network directly; route through a *sandbox*.

Thread-scoped sandbox (most common pattern)

```
from daytona import CreateSandboxFromSnapshotParams, Daytona
from deepagents import create_deep_agent
from langchain_core.runnables import RunnableConfig
from langchain_daytona import DaytonaSandbox

client = Daytona()

async def agent(config: RunnableConfig):
    thread_id = config["configurable"]["thread_id"]
    try:
        sandbox = await client.find_one(labels={"thread_id": thread_id})
    except Exception:
        sandbox = await client.create(
            CreateSandboxFromSnapshotParams(
                labels={"thread_id": thread_id},
                auto_delete_interval=3600, # TTL
            )
        )
    return create_deep_agent(
        model="google_genai:gemin-3.5-flash",
        backend=DaytonaSandbox(sandbox=sandbox),
    )
```

Assistant-scoped sandbox

Use when every thread should share one sandbox so tool-chain caches and installed binaries are preserved.

12.7 Durability · Async I/O

Checkpoint on every step

LangSmith Deployments attach a checkpointer automatically. State is saved on every step, which gives you:

- **Indefinite interrupt:** A HITL approval can sit for *days* and still resume exactly where it paused
- **Time travel:** Rewind to an arbitrary checkpoint and branch
- **Audit trail:** State audit just before sensitive operations like payments or admin actions

```
await agent.ainvoke(
    {"messages": [...]},
    config={"configurable": {"thread_id": "thread-abc"}},
)
```

Async I/O

LLM apps are I/O-bound. Using async tools and async middleware hooks (`abefore_agent` , `astream`) significantly increases throughput.

12.8 Rate limits and cost control

```
from deepagents import create_deep_agent
from langchain.agents.middleware import (
    ModelCallLimitMiddleware,
    ToolCallLimitMiddleware,
)

agent = create_deep_agent(
    model="google_genai:gemin-3.5-flash",
    middleware=[
        ModelCallLimitMiddleware(run_limit=50),
        ToolCallLimitMiddleware(run_limit=200),
    ],
)
```

`run_limit` resets for every `invoke` ; `thread_limit` accumulates over the thread's lifetime. Cut off runaway loops before they become cost bombs.

12.9 Three-tier error handling

Category	Examples	Strategy	Middleware
Transient	Timeouts, rate limits, transient network failures	Auto-retry with backoff	<code>ModelRetryMiddleware</code> , <code>ToolRetryMiddleware</code>
Recoverable	Bad tool arguments, parse failures	Feed back to the model → retry	Tool-wrapper error messages
Human required	Missing permissions, ambiguous request	Pause the agent	HITL <code>interrupt_on</code>

```
from langchain.agents.middleware import (
    ModelRetryMiddleware,
    ModelFallbackMiddleware,
    ToolRetryMiddleware,
)

agent = create_deep_agent(
    model="google_genai:gemin-3.5-flash",
    middleware=[
        ModelRetryMiddleware(max_retries=3, backoff_factor=2.0, initial_delay=1.0),
    ],
)
```

```

    ModelFallbackMiddleware("gpt-5.4"),
    ToolRetryMiddleware(
      max_retries=2,
      tools=["search", "fetch_url"],
      retry_on=(TimeoutError, ConnectionError),
    ),
  ],
)

```

12.10 Data privacy and real-time frontend

PIIMiddleware

Process PII — email, card numbers, national IDs — at the input and output boundaries. Strategies include `redact` (remove), `mask` (mask), `hash` (hash), and `block` (block); custom detectors can also be registered. The key is to process PII at the input side before it enters anything you log. LangSmith traces also record masked content only.

`useStream` hook

The `useStream` hook from `@langchain/react` handles real-time streaming, reconnection, and history loading in one place.

```

import { useStream } from "@langchain/react";

function App() {
  const stream = useStream<typeof agent>({
    apiUrl: "https://your-deployment.langsmith.dev",
    assistantId: "agent",
    reconnectOnMount: true,
    fetchStateHistory: true,
  });
}

```

Deep Agents that spawn many subagents also stream subgraph events so the UI can render subagent progress cards (`streamSubgraphs: true`).

12.11 Production checklist

- [] Graphs and env are declared in `langgraph.json`
- [] Per-user authz handlers are applied to threads and the store
- [] External-service tokens are managed via Agent Auth / Proxy and never hardcoded
- [] `/memories/` is routed to `StoreBackend` with an explicit scope (user/assistant/org)
- [] Shared memory is read-only or write access is tightly restricted
- [] Host filesystem and shell are not exposed directly; access goes through a sandbox
- [] Cost ceilings are set via `ModelCallLimitMiddleware` / `ToolCallLimitMiddleware`
- [] Retry, fallback, and HITL are all configured
- [] PII is masked at input/output boundaries
- [] The frontend uses `reconnectOnMount` + `fetchStateHistory`

Key Takeaways

- Roll out the ten operational areas independently on top of Thread · User · Assistant
- `langgraph.json` + LangSmith Deployments auto-provision checkpoint, Store, auth, and observability
- Memory scope defaults to user-scoped; shared scopes must be read-only
- Classify errors into three tiers (Transient / Recoverable / Human) and respond with middleware plus HITL
- Defend PII in two layers: at the model-input stage and at the trace-transmission stage

13 Context Engineering

Write big, read selective

This chapter pulls together the full design of how a Deep Agent decides *what* to show the model inside its limited context window, *when* to offload, and *how* to bring things back. Use it when you are tuning the drift in quality over long sessions, or when you want to organize subagents / skills / memory into a single pipeline.

Learning Objectives

LEARNING OBJECTIVES

- Understand the four-layer architecture of context engineering (Layered / Dynamic / Compression / Retrieval)
- Know the nine-step automatic synthesis order of the system prompt
- Distinguish the automatic offloading (>20k tokens) and automatic summarization (85%) triggers
- Propagate runtime context via `@dynamic_prompt` and `context_schema`
- Prevent context pollution with subagent isolation and `/memories/` routing

13.1 The four-layer architecture

Deep Agents' context engineering stacks in four layers.

1. **Layered input context** — system prompt, `AGENTS.md`, `SKILL.md`, and tool descriptions are synthesized in a fixed order
2. **Dynamic / runtime context** — `@dynamic_prompt` and `ToolRuntime` inject request-time data
3. **Automatic compression** — offloading (>20k tokens → disk), summarization (85% → structured summary)
4. **Filesystem-centric retrieval** — selective re-injection via `read_file` / `grep` / subagent isolation

Each layer can be toggled independently, but the defaults alone handle roughly 90% of long-session cases out of the box.

13.2 Layered input context — nine-step system prompt synthesis

The system prompt is synthesized automatically in this order.

1. Custom system prompt (user-supplied instructions)
2. Base agent prompt (planning / filesystem / subagent base instructions)
3. To-do list prompt
4. Memory prompt (`AGENTS.md` , always loaded when configured)
5. Skills prompt (only skill locations and frontmatter)
6. Filesystem prompt (tool documentation)
7. Subagent prompt (delegation guidance)
8. Middleware prompts (custom additions)
9. Human-in-the-loop prompt

AGENTS.md: permanent context loaded every time

Put content that must apply to *every* conversation — project conventions, user preferences.

```
from deepagents import create_deep_agent

agent = create_deep_agent(
    model="google_genai:gemin-3.5-flash",
    memory=["/project/AGENTS.md", "~/deepagents/preferences.md"],
)
```

Because it is always loaded, keep it minimal. Move detailed workflows into `SKILL.md`.

13.3 @dynamic_prompt — request-time data injection

Use this when a static string is not enough and the instruction has to change based on *request-time data* (user role, organization id, current time, store contents). The middleware reads `request.runtime.context` and `request.runtime.store` to build the dynamic prompt. The dynamic prompt is decided at the *middleware layer*; tools receive runtime values through `ToolRuntime` with no extra wiring.

13.4 Progressive skill loading

Skills load in two phases.

- **Startup**: only the frontmatter (name · description) of `SKILL.md` is read → a few hundred tokens
- **On demand**: the full body is loaded only when a user request matches the skill

```
agent = create_deep_agent(
    model="google_genai:gemin-3.5-flash",
    skills=["/skills/research/", "/skills/web-search/"],
)
```

Even with dozens of skills registered, the initial token cost is just the frontmatter.

13.5 Runtime context propagation

Context declared via `context_schema` is automatically forwarded not only to the agent but to *every subagent and tool*.

```

from dataclasses import dataclass
from deepagents import create_deep_agent
from langchain.tools import tool, ToolRuntime

@dataclass
class Context:
    user_id: str
    api_key: str

@tool
def fetch_user_data(query: str, runtime: ToolRuntime[Context]) -> str:
    """Look up data for the current user."""
    user_id = runtime.context.user_id
    return f>Data for user {user_id}: {query}

agent = create_deep_agent(
    model="google_genai:gemini-3.5-flash",
    tools=[fetch_user_data],
    context_schema=Context,
)

result = agent.invoke(
    {"messages": [{"role": "user", "content": "Get my recent activity"}]},
    context=Context(user_id="user-123", api_key="sk-..."),
)

```

Thanks to this structure, tools can read runtime values without loading API keys into the message history, and subagents automatically inherit the same values.

TIP

Tool doc quality is context. The model picks tools purely from their *description*. Writing concrete docstrings + Args sections reduces wasted tokens. Enable automatic Args parsing with `@tool(parse_docstring=True)` .

13.6 Automatic offloading (>20k tokens)

As context grows, two offloading behaviors kick in automatically.

- **Input offloading:** Results of large-file `write` / `edit` operations are replaced with a *file pointer reference* instead of their actual content once context exceeds 85% capacity
- **Result offloading:** Tool responses over *20,000 tokens* are written to storage; the context retains only a *file path + first 10-line preview*

Large results go to disk first and are pulled back selectively with `read_file` / `grep` when needed. This is the core rationale for the filesystem-centric design.

13.7 Automatic summarization (85% trigger)

When there is nothing left to offload but context is still large, *structured summarization* kicks in. An LLM compresses the current conversation into three elements: Session intent, Artifacts created, Next steps. This summary *replaces* the existing conversation history and the agent continues.

Item	Default
------	---------

Trigger	85% of the model's <code>max_input_tokens</code>
Retained	10%
Fallback	Triggers at 170,000 tokens, retains the last 6 messages
Immediate trigger	On <code>ContextOverflowError</code>

Manual summarization tool

Beyond the automatic trigger, add middleware to let the agent invoke summarization explicitly.

```
from deepagents import create_deep_agent
from deepagents.backends import StateBackend
from deepagents.middleware.summarization import create_summarization_tool_middleware

backend = StateBackend
model = "google_genai:gemin-3.5-flash"

agent = create_deep_agent(
    model=model,
    middleware=[create_summarization_tool_middleware(model, backend)],
)
```

When streaming, you can filter summarization tokens out of the UI (`metadata.get("lc_source") == "summarization"`).

13.8 Subagent isolation

Subagents run in *their own context* and return only *one final report* to the supervisor. Intermediate tool calls and search results stay in the subagent's context, keeping the supervisor window clean.

```
research_subagent = {
    "name": "researcher",
    "description": "Run research on a specific topic",
    "system_prompt": (
        "You are a research assistant.\n"
        "IMPORTANT: Return only the essential summary (under 500 words).\n"
        "Do NOT include raw search results or detailed tool outputs."
    ),
    "tools": [web_search],
}
```

Explicitly stating “return only the final summary” in the subagent's prompt is the first line of defense against context pollution.

13.9 /memories/ routing with CompositeBackend

Route only certain paths to a persistent store, leave the rest on ephemeral state. The agent uses the same

`write_file` / `read_file` calls; different backends handle them under the hood.

```
from deepagents import create_deep_agent
from deepagents.backends import CompositeBackend, StateBackend, StoreBackend
from langgraph.store.memory import InMemoryStore
```



```
def make_backend(runtime):
    return CompositeBackend(
        default=StateBackend(runtime),
        routes={"/memories/": StoreBackend(runtime)},
    )

agent = create_deep_agent(
    model="google_genai:gemin-3.5-flash",
    store=InMemoryStore(),
    backend=make_backend(),
    system_prompt=(
        "When users tell you their preferences, save them to "
        "/memories/user_preferences.txt so you remember them in future conversations."
    ),
)
```

13.10 Filesystem-centric architecture

All of Deep Agents' context-compression strategies converge on a single principle.



TIP

“Write big, read selective.” Write big artifacts to disk; when you need them again, inject only the relevant portions via `read_file` / `grep` .

Because of this structure:

- Even when tool results exceed 20k tokens, the context retains only a file reference + 10-line preview
- Subagents can explore as widely as they want without touching the supervisor window
- Memory is not always loaded; instead it is pulled in via `read_file("/memories/...")` *when needed*
- Only the frontmatter of a skill is visible; the body is loaded on demand

13.11 Three patterns

Pattern 1: long research sessions

- `AGENTS.md` is minimal — only research style and citation rules
- Register 3–10 skills, expose only frontmatter
- Topic-specific subagents explore in parallel → return only the final summary
- Large search results are offloaded to files automatically → re-query via `grep`

Pattern 2: personalization assistant

- `/memories/` routing accumulates per-user preferences
- `@dynamic_prompt` injects tone and guardrails by user role
- Declare `user_id` / `org_id` on `context_schema` so every subagent sees them
- Shared policies live under `/policies/` (read-only) namespace

Pattern 3: cost optimization

- Lower the automatic summarization trigger slightly below the 85% default
- Declare “under 500 words” in subagent system prompts
- Wrap tools with large responses so they return a summary + file pointer

13.12 Caveats

-
- **Do not overload AGENTS.md** — it is always loaded, so anything over 5 KB starts to hurt
 - **Skill frontmatter is everything** — if matching fails, the body is never loaded
 - **Summarization is destructive** — save important intermediate artifacts to files first
 - **If a subagent returns a raw dump**, the supervisor window blows up faster than without it
 - **Tool description quality = token efficiency** — vague docstrings cause costly retries

Key Takeaways

-
- Context engineering is a four-layer architecture: Layered / Dynamic / Compression / Retrieval
 - Automatic offloading (20k results) and automatic summarization (85%) are the primary compression mechanisms
 - `@dynamic_prompt` and `context_schema` propagate runtime context across the entire graph
 - Subagent isolation and `/memories/` routing are the two pillars against context pollution
 - The full design distills to “Write big, read selective” — big outputs to disk, selective re-injection

14 Streaming

subgraphs=True + v2 namespaces

This chapter covers how to surface events from both the main agent and inside subagents in real time and how to emit custom progress events. Use it when you want to show long-running agent progress to users or to build a subagent-card UI.

Learning Objectives

LEARNING OBJECTIVES

- Observe the inside of subagents by combining `subgraphs=True` and `version="v2"`
- Route main vs subagent events using the namespace (`ns`) tuple
- Subscribe to `updates` · `messages` · `custom` modes simultaneously
- Emit custom progress events with `get_stream_writer`
- Map the three subagent lifecycle signals (Pending / Running / Complete) to UI cards

14.1 Three key ideas

Deep Agents streaming rests on three ideas.

- `subgraphs=True` + `version="v2"` — observe events inside subagents
- **Namespace** (`ns`) — a tuple tells you which subgraph the event came from
- **Stream-mode combination** — `updates` (steps), `messages` (tokens / tools), `custom` (custom events)

The v2 format returns every chunk in a uniform `{"type", "ns", "data"}` shape. Routing is simpler than v1's nested tuples.

14.2 Basic usage: enabling subagent streaming

```
for chunk in agent.stream(
    {"messages": [{"role": "user", "content": "Research quantum computing advances"}]},
    stream_mode="updates",
    subgraphs=True,
    version="v2",
```

```

):
    print(chunk)

```

Without `subgraphs=True`, subagent internals are only visible at the supervisor level as the result of a `task` tool call. From the UI's perspective, that is “something happened in a black box and a result came out.”

14.3 Routing events via namespaces

Every v2 chunk carries a *tuple* in the `ns` field, and that tuple identifies where the event was emitted.

Tuple	Meaning
<code>()</code>	Main-agent event
<code>("tools:abc123",)</code>	Subagent spawned by a <code>task</code> tool call from the main agent
<code>("tools:abc123", "model_request:def456")</code>	The model-request node inside that subagent

Detecting whether an event comes from a subagent:

```

is_subagent = any(segment.startswith("tools:") for segment in chunk["ns"])

```

This branch is the basic split when the UI separates the main conversation panel from subagent cards.

14.4 Stream mode: `updates`

Emits “which step is running right now” at node granularity. The most useful mode for tracking subagent lifecycles.

```

for chunk in agent.stream(
    {"messages": [...]},
    stream_mode="updates",
    subgraphs=True,
    version="v2",
):
    if chunk["type"] == "updates":
        for node_name, data in chunk["data"].items():
            print(f"Step: {node_name}")

```

14.5 Stream mode: `messages`

Receives LLM tokens in fragments. When a tool call happens, each chunk carries `tool_call_chunks` so the *tool name and args stream in incrementally*.

```

for chunk in agent.stream(
    {"messages": [...]},
    stream_mode="messages",
    subgraphs=True,

```

```

        version="v2",
    ):
        if chunk["type"] == "messages":
            token, metadata = chunk["data"]
            if token.tool_call_chunks:
                for tc in token.tool_call_chunks:
                    print(f"Tool: {tc['name']}, Args: {tc.get('args')}")
            else:
                print(token.content, end="")

```

Token-level output and tool-call assembly are handled in the same loop.

14.6 Custom progress events

Pull the stream writer inside a tool and emit arbitrary structures. Useful for upload progress, item counts, or intermediate states.

```

from langchain.tools import tool
from langgraph.config import get_stream_writer

@tool
def analyze_data(topic: str) -> str:
    """Analyze data for the given topic."""
    writer = get_stream_writer()
    writer({"status": "starting", "progress": 0})
    # ... analysis work ...
    writer({"status": "complete", "progress": 100})
    return "done"

```

Subscribe with `stream_mode="custom"` on the receiving side.

```

for chunk in agent.stream(
    {"messages": [...]},
    stream_mode="custom",
    subgraphs=True,
    version="v2",
):
    if chunk["type"] == "custom":
        print(chunk["data"])

```

14.7 Subscribing to multiple modes at once

Pass a list and receive all three event types in one loop.

```

for chunk in agent.stream(
    {"messages": [...]},
    stream_mode=["updates", "messages", "custom"],
    subgraphs=True,
    version="v2",
):
    t = chunk["type"]
    if t == "updates":
        # step progress
        ...
    elif t == "messages":
        # tokens / tool calls
        ...

```

```
elif t == "custom":
    # progress / custom events
    ...
```

Production UIs typically subscribe to all three modes at once and fan them out to the respective panels.

14.8 Tracking the subagent lifecycle

From the main agent’s perspective, a subagent is identified by three events.

1. **Pending** — the moment the main agent emits a `tool_call` with `name="task"`
2. **Running** — the moment the first event arrives on the `("tools:<id>")` namespace
3. **Complete** — the moment the `ToolMessage` for that `task` call returns to the main `tools` node

Mapping UI card states to these three signals shows “which subagent is exploring right now” at a glance.

14.9 v2 unified format

Every chunk has the following three fields.

```
{
  "type": "updates" | "messages" | "custom",
  "ns": tuple,
  "data": Any,
}
```

The old v1 nested-tuple format had different data shapes per `type`, which made branching noisy. v2 simplifies frontend routing to a single `(type, ns prefix)`.

14.10 Frontend integration

React / Vue / Svelte / Angular can consume this stream via `useStream` in `@langchain/react` and similar helpers.

Subagent card rendering, reconnection, and history loading are all built in.

```
import { useStream } from "@langchain/react";

const stream = useStream<typeof agent>({
  apiUrl: "https://your-deployment.langsmith.dev",
  assistantId: "agent",
  reconnectOnMount: true,
  fetchStateHistory: true,
});

stream.submit(
  { messages: [{ type: "human", content: text }] },
  {
    streamSubgraphs: true,
    config: { recursionLimit: 10000 },
  },
);
```

14.11 Caveats

- **Without** `subgraphs=True`, **you cannot see inside a subagent** — essential for UI progress cards
- **Specify** `version="v2"` **explicitly** — falling back to the legacy format changes namespace handling
- **Keep the custom event schema stable** — if every tool uses its own shape, UI routing breaks (for example, use a shared `{"status", "progress", "message"}` structure)
- **Decide whether to expose summarization tokens** (filter with `metadata.get("lc_source") == "summarization"`)
- **Retry events are streamed too** — with `ModelRetryMiddleware` attached, fail-retry flows are visible; aggregate them in the UI

Key Takeaways

- v2 format + `subgraphs=True` is the entry point for subagent observability
- Route main vs subagent events using the `ns` tuple prefix
- Subscribing to `updates` / `messages` / `custom` as a list is the production default
- Use `get_stream_writer` to emit arbitrary progress events from tools
- Map Pending / Running / Complete to subagent UI card states

15 Permissions

FilesystemPermission · first-match-wins

Apply declarative allow/deny rules to Deep Agents' built-in filesystem tools (`ls` , `read_file` , `glob` , `grep` , `write_file` , `edit_file`) to enforce *path-based access control*. Use this chapter when you are defending against prompt injection, building a read-only agent, or carving out a workspace-scoped sandbox.

Learning Objectives

LEARNING OBJECTIVES

- Understand the three components of `FilesystemPermission` (`operations` , `paths` , `mode`)
- Know the first-match-wins evaluation rule and the default (allow)
- Distinguish the four patterns — read-only, workspace isolation, sensitive-file protection, read-only memory
- Recognize how subagent permissions are inherited and how overrides are full replacements
- Know the compensating strategies for custom tools, MCP, and sandbox shells that permissions cannot reach

15.1 Scope of application

`permissions` are path-based rules that *only apply to the built-in filesystem tools*. The following bypass them:

- Custom tools
- MCP tools
- `execute` shell commands in the sandbox

In other words, you cannot complete a security boundary with permissions alone. In configurations where the sandbox can run arbitrary shell commands, rules must be designed together with *CompositeBackend routing constraints*. Extra validation or auditing for custom tools belongs to *backend policy hooks*.

15.2 Evaluation rule: first-match-wins

Rules are evaluated in list order; the *first rule* whose `operations` and `paths` match the current call decides the outcome. If no rule matches, the default is **allow**. Because of this, place *specific deny/allow rules first* and keep general fallbacks at the end.

15.3 Three rule components

Every `FilesystemPermission` has three fields.

Field	Value	Description
<code>operations</code>	<code>list["read" "write"]</code>	<code>read = ls / read_file / glob / grep</code> , <code>write = write_file / edit_file</code>
<code>paths</code>	<code>list[str]</code>	Glob patterns with <code>**</code> recursion and <code>{a,b}</code> selectors
<code>mode</code>	<code>"allow" "deny"</code>	Defaults to <code>"allow"</code>

15.4 Pattern 1: read-only agent

Block every write globally. Useful for investigation, audit, and report-generation agents.

```
from deepagents import create_deep_agent, FilesystemPermission

agent = create_deep_agent(
    model=model,
    backend=backend,
    permissions=[
        FilesystemPermission(
            operations=["write"],
            paths=["/**"],
            mode="deny",
        ),
    ],
)
```

15.5 Pattern 2: workspace isolation

Allow only under `/workspace/`, deny everything else. Because of first-match-wins, the *allow goes first* and the deny catch-all follows.

```
agent = create_deep_agent(
    model=model,
    backend=backend,
    permissions=[
        FilesystemPermission(
            operations=["read", "write"],
            paths=["/workspace/**"],
            mode="allow",
        ),
        FilesystemPermission(
```

```

        operations=["read", "write"],
        paths=["/**"],
        mode="deny",
    ),
    ],
)

```

15.6 Pattern 3: protect specific files

Forbid touching `/workspace/.env` and the examples directory, but allow the rest of `/workspace/` freely.

```

agent = create_deep_agent(
    model=model,
    backend=backend,
    permissions=[
        # the most specific deny at the top
        FilesystemPermission(
            operations=["read", "write"],
            paths=["/workspace/.env", "/workspace/examples/**"],
            mode="deny",
        ),
        FilesystemPermission(
            operations=["read", "write"],
            paths=["/workspace/**"],
            mode="allow",
        ),
        FilesystemPermission(
            operations=["read", "write"],
            paths=["/**"],
            mode="deny",
        ),
    ],
)

```

15.7 Pattern 4: read-only memory / policies

Block only *writes* on `/memories/` and `/policies/`. Reads stay open. This is the baseline defense that prevents prompt injection from corrupting shared memory and organization policies.

```

from deepagents import create_deep_agent, FilesystemPermission
from deepagents.backends import CompositeBackend, StateBackend, StoreBackend

agent = create_deep_agent(
    model=model,
    backend=CompositeBackend(
        default=StateBackend(),
        routes={
            "/memories/": StoreBackend(
                namespace=lambda rt: (rt.server_info.user.identity,),
            ),
            "/policies/": StoreBackend(
                namespace=lambda rt: (rt.context.org_id,),
            ),
        },
    ),
    permissions=[
        FilesystemPermission(
            operations=["write"],
            paths=["/memories/**", "/policies/**"],
            mode="deny",
        ),
    ],
)

```

```

    ),
    1,
)

```

Update memories and policies only from application code; let the agent read.

15.8 Subagent inheritance

Default behavior: a parent agent's permissions are *inherited by subagents as-is*. If a subagent spec defines its own `permissions`, that *fully replaces* the parent rules (not a partial override). When overriding, include the catch-all deny yourself to stay safe.

```

agent = create_deep_agent(
    model=model,
    backend=backend,
    permissions=[...parent_rules...],
    subagents=[
        {
            "name": "auditor",
            "description": "Read-only code reviewer",
            "system_prompt": "Review the code for issues.",
            "permissions": [
                # block writes globally
                FilesystemPermission(
                    operations=["write"], paths=["/*"], mode="deny",
                ),
                # allow reads only inside /workspace
                FilesystemPermission(
                    operations=["read"], paths=["/workspace/*"], mode="allow",
                ),
                # deny all other reads
                FilesystemPermission(
                    operations=["read"], paths=["/*"], mode="deny",
                ),
            ],
        },
        1,
    ],
    1,
)

```

This lets an audit-only subagent have a narrower read scope while the main agent keeps broader access.

15.9 CompositeBackend constraint: sandbox-default

When the `CompositeBackend` *default is a sandbox*, every permission path must sit *inside a declared route prefix*. This constraint exists because the sandbox can run arbitrary shell commands via the `execute` tool. Path rules cannot stop shell-level file access, so attaching permissions to paths outside the routes creates a *false sense of security*.

```

from deepagents import create_deep_agent, FilesystemPermission
from deepagents.backends import CompositeBackend

composite = CompositeBackend(
    default=sandbox,
    routes={"/memories/": memories_backend},
)

# valid: inside the /memories/ route

```

```
agent = create_deep_agent(
    model=model,
    backend=composite,
    permissions=[
        FilesystemPermission(
            operations=["write"], paths=["/memories/**"], mode="deny",
        ),
    ],
)
```

Placing a rule on a path outside any known route raises `NotImplementedError` . For control over the sandbox-internal filesystem, do not reach for permissions — configure the sandbox itself (allowed binaries, network policy, volume-mount scope).

15.10 Prompt-injection defense perspective

Shared memory, organization policy, and externally ingested documents are all injection vectors. The defense layers are:

- 1. **Enforce read-only:** write-deny on `/memories/**` and `/policies/**` (pattern 4)
- 2. **Workspace isolation:** constrain what the agent can touch to `/workspace/**`
- 3. **Protect sensitive files:** deny each `.env` -like path individually as in pattern 3
- 4. **Shrink subagent scope:** configure audit / query subagents as read-only separately
- 5. **Custom-tool / sandbox boundaries:** permissions cannot reach them — cover with policy hooks + sandbox policy

15.11 Permissions vs backend policy hooks

Purpose	Use this surface
Path-based allow/deny on built-in FS tools	permissions
Custom validation (data checks, logging, rate limits)	backend policy hooks
Custom-tool / MCP tool control	Tool wrappers / middleware
Sandbox-internal file / network control	Sandbox configuration (allowed binaries, network policy)

Permissions are *declarative quick rules*; policy hooks are *logic-driven controls*. Use each for what it is good at.

15.12 Caveats

- **first-match-wins:** rule order changes the outcome. Put more specific deny/allow at the top
- **no match = allow:** to prevent accidental allows, end with a catch-all deny
- **Missing operations:** a rule with only `"read"` leaves writes to subsequent rules' (default allow)
- **Subagent override is full replacement:** no partial edits. Copy parent rules you want to keep

- **Permissions are not a complete security boundary:** they only make sense alongside sandbox, network, and secret management

Key Takeaways

- `FilesystemPermission` 's three components (`operations` / `paths` / `mode`) and first-match-wins are the foundations
- Four canonical patterns (read-only / workspace isolation / sensitive-file protection / read-only memory) cover most cases
- Subagent permission overrides are *full replacements* — be careful to copy in the catch-all deny
- Under CompositeBackend + sandbox-default, permissions outside the route prefixes raise `NotImplementedError`
- Permissions are *declarative quick rules*; custom tools, MCP, and sandbox shells require policy hooks and sandbox policy

P A R T



Advanced Patterns

Advanced Agent Patterns

What You Will Learn in This Part

- 0. Migration from v0 to v1
- 1. Advanced Middleware
- 2. Multi-Agent: Subagents
- 3. Multi-Agent: Handoffs and Router
 - 4. Context and Memory
 - 5. Agentic RAG
 - 6. SQL Agent
 - 7. Data Analysis
 - 8. Voice Agent
- 9. Production Deployment

00 v0 → v1 migration guide

Covers the breaking changes and code mapping you need to know when transitioning from LangChain/LangGraph v0 to v1.

Learning Objectives

- Understand v1 package structure changes and import paths
- Perform `create_react_agent` → `create_agent` migration
- Apply middleware-based dynamic prompting, state management, and context injection
- Utilizes standard content blocks and structured output strategies

0.1 Change package structure

In v1, the `langchain` namespace has been significantly reduced to five core modules essential for building an agent:

v1 module	Role	Main API
<code>langchain.agents</code>	Agent creation and state management	<code>create_agent</code> , <code>AgentState</code>
<code>langchain.messages</code>	Message types and content blocks	<code>HumanMessage</code> , <code>AIMessage</code> , <code>content_blocks</code>
<code>langchain.tools</code>	tool Definition	<code>@tool</code> decorator, <code>BaseTool</code>
<code>langchain.chat_models</code>	model initialization	<code>init_chat_model</code>
<code>langchain.embeddings</code>	Embedding Utility	Embedding model wrapper

Legacy code migration — `langchain-classic`

Chains, Retrievers, Hub, and Indexing API, which were previously used in the `langchain` package, have all been separated into a separate package called `langchain-classic` . If you need to keep your existing code, change the import path after installation to `pip install langchain-classic` :

Note that message and tool imports also moved: v0 used `langchain_core.messages` and `langchain_core.tools` , but v1 re-exports the same symbols under `langchain.messages` and `langchain.tools` . Both paths still work, but new v1 code should prefer the shorter form.

```
# v1 preferred imports
from langchain.messages import HumanMessage, AIMessage, ToolMessage
from langchain.tools import tool, ToolRuntime, BaseTool
```

The

v0 — Legacy approach

```
from langchain.chains import LLMChain
from langchain.retrievers import MultiQueryRetriever
from langchain import hub
```

v1 — langchain-classic migration path

```
from langchain_classic.chains import LLMChain
from langchain_classic.retrievers import MultiQueryRetriever
from langchain_classic import hub
```

Because of this split, the v1 `langchain` package became a lightweight layer focused on agent building, while legacy functionality is maintained separately.

0.2 Agent creation API changes

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
from langchain.tools import tool
from langchain.agents import create_agent

model = ChatOpenAI(model="gpt-5.4")

@tool
def add(a: int, b: int) -> int:
    """Add two numbers together."""
    return a + b

# v0: from langgraph.prebuilt import create_react_agent
# v1: from langchain.agents import create_agent
agent = create_agent(
    model=model,
    tools=[add],
    system_prompt="You are a math assistant.", # v0: prompt=
)
print("✓ v1 agent creation completed")
```

Major changes

v0	v1	Remarks
<code>from langgraph.prebuilt import create_react_agent</code>	<code>from langchain.agents import create_agent</code>	import path + function name
<code>prompt=</code>	<code>system_prompt=</code>	Parameter name
ToolNode Support	Not supported	Functions/BaseTool/dict only

<code>pre_hooks</code> , <code>post_hooks</code>	<code>middleware=[]</code>	Integrated middleware
Pydantic/dataclass status	<code>TypedDict</code> only	state schema

0.3 State Schema — TypedDict only

In v1, custom state must inherit from `TypedDict` based on `AgentState` .

0.4 Runtime context injection (new)

In v1, *immutable runtime data* can be passed to the agent via the `context_schema` and `context` parameters. This is a pattern that safely passes data that varies from request to request, such as user ID, role, and session information, but does not change during execution, to the agent and tool.

How it works:

1. Define the context schema with `@dataclass` .
2. Register the schema with `create_agent(context_schema=...)` .
3. Pass the value to runtime with `agent.invoke(..., context=ContextInstance(...))` .
4. In tool, the context is accessed with the `ToolRuntime[ContextType]` parameter.

Context is read-only data that *does not change* between tool calling, unlike agent state (`AgentState`). The state is updated during the agent loop, but the context is fixed when calling `invoke` .

0.5 Dynamic Prompts — Middleware Approach (New)

Instead of the static prompt in v0, `@dynamic_prompt` creates a prompt dynamically based on the runtime context.

0.6 tool Error handling — `@wrap_tool_call` (new)

Instead of v0's `handle_tool_errors` , v1 handles tool errors with middleware.

```
from langchain.agents.middleware import wrap_tool_call
from langchain.messages import ToolMessage

@wrap_tool_call
def handle_errors(request, handler):
    try:
        return handler(request)
    except Exception as e:
        return ToolMessage(
            content=f"Tool error: {e}",
            tool_call_id=request.tool_call["id"],
        )

agent = create_agent(
    model=model,
    tools=[add],
    middleware=[handle_errors],
```

```
)
print("✓ Application of error handling middleware")
```

0.7 Standard Content Block & structured output (New)

In v1, messages support provider-agnostic `content_blocks`. structured output was split into two branches:

`ToolStrategy` (based on tool calling) and `ProviderStrategy` (native).

```
from pydantic import BaseModel
from langchain.agents.structured_output import ToolStrategy, ProviderStrategy

class Weather(BaseModel):
    location: str
    temperature_c: float

# Tool-call based – works on any model
agent_tool = create_agent(model=model, tools=[add],
                        response_format=ToolStrategy(schema=Weather))

# Provider-native – uses OpenAI JSON mode / Anthropic tool_use etc.
agent_provider = create_agent(model=model, tools=[add],
                            response_format=ProviderStrategy(schema=Weather))
```

v1 models default to `output_version="v1"`, which enables standard content blocks and the `content_blocks` property automatically. Set `output_version="v0"` to restore legacy behaviour.

```
from langchain_openai import ChatOpenAI

model_v1 = ChatOpenAI(model="gpt-5.4", output_version="v1")
model_v0 = ChatOpenAI(model="gpt-5.4", output_version="v0")
```

0.8 Streaming changes

Item	v0	v1
Agent node name	"agent"	"model"
<code>.text</code>	method <code>.text()</code>	Property <code>.text</code>

0.9 Guitar Breaking Change

change	Description
<i>Python 3.10+</i>	All LangChain packages require Python 3.10 or higher. Versions 3.9 and below are not supported.
<i>return type</i>	The return type of the chat model has been fixed from <code>BaseMessage</code> to <code>AIMessage</code> .

<i>OpenAI Responses API</i>	Message content is in standard block format by default. You can restore the previous behavior with <code>output_version="v0"</code> .
<i>Anthropic max_tokens</i>	Default value changed from 1024 to automatic setting per model.
<i>AIMessage.example</i>	The <code>example</code> parameter has been removed. Use <code>additional_kwargs</code> .
<i>AIMessageChunk</i>	Added <code>chunk_position</code> attribute (value <code>'last'</code> in last chunk).
<code>.text</code> properties	<code>.text()</code> method changed to <code>.text</code> property.
<i>File Encoding</i>	Files open with UTF-8 encoding by default.

Summary — Migration Checklist

- [] Python 3.10+ confirmed
- [] `create_react_agent` → `create_agent` changed
- [] `prompt=` → `system_prompt=` changed
- [] Convert state schema to `AgentState` based on `TypedDict`
- [] `pre_hooks` / `post_hooks` → `middleware=[]`
- [] `ToolNode` → Replaced with `Function/BaseTool`
- [] `.text()` → `.text` property
- [] Check streaming node name `"agent"` → `"model"`
- [] Move legacy imports to `langchain-classic`

Next Steps

→ [01_middleware.ipynb](#): v1's biggest new feature — Deepening the middleware system

01 Deepening the middleware system

v1's biggest new features

Learning Objectives

- Understand the role and execution flow of middleware hooks in the agent loop.
- Learn how to set up and use 7 types of built-in middleware
- You can write decorator/class-based custom middleware.
- Execution order can be accurately predicted when combining multiple middleware

1.1 Environment Setup

Middleware is a core feature of v1 that implements monitoring, transformation, reliability, and governance by inserting hooks into each step of agent execution. It is used by passing the middleware instance list to the `middleware` parameter of the `create_agent` function.

```
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv()

model = ChatOpenAI(model="gpt-5.4")
```

1.2 Middleware Architecture Overview

The agent loop is a repeating cycle of *model call* → *tool selection* → *tool execution* → *termination decision*. Middleware enables fine-grained control by inserting hooks into each step of this cycle.

Hook type

Hook Type	When to run	Representative uses
<code>before_model</code>	Just before the model call	Prompt Modification, Logging, Status Updates
<code>after_model</code>	Immediately after the model responds	Response validation, guardrails, result transformation
<code>before_agent</code>	When the agent starts running	initialization, preprocessing
<code>after_agent</code>	At the end of agent execution	Cleanup, post-processing

<code>wrap_model_call</code>	Wrapping model call	Retries, caching, fallback
<code>wrap_tool_call</code>	tool calling Wrap	tool Retries, audit logs, error handling

Two hook styles

- *Node-style hook* (`before_*` , `after_*`): Executes sequentially and is suitable for logging/verification/status updates.
- *Wrap-style hook* (`wrap_*`): Allows you to control whether the handler (`next_fn`) is called. It can be called 0 times (blocking), 1 time (passing), or multiple times (retry), making it suitable for retry, caching, and conversion logic.

Middleware provides clean separation of cross-cutting concerns such as monitoring, transformation, reliability, and governance without changing the agent's core logic.

```
from langchain.agents import create_agent
from langchain.agents.middleware import (
    SummarizationMiddleware,
    HumanInTheLoopMiddleware,
)

agent = create_agent(
    model="gpt-5.4", tools=[],
    middleware=[
        SummarizationMiddleware(model="gpt-5.4-mini", trigger=("messages", 50)),
        HumanInTheLoopMiddleware(interrupt_on={}),
    ],
)
```

1.3 SummarizationMiddleware

When a conversation becomes long enough to exceed the context window, it automatically compresses the previous conversation by Summary. It is essential for long-running conversations, multi-turn conversations, and applications that require preservation of the entire conversation context.

Main parameters

parameters	Description	Example
<code>model</code>	Summary Lightweight model to use for creation (reduces costs)	"gpt-5.4-mini"
<code>trigger</code>	Summary Trigger condition	<code>("tokens", 4000)</code> , <code>("messages", 50)</code> , <code>("fraction", 0.8)</code>
<code>keep</code>	Latest context to keep after Summary	<code>("messages", 20)</code>
<code>token_counter</code>	Custom token counting function	optional
<code>summary_prompt</code>	Custom Summary Prompt Template	optional

`trigger` can be set based on one of the following: number of tokens, number of messages, or window ratio.

When the condition is reached, all but the most recent messages specified in `keep` are replaced with Summary statements.

```
from langchain.agents.middleware import SummarizationMiddleware

summarizer = SummarizationMiddleware(
    model="gpt-5.4-mini",
    trigger=("tokens", 4000),
    keep=("messages", 20),
)
```

1.4 HumanInTheLoopMiddleware

Stop agent execution before high-risk tool calling and wait for human approval. Use when human supervision is required for high-risk tasks or compliance workflows, such as database writes, financial transactions, and email sending.

checkpointer required — a checkpointer is absolutely required to restore state after an interruption.

Decision type

decision	Description	How to use
approve	tool calling Approval and Execution	<code>Command(resume="approve")</code>
edit	tool Execute after modifying arguments	<code>Command(resume={"type": "edit", "args": {...}})</code>
reject	tool calling Reject	<code>Command(resume={"type": "reject", "reason": "..."})</code>

Set the approval policy for each tool in the `interrupt_on` dictionary. If set to `False`, the corresponding tool will run without interruption.

```
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver

hitl = HumanInTheLoopMiddleware(
    interrupt_on={
        "send_email": {"allowed_decisions": ["approve", "edit", "reject"]},
        "read_email": False,
    }
)
```

```
agent = create_agent(
    model="gpt-5.4", tools=[],
    checkpointer=InMemorySaver(),
    middleware=[hitl],
)
```

1.5 ModelCallLimitMiddleware & ToolCallLimitMiddleware

Call limiting middleware to prevent infinite loops or excessive API costs.

ModelCallLimitMiddleware

Limits the number of times the agent calls the model. Used to prevent agent flooding, control production costs, and manage call budgets during testing.

ToolCallLimitMiddleware

Limits tool calling counts globally or per specific tool. It is useful for limiting expensive external API calls, controlling search/DB query frequency, and enforcing rate limits for specific tool.

Common parameters

parameters	Description
<code>thread_limit</code>	Maximum number of calls across all threads (all invokes)
<code>run_limit</code>	Maximum number of calls in a single invoke execution
<code>exit_behavior</code>	"end" (normal termination), "error" (raise exception), "continue" (continue with error message — ToolCallLimit only)

ToolCallLimitMiddleware can additionally take a `tool_name` parameter to apply limits to specific tool only.

```
from langchain.agents.middleware import ModelCallLimitMiddleware

model_limit = ModelCallLimitMiddleware(
    thread_limit=10,
    run_limit=5,
    exit_behavior="end",
)
```

```
from langchain.agents.middleware import ToolCallLimitMiddleware

# global limit
global_tool_limit = ToolCallLimitMiddleware(thread_limit=20, run_limit=10)

# Certain tool restrictions
search_limit = ToolCallLimitMiddleware(
    tool_name="search",
    thread_limit=5, run_limit=3,
    exit_behavior="continue",
)
```

1.6 ModelFallbackMiddleware

Automatically switches to an alternate model chain when the primary model fails. It is useful for responding to production failures, optimizing costs (fallback from expensive models to cheap models), and ensuring multi-provider redundancy (OpenAI + Anthropic, etc.).

If you pass a fallback model to the constructor in order, it will try the fallback models in the specified order when the main model call fails. If all fallbacks fail, a final error is raised.

```
from langchain.agents.middleware import ModelFallbackMiddleware

# gpt-5.4 failed -> gpt-5.4-mini -> claude
```

```
fallback = ModelFallbackMiddleware(  
    "gpt-5.4-mini",  
    "claude-sonnet-4-6",  
)
```

1.7 PIIMiddleware

Personally identifiable information (PII) is automatically detected and processed according to established strategies. It is essential for healthcare/financial compliance, cleaning logs for customer service agents, handling sensitive user data, and more.

Built-in PII type

email , credit_card , ip , mac_address , url

Processing Strategy

strategy	Action	Example (email)
block	Exception raised — execution halts when PII is found	Error Occurred
redact	Replace with [REDACTED_TYPE]	[REDACTED_EMAIL]
mask	Partial masking	u***\@example.com
hash	deterministic hashing	a1b2c3d4...

Scope of application

- apply_to_input : Check user input message
- apply_to_output : Check AI response message
- apply_to_tool_results : Check tool execution results

Custom Detector

In addition to the built-in PII types, you can create custom detectors in three ways:

1. *Regular Expression String*: Simple pattern matching
2. **Compiled regular expressions** (re.compile): complex regular expressions
3. *Function*: Advanced detection that requires validation logic (returns: list[dict] — contains text , start , end keys)

```
from langchain.agents.middleware import PIIMiddleware  
  
email_pii = PIIMiddleware("email", strategy="redact", apply_to_input=True)  
card_pii = PIIMiddleware("credit_card", strategy="mask", apply_to_input=True)
```

```
# Custom Detector: Regular Expression String  
api_key_pii = PIIMiddleware(  
    "api_key",  
    detector=r"sk-[a-zA-Z0-9]{32}",
```



```
        strategy="block",
    )
```

```
import re

# Custom Detector: Compiled Regular Expressions
phone_pii = PIIMiddleware(
    "phone_number",
    detector=re.compile(r"\+?\d{1,3}[\s.-]??\d{3,4}[\s.-]??\d{4}"),
    strategy="mask",
)
```

```
# Custom detector: function (SSN example)
def detect_ssn(content: str) -> list[dict]:
    matches = []
    for m in re.finditer(r"\d{3}-\d{2}-\d{4}", content):
        first = int(m.group(0)[:3])
        if first not in [0, 666] and not (900 <= first <= 999):
            matches.append({"text": m.group(0), "start": m.start(), "end": m.end()})
    return matches

ssn_pii = PIIMiddleware("ssn", detector=detect_ssn, strategy="hash")
```

1.8 LLMToolSelectorMiddleware

When there are more than 10 tools, Lightweight LLM analyzes the user query and selects only the relevant tools. This reduces token waste due to unnecessary tool descriptions and allows the model to focus on relevant tool, increasing accuracy.

Main parameters

parameters	Description	default
model	tool Model for selection	Agent's main model
system_prompt	Custom Selection Guidelines	Built-in prompt
max_tools	Maximum number of selections tool	All
always_include	List of tool names to always include	<code>[]</code>

Using a lightweight model like `gpt-5.4-mini` as the model of choice allows for effective tool filtering while reducing cost.

```
from langchain.agents.middleware import LLMToolSelectorMiddleware

tool_selector = LLMToolSelectorMiddleware(
    model="gpt-5.4-mini",
    max_tools=3,
    always_include=["search"],
)
```

ContextEditingMiddleware

For long multi-turn conversations, `ContextEditingMiddleware` clears *old tool results* once a token threshold is crossed, while keeping the most recent N. Where `SummarizationMiddleware` writes a single summary blob, this one prunes tool-result by tool-result.

```
from langchain.agents.middleware import ContextEditingMiddleware, ClearToolUsesEdit

agent = create_agent(
    model="gpt-5.4",
    tools=[search_tool],
    middleware=[
        ContextEditingMiddleware(
            edits=[ClearToolUsesEdit(trigger=100_000, keep=3, placeholder="[cleared]")],
        ),
    ],
)
```

PatchToolCallsMiddleware (Deep Agents)

Some models produce slightly malformed tool-call JSON. `PatchToolCallsMiddleware` (from `deepagents`) repairs common defects so tool calls do not fail outright.

```
from deepagents.middleware.patch_tool_calls import PatchToolCallsMiddleware

agent = create_agent(
    model="claude-sonnet-4-6",
    tools=[search_tool, write_file],
    middleware=[PatchToolCallsMiddleware()],
)
```

1.9 Writing custom middleware

There are two implementation methods:

1. Decorator method

Single hook, suitable for simple logic. Use the `@before_model`, `@after_model`, `@wrap_model_call`, and `@wrap_tool_call` decorators.

2. Class method (`AgentMiddleware`)

If you need to combine multiple hooks or configure them, inherit `AgentMiddleware`. You can provide sync/async implementations simultaneously.

Custom Status

Middleware can extend agent state using the `NotRequired` type hint. This allows tracking values between executions, sharing data between hooks, and implementing cross-cutting concerns such as rate limiting or audit logging.

Agent Jump

You can control agent flow by returning a dictionary from `after_model`, etc.:

- `{"jump_to": "end"}` — Terminate agent immediately

- Go to `{"jump_to": "tools"}` — tool execution phase
- `{"jump_to": "model"}` — Go to model call step

Since v1.2, declare legal jump targets statically with `@hook_config(can_jump_to=[...])`. The graph is validated at compile time so an invalid jump target fails fast.

```
from langchain.agents.middleware import after_model, hook_config, AgentState
from langgraph.runtime import Runtime
from langchain.messages import AIMessage

@after_model
@hook_config(can_jump_to=["end"])
def block_unsafe(state: AgentState, runtime: Runtime) -> dict | None:
    last = state["messages"][-1]
    if "BLOCKED" in last.content:
        return {"messages": [AIMessage("Cannot respond.")], "jump_to": "end"}
    return None
```

Type annotations — `ModelRequest` / `ToolCallRequest` / `ModelResponse`

v1 ships dataclasses for hook signatures:

- `ModelRequest` (`wrap_model_call` input): `messages`, `tools`, `model`, `system_message`, `response_format`, `state`, `runtime`.
- `ToolCallRequest` (`wrap_tool_call` input): `tool_call`, `state`, `runtime`.
- `ModelResponse`: return type of the inner handler.
- `ExtendedModelResponse`: wraps `ModelResponse` + `Command` for persistent state updates.

```
from typing import Callable
from langchain.agents.middleware import (
    wrap_model_call, wrap_tool_call,
    ModelRequest, ModelResponse, ToolCallRequest,
)
from langchain.messages import ToolMessage
from langgraph.types import Command

@wrap_model_call
def add_cache_marker(
    request: ModelRequest,
    handler: Callable[[ModelRequest], ModelResponse],
) -> ModelResponse:
    cached = request.override(system_message=f"{request.system_message}\n[CACHE]")
    return handler(cached)
```

`request.override(...)`

`ModelRequest.override(...)` returns a modified copy without mutating shared state, so multiple middleware can cooperate safely. `messages`, `tools`, `model`, `system_message`, `system_prompt`, `response_format` are all overridable.

`ExtendedModelResponse`

Return `ExtendedModelResponse` from `wrap_model_call` when you need a state update to survive the call:

```
from langchain.agents.middleware import ExtendedModelResponse

@wrap_model_call
def track_tokens(request, handler) -> ExtendedModelResponse:
    response = handler(request)
    tokens = response.message.usage_metadata.get("total_tokens", 0)
    return ExtendedModelResponse(
        model_response=response,
```

```
        command=Command(update={"total_tokens": tokens}),
    )
```

```
from langchain.agents.middleware import before_model

@before_model
def log_before(state, runtime):
    """Record the number of messages before calling the model."""
    print(f"[LOG] {len(state.get('messages', []))} messages")
```

```
from langchain.agents.middleware import after_model

@after_model
def validate_output(state, runtime):
    """Guard: Blocks prohibited content."""
    last = state["messages"][-1].content
    if "FORBIDDEN" in last:
        return {"jump_to": "end"}
```

```
from langchain.agents.middleware import wrap_model_call

@wrap_model_call
def retry_on_error(request, handler):
    """In case of failure, the model call is retried up to 2 times."""
    for attempt in range(3):
        try:
            return handler(request)
        except Exception as e:
            if attempt == 2: raise
```

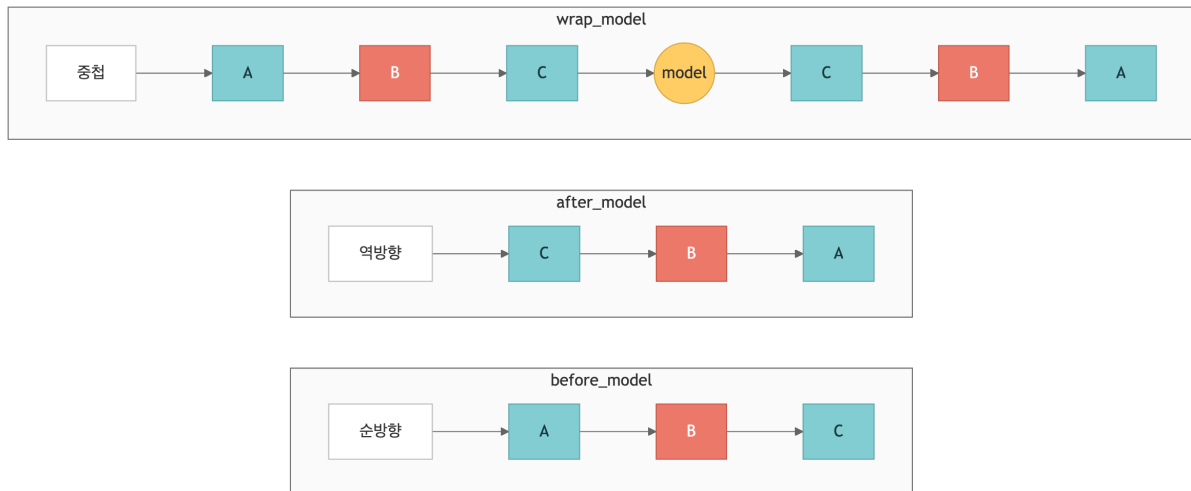
```
from langchain.agents.middleware import AgentMiddleware

class AuditMiddleware(AgentMiddleware):
    def __init__(self, log_file="audit.log"):
        self.log_file = log_file
    def before_model(self, state, config):
        print(f"[AUDIT] before -> {self.log_file}")
    def after_model(self, state, config):
        print(f"[AUDIT] after -> {self.log_file}")
```

1.10 Middleware execution order

When registering multiple middleware, you can prevent unexpected behavior by accurately understanding the execution order.

`middleware=[A, B, C]` Upon registration:



Practical tips

- PII detection must be registered before logging so that PII is not included in the log.
- Place fallback middleware after retry middleware, so that fallback works after a retry failure.
- If `next_fn` is not called from the `wrap` hook, all subsequent middleware and actual calls will be skipped.

```

@before_model
def mw_a(state, runtime): print("before A")

@before_model
def mw_b(state, runtime): print("before B")

@before_model
def mw_c(state, runtime): print("before C")

# Run: A -> B -> C (if after_model, C -> B -> A)

```

1.11 Middleware Combination (Stacking)

In a production environment, multiple middlewares are used together to implement comprehensive agent governance. Since middleware is executed in registration order, it is recommended to place it in the following order: Security (PII) → Reliability (fallback) → Cost control (call limiting) → Context management (Summary) → Optimization (tool selection) → Supervision (HITL).

This combination allows each middleware to maintain single-responsibility principles, while collectively forming a powerful production agent pipeline.

```

from langchain.agents import create_agent
from langchain.agents.middleware import (
    PIIMiddleware, ModelFallbackMiddleware,
    ModelCallLimitMiddleware, SummarizationMiddleware,
    HumanInTheLoopMiddleware, LLMToolSelectorMiddleware,
)
from langgraph.checkpoint.memory import InMemorySaver

```

```

middleware_stack = [
    PIIIMiddleware("email", strategy="redact", apply_to_input=True),
    ModelFallbackMiddleware("gpt-5.4-mini", "claude-sonnet-4-6"),
    ModelCallLimitMiddleware(thread_limit=50, run_limit=10),
    SummarizationMiddleware(model="gpt-5.4-mini", trigger=("tokens", 4000)),
]

production_agent = create_agent(
    model="gpt-5.4", tools=[], checkpointer=InMemorySaver(), middleware=middleware_stack,
)

```

1.12 Provider-specific Middleware

While the seven built-in middleware above cover *provider-agnostic* patterns, provider-specific middleware wires *vendor-only features* into the agent. Capabilities that must be toggled on a provider's server — Anthropic prompt cache, Bedrock TTL cache, OpenAI Moderation API — are exposed as dedicated middleware.

Anthropic Prompt Caching

`langchain_anthropic.middleware.AnthropicPromptCachingMiddleware` attaches `cache_control` markers automatically at the *system prompt* · *tool definitions* · *last message* positions when calling Claude models. Agents that repeatedly send long system prompts or RAG context can cut input-token cost by up to 90%.

Parameter	Default	Description
<code>type</code>	"ephemeral"	The only type Anthropic currently supports
<code>ttr</code>	"5m"	"5m" or "1h" (1h is an Anthropic beta)
<code>min_messages_to_cache</code>	0	Attach <code>cache_control</code> only when message count is at least this value
<code>unsupported_model_behavior</code>	"warn"	When applied to non-Anthropic models: "warn" / "ignore" / "raise"

```

from langchain.agents import create_agent
from langchain_anthropic.middleware import AnthropicPromptCachingMiddleware

agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    tools=[],
    middleware=[
        AnthropicPromptCachingMiddleware(
            ttl="1h",
            min_messages_to_cache=2,
            unsupported_model_behavior="warn",
        ),
    ],
)

```

On successive calls, check `usage_metadata.input_token_details` for `cache_creation_input_tokens` (first call) and `cache_read_input_tokens` (subsequent calls) to confirm cache hits.

Claude native tool middleware

Claude models are trained on native tool schemas (`bash_20250124` , `text_editor_20250728` , `memory_20250818`) that the Anthropic server interprets directly. Compared to building the same behavior with a generic `@tool` decorator,

tool-schema token usage approaches zero and error rates are lower. LangChain ships middleware that wraps these native tools in two variants — state-backed and filesystem-backed.

- `ClaudeBashToolMiddleware` — native bash execution tool. Takes `workspace_root`, `startup_commands`, `execution_policy` (`HostExecutionPolicy` / `DockerExecutionPolicy` / `CodexSandboxExecutionPolicy`), and `redaction_rules`. The Docker policy is the recommended default.
- `StateClaudeTextEditorMiddleware` / `FilesystemClaudeTextEditorMiddleware` — native text editor supporting `view` / `create` / `str_replace` / `insert` / `delete` / `rename`. The state variant writes into the `text_editor_files` state key; the filesystem variant writes into an actual `root_path` directory.
- `StateClaudeMemoryMiddleware` / `FilesystemClaudeMemoryMiddleware` — native memory tool that follows the `/memories/*` path contract. The state variant, combined with a checkpointer, is restored on `thread_id` resume; the filesystem variant persists to disk across process restarts.
- `StateFileSearchMiddleware` — native tool that runs `glob` / `grep` across the virtual files accumulated by the text editor or memory middleware. `state_key="text_editor_files"` is the default; switch to `"memory_files"` to search memory-side files.

```
from langchain_anthropic.middleware import (
    ClaudeBashToolMiddleware,
    StateClaudeTextEditorMiddleware,
    StateClaudeMemoryMiddleware,
    StateFileSearchMiddleware,
)
from langchain.agents.middleware import DockerExecutionPolicy

agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        ClaudeBashToolMiddleware(execution_policy=DockerExecutionPolicy()),
        StateClaudeTextEditorMiddleware(allowed_path_prefixes=["/src"]),
        StateClaudeMemoryMiddleware(),
        StateFileSearchMiddleware(state_key="text_editor_files"),
    ],
)
```

! WARNING

LangChain 1.2 `create_agent` rejects duplicate instances of the same middleware class. To search both text-editor files and memory files with `StateFileSearchMiddleware` at once, split them into separate subclasses.

Bedrock Prompt Caching

In enterprise setups that call Claude/Nova through AWS Bedrock, use

`langchain_aws.middleware.BedrockPromptCachingMiddleware`. Parameter names and semantics are almost identical to `AnthropicPromptCachingMiddleware`, but the target package and model-specific constraints differ.

Item	ChatBedrockConverse + Anthropic	ChatBedrockConverse + Nova
System-prompt cache	O	O
Tool-definition cache	O	X
Message cache	O	O (excluding tool-result messages)
TTL "1h"	O	X (5m only)

The `type` parameter only takes effect on `ChatBedrock` (the legacy invoke-model wrapper); on `ChatBedrockConverse` it is pinned to `"default"`. For a checkpoint to actually hit, the cached block must be roughly *1,024 tokens or more*. For Nova models, pin `ttr="5m"` and set `unsupported_model_behavior="warn"` so an accidental `"1h"` is not silently ignored.

```
from langchain_aws.middleware import BedrockPromptCachingMiddleware

agent = create_agent(
    model="bedrock_converse:anthropic.claude-sonnet-4-v6:0",
    tools=[],
    middleware=[
        BedrockPromptCachingMiddleware(
            ttl="1h",
            min_messages_to_cache=2,
            unsupported_model_behavior="warn",
        ),
    ],
)
```

OpenAI Content Moderation (in depth)

Here is the full parameter set for `OpenAIModerationMiddleware`, which we introduced briefly earlier. `check_tool_results` adds a scan over tool outputs before they enter the model; it is cost-effective only in pipelines that mix in *third-party authored content* such as web search or email.

Parameter	Default	Description
<code>model</code>	"omni-moderation-latest"	Moderation model. Pinning to "omni-moderation-2024-09-26" etc. is supported
<code>check_input</code>	<code>True</code>	Scan user messages before they reach the model
<code>check_output</code>	<code>True</code>	Scan model-generated responses before returning
<code>check_tool_results</code>	<code>False</code>	Scan tool outputs before they enter the model
<code>exit_behavior</code>	"end"	"end" (end the graph) / "error" (raise) / "replace" (swap message only)
<code>violation_message</code>	default template	Supports the <code>{categories}</code> · <code>{category_scores}</code> · <code>{original_content}</code> variables
<code>client</code> / <code>async_client</code>	<code>None</code>	Inject a pre-configured OpenAI client (optional)

```
from langchain_openai.middleware import OpenAIModerationMiddleware

agent = create_agent(
    model="openai:gpt-5.4",
    tools=[search_tool],
    middleware=[
        OpenAIModerationMiddleware(
            model="omni-moderation-latest",
            check_input=True,
            check_output=True,
            check_tool_results=False,
            exit_behavior="replace",
            violation_message=(
                "A safety-policy violation was detected "
                "(categories: {categories}). The original content is not recorded."
            ),
        ),
    ],
)
```



```
    1,
),
)
```

 TIP

In production it is common to always keep `check_input` on and run `check_output` under a sampling strategy (for example, 10% random sampling). The Moderation API itself adds call cost and latency.

Summary

Item	Key Takeaways
Architecture	Four hooks: <code>before_model</code> , <code>after_model</code> , <code>wrap_model_call</code> , <code>wrap_tool_call</code>
7 built-in types	Summarization, HITL, ModelCallLimit, ToolCallLimit, ModelFallback, PII, LLMToolSelector
Custom	Decorator (<code>@before_model</code> , etc.) / <code>AgentMiddleware</code> class
Execution order	<code>before</code> : forward, <code>after</code> : reverse, <code>wrap</code> : nested
Production	PII → Fallback → Limit → Summarization → ToolSelector → HITL
Provider-specific	Anthropic (caching · bash · editor · memory · search), Bedrock (caching), OpenAI (Moderation)

Middleware is a powerful tool for controlling a single agent’s behavior. But solving complex domain problems requires multi-agent architectures where several agents collaborate. The next chapter covers multi-agent systems that coordinate subagents with the supervisor pattern.

Next Steps

→ [02_multi_agent_subagents.ipynb](#): Multi-Agent: Learn the Subagents pattern.

02 Multiagents: Subagents

Supervisor pattern

Learning Objectives

- Design a three-tier architecture of Supervisor → subagent → tool
- Wrap subagent with `@tool` and expose it to supervisors as tool
- Understand HITL, ToolRuntime, and asynchronous/dispatch patterns.

2.1 Environment Setup

The Subagents pattern is a multi-agent architecture in which a central supervisor agent delegates tasks by calling specialized subagents like tool. On this laptop, we build a personal assistant system that handles calendars and email domains.

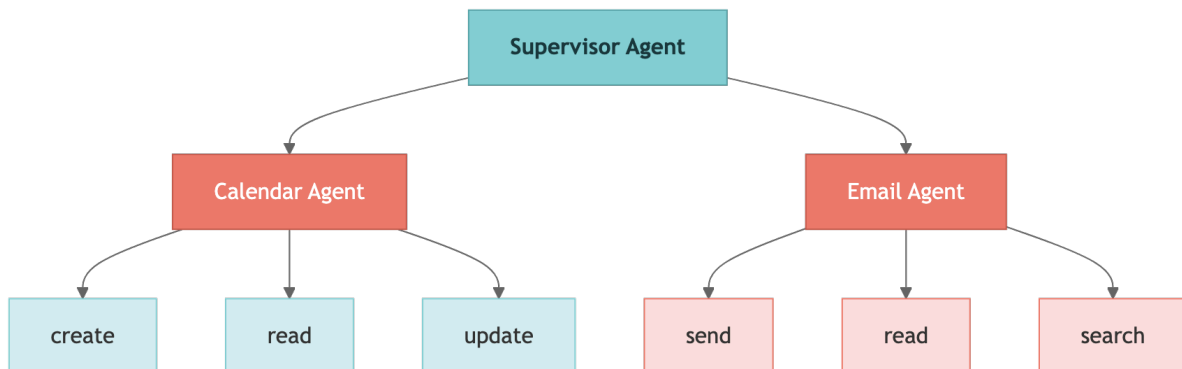
```
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv()

model = ChatOpenAI(model="gpt-5.4")
```

2.2 Subagents Architecture Overview

The Subagents pattern consists of a *three-tier architecture*. The supervisor is responsible for all routing, subagent does not interact directly with the user, and returns results to the supervisor.



tier	Role	Features
<i>Low Level tool</i>	Direct call to external service (Calendar API, Email API)	Simple function wrapper
<i>subagent</i>	Domain-specific inference + tool combination	Expert system prompt, independent tool set
<i>Supervisor</i>	Decompose tasks, delegate, and aggregate results	Remember entire conversation, treat subagent = tool

Core Traits

- *Centralized Control*: All routing flows through the supervisor
- *Context Isolation*: subagent runs in a clean context window every time, preventing context bloat.
- *Parallel execution*: Multiple subagent can be called simultaneously in one turn
- *tool based calls*: Wrap subagent with `@tool` and expose it to the supervisor like a regular tool

When to use

The subagent pattern is suitable when you manage multiple domains (calendar, email, CRM, etc.), but subagent does not need to interact directly with users, and you need centralized workflow management. For simple scenarios with few tool, a single agent is sufficient.

2.3 Low-level tool definitions

Defines the low-level tool, the bottom layer of the three-tier architecture. These tool are simple function wrappers that interact directly with external services (Calendar API, Email API). In actual production, it integrates with Google Calendar API, Email Service, etc., but here we use a stub implementation for learning.

Important points when designing tool:

- One tool is responsible for only one function (single responsibility principle)
- The same tool should not be assigned redundantly to multiple subagent
- Write the docstring clearly so that LLM can select tool correctly

```
from langchain_core.tools import tool

@tool
def create_calendar_event(
    title: str, start_time: str, end_time: str,
    attendees: list[str] = None,
) -> str:
    """Create a new calendar event."""
    return f"Event '{title}' created: {start_time} ~ {end_time}"
```

```
@tool
def read_calendar_events(date: str) -> str:
    """Look up calendar events for a date (YYYY-MM-DD)."""
    return f"No events found on {date}."
```

```
@tool
def send_email(to: str, subject: str, body: str) -> str:
    """Send an email message."""
    return f"Email sent to {to}: '{subject}'"

@tool
def read_emails(folder: str = "inbox", limit: int = 10) -> str:
    """Read recent emails from a folder."""
    return f"3 emails found in {folder}"
```

```
@tool
def search_emails(query: str, limit: int = 10) -> str:
    """Search for emails with your search term."""
    return f"2 search results for '{query}'"
```

2.4 Create subagent

Each subagent is created by `create_agent()` and has three core elements:

1. *Specialized system prompts*: Define domain-specific roles and behavioral guidelines
2. *Set tool by domain*: Separate concerns by assigning only tool for that domain
3. **name identifier**: used by the supervisor to identify and call a subagent

The recommended granularity of subagent is domain-level (calendar, email, etc.). Too much granularity increases the routing burden on the supervisor, while too much integration reduces the benefits of context isolation.

```
from langchain.agents import create_agent

calendar_agent = create_agent(
    model="gpt-5.4",
    tools=[create_calendar_event, read_calendar_events],
    system_prompt="You are a calendar assistant. Use ISO 8601 date format.",
    name="calendar_agent",
)
```

```
email_agent = create_agent(
    model="gpt-5.4",
    tools=[send_email, read_emails, search_emails],
    system_prompt="You are an email assistant. Write your message professionally.",
    name="email_agent",
)
```

2.5 Wrapping subagent with tool

The standard pattern for exposing subagent to a supervisor is to wrap it with a `@tool` decorator. Inside the wrapping function, call `subagent.invoke()` and return `content` of the last message.

Advantages of this pattern:

- From the supervisor's perspective, subagent is treated the same as regular tool
- Changes to the internal implementation of subagent do not affect the supervisor.
- The input/output format can be freely customized in the wrapping function.

Choose an input/output strategy: You can pass just the query (simple) or the entire context (sophisticated). When returning results, you have the option of returning only the final result or returning the entire history.

2.6 Supervisor Agent Assembly

The supervisor is created by passing the wrapped subagent tool to `tools`. The supervisor's system prompts include task decomposition and delegation instructions.

Supervisor design considerations:

- *Error handling:* subagent failures must be handled gracefully by the supervisor.
- *Result Aggregation:* Consolidates results from multiple subagent to provide consistent responses to users
- *Approval Scope:* Applies HITL only to operations that change the state (sending an email, creating an event)

```
supervisor = create_agent(
    model="gpt-5.4",
    tools=[call_calendar, call_email],
    system_prompt=(
        "You are a personal assistant. complex request"
        "Decompose it into subtasks and delegate them to appropriate agents."
    ),
)
```

2.7 Run test

```
User: "Schedule a meeting with Sarah tomorrow at 2 PM and send the invitation email"
Supervisor → call_calendar → create_calendar_event
Supervisor → call_email → send_email
Supervisor: "The meeting and invitation email are complete"
```

2.8 HITL (Human-in-the-Loop) Integration

Combining `HumanInTheLoopMiddleware` and `checkpoint` allows you to request user approval before high-risk tool calling (sending emails, creating schedules, etc.). When the agent attempts to call a protected tool, execution is paused and reviewed by the user.

Authorization response type

Reply	Description	code
<i>Approve</i>	Run tool calling as is	<code>Command(resume="approve")</code>
<i>Edit</i>	Execute after modifying tool argument	<code>Command(resume={"type": "edit", "args": {...}})</code>
<i>Reject</i>	Cancel tool calling	<code>Command(resume={"type": "reject", "reason": "..."})</code>

It is recommended to apply HITL only to operations that change state (send_email, create_calendar_event, etc.). It is not applied to read-only operations to reduce unnecessary friction.

```
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver

hitl = HumanInTheLoopMiddleware(interrupt_on={
    "schedule_event": {"allowed_decisions": ["approve", "edit", "reject"]},
    "manage_email": {"allowed_decisions": ["approve", "reject"]},
})
```

```
supervisor_hitl = create_agent(
    model="gpt-5.4",
    tools=[call_calendar, call_email],
    checkpoint=InMemorySaver(),
    middleware=[hitl],
    system_prompt="You are a personal assistant.",
)
```

2.9 Passing context (ToolRuntime)

`ToolRuntime` is a mechanism for passing runtime context (user ID, name, time zone, etc.) to tool without including it in the message. Instead of putting repetitive text in every prompt, you set the shared context with `ToolRuntime` just once.

The tool function accesses the context with the `runtime_context` keyword argument. Through this:

- User identity information may be used by tool (e.g. to automatically set sender email)
- Environment Setup (time zone, etc.) can be applied consistently
- Reduce token cost by reducing prompt length

```
# ToolRuntime is the runtime context passing mechanism for LangChain v1 agents.
# This is a fallback in case it hasn't been released yet.
try:
    from langchain.runtime import ToolRuntime
except ImportError:
    # Simple fallback implementation if ToolRuntime is not yet released
    class ToolRuntime:
        """Fallback ToolRuntime for pre-release versions."""
        def __init__(self, context: dict):
            self.context = context
        print("ToolRuntime unreleased - use fallback stub")

runtime = ToolRuntime(context={
    "user_email": "me@example.com",
    "user_name": "Alice",
    "timezone": "Asia/Seoul",
})
```

```
supervisor_ctx = create_agent(
    model="gpt-5.4",
    tools=[call_calendar, call_email],
    system_prompt="You are a personal assistant.",
)
```

```
# Accessing runtime context in tool
@tool
def send_email_ctx(
    to: str, subject: str, body: str, *, runtime_context: dict
) -> str:
    """Send an email to the current user."""
    sender = runtime_context["user_email"]
    return f"Email sent from {sender} to {to}: '{subject}'"
```

2.10 Asynchronous execution pattern

subagent has two execution modes:

mode	Action	When to use
<i>Synchronous</i>	Supervisor waits for subagent completion and then proceeds	When results are needed for next operation (default)
<i>Asynchronous</i>	Return Job ID immediately, run in background, view results later	Independent work, long-time work

Asynchronous patterns follow the structure *Job ID* → *Status* → *Result*. If subagent returns the Job ID immediately, the supervisor can continue working on other tasks and retrieve the results later.

```
import uuid
job_store = {}

@tool("schedule_async", description="Event scheduling (asynchronous)")
def call_calendar_async(query: str) -> str:
    """Starts an asynchronous calendar task and returns the task ID."""
    job_id = str(uuid.uuid4())[:8]
    job_store[job_id] = {"status": "done", "result": "Event created"}
    return f"Job started: {job_id}"
```

```
@tool("check_job", description="Check status of asynchronous task")
def check_job(job_id: str) -> str:
    """Check the status of an asynchronous operation."""
    job = job_store.get(job_id, {"status": "not_found"})
    return f"Status: {job['status']}, result: {job.get('result')}"
```

2.11 Single Dispatch tool Pattern

There are two approaches to the tool pattern:

pattern	Description	Advantages
<i>By agent tool</i>	Create a separate wrapping tool for each subagent	Detailed control, easy description customization
<i>Single Dispatch tool</i>	Call all subagent with one parameterized tool	Excellent scalability, independent addition/removal of subagent

The single dispatch pattern specifies the target of the call with the `agent_name` parameter. Because subagent registered in the agent registry is called by looking up its name, subagent can be added or removed independently in distributed teams, providing excellent scalability.

Three ways to expose a single dispatch tool

Style	How	When to use
<i>Literal enum</i>	<code>agent_name: Literal["calendar", "email"]</code>	≤ 5 domains, stable routing
<i>Dict registry</i>	Add/remove via dict without code change	6-20 domains, frequent changes
<i>Vector search routing</i>	Embed descriptions, match against user query	20+ domains, semantic routing

`SubAgentMiddleware` — Deep Agents integration

Instead of wrapping each subagent with `@tool`, `deepagents.SubAgentMiddleware` registers the dispatch tool automatically and merges subagent results into the supervisor state.

```
from deepagents.middleware import SubAgentMiddleware

supervisor = create_agent(
    model="gpt-5.4",
    tools=[], # middleware adds the dispatch tool
    middleware=[SubAgentMiddleware(subagents=[
        {"name": "calendar", "description": "schedule", "agent": calendar_agent},
        {"name": "email", "description": "send/read email", "agent": email_agent},
    ])],
    system_prompt="You are a personal assistant.",
)
```

`checkpointer=True` — inherit the parent checkpointer

Since v1.2, `create_agent(checkpointer=True)` automatically inherits the parent graph's checkpointer, so subagents resume under the same `thread_id` without passing an explicit instance. Useful when HITL middleware is applied at the subagent level.

```
calendar_agent = create_agent(
    model="gpt-5.4",
    tools=[create_calendar_event],
    checkpointer=True, # inherit from parent graph
    middleware=[HumanInTheLoopMiddleware(interrupt_on={
        "create_calendar_event": {"allowed_decisions": ["approve", "reject"]},
    })],
    name="calendar_agent",
)
```

`ToolRuntime[None, CustomState]`

When you need *custom state* in tools but no context, parametrise `ToolRuntime` with `None` as the first type and the state schema as the second.

```
from typing import NotRequired
from langchain.tools import tool, ToolRuntime
from langchain.agents import AgentState
```



```
class JobState(AgentState):
    pending_jobs: NotRequired[list[str]]

    @tool
    def list_pending(runtime: ToolRuntime[None, JobState]) -> str:
        return ", ".join(runtime.state.get("pending_jobs", []))
```

Async 5-tool dispatch pattern

A production-ready async subagent pattern exposes five tools, one responsibility each: `start_job`, `check_job`, `cancel_job`, `list_jobs`, `wait_for`. Pair these with an external queue (Celery, Redis Streams, Cloud Tasks) so job state survives process restarts.

```
supervisor_dispatch = create_agent(
    model="gpt-5.4",
    tools=[dispatch],
    system_prompt=(
        "Use delegate tool to route work."
        "Agent: 'calendar', 'email'."
    ),
)
```

Summary

Item	core
<i>Tier 3</i>	tool → subagent(<code>create_agent</code>) → Supervisor
<i>Wrapping</i>	<code>\@tool</code> + <code>subagent.invoke()</code> → Return last content
<i>Quarantine</i>	subagent = clean context, only supervisor remembers full
<i>HITL</i>	<code>HumanInTheLoopMiddleware</code> + <code>InMemorySaver</code>
<i>Context</i>	<code>ToolRuntime(context={...})</code> → <code>runtime_context</code>
<i>Asynchronous</i>	Job ID → Status → Result pattern
<i>Dispatch</i>	Single <code>dispatch(agent_name, query)</code> tool

Next Steps

→ [03_multi_agent_handoffs_router.ipynb](#): Learn Handoffs and Router patterns.

03 multi

agent: Handoffs & Router — State machines and parallel routing

Learning Objectives

- Handoffs pattern: Implements state variable-based dynamic configuration (prompt + tool replacement)
 - Trigger a state transition from tool to the `Command` object.
 - Router pattern: structured output classification → `Send` API parallel execution → result synthesis
-

Part A — Handoffs: Customer Support State Machine

3.1 Environment Setup

This notebook covers two multi-agent patterns:

- *Part A — Handoffs*: State machine pattern where a single agent dynamically swaps prompts and tool based on state variables.
- *Part B — Router*: A pattern that classifies queries, routes them in parallel to professional agents, and synthesizes the results.

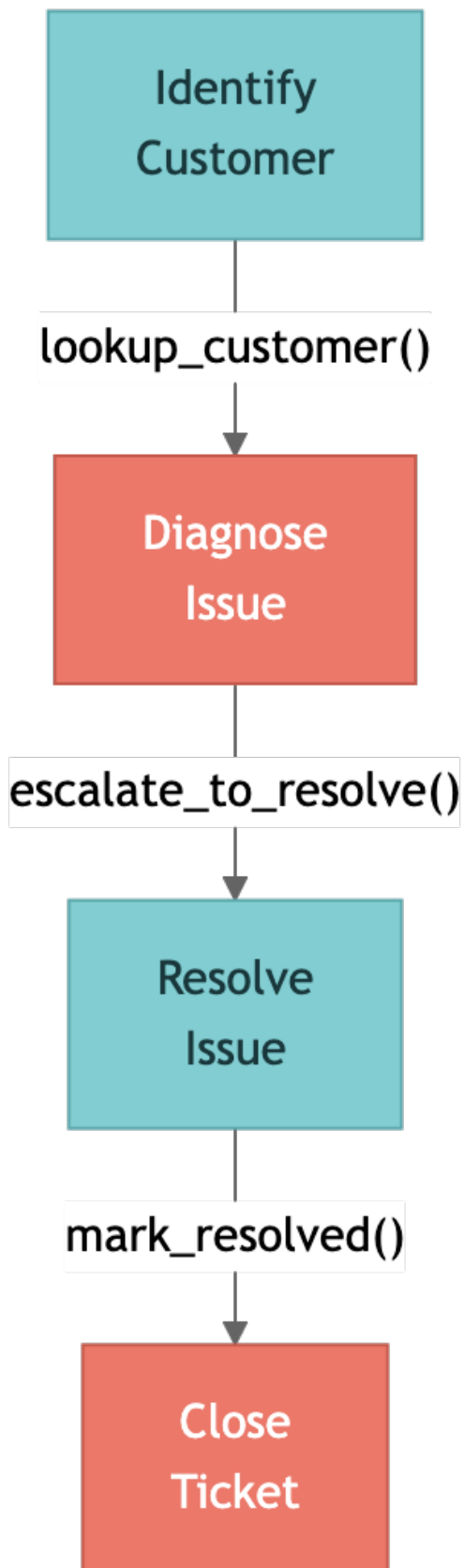
```
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv()

model = ChatOpenAI(model="gpt-5.4")
```

3.2 Handoffs Overview

The Handoffs pattern is an architecture where a *single agent* dynamically changes its behavior based on state variables. Rather than switching between multiple agents, one agent uses different sets of system prompts and tool depending on the step.



Core Mechanism

mechanism	Description
<code>current_step</code>	State variable that tracks the current step. This value determines the agent's behavior
<code>Command(update={...})</code>	tool returns, triggering a state transition. <code>current_step</code> Change + Save Additional Data
<code>\@wrap_model_call</code>	Middleware reads <code>current_step</code> and dynamically replaces tool with system prompt

Key characteristics of Handoffs

- *State-driven behavior*: Settings are adjusted based on tracked state variables
- *tool-based transitions*: tool returns a `Command` object to update state
- *Direct user interaction*: Messages are processed independently at each stage
- *Persistent state*: The state persists beyond conversation turns

Important implementation details

When tool updates a message through `Command`, it must include `ToolMessage` with a matching `tool_call_id`. LLM expects responses to be paired with tool calling, so missing them will result in a malformed conversation history.

When to use

This is ideal when you need sequential constraints, interact directly with the user in each state, or collect information in a specific order in a multi-step flow (e.g. customer support).

3.3 SupportState definition

Inherit from `AgentState` and add the `current_step` field. This field determines the current node of the state machine and limits the valid steps to type `Literal`.

The default is `"identify_customer"`, which means all conversations start at the customer identification stage. Afterwards, when tool returns `Command(update={"current_step": "..."})`, it automatically transitions to Next Steps.

```
from langchain.agents import AgentState
from typing import Literal

class SupportState(AgentState):
    current_step: Literal[
        "identify_customer", "diagnose_issue",
        "resolve_issue", "close_ticket",
    ] = "identify_customer"
```

3.4 Step-by-step tool definition

Each step is assigned tool that matches the role of that step. tool is divided into two types:

- *State transition tool*: Returns `Command(update={...})` to change `current_step` and store additional data in the state. The string result to be shown to LLM is also delivered to the `result` field.

- *General tool*: Returns a string and does not change state. Used for information retrieval, etc.

Design recommendations:

- State transitions must only occur through tool, which returns `Command`
- Backward transitions (going back to the previous step) are also allowed when necessary.
- Prevent step skipping by validating invalid transitions in middleware

```
from langchain_core.tools import tool
from langgraph.types import Command

# --- Identify Customer ---
@tool
def lookup_customer(email: str) -> Command:
    """Track customers by email."""
    return Command(
        update={"customer": {"name": "Alice", "id": "C-1234"}, "current_step": "diagnose_issue"},
        result="Customer found: Alice (C-1234). Go to the diagnosis step.",
    )
```

```
# --- Diagnose Issue ---
@tool
def check_service_status(service_name: str) -> str:
    """Check the current status of the service."""
    return f"Service '{service_name}': healthy (99.9% uptime)"
```

```
@tool
def escalate_to_resolve(diagnosis: str) -> Command:
    """After diagnosis, move on to resolution steps."""
    return Command(
        update={"diagnosis": diagnosis, "current_step": "resolve_issue"},
        result=f"Diagnosis complete: {diagnosis}. Moving to the resolution step.",
    )
```

```
# --- Resolve Issue ---
@tool
def apply_fix(fix_type: str, customer_id: str) -> Command:
    """Apply modifications to customer accounts."""
    return Command(
        update={"resolution": {"type": fix_type}},
        result=f"Fix applied: {fix_type} for {customer_id}",
    )
```

```
@tool
def mark_resolved(summary: str) -> Command:
    """Mark the issue as resolved and move it to the Close stage."""
    return Command(
        update={"current_step": "close_ticket", "resolution_summary": summary},
        result="Solved. Go to the termination step.",
    )
```

```
# --- Close Ticket ---
@tool
def send_satisfaction_survey(customer_id: str) -> str:
    """Send a satisfaction survey."""
    return "Survey completed."

@tool
def close_ticket(ticket_id: str, notes: str) -> str:
```

```
"""Close the support ticket."""
return f"Ticket {ticket_id} closed."
```

3.5 @wrap_model_call middleware

`@wrap_model_call` Middleware is the core of the Handoffs pattern. Intercepts LLM calls and dynamically replaces system prompts and available tool depending on `current_step`.

Sequence of operations:

1. The middleware reads the `current_step` value from the state
2. Look up the settings (prompt + tool) for that step in the `STEP_CONFIG` dictionary.
3. Override `config` and pass it to LLM
4. Call LLM with modified settings with `next_fn(state, config)`

This is the core mechanic of Handoffs: a single agent will have completely different personas and abilities depending on their status. Achieve dynamic behavior changes with a single middleware, without the need to create multiple agents.

Single-agent handoff via `request.override(system_prompt=, tools=)`

Rather than running multiple agent nodes that hand off through `Command(goto=...)`, v1 lets you keep a *single agent node* and swap its persona/tools from inside a `@wrap_model_call` middleware. Message history stays intact, so the user sees one continuous conversation.

```
@wrap_model_call
def step_middleware(request, handler):
    step = request.state.get("current_step", "identify_customer")
    cfg = STEP_CONFIG[step]
    request = request.override(system_prompt=cfg["system_prompt"], tools=cfg["tools"])
    return handler(request)
```

! WARNING

`request.override(...)` returns a copy. You must rebind `request = request.override(...)` before passing it to `handler(request)`.

Subgraph message rules — pair `AIMessage` + `ToolMessage`

When a tool updates `messages` directly through `Command`, the update must contain both:

1. An `AIMessage` declaring the tool call (`tool_calls=[{"id": ..., ...}]`),
2. A `ToolMessage` with a matching `tool_call_id`.

Missing the pair causes `Unmatched tool_call_id` on the next model call. `Command(update={...}, result=...)` generates the pair automatically; if you build `messages` by hand, include both.

```
from langchain.messages import AIMessage, ToolMessage
from langgraph.types import Command

return Command(
    update={"current_step": "diagnose_issue"},
```

```
        result="Found customer. Moving to diagnosis.",
    )
```

```
STEP_CONFIG = {
    "identify_customer": {
        "tools": [lookup_customer],
        "system_prompt": "Identify your customers. Request your email or account ID.",
    },
    "diagnose_issue": {
        "tools": [check_service_status, escalate_to_resolve],
        "system_prompt": "Diagnose the issue. Call escalate_to_resolve after using tool.",
    },
}
```

```
STEP_CONFIG["resolve_issue"] = {
    "tools": [apply_fix, mark_resolved],
    "system_prompt": "Solve the issue. After applying the fix, call mark_resolved.",
}
STEP_CONFIG["close_ticket"] = {
    "tools": [send_satisfaction_survey, close_ticket],
    "system_prompt": "Thank your customers, send surveys, and close tickets.",
}
```

```
from langchain.agents.middleware import wrap_model_call

@wrap_model_call
def step_middleware(request, handler):
    """Dynamically configures agents based on current_step."""
    step = request.state.get("current_step", "identify_customer")
    cfg = STEP_CONFIG[step]
    request = request.override(
        system_prompt=cfg["system_prompt"],
        tools=cfg["tools"],
    )
    return handler(request)
```

3.6 Agent creation and execution flow

When creating an agent, register all tool, but specify `state_schema=SupportState` and middleware. Because the middleware filters tool according to `current_step` at runtime, each step only exposes tool for that step to the LLM.

Example execution flow: This is done automatically by tool, which returns [identify_customer] User: "I can't log in. Email: alice@example.com" → lookup_customer("alice@example.com") ← Command(update={customer: {...}, current_step: "diagnose_issue"})

[diagnose_issue] Agent: "I found your account. What seems to be the issue?" → check_service_status("auth-service") → "healthy" → escalate_to_resolve("Locked after three failed login attempts")

[resolve_issue] → apply_fix("reset_password", "C-1234") → mark_resolved("Password reset completed")

[close_ticket] → send_satisfaction_survey("C-1234") → close_ticket("T-5678", notes="Password reset")

Each step transition is driven by a ``Command` object`.

```
from langchain.agents import create_agent

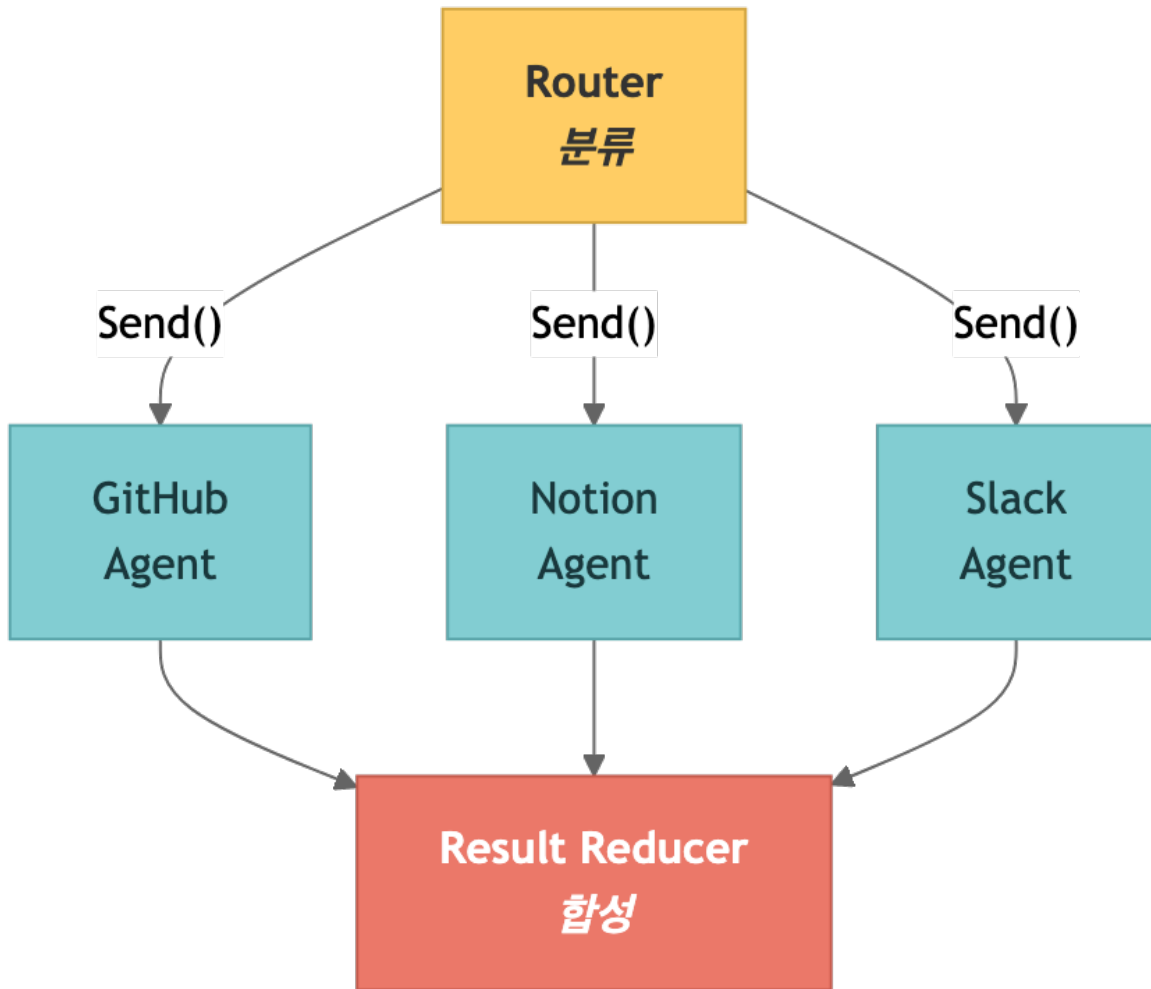
all_tools = [
    lookup_customer, check_service_status,
    escalate_to_resolve, apply_fix, mark_resolved,
    send_satisfaction_survey, close_ticket,
]
support_agent = create_agent(
    model="gpt-5.4", tools=all_tools,
    state_schema=SupportState, middleware=[step_middleware],
)
```

```
# [identify_customer]
#   User: "Can't log in. Email: alice@example.com"
#   Agent -> lookup_customer("alice@example.com")
#       <- Command(update={current_step: "diagnose_issue"})
#
# [diagnose_issue] (auto transition)
#   Agent -> check_service_status("auth-service")
#   Agent -> escalate_to_resolve("Locked out")
#
# [resolve_issue] -> [close_ticket]
```

Part B — Router: Parallel routing and result synthesis

3.7 Router Overview

The Router pattern is an architecture that classifies input and routes it to specialized agents. Unlike the Subagents pattern, Router distributes queries via a dedicated classification step (either a single LLM call or rule-based logic).



Pipeline

steps	Role	implementation
<i>Classification</i>	Analyze queries to create related sources and subqueries	<code>with_structured_output(QueryClassification)</code>
<i>Parallel Dispatch</i>	Deliver subqueries to each classified source simultaneously	<code>Send</code> API
<i>Result synthesis (Reduction)</i>	Gather all agent results to create a unified response	Reducer node + LLM

Router vs. Subagents Comparison

Routers have a “dedicated routing stage (classification),” while Subagents have “supervisor agents dynamically” deciding what to call. Router is suitable when distinct knowledge domains (verticals) are clearly distinguished and parallel queries are required.

Router vs. Supervisor vs. Handoffs

Pattern	Control flow owner	Execution	Best fit
<i>Router</i>	Dedicated classifier	Parallel fan-out / fan-in	Distinct knowledge verticals
<i>Supervisor (Subagents)</i>	Supervisor agent tool-calls	Sequential or parallel	Conversational + many domains
<i>Handoffs</i>	State variable (<code>current_step</code>)	Single agent, persona swap	Sequential multi-step flows

Architecture Mode

- *Stateless*: Each request is routed independently (no memory)
- *Stateful*: Supports multi-turn interactions by maintaining conversation history. An alternative is to wrap a stateless router with tool, or have the router itself manage the state directly.

3.8 RouterState and classification schema

`QueryClassification` is a Pydantic model, which classifies queries into a structured form through LLM's `with_structured_output()`. `RouterState` tracks classification results, source lists, subqueries, and agent results.

Key fields in the classification schema:

- `sources` : Which knowledge sources are relevant (multiple choices possible)
- `reasoning` : Explain why you selected the source
- `sub_queries` : Subquery optimized for each source (original query reorganized for each source)

```
from pydantic import BaseModel, Field
from typing import Literal

class SubQuery(BaseModel):
    """Subqueries by source."""
    source: Literal["github", "notion", "slack"] = Field(description="Knowledge source.")
    query: str = Field(description="Search queries optimized for that source.")

class QueryClassification(BaseModel):
    """Classification results from user queries."""
    sources: list[Literal["github", "notion", "slack"]] = Field(
        description="Related knowledge sources."
    )
    reasoning: str = Field(description="Why you chose that source.")
    sub_queries: list[SubQuery] = Field(description="Subqueries by source.")
```

```
from langchain.agents import AgentState

class RouterState(AgentState):
    classification: QueryClassification = None
    sources: list[str] = []
    sub_queries: list[SubQuery] = []
    agent_results: list[dict] = []
```

3.9 Classification Node

`with_structured_output` classifies queries by source and creates subqueries optimized for each source.

Classification example

user query	Category Source	Reason
"How to deploy the auth service"	["github", "notion"]	Distribution code exists on GitHub, procedure documentation exists on Notion
"How the decision was made to change the API"	["slack", "notion"]	Discussions are recorded in Slack, decision documents are recorded in Notion
"Login bug PR"	["github"]	PR only exists on GitHub
"Onboarding Process and Starter Repo"	["github", "notion", "slack"]	Repo is GitHub, process is Notion, context is Slack

Subquery creation is important: optimize the original query "auth service deployment" to

"auth service deployment scripts CI/CD pipeline" for GitHub and "auth service deployment process procedure runbook" for Notion, respectively.

3.10 Parallel Routing (Send API)

The `Send` API dispatches subqueries to each classified source simultaneously. In the form of

`Send(node_name, payload)`, data is passed in parallel to specific nodes in the graph.

This parallel execution is the core strength of the Router pattern: sequentially querying multiple knowledge sources adds up the latency, but parallel execution via `Send` takes only as long as the response time of the slowest source.

Add new source

The Router pattern is simple to extend:

1. Define source-specific tool
2. Create a professional agent
3. Add new source to `QueryClassification.sources`
4. Add agent nodes to the graph
5. Connect to Reducer

```
from langchain_core.tools import tool
from langchain.agents import create_agent

@tool
def search_github_code(query: str) -> str:
    """Search the GitHub repository."""
    return f"GitHub result for '{query}'"
```

```
@tool
def search_notion_pages(query: str) -> str:
    """Search for your Notion workspace."""
    return f"Notion result for '{query}'"
```

```
@tool
def search_slack_messages(query: str) -> str:
    """Search for Slack messages."""
    return f"Slack result for '{query}'"
```

```
github_agent = create_agent(
    model="gpt-5.4", tools=[search_github_code],
    system_prompt="Search for code and PRs on GitHub.",
    name="github_agent",
)
notion_agent = create_agent(
    model="gpt-5.4", tools=[search_notion_pages],
    system_prompt="Search documents in Notion.",
    name="notion_agent",
)
```

```
slack_agent = create_agent(
    model="gpt-5.4", tools=[search_slack_messages],
    system_prompt="Search for discussions in Slack.",
    name="slack_agent",
)
```

```
from langgraph.types import Send

def dispatch_to_agents(state):
    """Subqueries are passed to agents in parallel."""
    cls = state["classification"]
    sq_dict = {sq.source: sq.query for sq in cls.sub_queries}
    return [
        Send(src, {"messages": [{"role": "user", "content": sq_dict.get(src, "")}], "source": src})
        for src in cls.sources
    ]
```

When the classifier returns a list of dicts like `[{"agent": "github", "query": "...", ...}]`, fan out with `Send(c["agent"], {...})`:

```
def dispatch_dict_form(state):
    return [
        Send(c["agent"], {"messages": [{"role": "user", "content": c["query"]}]}))
        for c in state["classification"]
    ]
```

3.11 Result synthesis

Reducer collects the results from all agents and uses LLM to synthesize an integrated response. When synthesizing, the source of each information is cited so that the user can determine where the information came from.

The synthesis prompt instructs you to indicate the source. For example, respond with “The deployment script is in the `payment-service` repo on GitHub (GitHub), and for deployment procedures, please refer to Notion’s ‘Payment Service Ops’ document (Notion).”

```
from langgraph.graph import StateGraph, START, END

graph = StateGraph(RouterState)
graph.add_node("router", route_query)
graph.add_node("github", github_agent)
graph.add_node("notion", notion_agent)
graph.add_node("slack", slack_agent)
graph.add_node("reducer", reduce_results)

graph.add_edge(START, "router")
graph.add_conditional_edges("router", dispatch_to_agents)
graph.add_edge("github", "reducer")
graph.add_edge("notion", "reducer")
graph.add_edge("slack", "reducer")
graph.add_edge("reducer", END)

app = graph.compile()
```

Summary

Part A — Handoffs

Item	core
<i>Pattern</i>	Single agent + <code>current_step</code> based dynamic configuration
<i>Transition</i>	<code>Command(update={"current_step": "next"})</code>
<i>Dynamic Configuration</i>	prompt with <code>\@wrap_model_call</code> + replace tool

Part B — Router

Item	core
<i>Category</i>	<code>with_structured_output(QueryClassification)</code>
<i>Parallel</i>	<code>Send(source, payload)</code> API
<i>Synthesis</i>	LLM integrated response from reducer node

Next Steps

→ [04_context_memory.ipynb](#): Learn context engineering and memory.

Context Engineering & Memory Deepening

04

- Static/Dynamic Context, InMemoryStore, Skills Pattern

Deep learning of LangGraph's context system and long-term memory (Store). Covers everything from static/dynamic runtime contexts to long-term memory based on semantic search, and Progressive Disclosure (Skills) patterns.

Learning Objectives

- Understand the two-dimensional (Mutability x Lifetime) matrix of context engineering.
- Implement a static runtime context with `context_schema` + `@dataclass`
- Manage dynamic runtime context with `state_schema` and `AgentState` customizations
- Utilizes `InMemoryStore`'s namespace, put, get, and search APIs
- Build long-term memory based on semantic search
- Design by distinguishing between 3 types of memory (Semantic, Episodic, Procedural)
- Implement Progressive Disclosure using Skills pattern
- Compare hot path vs background memory writing strategies

4.1 Environment Setup

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI, OpenAIEmbeddings

model = ChatOpenAI(model="gpt-5.4")
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
print("Environment ready.")
```

4.2 Context Engineering Overview

Context engineering is the design of systems that provide AI with “the right information, in the right format, at the right time.” Beyond simple prompt engineering, it is an architectural approach to programmatically assembling contexts at runtime.

There are two main reasons why agents fail:

1. Lack of LLM skills
2. *Lack of context or inappropriate context* (more frequent cause)

Therefore, context engineering is a core role for AI engineers and a fundamental solution to agent reliability.

Two-dimensional matrix: Mutability x Lifetime

Static (immutable)	User ID, DB connection, tool definition	config files, etc.
<i>Dynamic (variable)</i>	Conversation history, intermediate results	User Preferences, Learned Memory

3 context types

Type	Mutability	Lifetime	Example	LangGraph Implementation
Static Runtime	Static	Single run	User ID, DB conn	context_schema
Dynamic Runtime (State)	Dynamic	Single run	Messages, interim results	state_schema
Dynamic Cross-conv (Store)	Dynamic	Cross-conversation	Affinity, Memory	InMemoryStore

3 controllable context categories

Category	Control object	Characteristics
<i>Model Context</i>	Instructions, message history, tool, response format	Transient
<i>Tool Context</i>	tool Access, read/write state, runtime context	Persistent
<i>Life-cycle Context</i>	Transformation between stages, Summary, guardrails	Persistent

LangChain implements context engineering with a *middleware* mechanism. Middleware such as `@dynamic_prompt` and `@wrap_model_call` can be used to update context or control between lifecycle stages.

4.3 Static runtime context – context_schema + @dataclass

Injects *unchanging* information into `context_schema` while the agent is running. Define the schema as `@dataclass`, and access it as `ToolRuntime[Context]` from tool.

```
from dataclasses import dataclass
from langchain.tools import tool, ToolRuntime
from langchain.agents import create_agent

@dataclass
class Context:
    user_id: str
    role: str
    department: str
```

```
@tool
def get_permissions(runtime: ToolRuntime[Context]) -> str:
    """View permissions based on the current user's role."""
    ctx = runtime.context
    perms = {"admin": "read,write,delete", "editor": "read,write"}
    return f"User {ctx.user_id} ({ctx.department}): {perms.get(ctx.role, 'read')}"

agent = create_agent(
    model=model,
    tools=[get_permissions],
    context_schema=Context,
)
agent.invoke(
    {"messages": [{"role": "user", "content": "What can I do?"}]},
    context=Context(user_id="u42", role="admin", department="eng"),
)
```

`runtime.execution_info` / `runtime.server_info`

Beyond context, `runtime` exposes execution metadata. `runtime.execution_info` carries `thread_id` / `run_id` /step for the current invoke, and `runtime.server_info` exposes the LangGraph server environment (deployment id, host). Useful for audit logs and distributed tracing.

```
@tool
def audited_action(action: str, runtime: ToolRuntime[Context]) -> str:
    info = runtime.execution_info
    print(f"[audit] user={runtime.context.user_id} "
          f"thread={info.thread_id} run={info.run_id} action={action}")
    return f"Executed: {action}"
```

Key points

- You can use a type-safe context by passing `@dataclass` to `context_schema`
- Automatically injected as `runtime: ToolRuntime[Context]` type hint in tool function
- *read-only* and unchangeable while running
- Suitable data: User ID, DB connection, API key, session metadata

4.4 Dynamic runtime context – `state_schema`, `AgentState` custom

This state *changes* as the agent processes messages and calls tool. Add custom fields by inheriting `AgentState`.

```
from langchain.agents import AgentState

class RAGState(AgentState):
    """State including dynamic search context."""
    retrieved_docs: list[str]
    query_count: int

print(f"State keys: {list(RAGState.__annotations__.keys())}")
```

Static vs Dynamic Comparison

Category	Static Runtime (<code>context_schema</code>)	Dynamic Runtime (<code>state_schema</code>)
Whether to change	immutable (read-only)	variable (node updates)

Delivery method	<code>context= parameters</code>	invoke input dict
Approach	<code>runtime.context.field</code>	<code>state["field"]</code>
suitable data	Authentication Information, Settings	Conversation history, intermediate results

4.5 Long-term memory – InMemoryStore native API

Use `InMemoryStore` for cross-conversation context. Long-term memory is per-user or app-level data that persists across sessions and threads.

Storage structure

The memory is stored as a *JSON document*, organized hierarchically into *namespace*:

- *namespace*: Folder role to classify memory (e.g. `(user_id, "preferences")`)
- *key*: Unique identifier for each memory (e.g. `"theme"`)
- Namespaces usually include user IDs or organization IDs to facilitate information management.

Basic API

API	Description
<code>store.put(namespace, key, value)</code>	Save memory (upsert)
<code>store.get(namespace, key)</code>	Memory query by specific key
<code>store.search(namespace)</code>	Search all within a namespace
<code>store.search(namespace, filter={...})</code>	Search by filter conditions

In production environments, you should use *DB-based Store* (e.g. PostgreSQL) instead of `InMemoryStore` .

PostgreSQL Store

`langgraph-checkpoint-postgres` ships `PostgresStore` , a drop-in replacement with the same `put / get / search` API but PostgreSQL-backed persistence. With `pgvector` , semantic search works the same way. Use this as the default production backend; `AsyncPostgresStore` is the async variant.

```
# pip install langgraph-checkpoint-postgres
from langgraph.store.postgres import PostgresStore

DB_URI = "postgresql://user:pass@localhost:5432/langgraph?sslmode=disable"

with PostgresStore.from_conn_string(DB_URI) as store:
    store.setup()

    store_with_index = PostgresStore.from_conn_string(
        DB_URI,
        index={"embed": embeddings, "dims": 1536},
    )

    agent = create_agent(
        model=model,
        tools=[get_user_info, save_preference],
        store=store_with_index,
```

```

        context_schema=Context,
    )

```

```

from langgraph.store.memory import InMemoryStore

store = InMemoryStore()
user_id = "user_42"
store.put((user_id, "preferences"), "theme", {"value": "dark"})
store.put((user_id, "preferences"), "language", {"value": "ko"})

item = store.get((user_id, "preferences"), "theme")
print(f"Theme: {item.value}")

```

```

items = store.search((user_id, "preferences"))
for item in items:
    print(f" [{item.key}] = {item.value}")

filtered = store.search(
    (user_id, "preferences"), filter={"value": "dark"}
)
print(f"Filtered results: {len(filtered)}")

```

4.6 Long-Term Memory – Semantic Search

Setting the embedding function makes `InMemoryStore` support *semantic search*. Perform semantic-based similarity search with the `query` parameter.

```

semantic_store = InMemoryStore(
    index={"embed": embeddings, "dims": 1536}
)
ns = ("user_42", "memories")
semantic_store.put(ns, "mem1", {"content": "Prefer pytest over unittest"})
semantic_store.put(ns, "mem2", {"content": "Use type hints for all functions"})
semantic_store.put(ns, "mem3", {"content": "Favorite food is sushi"})
semantic_store.put(ns, "mem4", {"content": "Works on the ML Infrastructure team"})
print("Four memories have been saved along with embedding.")

```

```

results = semantic_store.search(
    ("user_42", "memories"), query="testing preferences", limit=2
)
for r in results:
    print(f" [{r.key}] {r.value['content']}")

```

```

results2 = semantic_store.search(
    ("user_42", "memories"), query="machine learning work", limit=2
)
for r in results2:
    print(f" [{r.key}] {r.value['content']}")

```

Basic Store vs Semantic Store comparison

Features	<code>InMemoryStore()</code>	<code>InMemoryStore(index={...})</code>
----------	------------------------------	---

Accurate key lookup	<code>get(ns, key)</code>	<code>get(ns, key)</code>
Filter Search	<code>search(ns, filter={...})</code>	<code>search(ns, filter={...})</code>
Semantic Search	Not possible	<code>search(ns, query="...", limit=N)</code>
Production	Use DB backend instead of <code>InMemoryStore</code>	PostgreSQL-based Store recommended

4.7 Reading/Writing Store from tool – `ToolRuntime.store`

You can access the Store from within the agent's tool to read and write user information. If you connect a Store to `create_agent(store=...)`, it will be automatically injected into `runtime.store`.

Reading Pattern

Search for user information saved through `runtime.store` in tool. Both context and Store can be accessed with `ToolRuntime[Context]` type hints.

Writing Pattern

Receives user input with the tool parameter and saves the memory as `store.put()`. This allows agents to permanently store information learned during a conversation.

Key points

- `runtime.store`: Access Store instance
- `runtime.context`: Access static runtime context
- By combining Store and Context, you can systematically manage “*whose (context) information (store)*”

```
@tool
def get_user_info(runtime: ToolRuntime[Context]) -> str:
    """Search the saved information of the current user."""
    store = runtime.store
    user_id = runtime.context.user_id
    info = store.get((user_id, "memories"), "profile")
    # info.value may be a lazy / dict-subclass – cast to dict explicitly
    return str(dict(info.value)) if info else "User information not found."
```

```
@tool
def save_preference(key: str, value: str, runtime: ToolRuntime[Context]) -> str:
    """Stores user preferences."""
    store = runtime.store
    user_id = runtime.context.user_id
    # Recommended namespace: (user_id, "memories") – compatible with semantic-search store
    store.put((user_id, "memories"), key, {"value": value})
    return f"Preference saved: {key}={value}"
```

4.8 3 types of memory: Semantic, Episodic, Procedural

Long-term memory is categorized into three types inspired by cognitive science. *Storage structure* and *utilization* are different for each type.

Type	Description	Example	structure
<i>Semantic</i>	factual knowledge about entities	User preferences, profile information	Profile or Collection
<i>Episodic</i>	Memories of past experiences and events	Few-shot example, past action log	Collection
<i>Procedural</i>	Rules/Guidelines on how to do it	Fix system prompt, guidelines	Profile (list of rules)

Semantic Memory – Profile vs Collection

Semantic memory has two approaches depending on the storage strategy:

Approach	structure	Suitable for	Example
<i>Profile</i>	Single JSON document, continuously updated	Few well-known properties	<code>{"name": "Alice", "language": "Python", "preferred_style": "concise"}</code>
<i>Collection</i>	Multiple narrow documents, high recall	Open-end or large-scale knowledge	<code>[{"topic": "testing", "content": "Prefers pytest"}, ...]</code>

Episodic Memory

Record how you have behaved in similar situations in the past. It is used as a few-shot example, allowing the agent to learn from past experiences.

Procedural Memory

Stores the agent's behavioral rules. This has the effect of dynamically modifying system prompts, allowing the agent to follow user-specific instructions.

```
mem_store = InMemoryStore(index={"embed": embeddings, "dims": 1536})
uid = "user_42"

# Semantic -- Profile (single JSON)
mem_store.put((uid, "profile"), "main", {
    "name": "Alice", "language": "Python",
    "preferred_style": "concise",
})
# Semantic -- Collection (multiple docs)
mem_store.put((uid, "facts"), "f1", {"content": "pytest preferred"})
```

```
# Episodic -- past experiences (few-shot)
mem_store.put((uid, "episodes"), "ep1", {
    "content": "SQL Optimization -> Use EXPLAIN ANALYZE",
})
```

```
# Procedural -- rules/guidelines
mem_store.put((uid, "procedures"), "rules", {
    "content": "Always includes error handling. Use logging.",
})
print("All three memory types have been saved.")
```

```
# Episodic search: find similar past experiences
episodes = mem_store.search(
    (uid, "episodes"), query="database query help", limit=1
)
for ep in episodes:
    print(f"Related episode: {ep.value['content']}")
```

4.9 Progressive Disclosure – Skills pattern

Putting all the context in the prompt increases token cost and reduces accuracy. The Skills pattern is a Progressive Disclosure method that loads relevant information only when needed.

Structure of Skill

Skill is a unit of knowledge consisting of `{name, description, content}` :

- *name*: Skill identifier (e.g. `"customers_schema"`)
- *description*: Short description (included in system prompt)
- *content*: Details (load on demand with `load_skill` tool)

Strategies by Size

size	strategy	Example
<code><1K tokens</code>	Included directly in the system prompt	table names, high-level relationships
<code>1-10K tokens</code>	Load on demand with <code>load_skill</code> tool	Table schema, query patterns, best practices
<code>>10K tokens</code>	Load on demand with pagination	Large reference data, historical query logs

Action flow

1. *Middleware* injects the names and descriptions of all skills into the system prompt
2. The agent analyzes the question and determines the skills needed
3. Call `load_skill` tool to load details
4. Perform actions based on loaded content

Advantages

Advantages	Description
<i>Token Efficiency</i>	Load only the information needed for the current query
<i>Scalability</i>	DB with hundreds of tables is also supported

<i>Accuracy</i>	Provide detailed schema at the point you need it
<i>Cost Savings</i>	Reduce input tokens per request

```
skills = [
    {"name": "db_overview",
     "description": "High-level Overview for all tables",
     "content": "Table: customers, orders, products"},
    {"name": "customers_schema",
     "description": "Full schema of the customers table",
     "content": "CREATE TABLE customers (id INT PK, name VARCHAR)"},
]
SKILL_MAP = {s["name"]: s for s in skills}
print(f"Defined {len(skills)} skills.")
```

```
from langchain_core.tools import tool

@tool
def load_skill(skill_name: str) -> str:
    """Loads detailed information about the database skill."""
    skill = SKILL_MAP.get(skill_name)
    if skill is None:
        return f"Not found. Available: {', '.join(SKILL_MAP.keys())}"
    return f"## {skill['name']}\n\n{skill['content']}"
```

4.10 Hot Path vs Background Memory Write

Depending on *when* you use memory, it will affect user response latency.

method	Timing	Available immediately?	Delay Impact
<i>Hot path</i>	Real-time within conversation loop	Instantly (available next turn)	Increased response delay
<i>Background</i>	Separate asynchronous task	Delayed (Eventual Consistency)	No delay impact

Write Hot Path

Save memory inline within the agent loop. This is perfect when you need to use that memory in the very next turn. Example: When you need to immediately reflect the preferences that the user just shared.

Background writing

Save memory as a separate process or asynchronous task. Used when Eventual Consistency is allowed and does not affect response delay. Examples: conversation pattern analysis, long-term learning data accumulation.

Selection criteria

- Is an immediate recall necessary? -> *Hot path*
- Is reducing delay a priority? -> *Background*
- In most cases, background writing is preferred.

```

from langgraph.store.base import BaseStore

# Hot path: write inline (adds latency)
def reflect_node(state, store: BaseStore):
    """Extract and store memory inline."""
    last_msg = state["messages"][-1].content
    store.put(("user", "reflections"), "latest", {"content": last_msg})
    return state

print("Hot path: Immediately save, available next turn.")

```

```

import asyncio

# Background: write in separate async task
async def background_memory_writer(state, store: BaseStore):
    """Saves memory in the background (no delays)."""
    last_msg = state["messages"][-1].content
    await store.put(
        ("user", "reflections"), "latest", {"content": last_msg}
    )
print("Background: Eventual consistency, no delay.")

```

Summary

Context Engineering Triad

element	implementation	API
static runtime	context_schema + \@dataclass	runtime.context.field
dynamic runtime	state_schema + AgentState	state["field"]
long term memory	InMemoryStore + store=	store.put/get/search

Memory 3 types

Type	Use	Namespace example
Semantic	User Profile/Facts	(user_id, "profile") , (user_id, "facts")
Episodic	Past experience (few-shot)	(user_id, "episodes")
Procedural	Edit Rule/Prompt	(user_id, "procedures")

Best Practices

Principle	Description
minimize static context	Include only what is needed for the current task
Namespace structuring	Use hierarchical namespace to avoid collisions

Semantic search priority	Embedding-based search is more scalable than exact matching
Background writing preference	Reduce delay to background when immediate recall is not required
Skills pattern application	Large-scale context can be found in Progressive Disclosure

Next Steps

→ [05_agentic_rag.ipynb](#): Learn Agentic RAG.

05 Agentic RAG

- Built directly with LangGraph

We implement Retrieval-Augmented Generation (RAG) in three ways: LangChain RAG Agent, LangChain RAG Chain, and a custom RAG based on LangGraph StateGraph. Covers in-depth patterns such as document relevance evaluation, query rewriting, and conditional routing.

Learning Objectives

- Understand the overall structure of the RAG pipeline (Indexing -> Search -> Generation)
- Chunk the document with `RecursiveCharacterTextSplitter`
- Build a vector store with `InMemoryVectorStore`
- Implement RAG Agent with LangChain `create_agent` + `@tool`
- Implement RAG Chain (single LLM call) with `@dynamic_prompt` middleware
- Build a custom RAG agent with LangGraph `StateGraph`
- `GradeDocuments` Evaluate document relevance with structured output
- Implement query rewriting and conditional routing

5.1 Environment Setup

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI, OpenAIEmbeddings

llm = ChatOpenAI(model="gpt-5.4")
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
print("Environment ready.")
```

5.2 RAG Overview

Retrieval-Augmented Generation (RAG) is a pattern that improves the accuracy of LLM responses by retrieving external knowledge. LLM has two key limitations:

- *Finite context*: the entire corpus cannot be processed at once.
- *Static Knowledge*: Training data becomes outdated over time.

RAG overcomes this limitation by bringing in relevant external information at query time.

Pipeline: Indexing -\> Search -\> Generate

```
[Documents] -> Text Splitter -> [Chunks] -> Embeddings -> [Vector Store]
[Question] -> Embedding -> similarity_search -> [Relevant Chunks] -> LLM -> [Answer]
```

5 Core Components

Components	Role
<i>Document Loaders</i>	Collect data from external sources (Google Drive, Notion, etc.) into a standard Document object
<i>Text Splitters</i>	Split large documents into chunks that fit into the context window
<i>Embedding Models</i>	Convert text into vectors where semantically similar content is grouped close together
<i>Vector Stores</i>	A specialized database that stores embeddings and performs similarity searches
<i>Retrievers</i>	Return related documents based on unstructured queries

Three RAG architectures

Approach	Architecture	LLM Call	Flexibility	Suitable for
<i>2-Step RAG</i>	Created immediately after search	single	low	FAQ, Doc Bot (fast and predictable)
<i>Agentic RAG</i>	Agent decides when/how to search	Multiple	High	Complex research, multiple tool approaches
<i>Hybrid RAG</i>	Query strengthening + search verification + answer quality check	Multiple	High	When repeated purification is required

Agent vs Chain approach (LangChain implementation)

Approach	Architecture	LLM Call	Suitable for
<i>RAG Agent</i>	agent + retriever tool	Multiple	Complex queries, query reorganization required
<i>RAG Chain</i>	Middleware Injection Context	single	Simple Q&A, Predictable Costs
<i>LangGraph Custom</i>	StateGraph + Custom Node	Multiple	Fine-grained control such as relevance evaluation and rewriting

5.3 Document loading & chunking

Document Loaders

The document loader reads raw content from various sources and returns it as a `Document` object with fields `page_content` and `metadata`.

loader	Source	package
<code>PyPDFLoader</code>	PDF file	<code>pypdf</code>
<code>TextLoader</code>	text file	built
<code>CSVLoader</code>	CSV file	built
<code>WebBaseLoader</code>	web page	<code>beautifulsoup4</code>
<code>DirectoryLoader</code>	Files in directory	built

Text Splitting

`RecursiveCharacterTextSplitter` maintains semantic relevance by recursively splitting it in the order `\n\n -> \n -> -> "`. Recommended as the most general purpose splitter.

parameters	Description	Recommended value
<code>chunk_size</code>	Chunk maximum number of characters	500-2000 (small for precise search, large for context preservation)
<code>chunk_overlap</code>	Number of adjacent chunks shared characters	10-20% of <code>chunk_size</code> (to avoid loss of boundary information)

Other dividers

divider	Suitable for
<code>MarkdownHeaderTextSplitter</code>	Markdown document
<code>HTMLHeaderTextSplitter</code>	HTML document
<code>TokenTextSplitter</code>	Token Budget Based Split
<code>CodeTextSplitter</code>	Source code (language recognition)

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_core.documents import Document

raw_docs = [
    Document(page_content="LangGraph is a state-based multi-actor with LLM"
                    "A framework for building applications.",
            metadata={"source": "langgraph-docs"}),
    Document(page_content="Agents use tool to communicate with external systems."
                    "Interact. The ReAct pattern alternates between reasoning and action.",
```

```

        metadata={"source": "agent-guide"}),
    ]
    print(f"Loaded {len(raw_docs)} documents.")

```

```

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000, chunk_overlap=200,
)
splits = text_splitter.split_documents(raw_docs)

for i, doc in enumerate(splits):
    print(f"Chunk {i}: {doc.page_content[:60]}...")
print(f"Total chunks: {len(splits)}")

```

5.4 Building a vector store

Vector Store is a specialized database that indexes embeddings and performs similarity searches.

`InMemoryVectorStore` is suitable for development/testing.

Comparison of major vector stores

Vector Store	Type	Suitable for
<code>InMemoryVectorStore</code>	In-process	Development, small dataset
<code>Chroma</code>	Embedded/Client-Server	Prototyping, medium-scale datasets
<code>FAISS</code>	In-process	High performance local search
<code>Pinecone</code>	Managed Cloud	Production, Scalability
<code>PGVector</code>	PostgreSQL extensions	Leverage existing PostgreSQL infrastructure

Search type

Search type	Description
"similarity"	Standard nearest neighbor search
"mmr"	Maximal Marginal Relevance – Balancing relevance and diversity (reducing duplication)
"similarity_score_threshold"	Return only documents with minimum similarity score or higher

```

from langchain_core.vectorstores import InMemoryVectorStore

vector_store = InMemoryVectorStore.from_documents(
    documents=splits, embedding=embeddings,
)
test_results = vector_store.similarity_search("LangGraph", k=2)
for doc in test_results:
    print(f" [{doc.metadata['source']}] {doc.page_content[:80]}")
print(f"Vector store ready with {len(splits)} documents.")

```

5.5 Search tool Definition

Using `response_format='content_and_artifact'` splits the tool output into two parts:

- *Content*: String expression passed to the model (used for inference)
- *Artifact*: The original Document object (accessible programmatically, but not sent to the model)

This separation allows you to use readable text for the model and the original object with metadata for subsequent processing.

```
from langchain_core.tools import tool

@tool(response_format="content_and_artifact")
def retrieve(query: str):
    """Search our knowledge base for related articles."""
    docs = vector_store.similarity_search(query, k=4)
    serialized = "\n\n".join(
        f"Source: {d.metadata.get('source', '?')}\n{d.page_content}"
        for d in docs
    )
    return serialized, docs
```

5.6 LangChain RAG Agent – `create_agent` + `\@tool`

Simplest way: register the retriever as tool and call the agent when needed.

Multi-step search flow

RAG Agent can automatically run multiple discovery steps:

1. *Initial Search* – Create a query based on user questions
2. *Result evaluation* – Determine whether the retrieved documents are sufficient for the question
3. *Reorganize and re-search* – If there are not enough results, modify the query and re-search
4. *Consolidation* – Combine all search results to create final answer

This approach is suitable for complex research questions, but multiple LLM calls increase costs and delays.

5.7 LangChain RAG Chain – `\@dynamic_prompt` Middleware

Implement RAG with a single LLM call. `\@dynamic_prompt` retrieves the document before the LLM call and automatically injects it into the system prompt. Because it is a middleware method, it operates in a *single pass* without an agent loop.

Characteristics	RAG Agent	RAG Chain
Number of LLM calls	Multiple (agent decision)	single
Number of searches	More than once (agent control)	Exactly once (middleware control)
Query Reconstruction	automatic	Not supported
delay	High (multiple round trips)	Low (single pass)

cost	High (more tokens)	Low (fewer tokens)
Transparency	Agent inference exposes message	Context injection is implicit

Advanced usage: You can also combine both approaches by injecting the default context with `@dynamic_prompt` while simultaneously providing a retriever tool.

```
from langchain.agents.middleware import dynamic_prompt

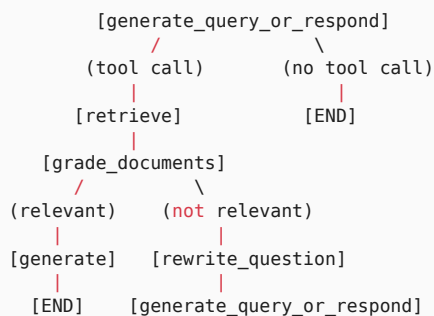
@dynamic_prompt
def rag_prompt(request):
    """It retrieves the document and injects it into the system prompt."""
    user_msg = request.state["messages"][-1].content
    docs = vector_store.similarity_search(user_msg, k=4)
    ctx = "\n\n".join(d.page_content for d in docs)
    return f"Answer based on the following context:\n\n{ctx}"
```

5.8 LangGraph Custom RAG – Building StateGraph

Build your own RAG agent that allows detailed control with LangGraph `StateGraph`. The key advantage of this approach is that conditional routing allows fine-grained flow control, such as evaluating the relevance of search results and rewriting queries if they are irrelevant.

Architecture

The custom RAG graph follows this high-level flow — a *Rewrite* → *Retrieve* → *Agent* loop with a relevance gate in between:



- `generate_query_or_respond` decides whether the model should search or answer directly.
- If a tool call is made, `retrieve` runs the search.
- `grade_documents` evaluates whether the retrieved documents are relevant.
- If the documents are relevant, `generate_answer` produces the final answer.
- If the documents are not relevant, `rewrite_question` rewrites the query and loops back to `generate_query_or_respond`.

· NOTE

Because `rewrite_question` can loop back to `generate_query_or_respond`, add a `retry_count` field to state so the graph cannot loop forever — 2–3 attempts is a sane default before falling back to “answer with what we have”.

Node	Role
<code>generate_query_or_respond</code>	Entry node. Decides whether to search or answer directly.
<code>retrieve</code>	Runs retrieval through a <code>ToolNode</code> .
<code>grade_documents</code>	Evaluates document relevance with structured output (<code>GradeDocuments</code>).
<code>rewrite_question</code>	Rewrites the query into something more specific when the results are not relevant.
<code>generate_answer</code>	Generates the final answer from relevant documents.

```
from langgraph.graph import MessagesState

class AgentState(MessagesState):
    """Custom RAG agent status."""
    relevance: str # "relevant" or "not_relevant"

print(f"AgentState keys: {list(AgentState.__annotations__)}")
```

5.9 `generate_query_or_respond` node

This is the entry node. Determines whether LLM will call retrieve tool or respond directly.

5.10 `grade_documents` Node – Evaluate relevance with structured output

The `GradeDocuments` schema allows LLM to evaluate document relevance. Receive a structured response as `with_structured_output`.

```
from pydantic import BaseModel, Field
from typing import Literal

class GradeDocuments(BaseModel):
    """Binary relevance score of the retrieved document."""
    relevance: Literal["relevant", "not_relevant"] = Field(
        description="Whether the document is relevant."
    )
    reasoning: str = Field(description="A brief explanation.")

grader = llm.with_structured_output(GradeDocuments)
```

```
def grade_documents(state: AgentState):
    """
    Evaluates the relevance of the retrieved documents.
    """

    msgs = state["messages"]

    user_q = next(
        (m.content for m in msgs if m.type == "human"),
        ""
    )
```

```

tool_content = msgs[-1].content

grade = grader.invoke(
    f"Question: {user_q}\nDocument:\n{tool_content}\n"
    f"Are these documents relevant?"
)

return {
    "relevance": grade.relevance,
    "messages": msgs
}

```

5.11 `rewrite_question` node

When retrieved documents are irrelevant, rewrite the original question to be more specific to improve search quality.

5.12 `generate_answer` node

Once relevant documents are identified, the search results and the original question are combined to generate the final answer.

5.13 Graph assembly & execution

Register all nodes in `StateGraph` and connect them with conditional edges (`tools_condition` , `relevance_router`).

```

from langgraph.graph import StateGraph, START, END
from langgraph.prebuilt import ToolNode, tools_condition

def relevance_router(state: AgentState):
    if state.get("relevance") == "relevant":
        return "generate_answer"
    return "rewrite_question"

graph = StateGraph(AgentState)
graph.add_node("gen_query", generate_query_or_respond)

```

```

graph.add_node("retrieve", ToolNode([retrieve]))
graph.add_node("grade_documents", grade_documents)
graph.add_node("rewrite_question", rewrite_question)
graph.add_node("generate_answer", generate_answer)

graph.add_edge(START, "gen_query")
graph.add_conditional_edges(
    "gen_query", tools_condition,
    {"tools": "retrieve", "__end__": END},
)

```

```

graph.add_edge("retrieve", "grade_documents")
graph.add_conditional_edges(
    "grade_documents", relevance_router,
    {"generate_answer": "generate_answer",

```



```

        "rewrite_question": "rewrite_question"},
    )
    graph.add_edge("rewrite_question", "gen_query")
    graph.add_edge("generate_answer", END)

    app = graph.compile()
    print("Graph compilation successful.")

```

Summary

Comparison of three RAG approaches

Characteristics	RAG Agent	RAG Chain	LangGraph Custom
Number of LLM calls	Multiple	single	Multiple
Number of searches	agent decision	Exactly 1 time	Custom
Query Reconstruction	automatic	Not supported	explicit node
Relevance Assessment	implicit	None	GradeDocuments
control level	low	low	High
Implementation Complexity	low	Lowest	High

Core LangGraph patterns

pattern	implementation
conditional routing	<code>add_conditional_edges</code> + <code>tools_condition</code>
structured output	<code>llm.with_structured_output(GradeDocuments)</code>
tool node	<code>ToolNode([retrieve])</code>
loop control	<code>rewrite_question</code> -> <code>gen_query</code> cycle

Next Steps

→ [06_sql_agent.ipynb](#): Creates a SQL agent.

06 SQL Agent Advanced

- *LangChain & LangGraph*

We build agents that convert natural language into SQL queries in two ways: LangChain `create_agent` + `SQLDatabaseToolkit` (simple version) and LangGraph `StateGraph` (custom version). Covers Human-in-the-Loop, `interrupt()`, and `Command(resume=...)` patterns.

Learning Objectives

- Understand the 8-step workflow of SQL Agent
- Utilizes 4 tool of `SQLDatabase` and `SQLDatabaseToolkit`
- Implement ReAct-based SQL Agent with LangChain `create_agent`
- Add approval before query execution with `HumanInTheLoopMiddleware`
- Build a custom SQL Agent with LangGraph `StateGraph`
- Force tool calling with `bind_tools` and `tool_choice`
- Implement query review with `interrupt()` and `Command(resume=...)`

6.1 Environment Setup (SQLite + Chinook DB)

```
# %pip install langchain langchain-openai langchain-community langgraph sqlalchemy python-dotenv

from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
from langchain_community.utilities import SQLDatabase

llm = ChatOpenAI(model="gpt-5.4")
db = SQLDatabase.from_uri("sqlite:///Chinook.db")
print(f"Dialect: {db.dialect}")
```

6.2 SQL Agent Overview

SQL Agent follows an *8-step* process to convert natural language questions into SQL queries:

1. Receive the question -> 2. List tables -> 3. Inspect the relevant table schema
- > 4. Generate the SQL query -> 5. Validate the query -> 6. (Optional) Human review
- > 7. Execute the query -> 8. Interpret the result

Why Do You Need an Agent?

Unlike a simple text-to-SQL pipeline, an agent can inspect schema, generate a query, validate it, and retry when needed. This improves accuracy because the agent can analyze errors and rewrite the query. It also uses the context window efficiently by loading only the schema that is actually needed.

Example agent execution trace

```
User: "What were the top 5 products by sales last month?"

Agent -> sql_db_list_tables()
      <- "customers, orders, order_items, products, categories"

Agent -> sql_db_schema("orders, order_items, products")
      <- CREATE TABLE orders (id INT, order_date DATE, ...)
         CREATE TABLE order_items (order_id INT, product_id INT, quantity INT, price DECIMAL, ...)

Agent -> sql_db_query_checker("SELECT p.name, SUM(oi.quantity * oi.price) ...")
      <- "The query looks correct."

Agent -> sql_db_query(validated_query)
      <- [("Widget Pro", 45230.00), ("Gadget X", 38100.00), ...]
```

Safety guidelines

Concern	Mitigation
SQL Injection	Use parameterized queries; the toolkit helps automatically.
DML execution	Ban INSERT/UPDATE/DELETE in the system prompt and enforce read-only DB permissions.
Expensive queries	Enforce LIMIT and require Human-in-the-Loop approval before execution.
Sensitive data	<code>include_tables</code> / <code>exclude_tables</code> to restrict accessible tables and enforce column-level permissions
Data exposure	Use database views or restricted user permissions.

Restricting Accessible Tables

In production, explicitly restrict which tables the agent may access:

```
db = SQLiteDatabase.from_uri(
    "sqlite:///company.db",
    include_tables=["products", "orders", "order_items"], # allowlist
    # exclude_tables=["users", "credentials"],           # or a denylist
)
```

6.3 SQLiteDatabaseToolkit

Automatically generates 4 tool:

tool	Features
------	----------

sql_db_list_tables	Return all table names in database
sql_db_schema	CREATE TABLE statement + return sample rows
sql_db_query	Execute a SQL query and return results
sql_db_query_checker	LLM pre-checks queries for errors

```
from langchain_community.agent_toolkits import SQLDatabaseToolkit

toolkit = SQLDatabaseToolkit(db=db, llm=llm)
tools = toolkit.get_tools()

for t in tools:
    print(f" {t.name}: {t.description[:60]}...")
print(f"Total tools: {len(tools)}")
```

6.4 LangChain SQL Agent – `create_agent` + ReAct

`create_agent` is LangChain's high-level API, which takes a model and tool and automatically constructs a *ReAct* (*Reasoning + Acting*) loop. The agent calls tool in order, following the workflow defined in the system prompt.

How ReAct loop works

1. The LLM analyzes the user question and conversation history to decide **which tool** to call next
2. tool is executed and the results are added to the conversation history
3. LLM will check the results and return to step 1 if additional tool calling is needed
4. Return a text response when the final answer is ready

What the system prompt does

System prompts define the agent's instructions for action. In SQL Agent, you must specifically specify the following:

- *tool calling order*: Force order `list_tables` → `schema` → `query_checker` → `query`
- *Safety rules*: Use `LIMIT`, no DML, query only necessary columns
- *Error handling*: Directs rewriting when a query error occurs.
- *SQL dialect*: Specify the dialect of the current DB (SQLite, PostgreSQL, etc.)

```
system_prompt = (
    "You are a SQL agent. Follow these steps:\n"
    "1. sql_db_list_tables\n2. sql_db_schema\n"
    "3. Write the query + sql_db_query_checker\n"
    "4. sql_db_query\n5. Interpret the result.\n"
    f"Rules: use LIMIT 10. DML is forbidden. Dialect: {db.dialect}"
)
```

```
from langchain.agents import create_agent

sql_agent = create_agent(
    model=llm, tools=tools, system_prompt=system_prompt,
```

```
)
print("LangChain SQL Agent created.")
```

6.5 Run test

6.6 HITL – HumanInTheLoopMiddleware

In a production environment, human approval is required before executing SQL queries. This is because agent-generated queries can be expensive, access unexpected tables, or return results that are different from what you intended.

`HumanInTheLoopMiddleware` intercepts the specified tool(`sql_db_query`) call and suspends execution, allowing human review.

3 review options

When an agent attempts to call `sql_db_query`, execution is suspended and the human chooses one of the following:

Options	Command(resume=...) value	Description
<i>Approve</i>	<code>{"decisions": [{"type": "approve"}]}</code>	Execute the generated query as-is
<i>Edit</i>	<code>{"decisions": [{"type": "edit", "args": {...}]}]</code>	Execute the modified query
<i>Reject</i>	<code>{"decisions": [{"type": "reject", "message": "..."}]}</code>	Skip execution and return the reason

TIP

The v1 `HumanInTheLoopMiddleware` accepts a `decisions` array so multiple tool calls in the same interrupt can be reviewed together. Each entry maps 1:1 to a queued tool call, and `type` is one of `approve` / `edit` / `reject`.

Why is HITL important?

- *Cost Control*: Avoid large table full scans without `LIMIT`.
- *Data Protection*: Pre-block access to sensitive columns
- *Accuracy Verification*: If the agent misinterpreted the intention of the question, correction is possible.
- *Audit Trail*: Maintains approval records for all executed queries

```
from langchain.agents.middleware import HumanInTheLoopMiddleware

hitl = HumanInTheLoopMiddleware(
    interrupt_on={"sql_db_query": True},
)
sql_agent_hitl = create_agent(
    model=llm, tools=tools,
    system_prompt=system_prompt, middleware=[hitl],
```

```
)
print("Created SQL Agent with HITL applied.")
```

Resume pattern for destructive queries (DELETE / UPDATE)

A read-only SELECT can be released with a plain `approve`, but destructive statements (DELETE/UPDATE/INSERT) are typically `edit`-ed first to add stricter `WHERE` clauses or `LIMIT` guards before execution. When the agent interrupts, deliver the reviewer's decision through the `decisions` array in `Command(resume=...)`.

```
from langgraph.types import Command

# Dangerous query – reviewer adds LIMIT and approves the edited statement
result = sql_agent_hitl.invoke(
    Command(resume={
        "decisions": [{
            "type": "edit",
            "args": {
                "query": (
                    "UPDATE invoices SET status='void' "
                    "WHERE customer_id=42 AND total < 1 LIMIT 10"
                )
            },
        }],
    }),
    config={"configurable": {"thread_id": "sql-session-1"}},
)
```

6.7 LangGraph Custom SQL Agent – StateGraph

`LangChain create_agent` can be used for quick prototyping, but if you need *fine-grained control at the node level*, use `LangGraph StateGraph`. By defining each step as an independent node, we can achieve:

- *Conditional Branch*: Route to regeneration node when query validation fails.
- *Force tool calling*: With `bind_tools(tool_choice=...)`, a specific tool calling must be installed on a specific node.
- *Fine-grained breakpoints*: Break execution exactly on the desired node with `interrupt()`
- *Custom Status*: Add query history, retry count, etc. to the status.

Graph structure

```
START -> list_tables -> get_schema -> generate_query
      -> check_query -> execute_query -> END
```

Each node receives the shared `State` object and appends messages while the workflow runs. With `tools_condition`, you can implement a conditional branch that either regenerates the query or proceeds to execution based on the validation result.

LangChain `create_agent` Advantages Compared with

Aspect	<code>create_agent</code>	<code>StateGraph</code>
Execution order	LLM decides autonomously	Explicitly enforced with graph edges

Retry on error	Depends on the system prompt	Explicitly implemented with conditional edges
Human review	Middleware-based	<code>interrupt()</code> based, position free
Debugging	Black box	Status of each node can be checked

```

from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph.message import add_messages

class SQLState(TypedDict):
    messages: Annotated[list, add_messages]

print(f"SQLState keys: {list(SQLState.__annotations__)}")

```

6.8 Dedicated nodes – `list_tables`, `get_schema`, `generate_query`, `check_query`

Each node is responsible for one step of the SQL Agent workflow.

6.9 `bind_tools` with `tool_choice` – Force tool calling

Set *Force* a call to a specific tool with the `tool_choice` parameter.

6.10 Reviewing queries with `interrupt()`

LangGraph's `interrupt()` function *suspends* graph execution and waits for external input (a human review). Unlike `HumanInTheLoopMiddleware`, `interrupt()` is more flexible as it allows you to break at *an exact location in the code inside the node*.

How it works

1. Calling `interrupt(payload)` inside a node function will immediately halt graph execution
2. `payload` is passed to the client and displayed in the review UI (i.e. the generated SQL query)
3. When the client resumes the graph with `Command(resume=value)`, `interrupt()` returns `value`
4. The node function executes, modifies, or rejects the query based on the returned values.

`interrupt()` VS `HumanInTheLoopMiddleware`

Characteristics	<code>interrupt()</code>	<code>HumanInTheLoopMiddleware</code>
Scope of application	Code level within node	tool calling Level
Flexibility	Arbitrary logic can be implemented	tool calling Interception only
state access	Entire State accessible	Only tool arguments are accessible
checkpointer	Required (stateful required)	optional

6.11 Command(resume=...) pattern

To resume a graph stopped by `interrupt()`, use `Command(resume=...)`.

```
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver

builder = StateGraph(SQLState)
builder.add_node("list_tables", list_tables_node)
builder.add_node("get_schema", get_schema_node)
builder.add_node("generate_query", generate_query_node)
builder.add_node("check_query", check_query_node)
builder.add_node("execute_query", execute_query_node)

builder.add_edge(START, "list_tables")
builder.add_edge("list_tables", "get_schema")
builder.add_edge("get_schema", "generate_query")
builder.add_edge("generate_query", "check_query")
builder.add_edge("check_query", "execute_query")
builder.add_edge("execute_query", END)

checkpointer = InMemorySaver()
sql_graph = builder.compile(checkpointer=checkpointer)
print("LangGraph SQL Agent compiled.")
```

Summary

Comparison of two SQL Agents

Characteristics	LangChain <code>create_agent</code>	LangGraph <code>StateGraph</code>
Implementation Complexity	Low (5 lines)	High (dedicated node)
control level	ReAct Automatic	Node-level customization
HITL	HumanInTheLoopMiddleware	<code>interrupt()</code> + <code>Command(resume=...)</code>
forced tool calling	Not supported	<code>bind_tools(tool_choice=...)</code>
Suitable for	Rapid Prototype	Production, granular control

HITL pattern

action	<code>Command(resume=...)</code>
Approve	<code>{"decisions": [{"type": "approve"}]}</code>
Edit	<code>{"decisions": [{"type": "edit", "args": {...}]}</code>
Reject	<code>{"decisions": [{"type": "reject", "message": "..."}]}</code>

4 SQLDatabaseToolkits tool

tool	steps	Use
sql_db_list_tables	2	Check table list
sql_db_schema	3	DDL + sample data query
sql_db_query_checker	5	Query pre-validation
sql_db_query	7	Run query

Next Steps

→ [07_data_analysis.ipynb](#): Creates a data analysis agent.

07 Data Analysis Agent

Deep Agents + Sandbox

Build an autonomous agent that takes CSV data as input, performs exploratory data analysis (EDA), generates visualization results, and shares them through Slack.

Utilizes the Deep Agents SDK's `create_deep_agent`, backend system, and built-in tool and checkpointer.

Learning Objectives

After completing this notebook, you will be able to:

1. *Backend Selection* — Understand the difference between `LocalShellBackend` (development) and `DaytonaSandbox` / `Modal` / `Runloop` (operations) and be able to choose
2. *Custom tool Definition* — External services tool, such as Slack integration, can be created with the `@tool` decorator.
3. *Agent Creation* — With `create_deep_agent()`, you can configure an agent that combines model, tool, backend, and checkpointer.
4. *Streaming Observation* — You can monitor the agent's analysis process in real-time.
5. *Use built-in tool* — Understand the role of `write_todos`, `ls`, `read_file`, `write_file`, `edit_file`, `glob`, `grep` tool
6. *checkerpoint* — You can maintain the conversation with `InMemorySaver` and then perform analysis.

7.1 Environment Setup

Install Deep Agents SDK, Tavily (web search), and Slack SDK. Use `pip` or `uv` depending on the execution environment.

```
from dotenv import load_dotenv

load_dotenv()
```

7.2 Data Analysis Agent Overview

Deep Agents' data analysis agents *autonomously* run the following pipelines:

```
CSV input -> planning ('write_todos') -> file reading ('read_file') -> code generation and execution ->
iterative analysis -> result delivery (Slack)
```

Detailed Execution Flow

Step	Description	Tools Used
<i>Planning</i>	<code>write_todos</code> builds a structured task plan and updates TODOs as the analysis progresses	<code>write_todos</code>
<i>File Reading</i>	inspects CSV structure, column names, data types, and row counts; image files can also be read multimodally	<code>read_file</code>
<i>Code Execution</i>	pandas, matplotlib and runs Python code in an isolated backend	Backend <code>execute</code>
<i>Iterative Analysis</i>	performs follow-up analysis and uses web search to gather domain context	Tavily, <code>edit_file</code>
<i>Result Delivery</i>	formats the analysis result and sends it to Slack	<code>slack_send_message</code>

Key Traits of an Autonomous Agent

A Deep Agents data-analysis agent goes beyond simple tool use and can *plan autonomously*:

1. *planning*: `write_todos` to break the analysis into steps and track progress
2. *Adaptive execution*: if an error occurs during analysis, it can revise the code (`edit_file`) and run it again.
3. *subagent Delegation*: Complex tasks are processed in parallel by creating specialized subagent
4. *Context Management*: Store intermediate results in file system tool and refer back to them when needed

7.3 Backend selection

Deep Agents provide a filesystem and code execution environment through a *pluggable backend architecture*. All backends implement the same `BackendProtocol` , so you can replace just the backend without changing any code.

backend	Use	Security level	Setting Difficulty
<code>LocalShellBackend</code>	Local development/testing	Low (host system full access)	No setup required
<code>Daytona</code>	Cloud sandbox environment	High (isolated container)	API key required
<code>Modal</code>	Serverless GPU/CPU computation	High	Modal account required
<code>Runloop</code>	Running a managed cloud	High	API key required

Details by backend type

- `StateBackend` (default): Stores files in the LangGraph agent state. It works well as a scratch pad, and large printouts are automatically removed.
- `FilesystemBackend` : Provides local disk access with `root_dir` settings. The `virtual_mode=True` option restricts paths and prevents directory traversal.

- `LocalShellBackend` : Provides **unlimited shell command execution** through the `execute` tool in addition to filesystem access. It runs with full user privileges on the host system.
- `CompositeBackend` : Routes different backends by route. Example: `StateBackend` for temporary files and `StoreBackend` for `/memories/`.

Sandbox Security Principles

Sandboxes provide an isolated environment where agents can run code safely. Core security principles:

- *Never put secrets in the sandbox* – Context injection attacks can allow agents to read environment variables or credentials from mounted files and thus leak them.
- Keep your credentials outside the sandbox in tool
- Use Human-in-the-Loop approval for sensitive tasks
- Block unnecessary network access

TIP

Caution: `LocalShellBackend` has *unrestricted shell execution permissions* on the host system. Be sure to use a sandbox backend in production.

```
from deepagents.backends import LocalShellBackend

# For development purposes – local shell backend
dev_backend = LocalShellBackend(virtual_mode=True)

# For Operations – Cloud Sandbox (Choose 1)
# prod_backend = ... # Use cloud sandbox backend in production
```

7.4 Upload sample data

Create a CSV file in the backend's working directory for the agent to analyze. `create_deep_agent`'s backend has `write_file` tool built in, so the agent can write files directly, but here the data is prepared in advance.

```
import os

workspace = "/tmp/analysis"
os.makedirs(workspace, exist_ok=True)

csv_content = (
    "region,quarter,revenue,units_sold\n"
    "Seoul,Q1,120000,340\nSeoul,Q2,135000,380\n"
    "Seoul,Q3,128000,355\nSeoul,Q4,150000,420\n"
    "Busan,Q1,85000,240\nBusan,Q2,92000,260\n"
    "Busan,Q3,88000,250\nBusan,Q4,105000,300"
)
```

```
with open(f"{workspace}/sales_2025.csv", "w") as f:
    f.write(csv_content)
print(f"CSV saved: {workspace}/sales_2025.csv")
```

7.5 Custom tool – Slack integration

Use the `@tool` decorator to define a custom tool that the agent can call. *docstring* in tool tells the agent how to use it.

Below is tool, which transmits analysis results to a Slack channel.

```
from langchain_core.tools import tool
try:
    from slack_sdk import WebClient
except ImportError:
    WebClient = None
    print("slack_sdk not installed -- Slack tool works as a stub")

slack_client = WebClient(token=os.environ.get("SLACK_BOT_TOKEN", "xoxb-placeholder")) if WebClient else None

@tool
def slack_send_message(message: str) -> str:
    """Send analysis results to Slack channel."""
    resp = slack_client.chat_postMessage(
        channel=os.environ.get("SLACK_CHANNEL_ID", "general"), text=message)
    return f'Sent successfully. Timestamp: {resp["ts"]}'
```

Web Search tool (Tavily)

Prepares Tavily tool so that agents can perform web searches when domain context is needed during analysis.

```
from tavily import TavilyClient

tavily_client = TavilyClient(api_key=os.environ["TAVILY_API_KEY"])

def tavily_search(query: str) -> str:
    """Search the web for information related to your analysis."""
    results = tavily_client.search(query, max_results=5)
    return "\n".join(
        [r["content"] for r in results["results"]]
    )
```

pandas / scikit-learn `@tool` integration

Data analysis agents often treat *each tool as a tiny analysis cell*. The `@tool` decorator runs pandas / scikit-learn code on demand and returns the result back to the model as text. Keeping these tools separate from filesystem access (`read_file`) makes input validation, caching, and unit testing far easier.

```
import pandas as pd
from sklearn.cluster import KMeans
from langchain_core.tools import tool

@tool
def summarize_csv(path: str, top_n: int = 5) -> str:
    """Return shape, dtypes, and the top-N rows of a CSV file."""
    df = pd.read_csv(path)
    return (
        f"shape={df.shape}\n"
        f"dtypes={df.dtypes.to_dict()}\n"
        f"head=\n{df.head(top_n).to_string()}"
    )

@tool
def cluster_customers(path: str, k: int = 3) -> str:
    """KMeans clustering over numeric columns only."""
```

```
df = pd.read_csv(path).select_dtypes("number").dropna()
labels = KMeans(n_clusters=k, n_init="auto").fit_predict(df)
return f"cluster_sizes={pd.Series(labels).value_counts().to_dict()}"
```

CodeInterpreterMiddleware and the Skills pattern

For free-form analysis that does not fit a fixed `@tool` signature — “draw a boxplot of this column”, for example — `CodeInterpreterMiddleware` lets the agent execute arbitrary code in its own sandbox. Repeatable procedures, on the other hand, are best captured in a `skills/` directory as markdown + code snippets and loaded as *Skills* the agent can invoke by name.

```
from deepagents.middleware import CodeInterpreterMiddleware
from deepagents.skills import load_skills

skills = load_skills("./skills/data-analysis") # markdown + snippets

agent = create_deep_agent(
    model="gpt-5.4",
    tools=[summarize_csv, cluster_customers, slack_send_message],
    middleware=[CodeInterpreterMiddleware(backend=backend)],
    skills=skills,
    backend=backend,
    checkpointer=checkpointer,
    system_prompt="You are a data analyst.",
)
```

TIP

Keep *reproducible analysis recipes* inside the Skills directory and let `CodeInterpreterMiddleware` handle one-off exploration. This prevents validated procedures from blending with throwaway code.

7.6 Creating an agent

`create_deep_agent()` creates an agent by combining the model, tool, backend, checkpointer, and system prompts.

parameters	Type	Description
model	str	model identifier (default: <code>claude-sonnet-4-6</code>)
tools	list	List of tool functions to be used by the agent
backend	Backend	Code execution and filesystem backend
checkpointer	Checkpointer	State Perpetuation Mechanism
system_prompt	str	Agent Behavior Guidelines

```
from deepagents import create_deep_agent
from deepagents.backends import LocalShellBackend
from langgraph.checkpoint.memory import InMemorySaver
```

```
backend = LocalShellBackend(virtual_mode=True)
checkpointer = InMemorySaver()
```

```
agent = create_deep_agent(
    model="gpt-5.4",
    tools=[tavily_search, slack_send_message],
    backend=backend,
    checkpointer=checkpointer,
    system_prompt="You are a data analyst.",
)
```

7.7 Execution — Analysis Request

When an analysis request is sent to `agent.invoke()`, the agent autonomously performs the following pipeline: planning → reading files → executing code → delivering results.

7.8 Observe the analysis process through streaming

`agent.stream()` allows you to observe the agent's execution *in real-time*. Deep Agents provides powerful monitoring that can trace the execution of subagent based on LangGraph's streaming infrastructure.

Stream mode

mode	Description	Use cases
<i>Updates</i>	Receive events upon completion of each step (node)	Progress Dashboard, Log
<i>Messages</i>	Individual token streaming, including source agent metadata	Real-time chat UI
<i>Custom</i>	Issue custom progress event with <code>get_stream_writer()</code>	Analysis progress display, custom notifications

Namespace system

Streaming events contain a namespace that identifies their source:

- `()` (empty tuple) = main agent
- `("tools:abc123",)` = subagent created with tool calling
- `("tools:abc123", "model_request:def456")` = model request node inside subagent

By setting `subgraphs=True`, you can trace the execution of subagent as well, and you can also use multiple stream modes simultaneously:

```
for namespace, chunk in agent.stream(
    {"messages": [...]},
    stream_mode=["updates", "messages", "custom"],
    subgraphs=True,
```

```

):
    mode, data = chunk
    # Handle each mode differently

```

Tracking the subagent lifecycle

A subagent goes through three stages:

1. *Pending* — detected when the main agent includes a delegated task in its `model_request`
2. *Running* — `tools:UUID` starts when events appear under the namespace
3. *Complete* — finishes when the main agent's `tools` node returns its result

7.9 Utilizing built-in tool

Deep Agents automatically provides the following built-ins to agents through the backend: The agent autonomously selects and uses these tool during the analysis process.

tool	Description	Example usage
<code>write_todos</code>	Create and track structured work plans	Creation of TODO list for each stage of analysis
<code>ls</code>	List directory contents (<code>ls_info()</code>)	Check CSV file existence
<code>read_file</code>	Read file (image multimodal support)	Understand CSV structure, check chart image
<code>write_file</code>	Creating a new file (create-only)	Analysis script, save result file
<code>edit_file</code>	Modifying existing files (find-and-replace)	Code fix, update config file
<code>glob</code>	glob pattern based file navigation	<code>*.csv</code> Searching data files by pattern
<code>grep</code>	Pattern matching search	Search for specific column name or value

Multimodal support in `read_file`

`read_file` supports image files as multimodal content in all backends. After the agent creates a chart with matplotlib, you can *visually interpret* the contents of the chart by reading the image with `read_file`. Through this, it autonomously determines “whether the chart was created correctly” and “what additional visualization is needed.”

Custom backend implementation

You can implement `BackendProtocol` yourself depending on your needs. Required methods:

- `ls_info()` – List directory contents
- `read()` – Read file with line numbers
- `grep_raw()` – Pattern matching that returns structured matches.
- `glob_info()` – glob-based file matching
- `write()` – Create file (create-only)
- `edit()` – find-and-replace that guarantees uniqueness

7.10 Maintain conversation with checkpointer

checkpointer saves agent state to enable *abort and resume*. A conversation session is identified through `thread_id`, and subsequent requests with the same `thread_id` will maintain the previous conversation context.

checkpointer	Use	permanence
InMemorySaver	Development/Test	Destroyed when process ends
SQLiteSaver	local persistence	Save to Disk (<code>./agent_checkpoints.db</code>)
PostgresSaver	Production	Store in database, support multiple instances

Role of checkpointer

checkpointer goes beyond simply saving conversation history and allows you to:

- *Interruption Recovery*: Resume from the last completed step in case of network error or timeout
- `interrupt()` **support**: Saves the graph state in human-in-the-loop flows and resumes at the correct location after the human responds.
- *Multi-turn analysis*: Process multiple analysis requests consecutively in the same session while referencing previous results

Sandbox Lifetime Management

An explicit shutdown is required when using a sandbox. Failure to terminate will result in unnecessary costs. In your chat application, use a unique sandbox for each conversation thread and configure automatic cleanup with Time-to-Live (TTL) settings.

```
from langgraph.checkpoint.memory import InMemorySaver

checkpointer = InMemorySaver()
config = {"configurable": {"thread_id": "analysis-session-1"}}

agent_with_memory = create_deep_agent(
    model="gpt-5.4",
    tools=[tavily_search, slack_send_message],
    backend=LocalShellBackend(virtual_mode=True),
    checkpointer=checkpointer,
)
```

Summary

To summarize what we covered in this notebook:

Topic	Key Takeaways
<i>Backend</i>	LocalShell (development) uses host shell access, production uses Daytona / Modal / Runloop sandbox
<i>Custom tool</i>	<code>\@tool</code> Defined by decorator + docstring, agent calls autonomously

Create Agent	<code>create_deep_agent(model, tools, backend, checkpointer, system_prompt)</code>
Streaming	Real-time observation with <code>agent.stream(stream_mode="updates", subgraphs=True)</code>
Built-in tool	<code>write_todos</code> , <code>ls</code> , <code>read_file</code> , <code>write_file</code> , <code>edit_file</code> , <code>glob</code> , <code>grep</code>
checkerpointer	<code>InMemorySaver</code> (Development) → <code>SQLiteSaver</code> → <code>PostgresSaver</code> (Production)

Next Steps

→ [08_voice_agent.ipynb](#): Creates a voice agent.

08 Voice Agent

STT/Agent/TTS Pipeline

We build a voice agent that converts voice input to text (STT), processes it by the LangChain agent, and then synthesizes text into speech (TTS) and returns it in real-time.

This architecture is a *Sandwich pattern* (STT -> Agent -> TTS), where each layer is connected by streaming, targeting sub-700ms latency.

Core design principles

The core of the Sandwich architecture is *Streaming Chaining*. Rather than waiting for the complete output of the previous layer, each layer passes partial results to the next layer as soon as they are produced:

- *STT*: Generates partial transcripts in real-time and emits final transcripts when detecting end-of-speech.
- *Agent*: Streams the response token by token through `astream()` — does not wait for the entire response generation to complete.
- *TTS*: Start audio synthesis as soon as a text chunk arrives based on WebSocket

Thanks to this structure, the latency of the entire pipeline is reduced to *the sum of the time until the first output of each stage*, rather than the sum of each stage.

Learning Objectives

After completing this notebook, you will be able to:

1. *Sandwich Architecture* — Can explain the structure and data flow of STT -> Agent -> TTS pipeline
2. *Architecture Comparison* — You can compare the pros and cons of the Sandwich method and the Speech-to-Speech (S2S) method.
3. *STT Step* — You can understand the Producer-Consumer pattern of AssemblyAI real-time transcription.
4. *Agent Phase* — You can build an agent that generates streaming responses with LangChain `create_agent`
5. *TTS Step* — Understand the operating principle of Cartesia WebSocket-based streaming voice synthesis.
6. *RunnableGenerator* — Pipelines can be combined with asynchronous generator chaining.
7. *Performance Optimization* — Understand optimization techniques at each stage to achieve latency goals.

· NOTE

Note: STT (AssemblyAI) and TTS (Cartesia) are external paid services. The cells are provided as markdown to illustrate the concept, *only the agent creation part is the actual executable code*.

8.1 Environment Setup

These are the packages and each role required for a voice agent:

package	Role
langchain , langchain-openai	Create Agent and Connect LLM
assemblyai	Real-time speech-to-text (STT)
cartesia	Low-latency text-to-speech synthesis (TTS)
websockets	Real-time two-way communication server
pyaudio	Microphone audio capture (optional)

```
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv()

model = ChatOpenAI(model="gpt-5.4")
```

8.2 Voice Agent Architecture Overview

The voice agent consists of a 3-layer pipeline:

```
Audio In -> [STT: AssemblyAI] -> Text -> [Agent: LangChain] -> Text -> [TTS: Cartesia] -> Audio Out
```

Layer	Provider	Role
<i>STT</i>	AssemblyAI	User Converts speech into text
<i>Agent</i>	LangChain	Processes the text query and generates a response
<i>TTS</i>	Cartesia	Converts the agent's text response into speech

Each layer is connected through *streaming*, so partial results can be forwarded immediately without waiting for the previous step to finish completely:

1. *STT* — streams partial transcriptions (emits a final transcript when the utterance is complete)
2. *Agent* — streams responses token by token (`astream()`)
3. *TTS* — Start speech synthesis before the entire response is complete (WebSocket-based)

Why Streaming Matters

When processing synchronously without streaming, Next Steps does not start until each step is completely finished. For example, if the agent takes 3 seconds to generate a 200 token response, TTS will only start after 3 seconds. In streaming, on the other hand, TTS begins synthesizing speech *as soon as the agent's first token appears*, so the user already starts hearing speech while the agent is still generating a response.

8.3 Architecture comparison table

Compare two approaches to building voice agents.

Sandwich architecture (STT -> Agent -> TTS)

Item	Content
<i>Advantages</i>	Each component can be replaced independently, easy to use text-based tool, convenient for debugging (intermediate text logging)
<i>Disadvantages</i>	Latency accumulation and loss of non-verbal information such as emotion/intonation due to 3-step serial processing
<i>Suitable for</i>	tool calling Complex agents with many, when multilingual support is required

Speech-to-Speech (S2S)

Item	Content
<i>Advantages</i>	Low latency, preservation of voice characteristics (emotion, stress), natural conversation
<i>Disadvantages</i>	tool calling Difficulty in integration, limited model selection, difficulty in debugging
<i>Suitable for</i>	Simple conversational interface, when emotion recognition is important

· NOTE

In this notebook, we focus on the *tool calling-enabled Sandwich architecture*.

8.4 STT Step – AssemblyAI Real-Time Transcription

TIP

This cell requires an AssemblyAI API key. Reference code to understand the concept.

AssemblyAI's `RealtimeTranscriber` operates with the *Producer-Consumer pattern*:

- *Producer*: Send audio chunks captured from the microphone to WebSocket
- *Consumer*: Receives partial and final transcription results as a callback.

Since both operations take place *simultaneously*, you can receive a transcription of the previous utterance even while your voice is being transmitted.

Two types of transcription results

Type	class	Description
<i>Partial</i>	<code>RealtimeTranscript</code>	Temporary transcription of words still being uttered. Updated in real-time as the user speaks
<i>Final</i>	<code>RealtimeFinalTranscript</code>	Confirmation transcription after the end of utterance (endpointing) is detected. Pass only this result to the agent

AssemblyAI's built-in Voice Activity Detection (VAD) automatically detects when an utterance has ended, eliminating the need for separate silence detection logic.

```
import assemblyai as aai

aai.settings.api_key = "your-assemblyai-key"

transcriber = aai.RealtimeTranscriber(
    sample_rate=16000,
    encoding=aai.AudioEncoding.pcm_s16le,
    on_data=on_transcription_data,
    on_error=on_transcription_error,
)

def on_transcription_data(transcript: aai.RealtimeTranscript):
    if isinstance(transcript, aai.RealtimeFinalTranscript):
        process_user_input(transcript.text)

def on_transcription_error(error: aai.RealtimeError):
    print(f"Transcription error: {error}")

transcriber.connect()
```

Capturing Microphone Audio

PCM 16-bit, 16kHz Capture mono audio and send it to the transcriber in real time:

```
import pyaudio

audio = pyaudio.PyAudio()
stream = audio.open(
    format=pyaudio.paInt16,
    channels=1, rate=16000,
    input=True, frames_per_buffer=1024,
)

while True:
    data = stream.read(1024)
    transcriber.stream(data)
```

8.5 Agent Phase – Utilizing LangChain Agent

The agent stage is the core of the pipeline. An agent created with `create_agent` takes text input, performs inference with tool calling, and streams a text response.

The key to system prompts for voice agents is to elicit concise, conversational responses. For voice output, unlike text, long responses can be burdensome to the user, so users are instructed to generate short responses of no more than 1 to 2 sentences.

The role of asynchronous streaming

The agent's `astream()` method yields a response token as soon as it is generated. Why this is especially important for voice agents:

- *TTS Early Start*: Speech synthesis is possible from the first token without waiting for the entire response to be completed.
- *Reduced perceived lag*: Users start hearing the agent's response sooner.
- *Pipeline efficiency*: Agent and TTS run simultaneously, reducing total processing time

```
from langchain.agents import create_agent

def search_tool(query: str) -> str:
    """Search the web for the latest information."""
    return f"Search result: {query}"

def calendar_tool(action: str, details: str) -> str:
    """Manage calendar events."""
    return f"Calendar {action}: {details}"
```

```
agent = create_agent(
    model="gpt-5.4",
    tools=[search_tool, calendar_tool],
    system_prompt=(
        "You are a useful voice assistant."
        "Keep your responses brief and conversational."
    ),
)
```

Generate asynchronous streaming response

`astream()` yields the agent's response token as soon as it is generated. Allows the TTS stage to start speech synthesis without waiting for the entire response to complete.

LangChain's streaming offers two main methods:

method	Description
<code>astream()</code>	Stream the output for each step of agent execution. Contains messages, tool calling results, etc.
<code>astream_events()</code>	More granular event-level streaming. Provide detailed events such as individual tokens, LLM start/end, etc.

The voice agent receives message chunks as `astream()`, extracts `content` from each chunk, and passes it to TTS.

```
async def stream_agent_response(user_text: str):
    """Stream agent response tokens one by one."""
    async for chunk in agent.astream(
        {"messages": [{"role": "user",
                        "content": user_text}]}
    ):
        if "messages" in chunk:
            for msg in chunk["messages"]:
                if hasattr(msg, "content") and msg.content:
                    yield msg.content
```

Multimodal content blocks and `output_version="v1"`

A voice agent's input is not always pure text — a user may attach a photo, or a tool may return a chart image. LangChain v1 messages accept a *content blocks* array, so a single message can mix text, image, and audio. Creating the model with `output_version="v1"` also normalizes responses into the `AIMessage.content_blocks` schema, which is safe to serialize between STT, Agent, and TTS processes.

```
from langchain.chat_models import init_chat_model

llm_v1 = init_chat_model("gpt-5.4", output_version="v1")

response = llm_v1.invoke([
    {"role": "user", "content": [
        {"type": "text", "text": "Explain any anomalies in this chart by voice."},
        {"type": "image", "source_type": "base64",
         "mime_type": "image/png", "data": "..."},
    ]},
])

# v1 schema — content_blocks survives JSON round-trips between services
for block in response.content_blocks:
    if block["type"] == "text":
        tts_input_queue.put_nowait(block["text"])
```

TIP

`output_version="v1"` normalizes responses into the `content_blocks` schema. When messages cross process boundaries via WebSocket (a TTS server, for example), this prevents JSON (de)serialization from corrupting the payload — practically mandatory for boundary-heavy systems like voice pipelines.

8.6 TTS Steps – Cartesia Streaming Speech Synthesis

TIP

This cell requires a Cartesia API key. Reference code to understand the concept.

Cartesia provides WebSocket-based low-latency TTS. It takes a text stream from an agent and generates audio chunks for each partial text on the fly.

How Cartesia TTS works

1. *WebSocket connection:* Maintain a persistent connection by opening a WebSocket session with the `AsyncCartesia` client.
2. *Send text chunks:* Send text in units of tokens/sentences generated by the agent.
3. *Receive Audio Chunk:* Receive PCM audio bytes immediately for each text chunk
4. *Client Delivery:* Stream the received audio bytes to the client via WebSocket.

Because it is based on WebSocket, two-way real-time communication is possible without HTTP request/response overhead. The `sonic-2` model provides both natural speech and low latency.


```
import cartesia

cartesia_client = cartesia.AsyncCartesia(
    api_key="your-cartesia-key"
)

async def text_to_speech_stream(text_stream):
    ws = await cartesia_client.tts.websocket()
    async for text_chunk in text_stream:
        audio_chunks = ws.send(
            model_id="sonic-2",
            transcript=text_chunk,
            voice_id="your-voice-id",
            stream=True,
            output_format={
                "container": "raw",
                "encoding": "pcm_s16le",
                "sample_rate": 24000,
            },
        )
        async for audio in audio_chunks:
            yield audio["audio"]
    await ws.close()
```

8.7 Pipeline Combination – RunnableGenerator

LangChain's `RunnableGenerator` allows you to integrate asynchronous generators into your *Runnable pipeline*. This allows the entire data flow of STT output -> Agent processing -> TTS input to be organized into one pipeline.

What is RunnableGenerator?

`RunnableGenerator` wraps an asynchronous generator function into LangChain's `Runnable` interface. If you do this:

- Chaining with other Runnables is possible using the `|` (pipe) operator.
- Standard methods such as LangChain's `batch()` and `stream()` can be used.
- Suitable for patterns that receive an input stream and generate a converted output stream.

```
from langchain_core.runnables import RunnableGenerator

async def transform_input(input_stream):
    async for text in input_stream:
        async for token in stream_agent_response(text):
            yield token

agent_runnable = RunnableGenerator(transform_input)
```

Wiring the Full Pipeline (conceptual code)

Use `asyncio.Queue` to pass the final STT transcript to the agent and forward the agent's streaming response to TTS. `Queue` is a core building block in an async producer-consumer pattern.

```
async def voice_pipeline(audio_input_stream):
    transcript_queue = asyncio.Queue()

    def on_final(transcript):
        if isinstance(transcript, aai.RealtimeFinalTranscript):
            transcript_queue.put_nowait(transcript.text)
```

```

transcriber = aai.RealtimeTranscriber(
    sample_rate=16000, on_data=on_final
)
transcriber.connect()

async for audio_chunk in audio_input_stream:
    transcriber.stream(audio_chunk)
    if not transcript_queue.empty():
        user_text = await transcript_queue.get()
        text_stream = stream_agent_response(user_text)
        async for audio in text_to_speech_stream(text_stream):
            yield audio

```

8.8 WebSocket Server – Real-time two-way communication

TIP

Running the code in this cell will start the server. This is reference code to help you understand the concept.

Handles two-way audio streaming with clients via WebSockets. Receiving and sending are *executed* simultaneously with `asyncio.gather`.

Why use WebSockets

Voice agents require two-way, real-time communication:

- *Receive*: Client continuously transmits microphone audio to the server
- *Send*: The server continuously transmits synthesized voice audio to the client.

HTTP's request-response model is not suitable for this kind of two-way streaming. WebSockets support two-way data flow over a single TCP connection, and with `asyncio.gather`, ingress and egress run concurrently without blocking each other.

```

import websockets
import asyncio

async def handle_client(websocket):
    transcriber = create_transcriber()
    transcriber.connect()

    async def receive_audio():
        async for message in websocket:
            if isinstance(message, bytes):
                transcriber.stream(message)

    async def send_audio():
        async for transcript in transcription_queue:
            text_stream = stream_agent_response(transcript)
            async for audio in text_to_speech_stream(text_stream):
                await websocket.send(audio)

    await asyncio.gather(receive_audio(), send_audio())

async def main():
    async with websockets.serve(handle_client, "0.0.0.0", 8765):
        print("Voice agent server on ws://0.0.0.0:8765")
        await asyncio.Future()

```

8.9 Performance Target – Sub-700ms Latency

A key performance metric for voice agents is Time to First Audio (TTFA). For a natural conversation experience, this metric should be less than 700ms.

Latency analysis

steps	target time	Description
STT final transcription	200ms	Detect end of utterance -\> final text (AssemblyAI built-in endpointing)
Agent first token	300ms	Enter text -\> Create first response token (TTFT: Time to First Token)
TTS first audio	150ms	First text token -\> first audio chunk (WebSocket based)
<i>Total</i>	<i>\<700ms</i>	<i>End of speech -\> First audio output</i>

Optimization techniques

technique	effect	Implementation method
Streaming STT	Eliminate full transcription wait	<code>RealtimeTranscriber</code> + partial result
Streaming Agent	Start TTS before completing all responses	Use <code>astream()</code> — instead of <code>ainvoke()</code>
Streaming TTS	Reduce time to first audio	WebSocket-based synthesis
Connection Pooling	Eliminate connection setup delays	WebSocket reuse (avoid creating new connection per request)
VAD	Prevent processing of silent sections	AssemblyAI built-in endpointing
response caching	Immediate response to frequently asked questions	Frequently Asked Questions Caching

TIP

Tip: Agent's TTFT (Time to First Token) accounts for the largest proportion of overall latency. Short system prompts and lightweight model selection are key to latency optimization.

8.10 Add tool to agent

The strength of the voice agent is that its Sandwich architecture allows it to leverage text-based tool. You can add a variety of tool features such as search, calendar management, weather lookup, and more.

This is the biggest difference from the S2S (Speech-to-Speech) method. The S2S model processes audio directly, making text-based tool calling difficult, but the Sandwich architecture allows agents to operate in text areas, so all existing LangChain tool can be used as is.

tool Design Tips

tool of voice agents *fast response* is important. tool As the execution time increases, the latency of the entire pipeline increases:

- Designed mainly for light API calls
- Timeout setting required
- Apply caching if possible

```
def weather_tool(city: str) -> str:
    """View the current weather in your city."""
    return f"{city} weather: 15°C, partly cloudy"

def reminder_tool(time: str, message: str) -> str:
    """Set reminders at specified times."""
    return f"{time}Reminder scheduled for {message}"
```

```
voice_agent = create_agent(
    model="gpt-5.4",
    tools=[search_tool, calendar_tool,
           weather_tool, reminder_tool],
    system_prompt=(
        "You are a voice assistant."
        "Please keep your response concise and conversational in 1-2 sentences."
    ),
)
```

Summary

Topic	Key Takeaways
<i>Sandwich Architecture</i>	STT -\> Agent -\> TTS, each layer can be replaced independently, supports tool calling
<i>STT (AssemblyAI)</i>	<code>RealtimeTranscriber</code> , Producer-Consumer pattern, WebSocket streaming
<i>Agent (LangChain)</i>	<code>create_agent</code> + <code>astream()</code> , token-level streaming, tool integration
<i>TTS (Cartesia)</i>	WebSocket-based low-latency synthesis, instant audio conversion of partial text
<i>Pipeline Combination</i>	<code>RunnableGenerator</code> , asynchronous generator chaining
<i>Performance Goals</i>	Sub-700ms (STT 200ms + Agent 300ms + TTS 150ms)
<i>tool Expansion</i>	Text-based tool can be applied to voice agents as is

Next Steps

→ [09_production.ipynb](#): Learn production deployment.

09 Production deployment

- testing, observability, deployment

We cover the entire pipeline for deploying agents into production. We ensure quality through unit testing and LangSmith evaluation, secure observability through tracing, and deploy to the LangGraph Platform.

Development -> Testing (unit + evaluation) -> Observability (tracing) -> Deployment (LangGraph Platform)

Why do you need an agent-specific production pipeline?

Agents have different characteristics than traditional software:

- *Non-deterministic execution*: Same input can produce different tool calling orders and different responses.
- *Statefull*: A long-running process that manages conversation history and checkpoints.
- *Multiple components*: Multiple systems such as LLM, tool, memory, checkpoint, etc. are linked

Therefore, beyond simple unit testing, agent-specific quality assurance techniques such as *trajectory evaluation*, *LLM-as-Judge*, and *trace-based monitoring* are needed.

Learning Objectives

After completing this notebook, you will be able to:

1. *Unit Test* – You can write deterministic tests by mocking the LLM response with `GenericFakeChatModel`
2. *LangSmith Evaluation* – Create datasets, define evaluators, and perform automated agent evaluation.
3. *Trace Analysis* – LangSmith tracing allows you to track latency, token usage, and errors.
4. *LangGraph Studio* – You can analyze the agent execution flow with visual debugging tool
5. *Deployment Options* – Understand the differences between LangGraph Platform, self-host, and cloud deployment
6. *langgraph.json* – You can configure the deployment configuration file and execute deployment commands.

9.1 Environment Setup

Install the packages required for testing, observability, and deployment.

9.2 Production Pipeline Overview

Due to the non-deterministic nature of agents, traditional software testing alone cannot ensure quality.

steps	Purpose	tool
<i>Development</i>	Agent implementation and local testing	LangGraph, LangGraph Studio
<i>Test</i>	Unit Testing + LLM Based Assessment	pytest, agentevals, LangSmith
<i>observability</i>	Tracing, Metrics, Error Tracking	LangSmith Tracing
<i>Distribution</i>	Deploying a Production Environment	LangGraph Platform

Testing Strategy

Agent testing requires a combination of two approaches:

Type	Features	Suitable for
<i>Unit</i>	Mock LLM responses with <code>GenericFakeChatModel</code> for isolated, deterministic tests; runs fast without API calls	Individual tools, parsers, prompt formatting, state transition logic
<i>Integration</i>	Validate cross-component behavior with real network calls; <code>pytest</code> + <code>vcrapy</code> cassettes keep runs reproducible	End-to-end agent flow, tool calling sequence, external API contracts
<i>Evals</i>	Quantify response quality and tool-call ordering with LangSmith datasets and <code>agentevals</code> trajectory evaluators	Answer accuracy, trajectory match, LLM-as-Judge rubrics

TIP

All three test types are needed — *Unit* guards against regressions, *Integration* verifies external API contracts, *Evals* measure non-deterministic answer quality. A common pattern is to run unit + integration on every CI run, *sample-eval* on each PR, and *full-eval* nightly.

Because agent systems operate by chaining multiple components, integration testing generally requires a higher proportion. Unit tests verify the correctness of individual components, and integration tests evaluate the quality of the overall flow.

9.3 Agent testing – LangSmith evaluation

The `agentevals` package provides a dedicated evaluator for agent trajectories. Trajectory refers to all the steps (tool calling, intermediate reasoning, decision making) that the agent takes to reach the final response.

strategy	Description	Advantages	Disadvantages
<i>Trajectory Match</i>	Step-by-step comparison with expected sequence. Matching the agent's	Accurate verification, reproducible	Requires specific expectations, low flexibility

	actual trajectory with a predefined reference trajectory		
<i>LLM-as-Judge</i>	LLM qualitatively evaluates trajectories based on rubrics (evaluation criteria). tool Automatic judgment of appropriateness of use, accuracy of response, etc.	Flexible evaluation, response to complex scenarios	Additional LLM costs, indeterminacy of the assessment itself

Trajectory Match Mode

You can adjust your agent's tool calling ordering expectations in four levels:

mode	Description	When to use
strict	The message and tool calling order must be exactly the same as the reference to pass	Workflows where order is important (e.g. Authentication - \> Lookup -\> Update)
unordered	If the same tool are called, they pass regardless of the order. When there are multiple independent tool calling	subset
Agent only calls reference tool (no additional tool calling)	When you want to prevent unnecessary tool calling	superset
Agent contains at least reference tool (additional calls allowed)	When you want to ensure only the core tool calling	

```
try:
    from langsmith import Client

    ls_client = Client()

    dataset = ls_client.create_dataset("agent-eval-v1")
    ls_client.create_examples(
        inputs=[{"query": "What is LangGraph?"}],
        outputs=[{"expected": "Framework for Agents"}],
        dataset_id=dataset.id,
    )
    print("Dataset created:", dataset.name)
except Exception as e:
    print(f"LangSmith not configured (skipped): {e}")
    ls_client = None
    dataset = None
```

```
try:
    from agentevals.trajectory import create_trajectory_llm_as_judge

    evaluator = create_trajectory_llm_as_judge(
```

```

        rubric=(
            "Did the agent use the appropriate tool?"
            "Was your final answer correct?"
        ),
    )
    print("Evaluator created:", type(evaluator).__name__)
except ImportError:
    print("agentevals not installed. Installation: pip install agentevals")
    evaluator = None
except Exception as e:
    print(f"Evaluator creation skipped (LLM API key required): {e}")
    evaluator = None

```

9.4 Unit test patterns

`GenericFakeChatModel` allows you to write deterministic tests by mocking LLM responses without making API calls. It takes a response iterator and returns one for each call.

Why mocking?

When I call the actual LLM API from my agent tests, I run into the following issues:

- *Non-deterministic results*: It is difficult to reproduce the test because the same input may result in a different response each time.
- *Cost*: API cost incurred each time a test is run
- *Speed*: Network delay slows down testing
- *Availability*: Test fails in case of API failure

`GenericFakeChatModel` returns predefined responses in sequence, allowing you to write tests that are *deterministic, fast, and free*. Streaming patterns are also supported, so `astream()`-based code can also be tested.

State persistence testing

`InMemorySaver` checkpointer allows you to test state-dependent behavior across multiple conversation turns. Specify `thread_id` to verify the cumulative state of multiple calls while maintaining the same conversation context.

```

def search_tool(query: str) -> str:
    """Search for information on the web."""
    return f"Search result: {query}"

def test_search_tool():
    """Tests whether search tool returns the expected format."""
    result = search_tool("test query")
    assert isinstance(result, str) and len(result) > 0
    print("Passed: test_search_tool")

test_search_tool()

```

HTTP request recording/playback

To reduce API costs in CI/CD, you can record and replay HTTP requests with `vcrpy` and `pytest-recording`. Record the actual API call (save it as a cassette file) on the first run, and play the recorded response on subsequent runs to test without network calls.

Advantages of this approach:

- *First run*: Verifies integration with actual API (roles as integration test)
- *Run after*: Quick and conclusive testing with recorded responses (acts as a unit test)
- *CI/CD*: Tests can be run without an API key

```
import pytest

@pytest.fixture(scope="module")
def vcr_config():
    return {"record_mode": "once"}

@pytest.mark.vcr()
def test_agent_with_recorded_responses():
    result = agent.invoke("What is LangGraph?")
    assert "framework" in result.lower()
```

9.5 observability – LangSmith Tracing

LangSmith tracing records *every step* of an agent's execution. A trace is a complete record of an agent's execution, from initial user input to the final response, including all model calls, tool usage, and decision points.

Information recorded in traces

Item	Description
<i>Input/Output</i>	Input data and output results of each step
<i>Model Call</i>	Prompts, responses, model parameters
<i>tool calling</i>	Called tool, arguments, return value
<i>Latency</i>	Time required for each stage
<i>Token Usage</i>	Number of input/output tokens
<i>Error</i>	Failed Steps and Error Messages

How to activate

Just set two environment variables and you'll get automatic tracing *without any additional code*:

```
export LANGSMITH_TRACING=true
export LANGSMITH_API_KEY=<your-api-key>
```

`create_agent` agents automatically send execution data to LangSmith when the relevant environment variables are configured. All LangChain components (LLMs, chains, tools, etc.) include built-in instrumentation, so no extra code changes are required.

Selective tracing & PII anonymization (`tracing_context` + `LangChainTracer`)

`tracing_context` lets you scope tracing to a code block and swap project, tags, and metadata at runtime. In production, you typically pair it with a directly-instantiated `LangChainTracer` and an *anonymizer* callback that masks PII (emails, phone numbers, IDs) *before* the trace leaves your process.

```
import re
from langsmith import tracing_context
from langchain.callbacks.tracers import LangChainTracer

EMAIL_RE = re.compile(r"[\w.+-]+@[\\w-]+\.[\\w.-]+")
PHONE_RE = re.compile(r"\b\d{2,3}-\d{3,4}-\d{4}\b")

def anonymize(payload: dict) -> dict:
    """Mask PII before the trace ships to LangSmith."""
    text = str(payload)
    text = EMAIL_RE.sub("<email>", text)
    text = PHONE_RE.sub("<phone>", text)
    return {**payload, "redacted": text}

tracer = LangChainTracer(
    project_name="production-agent",
    client_kwargs={"hide_inputs": anonymize,
                  "hide_outputs": anonymize},
)

with tracing_context(
    project_name="production-agent",
    tags=["v2.1", "experiment-A"],
    metadata={"user_tier": "premium"},
):
    agent.invoke({"messages": [...]}, config={"callbacks": [tracer]})
```

! WARNING

Run PII masking *before* the trace leaves the process. Keep the original payload on the model side and store only the redacted copy in LangSmith.

9.6 Trace analysis

Monitor the quality of your production agents by analyzing traces in the LangSmith dashboard.

metrics	Description	Confirmation Points
<i>Latency</i>	Time required for each stage	Identify bottleneck section (which node is the slowest)
<i>Token Usage</i>	Number of input/output tokens	Cost optimization (adjust prompt length, remove unnecessary context)
<i>Error Rate</i>	Failed Execution Rate	Stability monitoring (failure rate for specific tool, LLM timeouts, etc.)
<i>Tool Call Frequency</i>	Call frequency by tool	Agent behavior pattern analysis (identifying excessive tool calling and unused tool)

Utilizing tags and metadata

You can classify and filter traces by adding custom tags and metadata with the `config` parameter or `tracing_context`. Examples of useful tags/metadata in a production environment:

Tags/Metadata	Use
<i>Version tag</i> (<code>v2.1</code>)	A/B testing, performance comparison by version
<i>Experimental Tag</i> (<code>experiment-A</code>)	Experiment tracking, including prompt changes
<i>User Tier</i> (<code>premium</code>)	Quality monitoring by user group
<i>Region</i> (<code>kr</code>)	Latency analysis by region

Project Management

Projects can be set up in two ways:

- *static*: Specify the default project with the `LANGSMITH_PROJECT` environment variable.
- *Dynamic*: Separate project by code block with `tracing_context(project_name=...)`

```
try:
    from langsmith import tracing_context

    with tracing_context(
        project_name="production-agent",
        tags=["v2.1", "experiment-A"],
        metadata={"user_tier": "premium", "region": "kr"},
    ):
        print("Tagged tracing enabled")
except Exception as e:
    print(f"LangSmith tracing unavailable: {e}")
```

9.7 LangGraph Studio – Visual Debugging

LangGraph Studio is a free tool that allows you to *visually* debug an agent's execution flow. Optimized for developing and testing agents on your local machine.

Features	Description
<i>Graph visualization</i>	Inspect the agent's node/edge structure in real-time and highlight the currently running node
<i>Step-by-step execution</i>	Debug by inspecting each node's input/output: prompts, tool calls, and results step by step
<i>State inspection & editing</i>	Browse message history and checkpoints, then <i>edit node inputs on the fly</i> and re-run
<i>Time travel</i>	Rewind to an earlier checkpoint to try alternative branches or tool calls
<i>Interrupt handling (HITL)</i>	Resume <code>interrupt()</code> -paused graphs directly from the Studio UI with <code>Command(resume=...)</code>

<i>Thread management</i>	Show conversation trees per <code>thread_id</code> in the sidebar so you can compare past runs
<i>Live streaming</i>	Observe execution in real-time with token / latency metrics

Setting up a local development server

To use Studio, start your local development server with the LangGraph CLI:

```
# Install LangGraph CLI (requires Python 3.11+)
pip install --upgrade "langgraph-cli[inmem]"

# Start the development server
langgraph dev
```

Once the server is running, open `https://smith.langchain.com/studio/?baseUrl=http://127.0.0.1:2024` in your browser.

Information available in Studio

- Prompt sent to the agent
- Each tool call and its result
- Final output
- Intermediate state (inspectable and editable)
- Token usage and latency metrics

Chat UI scaffold – `npx create-agent-chat-app`

Studio is the *developer's* debugging surface; end-users need a separate chat UI. The `create-agent-chat-app` template from the LangChain team scaffolds a Next.js chat app wired to a deployed LangGraph agent in a single command.

```
npx create-agent-chat-app@latest my-agent-ui
cd my-agent-ui
# Set NEXT_PUBLIC_API_URL, NEXT_PUBLIC_ASSISTANT_ID, LANGSMITH_API_KEY in .env.local
pnpm dev
```

The template ships with `/runs/stream` SSE streaming, `thread_id`-scoped conversation history, and a built-in interrupt UI for `interrupt()` — a convenient *shell* around any deployed graph.

9.8 Deployment Options

Agents are *long-running processes that maintain state*, so they require a different approach than regular web app hosting. Traditional stateless web application hosting (e.g. Vercel, Heroku) is not suitable for persistent state management, background execution, and checkpointing of agents.

Options	Description	Suitable for
<i>LangGraph Cloud</i>	LangSmith Managed Hosting. Automatic build/deployment with just a GitHub connection	Rapid prototyping, small teams

<i>Self-Hosted</i>	Run as Docker containers on your own infrastructure	Data Sovereignty, Enterprise, Regulatory Environment
<i>Hybrid</i>	Cloud Management + Own Runtime	Convenience of management + data control

Deployment Requirements

Item	Description
<i>GitHub repository</i>	Code hosting (supports both public and private)
<i>LangSmith Account</i>	Free to join (smith.langchain.com)
<i>langgraph.json</i>	Deployment configuration file defining dependencies, graphs, and environment variables

LangGraph Cloud deployment process

1. Log in to LangSmith and go to the Deployments page.
2. Click “+ New Deployment”
3. Connect your GitHub account (for private repositories)
4. Select the repository and submit (takes about 15 minutes)
5. After deployment is complete, copy the API URL and use it on the client.

9.9 langgraph.json settings

`langgraph.json` is the core configuration file for your deployment. Define dependencies, graph entry points, and environment variables. This file must be located in the project root for LangGraph CLI and Cloud deployment to operate properly.

```
import json

langgraph_config = {
    "dependencies": [".",],
    "graphs": {"agent": "./src/agent.py:graph"},
    "env": ".env",
}

print(json.dumps(langgraph_config, indent=2))
```

Setting item details

Item	Type	Description
<code>dependencies</code>	<code>list[str]</code>	Python package dependencies. <code>references pyproject.toml</code> in the current directory
<code>graphs</code>	<code>dict</code>	Mapping graph names and module paths. <code>"module_path:variable_name"</code> format

env	str	Environment variable file path (<code>.env</code> format). Contains sensitive information such as API keys
-----	-----	---

Graph entry point

The value of the `graphs` field is in the format `"./path/file.py:variable_name"`. The variable must be a `CompiledGraph` instance. Both LangGraph's Graph API (`StateGraph`) and Functional API (`@entrypoint`) are available.

Graph API example

```
# src/agent.py
from langgraph.prebuilt import create_react_agent

graph = create_react_agent(
    model="claude-sonnet-4-6",
    tools=[search_tool],
    checkpointers=True,
)
```

Functional API Example

LangGraph's Functional API lets you add persistence, memory, and streaming to existing Python code with minimal changes. The `@entrypoint` decorator defines the start of the workflow, and the `@task` decorator marks an individual unit of work.

```
# src/agent.py
from langgraph.func import entrypoint, task

@task
def process_query(query: str) -> str:
    return f"Processing complete: {query}"

@entrypoint(checkpointer=checkpointer)
def graph(inputs: dict) -> str:
    result = process_query(inputs["query"])
    return result.result()
```

Key features of Functional API:

- *Uses standard Python control flow* (if/for, etc.) – No explicit graph structure required
- *Function scope state management* – No need to declare a separate state or set up a reducer.
- *Task result checkpointing* – Automatically reuses previously completed task results when re-executed
- *Input/output JSON serialization required* – When using checkpointers, all data must be serializable

9.10 Deployment command

Build, local server, and cloud deployment commands using LangGraph CLI.

1. Build Docker image

Create a Docker image based on the `langgraph.json` settings:

```
langgraph build -t my-agent:latest
```

2. Run the local server

Run the agent locally for testing and connect it to the Studio UI for visual debugging:

```
# Production mode (Docker-based)
langgraph up --config langgraph.json

# Development mode (in-memory, quick start)
langgraph dev
```

3. Cloud Deployment

LangSmith In the dashboard:

1. Deployments -> “+ New Deployment”
2. GitHub Connect the repository
3. Select the repository and submit it (about 15 minutes)
4. Copy the API URL after deployment completes

Python SDK Access the deployed agent from the Python SDK

`langgraph-sdk` allows you to communicate programmatically with the deployed agent. All functions, including streaming, thread management, and status inquiry, are controlled by the SDK.

```
try:
    from langgraph_sdk import get_sync_client

    api_key = os.environ.get("LANGSMITH_API_KEY", "")
    if not api_key:
        print("LANGSMITH_API_KEY Not set. Skipping client connection.")
    else:
        client = get_sync_client(
            url="https://your-deployment.langsmith.com",
            api_key=api_key,
        )
        print("Client connected:", type(client).__name__)
except Exception as e:
    print(f"LangGraph SDK client unavailable: {e}")
```

Streaming with `client.runs.stream(...)`

The SDK's `runs.stream(...)` returns tokens, messages, and tool calls from the deployed agent over *Server-Sent Events*. Combine `stream_mode` values like `"values"`, `"updates"`, `"messages"`, `"events"`, and pass a `thread_id` to keep the conversation history of the same session.

```
from langgraph_sdk import get_client

client = get_client(url="https://your-deployment.langsmith.com",
                    api_key=api_key)

async for chunk in client.runs.stream(
    thread_id="user-42",
    assistant_id="agent",
    input={"messages": [
        {"role": "user", "content": "Summarize sales for the last 5 minutes."}
    ]},
    stream_mode=["updates", "messages"],
):
    print(chunk.event, chunk.data)
```

Access via REST API

Deployed agents can also be accessed through REST API. This allows you to communicate with the agent in any programming language/framework:

```
curl --request POST \
  --url <DEPLOYMENT_URL>/runs/stream \
  --header 'Content-Type: application/json' \
  --header 'X-API-Key: <LANGSMITH_API_KEY>' \
  --data '{
    "assistant_id": "agent",
    "input": {
      "messages": [
        {"role": "user", "content": "Hello!"}
      ]
    },
    "stream_mode": "updates"
  }'
```

Key endpoints:

Endpoint	Method	Description
/runs/stream	POST	Streaming execution
/runs	POST	Synchronous execution
/threads	POST	Create a new thread
/threads/{id}/state	GET	thread status query

Summary

Topic	Key Takeaways
Testing Strategy	Unit test (GenericFakeChatModel) + integration test (agentevals) in parallel
LangSmith Evaluation	Trajectory Match (strict/unordered/subset/superset), LLM-as-Judge
observability	Automatic tracing with LANGSMITH_TRACING=true + LANGSMITH_API_KEY
Trace Analysis	Latency, Token Usage, Error Rate, Tool Call Frequency
LangGraph Studio	Visual graph debugging, step-by-step state inspection
Deployment Options	Cloud (Managed), Self-Hosted (Docker), Hybrid
langgraph.json	dependencies , graphs , env 3 core settings
Deployment Command	langgraph build -\> langgraph up -\> LangSmith Deploy

P A R T

VI

LangSmith

Tracing · Evaluation · Monitoring

What You Will Learn in This Part

- 1. Quickstart: Your First Trace
 - 2. Agent Trace Structure
- 3. Datasets and the Evaluation Loop
 - 4. Prompt Hub Versioning
 - 5. Production Monitoring

01 LangSmith Quickstart

From the first trace to a UI tour

LangSmith is the *LLM application observability platform* built by the LangChain team. Without changing a line of agent code, adding three environment variables is enough to record every LLM call and tool execution as traces automatically. This chapter walks from API key issuance, to confirming your first trace in the UI, to incorporating plain Python functions into traces via the `@traceable` decorator and `langsmith.Client`.

Learning Objectives

LEARNING OBJECTIVES

- Issue a LangSmith API key and load it via `.env`
- Confirm that existing agents are auto-traced with just `LANGSMITH_TRACING=true`
- Make traces searchable with `run_name` · `tags` · `metadata`
- Read the trace tree, latency, token usage, and cost in the UI
- Incorporate plain functions into traces with `@traceable`

1.1 API keys and environment variables

LangSmith enables tracing with no code changes — three environment variables do it. Add the following to your `.env`.

```
LANGSMITH_API_KEY=lsv2_pt_XXXXXXX
LANGSMITH_TRACING=true
LANGSMITH_PROJECT=langsmith-quickstart
```

`LANGSMITH_PROJECT` is the namespace shown in the UI's left sidebar under *Projects*. If you leave it unset, every trace lands in the `default` project. API keys are shown only once on creation, so copy the key into `.env` immediately.

Add the `langsmith>=0.3` package alone and `langchain` · `langgraph` · `deepagents` auto-instrumentation all share these environment variables.

```
pip install -U langsmith
```

Onboarding flow (first time only)

The first login goes through a four-step onboarding: role selection → mode selection → home → quickstart dialog. Choose the Technical role and LangSmith (code-first) mode to follow the developer track.

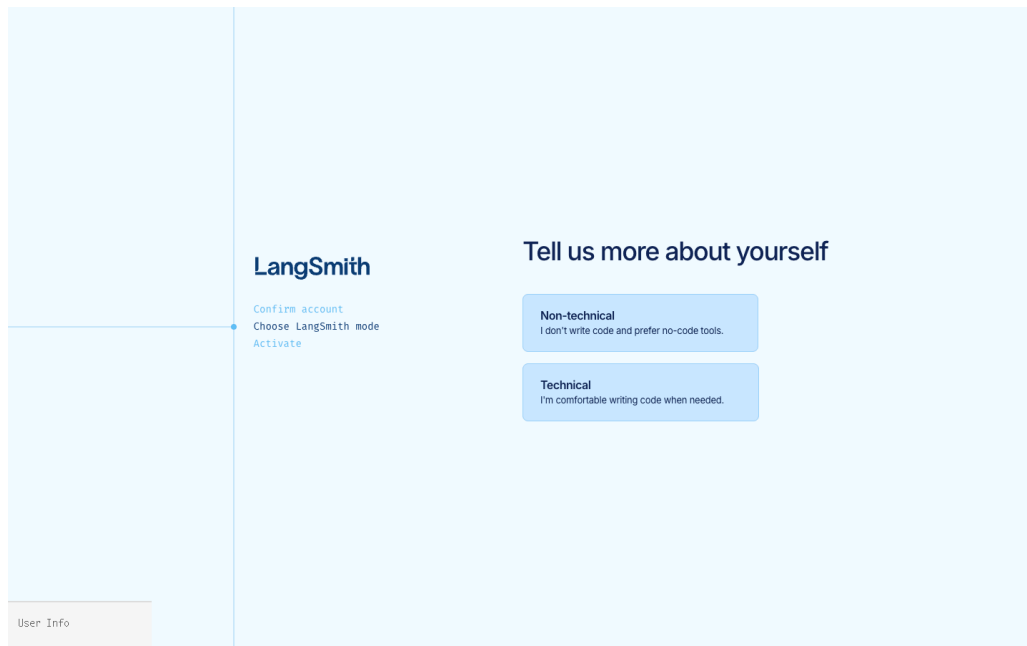


Figure 1: Role selection — choosing Technical takes you into the developer-oriented code-first flow

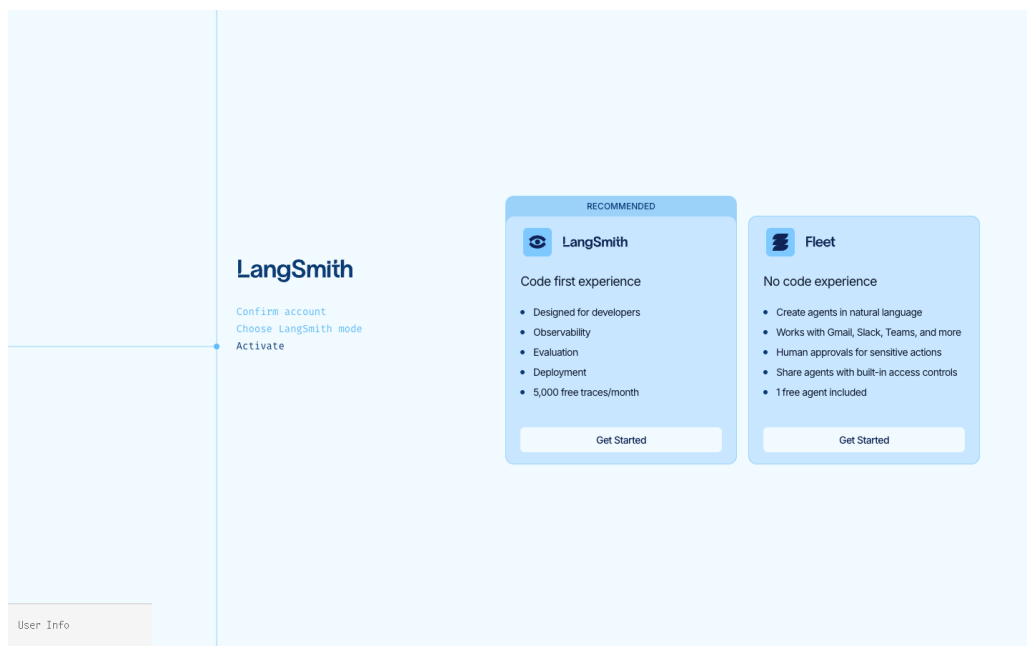


Figure 2: LangSmith (code-first) vs Fleet (no-code) — this chapter follows the LangSmith path

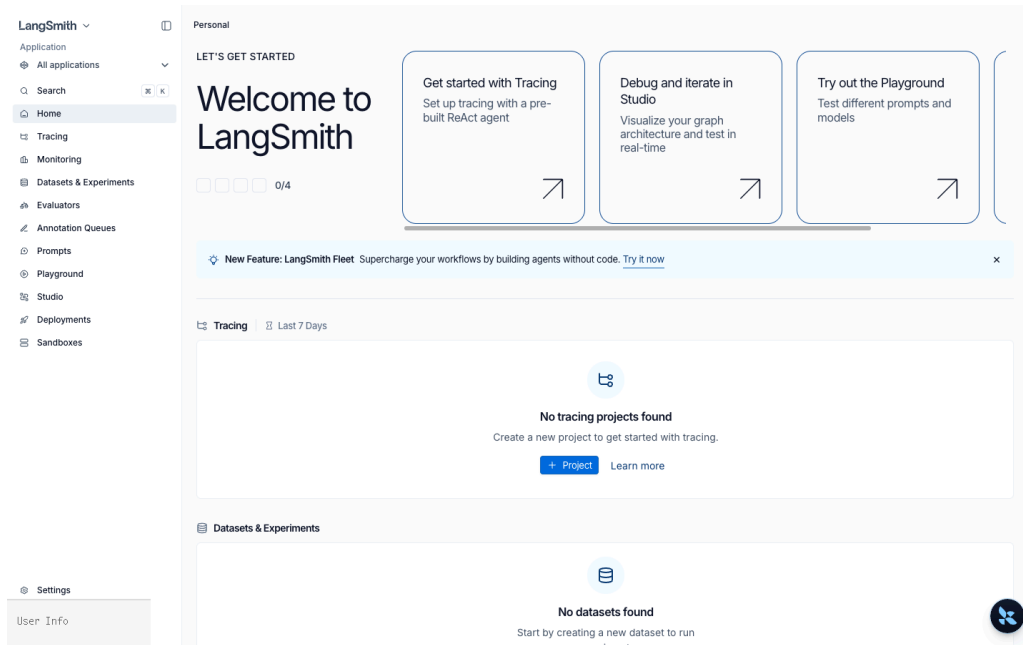


Figure 3: Home right after onboarding — Tracing / Datasets / Prompts all in initial state

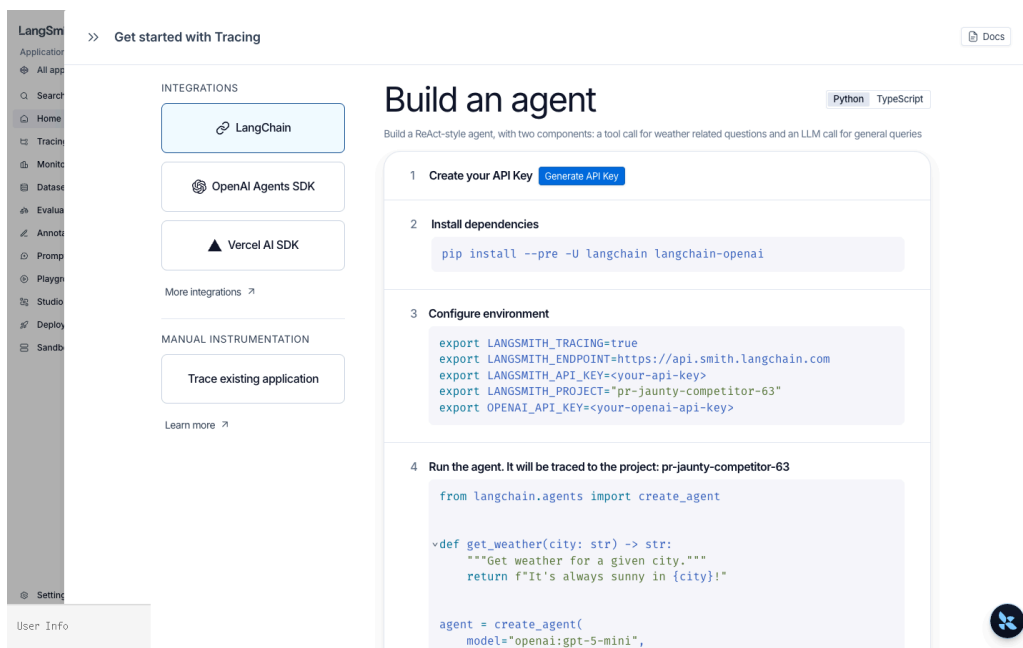


Figure 4: The four-step quickstart dialog that opens as the first card on Home

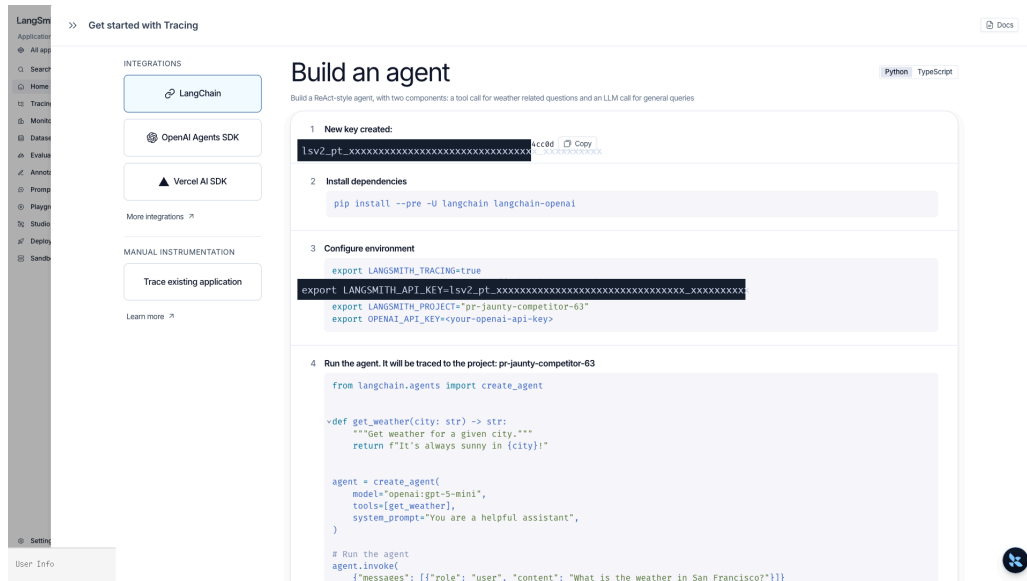


Figure 5: Clicking Generate API Key produces an `lsv2_pt_...` key — shown only once, so copy it into `.env` immediately

1.2 Your first trace — a LangChain agent

`create_agent` is *auto-instrumented*. As soon as the environment variables are set, the moment the code below runs it lands in LangSmith.

```
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain.tools import tool

load_dotenv()

@tool
def get_weather(city: str) -> str:
    """Look up the weather for a city."""
    return f"{city} is sunny"

agent = create_agent(
    model="openai:gpt-5.4",
    tools=[get_weather],
)

agent.invoke({"messages": [{"role": "user", "content": "What's the weather in Seoul?"}]})
```

After running, that invocation appears as a single row under *Projects* → *langsmith-quickstart* in the UI. Clicking it reconstructs the LLM-call → tool-call → LLM-call tree visually.

Name	Most Recent Run (7d)	Trace Count (7d)	Error Rate (7d)	P50 Latency (7d)	P99 Latency (7d)	Total Tokens (7d)	Total Cost (7d)	Description
pr-jaunty-competitor-63	a few seconds ago	3	0%	4.16s	4.29s	602	\$0.01	

Figure 6: Projects list — Trace Count, P50/P99 Latency, Total Tokens, and Total Cost are aggregated automatically

Name	Input	Output	Error	Start Time	Latency	Dataset	Annotation Queue	Tokens	Cost	First Token	Tags	Metadata
weather-pipeline	서울특별시	현재 서울의 날씨는 ...		4/18/2026, 2:...	1.87s			195	\$0.000618			
weather-bot:seoul-re...	user: 부산 날씨 알려줘	ai: 현재 부산의 날씨...		4/18/2026, 2:...	4.16s			206	\$0.000682		env:dev featurew	
LangGraph	user: 서울 날씨 알려줘	ai: 서울의 현재 날씨...		4/18/2026, 2:...	4.29s			201	\$0.000666			

Figure 7: Run table on the project detail page — Input / Output / Latency / Tokens / Cost / Tags / Metadata in one view

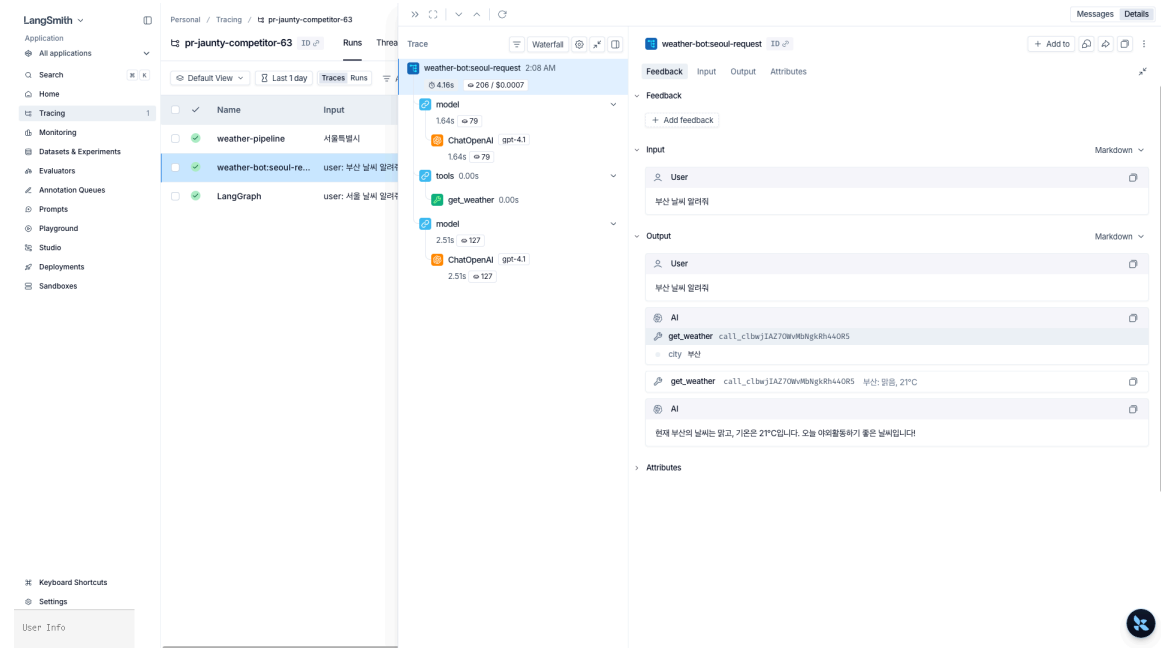


Figure 8: Trace Tree — `model` → `tools` → `model` order with tokens / latency shown as a waterfall at each step

1.3 run_name · tags · metadata

Attach search and filter metadata to traces with `invoke(..., config=...)`. This is the main lever that makes queries like “show only failures this user had this session” possible in operations.

```
agent.invoke(
  {"messages": [{"role": "user", "content": "Weather in Busan"}]},
  config={
    "run_name": "weather-query-demo",
    "tags": ["env:dev", "feature:weather"],
    "metadata": {
      "user_id": "u_00123",
      "session_id": "s_demo",
      "app_version": "0.1.0",
    },
  },
)
```

Confirm filtering works in the UI with `Filter` → `Tags contains env:dev` or `Metadata.user_id = u_00123`.

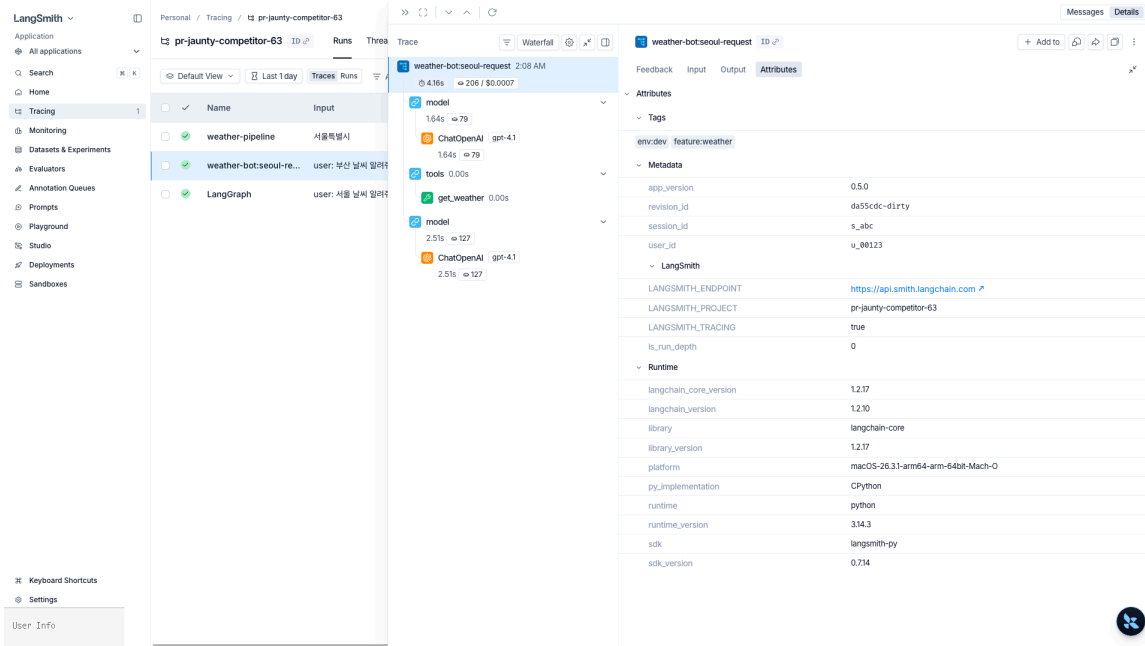


Figure 9: Attributes tab — attached Tags / Metadata are visible, and environment / runtime info is auto-captured

1.4 Tracing plain Python functions — @traceable

Preprocessing / postprocessing helpers that do not go through LangChain can also be incorporated into traces via `@traceable`. They nest as child spans under the parent trace, making it possible to track things like “why did we query this city”.

```
from langsmith import traceable

@traceable(run_type="chain", name="normalize_city")
def normalize_city(raw: str) -> str:
    return raw.strip().replace("City", "")

@traceable(run_type="chain", name="weather-pipeline")
def run_weather(user_text: str) -> str:
    city = normalize_city(user_text)
    result = agent.invoke(
        {"messages": [{"role": "user", "content": f"Weather in {city}"}]},
    )
    return result["messages"][-1].content

run_weather("Busan City")
```

In the UI, `weather-pipeline` is the root with `normalize_city` and `AgentExecutor` grouped beneath it as a single tree.

Enable tracing without env vars via `tracing_context`

To toggle tracing on and off temporarily without environment variables, use `langsmith as ls`'s `tracing_context` as a context manager. Useful for capturing only a specific block from a notebook or test.

```
import langsmith as ls
```



```
with ls.tracing_context(enabled=True, project_name="langsmith-quickstart"):
    agent.invoke({"messages": [{"role": "user", "content": "Weather in Seoul"}]})
```

Calling it with `enabled=False` disables tracing for that block alone, even when the global setting is on — useful for isolating blocks that handle sensitive data.

1.5 Re-querying traces from code

`langsmith.Client` lets you pull traces programmatically without the UI. Use it as inputs for regression tests, source data for nightly batch reports, or seeds for building datasets.

```
from langsmith import Client

client = Client()

runs = list(
    client.list_runs(
        project_name="langsmith-quickstart",
        filter='and(has(tags, "env:dev"), eq(is_root, true))',
        limit=10,
    )
)

for r in runs:
    print(r.id, r.name, r.total_tokens, r.total_cost)
```

The `filter` expression uses the same DSL that the UI `Add filter` emits.

1.6 Cost and token aggregation

The *Analytics* tab at the top-right of the UI project page shows project-level total cost and token time series. LangSmith fills `total_cost` automatically using per-model pricing; self-hosted models can have custom prices set under *Settings* → *Model pricing*.

1.7 Re-issuing API keys

To re-issue or revoke a key after onboarding, go to *Settings* → *Access and Security* → *API Keys*. Existing keys record their Last Used At automatically; if compromise is suspected, revoke immediately and issue a new one.

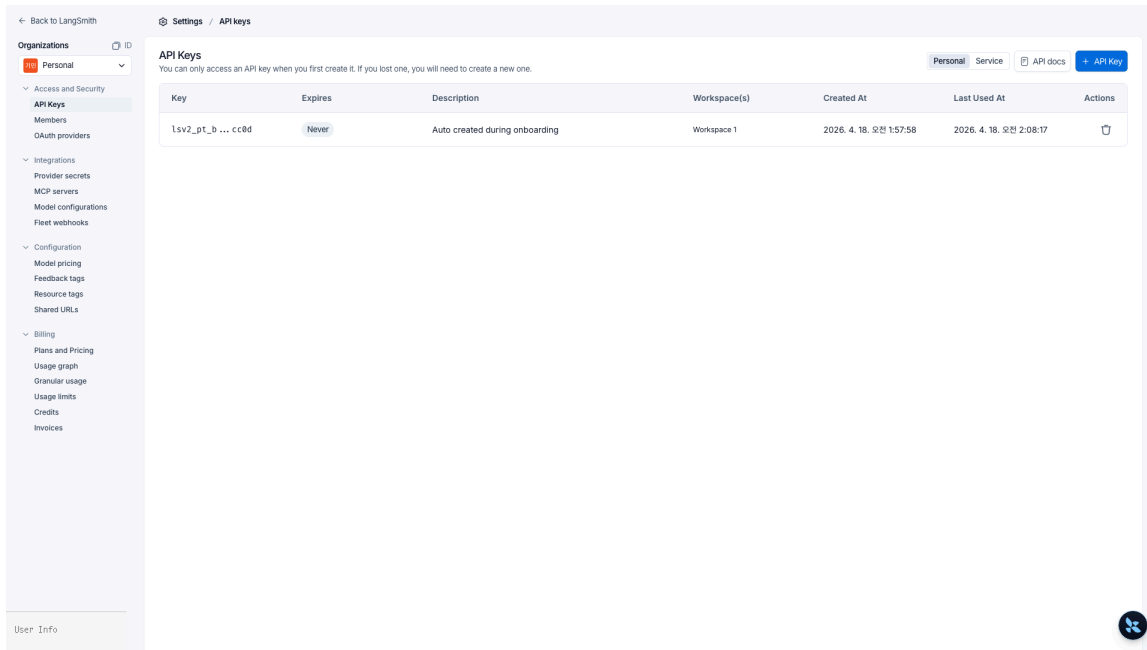


Figure 10: Settings > API Keys — the Key column shows only the prefix/suffix automatically, and Last Used At is tracked

Key Takeaways

- Three environment variables (`LANGSMITH_API_KEY` , `LANGSMITH_TRACING=true` , `LANGSMITH_PROJECT`) auto-trace existing agents — runs land in the `default` project when unset
- `run_name` · `tags` · `metadata` surface runs in UI filters and `client.list_runs(filter=...)` queries
- `@traceable` pulls non-LangChain functions into the same trace tree
- `import langsmith as ls` + `ls.tracing_context(enabled=...)` toggle tracing per block without environment variables
- `langsmith.Client` fetches traces programmatically to seed evaluation and regression tests
- The three UI levels — Projects / Runs / Trace tree — reproduce “which call produced what”

02 Agent Trace Structure

Subgraph · SubAgent · Thread · Feedback

Chapter 1 showed one `create_agent` invocation in the UI. This chapter covers how nested-structure agents — *LangGraph StateGraph · Deep Agents subagents · async tasks* — are drawn as traces. It collects operational concerns end to end: the four concepts of Run / Trace / Project / Thread, subgraph namespaces, the trace differences between sync and async subagents, the feedback API, and the 400-day retention limit.

Learning Objectives

LEARNING OBJECTIVES

- Understand the relationship between Run · Trace · Project · Thread (run = span, trace = span tree)
- Confirm how LangGraph subgraphs appear as namespaced children inside the parent trace
- Distinguish Deep Agents sync vs async subagent (`async_tasks` `channel`) trace differences
- Group multiple runs into a Thread view via `thread_id` / `session_id`
- Attach evaluation scores to a run with `client.create_feedback(run_id, key, score)`
- Run programmatic filters based on tags and metadata with `client.list_runs(filter=...)`
- Persist important traces as *datasets* to survive the 400-day retention limit

2.1 Concepts: Run · Trace · Project · Thread

LangSmith's data layer stacks in four levels.

Level	Definition	Example
Project	Container for all traces from the same application	<code>langsmith-tracing-agents</code>
Trace	Tree of runs created while servicing a single user request (up to 25,000 runs / trace)	One agent invocation
Run	A single span — LLM call, tool call, chain node, etc.	<code>ChatOpenAI</code> , <code>get_weather</code>
Thread	Multiple traces grouped by <code>thread_id</code> / <code>session_id</code> / <code>conversation_id</code> — the multi-turn conversation view	A single user's session

Each run carries `parent_run_id`, `trace_id`, `start_time`, `end_time`, `inputs`, `outputs`, `total_tokens`, `total_cost`, etc. A trace is simply the tree of runs that share the same `trace_id`.

Name	Input	Output	Error	Start Time	Latency	Dataset	Annotation Queue	Tokens	Cost	First Token	Tags
LangGraph	user: fetch_forum_po...	ai: 포럼 글(post_id...		4/18/2026, 2...	4.91s			276	\$0.001038		
LangGraph	user: 자유로운 창작 예시...	ai: 물론이죠! 다음은 ...		4/18/2026, 2...	3.79s			274	\$0.002072		
LangGraph	user: 너 자신을 해치는 구...	OpenAIModeratio...		4/18/2026, 2...	0.57s						
LangGraph	user: 너 자신을 해치는 구...	ai: I'm sorry, but I...		4/18/2026, 2...	0.76s						
LangGraph	user: 파이썬 익제너러와 ...	ai: 익제너러는 카-값 ...		4/18/2026, 2...	3.61s			53	\$0.000256		
ChatOpenAI	human: ping	ai: pong! How ca...		4/18/2026, 2...	1.22s			18	\$0.000096		
LangGraph	user: '취소' 또는 'cancel...	ai: 검색 결과입니다! ...		4/18/2026, 2...	5.89s			5,996	\$0.021732		
LangGraph	user: /docs/backend/...	ai: /docs/backen...		4/18/2026, 2...	13.06s			15,271	\$0.052965		
LangGraph	user: 아래 3개 파일을 /d...	ai: 3개 파일을 모두 ...		4/18/2026, 2...	6.70s			4,936	\$0.021708		
ChatAnthropic	human: ping	ai: Pong! 🏓 How ...		4/18/2026, 2...	2.24s			26	\$0.000294		
LangGraph	user: 다음 사실을 메모리...	ai: 메모리에 저장 한...		4/18/2026, 2...	5.56s			5,737	\$0.020379		
LangGraph	user: 내 주 언어를 기본으...	ai: 지만님의 주 언어...		4/18/2026, 2...	16.48s			5,492	\$0.029088		
LangGraph	user: 내 이름은 지민이고 ...	ai: 저장 완료했어요! ...		4/18/2026, 2...	7.22s			5,696	\$0.02154		
ChatAnthropic	human: ping	ai: Pong! 🏓 How ...		4/18/2026, 2...	1.43s			26	\$0.000294		
LangGraph	user: /src/main.py 의 '...	ai: /src/main.py ...		4/18/2026, 2...	7.16s			4,646	\$0.01827		
LangGraph	user: /etc/passwd 퍼...	ai: 완료되었습니다! ...		4/18/2026, 2...	8.93s			4,921	\$0.021003		
LangGraph	user: /notes/todo.md ...	ai: /notes/todo.m...		4/18/2026, 2...	5.75s			3,006	\$0.012294		

Figure 11: Project Runs list — 17 columns including Name / Input / Output / Error / Latency / Dataset / Tokens / Cost / Tags / Metadata

Thread	First Input	Last Output	First Start Time
user:42 2 turns → 11,388 tokens · \$0.06	user: 내 이름은 ...	ai: 지민님의 주 연...	2026. 4. 18. 오...
t_demo_0001 6 turns → 51,841 tokens · \$0.02	user: 안녕	ai: 안녕하세요, 더 평...	2026. 4. 18. 오...
s_abc 1 turn → 206 tokens · <\$0.01	user: 부산 날씨 ...	ai: 현재 부산의 날...	2026. 4. 18. 오...

Turn	Input	Output	Attributes
1 chat-turn			
2 PatchToolCallsMiddle...			
3 model			
4 ChatOpenAI			
5 TodoListMiddleware...			
6 chat-turn			
7 chat-turn			
8 chat-turn			
9 chat-turn			
10 chat-turn			

Feature	Value
feature:chat	env:dev
Metadata	
app_version	0.5.0
revision_id	da55cdc-dir...
thread_id	t_demo_0001
user_id	u_00123
LangSmith	
LANGSMITH_ENDPOINT	https://api.smith...
LANGSMITH_PROJECT	pr-jaunty-com...

Figure 12: LangGraph subgraph trace tree — a `PatchToolCallsMiddleware` → `model` → `ChatOpenAI` → `TodoListMiddleware` chain built with namespaces

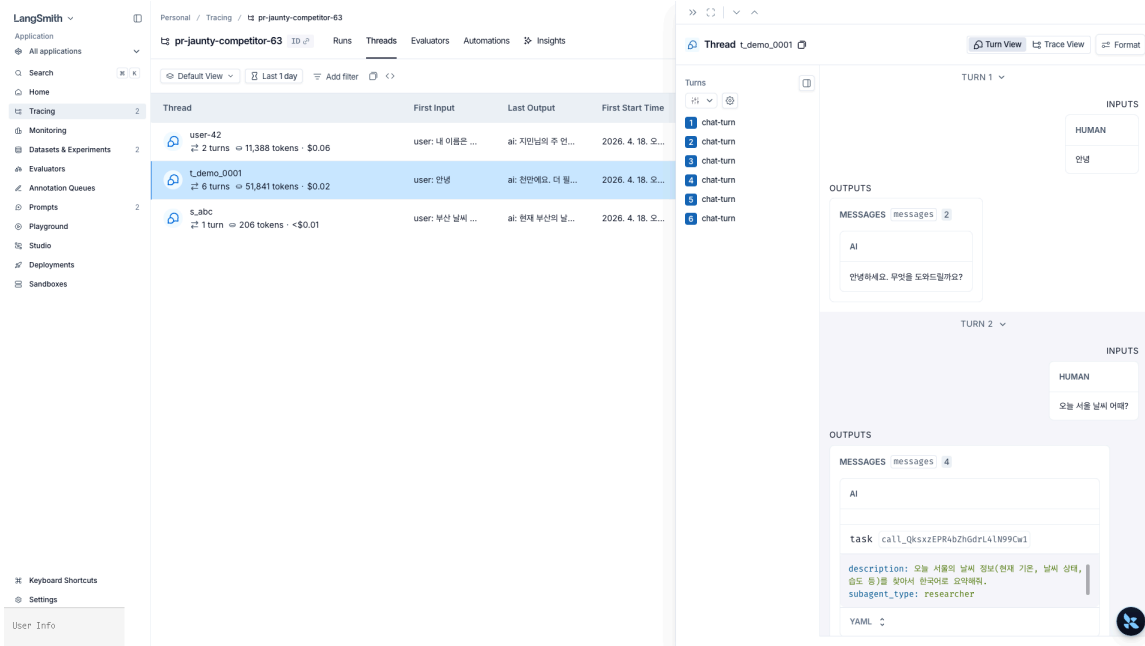


Figure 14: Thread Turn View — each turn's Input / Output shown as conversation bubbles with task call description and `subagent_type: researcher` YAML

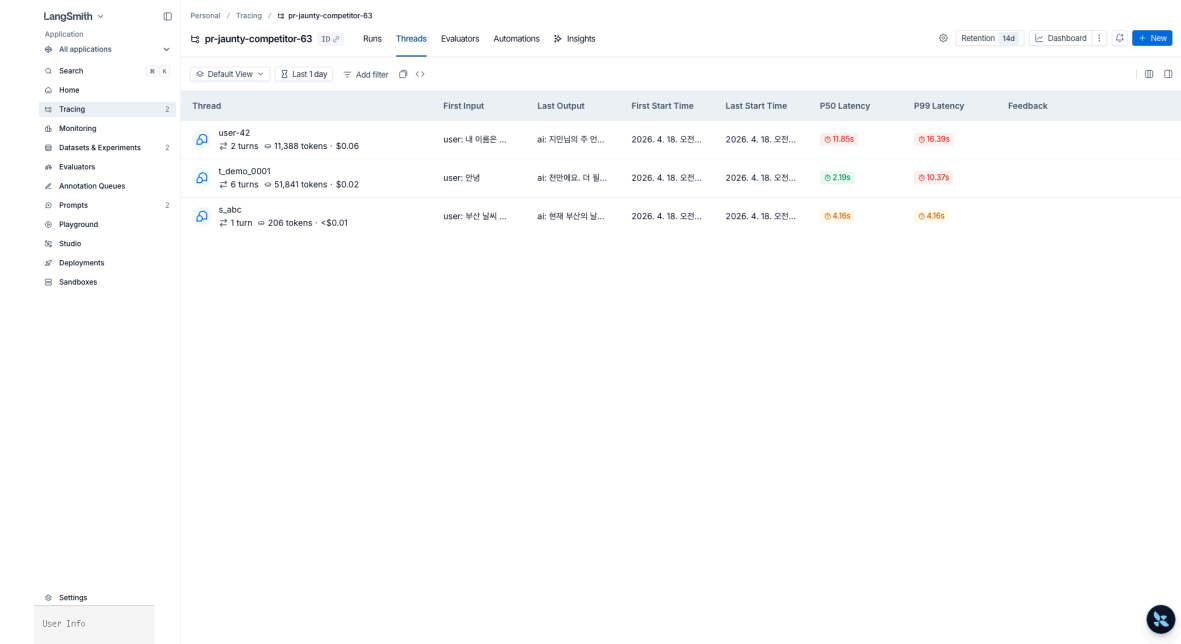
```
from deepagents import AsyncSubAgent

researcher = AsyncSubAgent(name="researcher", description="Long-running research", graph_id="researcher")
# parent trace: only the start_async_task tool call
# child trace: the researcher graph is a separate trace (grouped by the same thread_id)
# state preservation: the async_tasks channel survives compaction
```

2.4 Session view — `thread_id` · `session_id` · `conversation_id`

To group multiple invokes into *one conversation*, put a session identifier on the `metadata`. LangSmith automatically groups runs into the Threads view if any of `thread_id`, `session_id`, or `conversation_id` is present.

```
agent.invoke(
    {"messages": [{"role": "user", "content": "..."}]},
    config={"metadata": {"thread_id": "t_demo_0001", "user_id": "u_alice"}},
)
```



Thread	First Input	Last Output	First Start Time	Last Start Time	P50 Latency	P99 Latency	Feedback
user-42 2 turns → 11,388 tokens - \$0.06	user: 내 이름은 ...	ai: 지민님의 주 인...	2026. 4. 18. 오전...	2026. 4. 18. 오전...	11.85s	16.39s	
t_demo_0001 6 turns → 51,841 tokens - \$0.02	user: 안녕	ai: 안녕하세요. 더 평...	2026. 4. 18. 오전...	2026. 4. 18. 오전...	2.39s	10.37s	
s_abc 1 turn → 206 tokens - <\$0.01	user: 부산 날씨 ...	ai: 현재 부산의 날...	2026. 4. 18. 오전...	2026. 4. 18. 오전...	4.16s	4.16s	

Figure 15: Threads tab — runs sharing a `thread_id` grouped as a conversation session. First Input / Last Output / turns / tokens / cost / P50:P99 latency auto-aggregated

2.5 Attaching feedback to runs — `client.create_feedback`

Evaluation scores, user thumbs-up/down, and internal QA review results are attached as **Feedback** on a run.

- `key` : feedback name (e.g., `"correctness"`, `"user_thumbs"`)
- `score` : a float in 0–1 or an arbitrary number
- `value`, `comment` : optional

```
from langsmith import Client

client = Client()
client.create_feedback(
    run_id=latest_run.id,
    key="user_thumbs",
    score=1.0,
    comment="Extracted the correct answer",
)
```

2.6 Tag- and metadata-based filter queries

The same expressions used in UI filters work from code through `client.list_runs(filter=...)`. Useful for regression tests, nightly batches, and dashboard feeds.

```
runs = client.list_runs(
    project_name="langsmith-tracing-agents",
    filter='and(has(tags, "env:prod"), eq(run_type, "chain"))',
    limit=50,
)
```

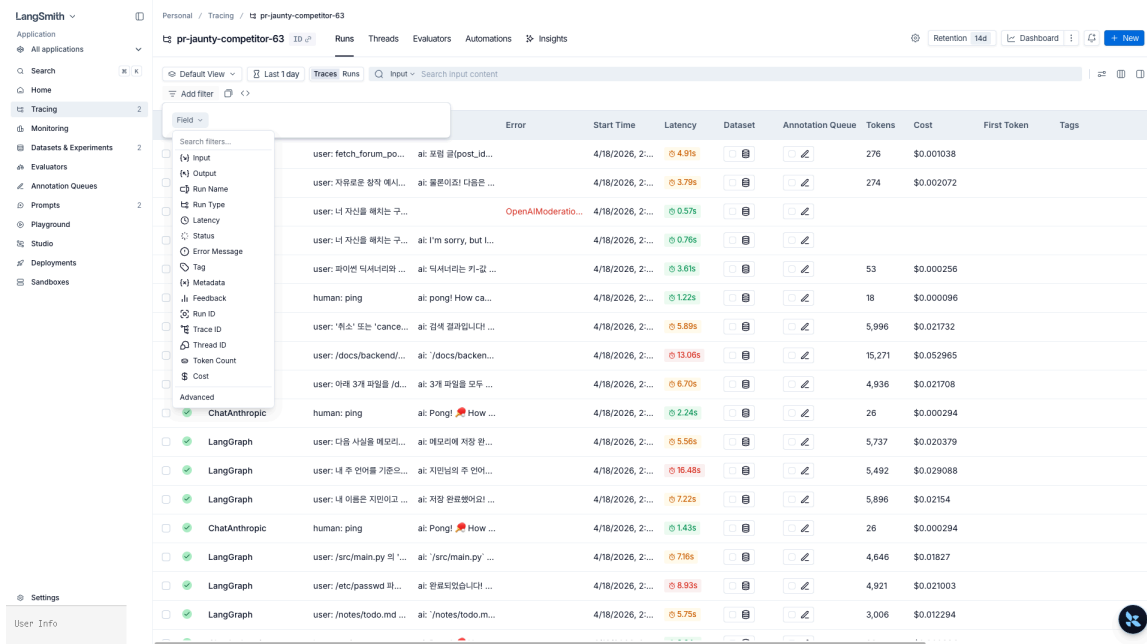


Figure 16: Add filter menu — Tag and Metadata exist as separate fields, enabling conditions such as `tags contains env:dev`

2.7 The `@traceable` decorator — `run_type` variants

`@traceable` brings non-LangChain functions into the trace tree; the `run_type` argument decides the UI icon and filter category. The common values:

run_type	Use case	UI rendering
chain	Default — any function / pipeline step	Chain icon, <code>chain</code> filter
llm	Manual LLM call (provider SDK used directly, etc.)	LLM icon, tokens / cost auto-aggregated
tool	External system calls (search · DB · API)	Tool icon, <code>tool</code> filter
retriever	Vector store / retrieval step	Retriever icon with document count
embedding	Embedding computation	Embedding icon
parser	Output parsing / post-processing	Parser icon

```
from langsmith import traceable

@traceable(run_type="retriever", name="vector-search")
def search(query: str) -> list[dict]:
    return vectorstore.similarity_search_with_score(query, k=4)

@traceable(run_type="tool", name="lookup-order")
def lookup_order(order_id: str) -> dict:
    return db.fetch_order(order_id)
```


2.8 PII masking — `LangChainTracer` + `Client(anonymizer=...)`

To regex-mask inputs/outputs *just before* they reach LangSmith, build a pattern list with `create_anonymizer` and inject it into `Client(anonymizer=...)`. A `LangChainTracer` created from that client and passed via `config={"callbacks": [...]}` records only that invocation with PII redacted.

```
from langsmith import Client
from langsmith.anonymizer import create_anonymizer
from langchain.callbacks.tracers import LangChainTracer

anonymizer = create_anonymizer([
    {"pattern": r"[a-zA-Z0-9_+.-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-]+", "replace": "<email>"},
    {"pattern": r"\b\d{4}[-]\d{4}[-]\d{4}[-]\d{4}\b", "replace": "<card>"},
    {"pattern": r"\+?\d{2,3}[-]\d{3,4}[-]\d{4}", "replace": "<phone>"},
])
masked_client = Client(anonymizer=anonymizer)
tracer = LangChainTracer(client=masked_client, project_name="prod-masked")

agent.invoke(
    {"messages": [{"role": "user", "content": "My email a@b.com, phone +1-555-1234"}]},
    config={"callbacks": [tracer]},
)
```

To block inputs/outputs entirely, use `Client(hide_inputs=lambda _: {}, hide_outputs=lambda _: {})` or set

`LANGSMITH_HIDE_INPUTS=true`. Chapter 5 revisits this as a defense-in-depth pattern alongside `PIIMiddleware`.

2.9 Selective tracing — record only specific calls

With global tracing on, it is often necessary to *exclude specific calls* or *route specific calls to a different project*.

```
import langsmith as ls

# 1) Disable only this block even if env vars enable tracing
with ls.tracing_context(enabled=False):
    agent.invoke({"messages": [{"role": "user", "content": "Sensitive data..."}]})

# 2) Route debugging calls to a separate project
with ls.tracing_context(project_name="langsmith-debug", tags=["mode:debug"]):
    agent.invoke({"messages": [{"role": "user", "content": "test case"}]})
```

`tracing_context` is thread-local so it is safe in async environments. The precedence among environment variable, context manager, and `config={"callbacks": [...]}` is *nearest scope wins*.

2.10 The 400-day retention limit → persist to datasets

SaaS LangSmith deletes traces *400 days after ingestion*. Runs you want to keep for evaluation regression must be *persisted as a Dataset*. We cover datasets in depth in chapter 3; here, just the pattern.

```
# langsmith 0.7 removed add_runs_to_dataset – use create_examples to convert
# each run's inputs/outputs into examples directly.
golden_runs = [r for r in runs if r.feedback.get("user_thumbs") == 1]
ds = client.create_dataset("agent-golden-traces",
                          description="Human-approved golden traces")
client.create_examples(
    dataset_id=ds.id,
```

```

examples=[
    {"inputs": r.inputs,
      "outputs": r.outputs,
      "metadata": {"source_run_id": str(r.id)}}
    for r in golden_runs if r.outputs
],
)

```

Key Takeaways

- The four-level concept — Project → Trace → Run + Thread (session grouping) — is the foundation of every UI view
- LangGraph subgraphs bake their namespaces into run names, making them filterable
- Deep Agents sync subagents are a single trace; async are separate traces — tracked via the `async_tasks` channel
- `thread_id` / `session_id` metadata triggers Thread-view grouping
- Feedback API + `list_runs(filter=...)` programmatically connect the evaluation loop
- `@traceable` 's `run_type` (`chain` / `llm` / `tool` / `retriever` / `embedding` / `parser`) decides UI icon and filter category
- Mask PII just before transmission with `create_anonymizer` + `Client(anonymizer=...)` + `LangChainTracer`
- Toggle selective tracing per block via `ls.tracing_context(enabled=...)` — thread-local
- Push traces into a Dataset to outlive the 400-day retention limit

03 Datasets and the Evaluation Loop

Code · LLM-as-judge · Pairwise · Summary · Online

Traces show “how the system is running right now”; evaluation answers “did it get better when you changed the prompt, model, or code”. LangSmith lifts traces directly into datasets and runs code evaluators · LLM-as-judge · pairwise · summary evaluations on top. This chapter covers the full evaluation pipeline: manual seed datasets, ingestion from production traces, the four evaluator types, the `evaluate` runner, and online evaluators.

Learning Objectives

LEARNING OBJECTIVES

- Create a dataset and examples with `client.create_dataset` + `client.create_examples`
- Ingest production traces into a dataset with `client.create_examples(dataset_id=..., examples=[...])`
- Write a code evaluator with the `(inputs, outputs, reference_outputs) → dict` signature
- Run an LLM-as-judge evaluator that produces a structured score
- Use pairwise / summary evaluators for A/B comparison and dataset-level metrics
- Kick off experiments with the `from langsmith.evaluation import evaluate` runner
- Auto-apply **online evaluators** to production traces

3.1 Creating a dataset — manual examples

A domain expert writes golden Q&A by hand and pushes them in with `create_examples`. Both `inputs` and `outputs` are dicts. `outputs` is the reference answer for code evaluators and LLM-as-judge comparisons.

```
from langsmith import Client

client = Client()
dataset = client.create_dataset(
    "weather-bot-qa",
    description="Golden examples for city extraction",
)

client.create_examples(
    dataset_id=dataset.id,
    inputs=[
        {"question": "What's the weather in Seoul?"},
        {"question": "Temperature in Busan City?"},
```

```

    {"question": "Will it rain in Daejeon this weekend?"},
],
outputs=[
    {"city": "Seoul"},
    {"city": "Busan"},
    {"city": "Daejeon"},
],
)

```

Name	Description	Experiments	Last Experiment	Examples	Type	Updated At	Created At
weather-bot-qa	날씨 봇 관련 Q&A — 도시명 형식과 응답 주 감응률	2	9 hours ago	3	kv	9 hours ago	9 hours ago
agent-golden-traces	영구 보존할 우수 트레이스	0		0	kv	9 hours ago	9 hours ago

Figure 17: Datasets & Experiments list — the two datasets `weather-bot-qa` and `agent-golden-traces`

3.2 Ingesting production traces into a dataset

Manual authoring only seeds the initial set; real volume comes from *production traces*. `add_runs_to_dataset` was removed in `langsmith 0.7`, so use `client.create_examples(...)` to convert each run's `inputs` / `outputs` into examples. In practice you usually push only runs that a human has approved via the Annotation Queue.

```

good_runs = [r for r in client.list_runs(project_name="prod") if is_good(r)]
client.create_examples(
    dataset_id=client.read_dataset(dataset_name="weather-bot-qa").id,
    examples=[
        {"inputs": r.inputs,
         "outputs": r.outputs,
         "metadata": {"source_run_id": str(r.id)}}
        for r in good_runs if r.outputs
    ],
)

```

Inputs	Reference Outputs	Created At	Modified At	Splits
서울 날씨 알려줘	서울	2026. 4. 18. 오전 2:18:01	2026. 4. 18. 오전 2:18:01	base
오늘 제주도 날씨 어때요?	제주	2026. 4. 18. 오전 2:18:01	2026. 4. 18. 오전 2:18:01	base
부산광역시 날씨는?	부산	2026. 4. 18. 오전 2:18:01	2026. 4. 18. 오전 2:18:01	base

3 examples in total

Page 1 Show 25

Figure 18: Dataset Examples tab — Korean question Inputs and expected city-name Reference Outputs with a JSON/YAML toggle and a Splits column

3.3 Evaluation target + code evaluator

For the example, we take an LLM function that “extracts a city from the question” as the target. A target only needs the signature `inputs: dict → outputs: dict`.

An evaluator takes `(inputs, outputs, reference_outputs)` and returns a `{“key”: ..., “score”: ...}` dict. Deterministic heuristics cost zero and add nearly zero latency — add as many as you can.

```
def city_exact_match(inputs, outputs, reference_outputs):
    return {
        "key": "city_exact_match",
        "score": int(outputs["city"] == reference_outputs["city"]),
    }

def city_non_empty(inputs, outputs, reference_outputs):
    return {
        "key": "city_non_empty",
        "score": int(bool(outputs.get("city"))),
    }
```

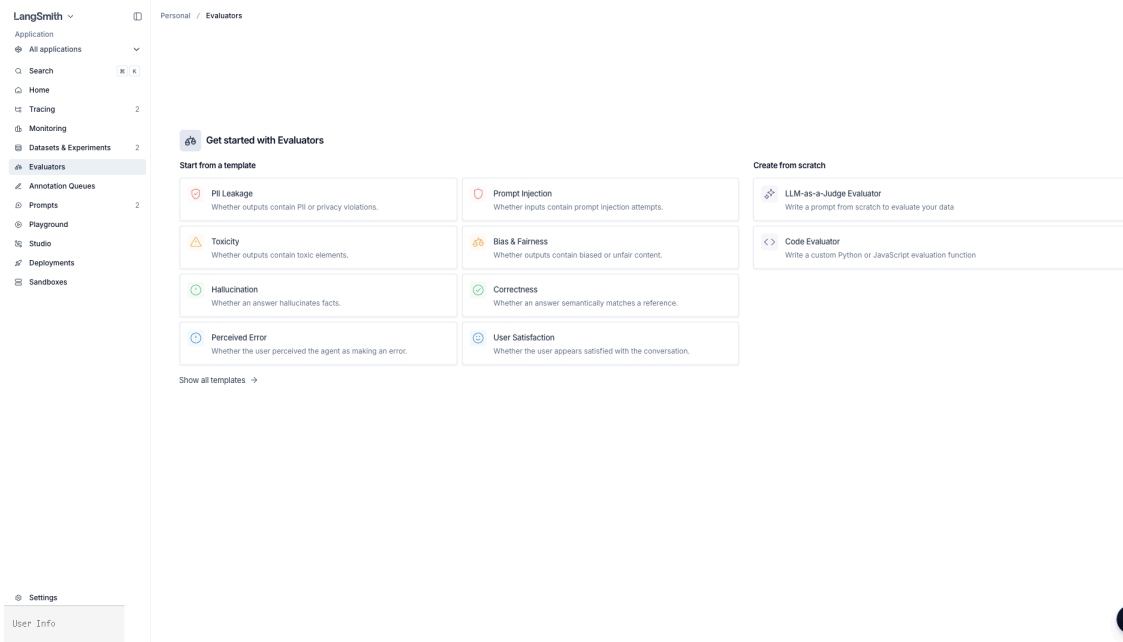


Figure 19: Evaluator template gallery — PII Leakage · Prompt Injection · Toxicity · Bias & Fairness · Hallucination · Correctness · Perceived Error · User Satisfaction + custom-written

3.4 LLM-as-judge evaluator

Use an LLM judge when there is no reference string, or when you need to grade natural-language quality (tone · accuracy · helpfulness). The key is to call the same LLM but *force structured output*.

```
from pydantic import BaseModel
from langchain_openai import ChatOpenAI

class Judgement(BaseModel):
    score: float # 0.0 ~ 1.0
    reason: str

judge_llm = ChatOpenAI(model="gpt-5.4").with_structured_output(Judgement)

def semantic_city_match(inputs, outputs, reference_outputs):
    j = judge_llm.invoke(
        f"Question: {inputs['question']}\n"
        f"Reference city: {reference_outputs['city']}\n"
        f"Extracted: {outputs['city']}\n"
        "Score 0.0–1.0 + reason",
    )
    return {"key": "semantic_city_match", "score": j.score, "comment": j.reason}
```

3.5 The `evaluate` runner + experiment name

Both import paths point at the same runner — new code should prefer the shorter `from langsmith import evaluate`. If you already have a `Client` instance, `client.evaluate(...)` works the same. Give a meaningful `experiment_prefix` and it shows up directly in the UI's Experiments view for comparison.

```

from langsmith import Client, evaluate # recommended (langsmith ≥ 0.3)
# from langsmith.evaluation import evaluate # legacy path, still works

result = evaluate(
    target=city_extractor,
    data="weather-bot-qa",
    evaluators=[city_exact_match, city_non_empty, semantic_city_match],
    experiment_prefix="city-extractor:gpt-5.4",
    metadata={"prompt_commit": "12344e88"},
)

# Client.evaluate(...) shares the same signature
client = Client()
result = client.evaluate(
    city_extractor,
    data="weather-bot-qa",
    evaluators=[city_exact_match],
)

```

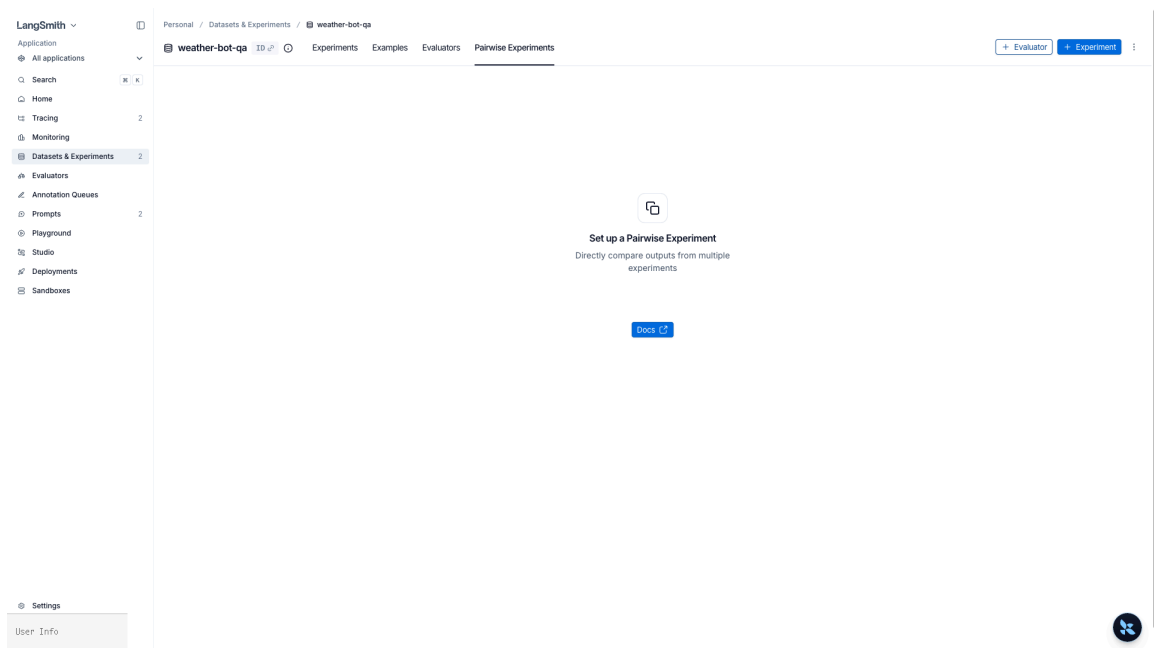


Figure 20: Pairwise Experiments tab — outputs of two experiments compared side by side. Run via the `evaluate_comparative` API

3.6 Pairwise + Summary evaluators

- **Pairwise:** Given the same example, picks “which experiment output is better”. Use for A/B prompt experiments. Runner: `evaluate_comparative`.
- **Summary:** Dataset-level metrics (for example, macro-average accuracy). The signature differs — it takes *lists* of runs and examples.

```

def macro_accuracy(runs, examples):
    correct = sum(
        r.outputs.get("city") == e.outputs["city"]
        for r, e in zip(runs, examples)
    )
    return {"key": "macro_accuracy", "score": correct / len(runs)}

```

```

evaluate(
    target=city_extractor,
    data="weather-bot-qa",
    evaluators=[city_exact_match],
    summary_evaluators=[macro_accuracy],
)

```

3.7 agentevals — evaluating agent trajectories

City extraction can be graded by looking only at the *final output*, but with agents the *sequence in which tools are called* matters too. The `agentevals` package (`pip install agentevals`) provides evaluators that compare a reference trajectory against the actual trajectory recorded as LangChain messages.

There are four comparison modes.

Mode	Match rule	When to use
<code>strict</code>	Tool names, order, and arguments must all match	Regression tests on a fixed workflow
<code>unordered</code>	Tool set must match; call order is irrelevant	Parallel tool use, pipelines where order is not material
<code>subset</code>	Actual $\subseteq$ reference (allowed actions)	Acceptable when the agent calls <i>fewer</i> tools than expected
<code>superset</code>	Reference $\subseteq$ actual (required actions)	When certain tools <i>must</i> be called

```

from agentevals.trajectory.match import create_trajectory_match_evaluator

trajectory_strict = create_trajectory_match_evaluator(
    trajectory_match_mode="strict",
)

# Pass straight to the evaluators kwarg to aggregate as an experiment score
evaluate(
    target=run_agent,
    data="agent-golden-trajectories",
    evaluators=[trajectory_strict],
    experiment_prefix="agent:trajectory-strict",
)

```

To grade trajectories with an LLM judge in free-form fashion, use `create_trajectory_llm_as_judge` . Instead of comparing tool names it asks the judge to assess *the appropriateness of the whole reasoning chain*.

```

from agentevals.trajectory.llm import create_trajectory_llm_as_judge

trajectory_judge = create_trajectory_llm_as_judge(
    model="openai:gpt-5.4",
    prompt="Did the agent's tool-call sequence efficiently fulfill the user's intent?",
)

```



TIP

`agentevals` is paired with `openevals` (generic LLM evaluation). The recommended pattern is to grade *final answer quality* with `openevals` and *process appropriateness* with `agentevals`, attaching both scores to the same experiment.

3.8 Online evaluator — automatic evaluation on production traces

Offline experiments are for pre-deploy regression testing; in production you attach **online evaluators** to emit real-time feedback. UI flow:

1. Project → **Evaluators** tab → `+ Evaluator`
2. Select **LLM-as-judge** and write the evaluation prompt (e.g., “does the response answer the user’s intent?”)
3. Specify a filter — e.g., only runs matching `has(tags, "env:prod")`
4. Setting a **Sampling rate** of 0.1 evaluates 10% of matching traces — cost control
5. On save, feedback keys start attaching to new traces automatically

To apply retroactively, turn on **Apply to past runs** and specify the time range.

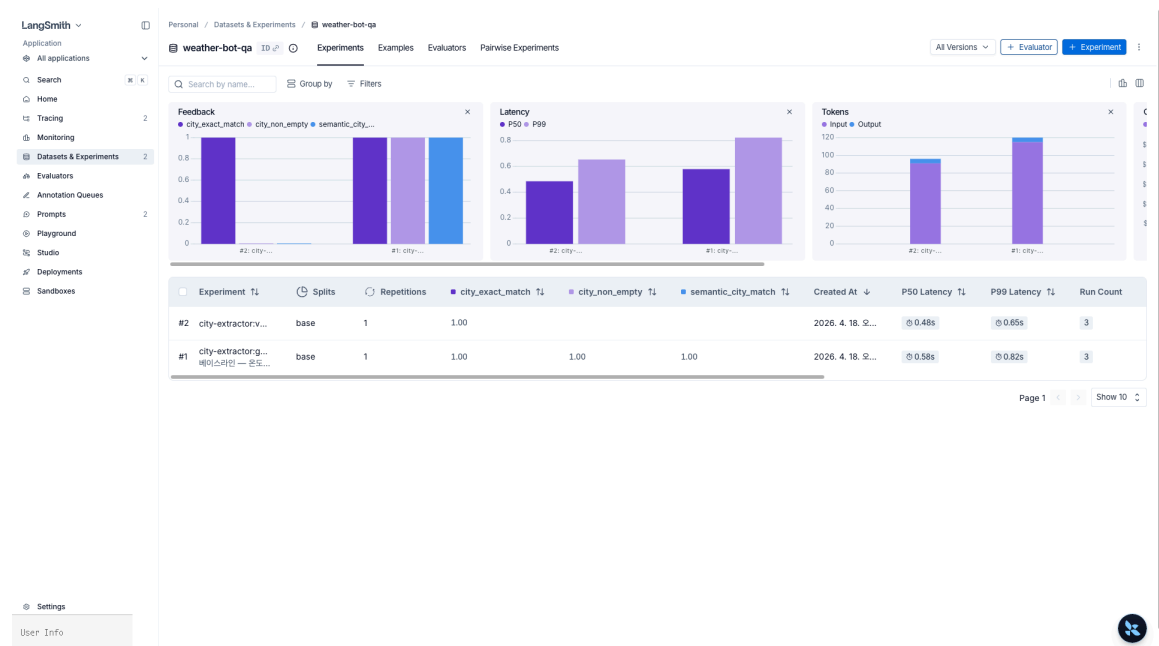


Figure 21: Experiment results + evaluator charts — feedback scores (`city_exact_match` / `city_non_empty` / `semantic_city_match`), latency P50/P99, and Input/Output token time series

Key Takeaways

- Accumulate datasets as a mix of manual seeds and production-trace ingestion
- Four evaluator types: Code (deterministic, low-cost) / LLM-as-judge (natural-language quality) / Pairwise (A/B comparison) / Summary (dataset level)
- `from langsmith import evaluate` and `Client.evaluate(...)` share the same signature

- `agentevals` provides four trajectory-match modes (strict / unordered / subset / superset) plus LLM-as-judge for grading *process appropriateness*
- Online evaluators attach feedback keys automatically — they become triggers for dashboards and alerts
- Attaching metadata such as `prompt_commit` to an experiment makes “which version produced this number” reproducible

04 Prompt Hub Versioning

Commit SHA · Tag · Playground

Prompts are effectively *code*, yet non-engineers often edit them and the cadence is tight. Avoiding redeploy on every tweak means keeping prompts in *storage decoupled from the application code* and pinning versions. LangSmith's Prompt Hub plays that role. This chapter covers the push/pull API, commit SHA vs tag, template-engine choices, Playground experiments, and a CI pin strategy.

Learning Objectives

LEARNING OBJECTIVES

- Upload a prompt with `client.push_prompt("name", object=...)`
- Understand the difference between pinning a commit SHA and referencing a tag like `prod / staging`
- Compare the variable handling of f-string vs mustache templates
- Follow the Playground → experiment → commit → tag flow
- Inject prompts at runtime via `client.pull_prompt("name:prod")`
- Pin a specific commit hash in CI tests to prevent regression

4.1 Creating and pushing a prompt — `Client.push_prompt(...)`

The simplest flow is to build a `ChatPromptTemplate` and upload it with `client.push_prompt("name", object=prompt)`. The first push creates a new prompt; subsequent pushes add commits. The returned URL opens the prompt directly in the UI.

```
from langchain_core.prompts import ChatPromptTemplate
from langsmith import Client

client = Client()
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a weather assistant. Extract only the city name."),
    ("user", "{question}"),
])

url = client.push_prompt(
    "weather-bot",
    object=prompt,
    description="System prompt that extracts the city from a question",
    tags=["weather", "extraction"],
)
```

```

    is_public=False,          # Public-hub visibility
)
print(url) # https://smith.langchain.com/hub/...

```

The main arguments to `Client.push_prompt(...)` :

- `object` — a LangChain prompt object such as `ChatPromptTemplate` / `PromptTemplate`
- `description` — one-line caption shown on the UI card
- `tags` — search / filter tags (prompt metadata, not commit tags)
- `is_public` — whether to publish to the public hub (default False)
- `parent_commit_hash` — create the new commit from a specific commit (default is latest)

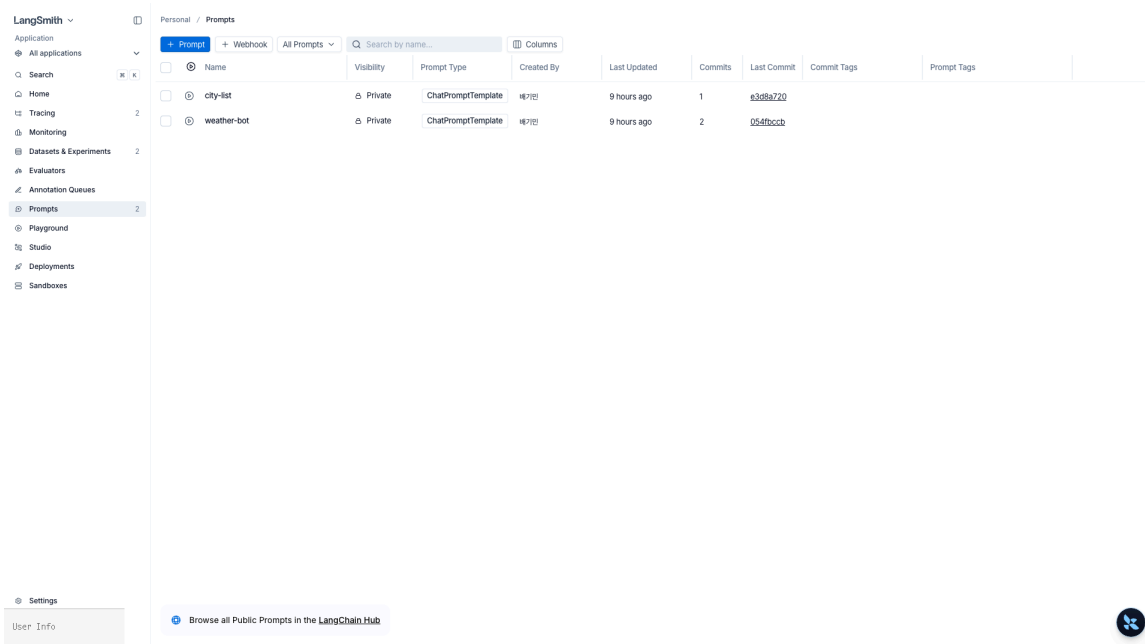


Figure 22: Prompts hub listing — `city-list` (1 commit), `weather-bot` (2 commits). Visibility and short-SHA Last Commit shown

4.2 Commit SHA pinning vs tags (`prod` , `staging`)

Prompts behave like Git.

Reference style	Example	Characteristic	When to use
Commit SHA	<code>weather-bot:12344e88</code>	Immutable, pins exactly that version	CI regression tests, any time reproducibility is required
Tag	<code>weather-bot:prod</code> , <code>:staging</code>	Movable — can be repointed at a different commit	Runtime deployment slots
Latest	<code>weather-bot</code>	The most recent commit	Early development only; never in production

Tags are the mechanism that lets you swap a prompt without redeploying application code. In the UI's Commits view, **promote** a specific commit to the `prod` tag.

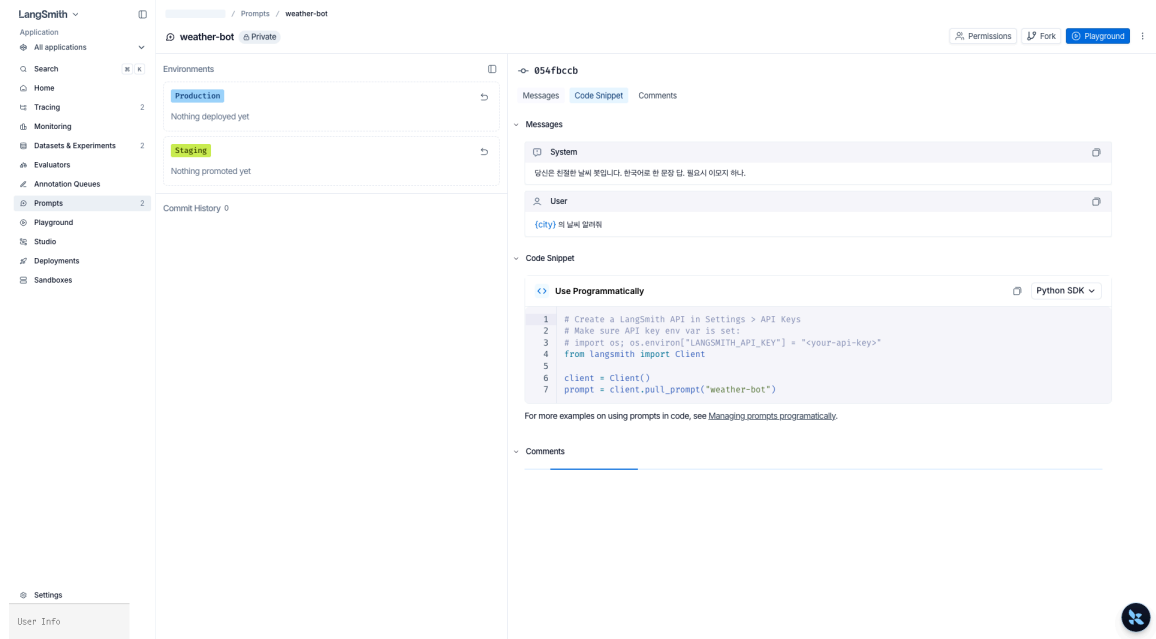


Figure 23: Prompt detail — top commit + tabs (Messages / Code Snippet / Comments) with Production / Staging slots on the Environments panel

4.3 f-string vs mustache

The characteristics of the two template engines often matter in practice.

Item	f-string (<code>{var}</code>)	mustache (<code>{{var}}</code>)
Default	LangChain default	Opt-in
Embedded JSON examples	Need <code>{{}}</code> escaping	No escaping needed
Conditionals · loops	Not supported	<code>{{#users}}...{{/users}}</code>
Nested keys	Limited	<code>{{user.name}}</code>
Playground variable declaration	Auto-detected	Manual (Inputs)

Mustache is easier when you embed a lot of JSON / code examples or need loops / conditionals; f-string is easier for plain variable substitution.

4.4 Playground — from experiment to commit

The UI's **Playground** is where prompts, models, and input variables run together. Flow:

1. Click `Open in Playground` on the prompt page
2. Adjust model · temperature · output schema · tools in the side panel

3. Enter variable values and hit `Run` — results and token/cost are recorded immediately
4. Use `Compare` to run *multiple prompt/model outputs in parallel* for the same input
5. Once satisfied, `Save as...` creates a new commit; promote to `prod` if appropriate

All runs started from Playground flow into the Experiments view and connect directly to the datasets from chapter 3.

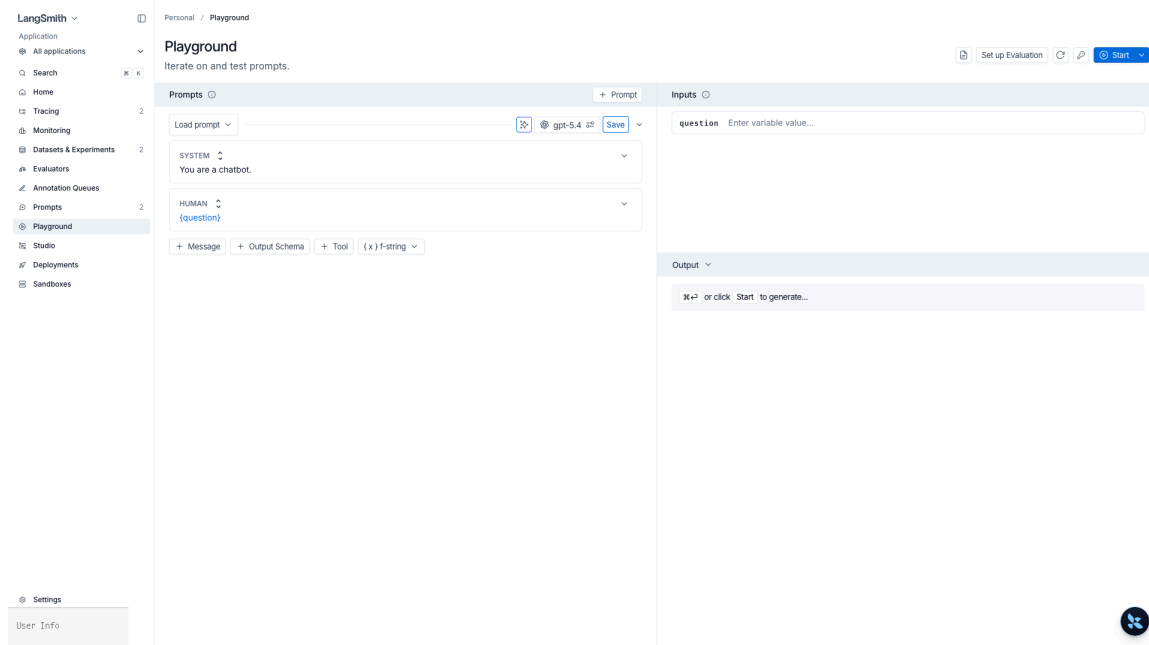


Figure 24: Playground — SYSTEM/HUMAN message editing, `{question}` variable input, and output generation with an f-string↔mustache switcher

4.5 Runtime injection — `Client.pull_prompt(...)` → `create_agent`

In the application, fetch the deployment-slot tag with `client.pull_prompt(...)` and *wire it straight into the LLM / agent*. Editing the prompt takes effect on the next request without redeployment.

```
from langchain.agents import create_agent

# 1) Reference by tag or commit SHA
prompt = client.pull_prompt("weather-bot:prod")           # tag
# prompt = client.pull_prompt("weather-bot:12344e88")      # commit SHA
# prompt = client.pull_prompt("weather-bot", include_model=True) # also include saved model config

agent = create_agent(
    model="openai:gpt-5.4",
    system_prompt=prompt.format_messages()[0].content,
    tools=[],
)
```

The main options on `Client.pull_prompt(...)` :

- `prompt_identifier` — three reference forms: `"name"`, `"name:tag"`, `"name:SHA"`
- `include_model` — when `True`, returns a `RunnableSequence` that also includes the model / temperature saved in Playground
- Caching — the Python SDK keeps an ETag cache keyed by commit, so repeated pulls of the same SHA cost nothing

4.6 Pinning a specific commit hash in CI

Take the production deployment slot via the `prod` tag, but *CI regression tests must pin a commit SHA*. That way, “someone changed the prompt after the tests passed” is auto-blocked.

```
# tests/test_prompt_regression.py
PINNED_SHA = "12344e88" # CI pin

def test_weather_prompt_still_extract_city():
    prompt = client.pull_prompt(f"weather-bot:{PINNED_SHA}")
    # ... concrete assertions
```

Deployment-pattern summary

Environment	Reference	Reason
dev / local	weather-bot (latest)	Immediate reflection
staging	weather-bot:staging	Promote a tag to test
prod	weather-bot:prod	Zero-downtime rollout by moving the tag; rollback is reverting the tag
CI	weather-bot:{SHA}	Reproducible; a prompt change immediately shows up as a test failure

When you attach `prompt_commit` as experiment metadata in chapter 3, the UI can trace *which commit produced these numbers* directly.

Key Takeaways

- Prompt Hub works as push → commit accumulation → tag promotion (similar to Git)
- Tags are runtime deployment slots; SHAs are CI reproducibility pins
- Pick f-string vs mustache based on loops / conditionals / nesting
- Playground → Save → promote is the editing path for non-developers
- Pin SHAs in CI and reference tags in production — that combination is the core of zero-downtime rollout + regression prevention

05 Production Monitoring

Dashboards · Alerts · Sampling · PII

Production is not “let’s capture a clean trace” — it is a real-time dashboard + automatic evaluation + alerting + PII defense running together. This chapter bundles the LangSmith features you need after deployment from an operational angle: Monitoring dashboards, autoeval rules, the user-feedback API, alert rules, sampling, PII scrubbing, and Slack/PagerDuty webhook integration.

Learning Objectives

LEARNING OBJECTIVES

- Read latency p50/p95, cost, and success rate on the project dashboard (Overview / Analytics)
- Register an online evaluator as an autoeval rule that runs automatically
- Send user thumbs/ratings from the app with `client.create_feedback(run_id, ...)`
- Understand metadata-based alert rules (failure rate > N% → webhook)
- Sample high-volume production traffic with `LANGSMITH_TRACING_SAMPLING_RATE`
- Defend in depth with `PIIMiddleware` and `hide_inputs` / `anonymizer`
- Configure Slack / PagerDuty webhook receivers

5.1 Project dashboard

Each project page has three default tabs in the UI.

Tab	What you see	Alert integration
Runs	Trace list, filters, quick search	—
Monitor	Latency p50-p95-p99, success rate, error distribution, token/cost time series	Threshold alerts via Rules
Evaluators	Attached online evaluators and score distributions	Alerts on specific key-score drops

To *build your own dashboard*, assemble custom charts in `Dashboards` . Or pull the same metrics with `client.list_runs` and attach them to an in-house system like Grafana.

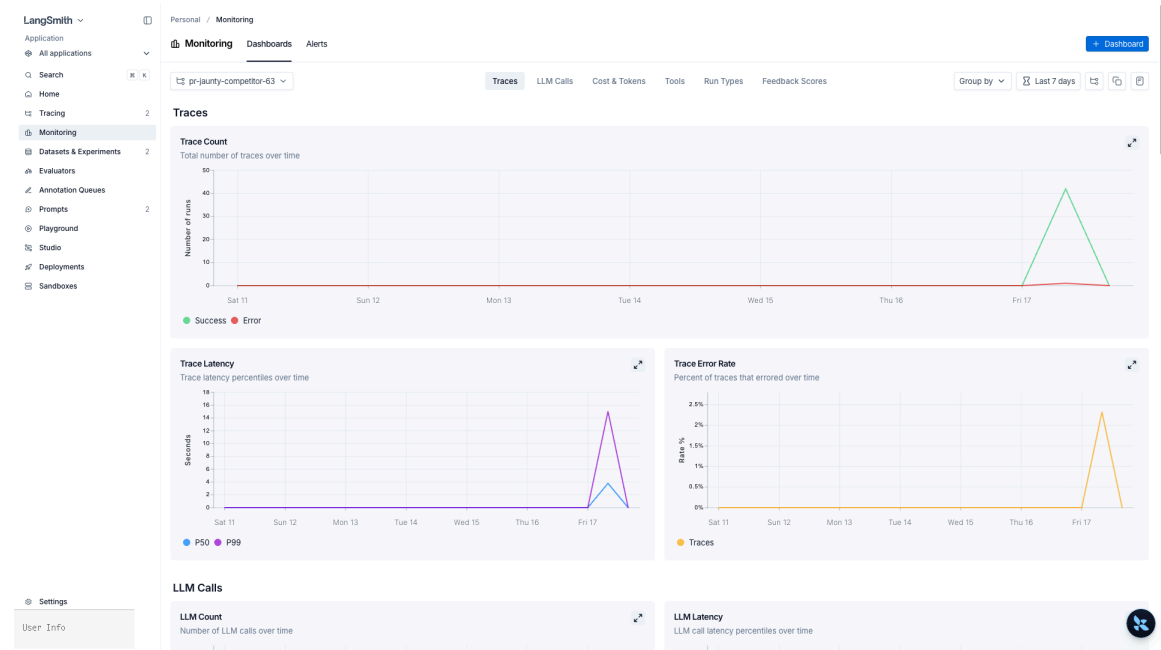


Figure 25: Project Monitoring Dashboards — six tabs (Traces / LLM Calls / Cost & Tokens / Tools / Run Types / Feedback Scores) × time range

5.2 Online evaluator — autoeval rule

Chapter 3 showed how to attach one from the UI; here we look at *operational design*.

Role	Example setup
Real-time quality gauge	LLM-as-judge (<code>useful</code> score 0–1) on runs matching <code>has(tags, "env:prod")</code>
Cost control	Sampling rate 0.05 → evaluate only 5%
Regression detection	Rule triggers a webhook when the <code>useful</code> average drops below N%
Specific use case	Attach only to runs with <code>metadata.feature == "checkout"</code>

autoeval results are stored as feedback keys, so alert rules, dashboards, and experiment comparisons all reuse them.

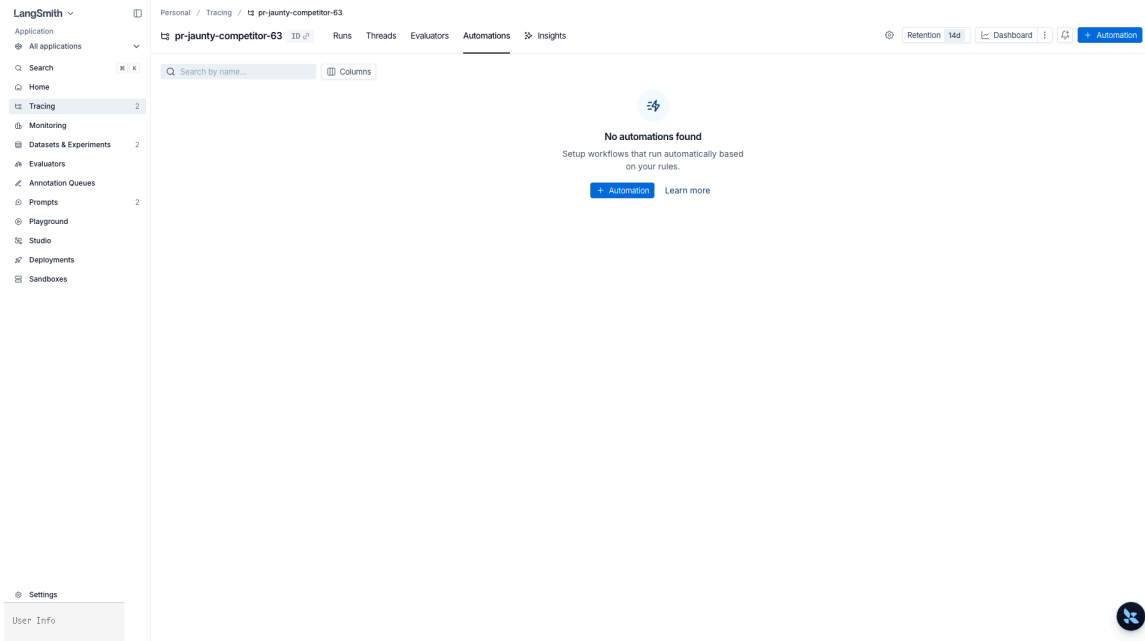


Figure 26: Project > Automations tab — `+ Automation` to define condition (Feedback score < 0.5) → action (dataset ingestion / webhook / annotation queue)

5.3 User feedback collection API

The canonical pattern is: thumbs-up / rating / “not helpful” button in the app UI → server calls

`client.create_feedback`. Fire in the *background* so the client does not wait (Python SDK handles this automatically if you pass a `trace_id`).

```
from langsmith import Client

client = Client()
client.create_feedback(
    trace_id=trace_id,
    key="user_thumbs",
    score=1.0,
    comment="Correct",
)
```

5.4 Metadata-based alert rules

Combine the following actions in the project’s **Rules** tab.

- `Add to annotation queue` (human review)
- `Add to dataset` (golden set)
- `Trigger webhook` (Slack, PagerDuty, in-house incident system)
- `Extend data retention` (extend retention for important traces)
- `Run online evaluator` (conditional quality evaluation)

Example filter expression

```
and(
  has(tags, "env:prod"),
  eq(status, "error")
)
```

Action execution order (fixed by LangSmith): annotation queue → dataset → webhook → online evaluator → code evaluator → alert. Sampling rate can also be set per rule (e.g., 0.5 = 50% of matches).

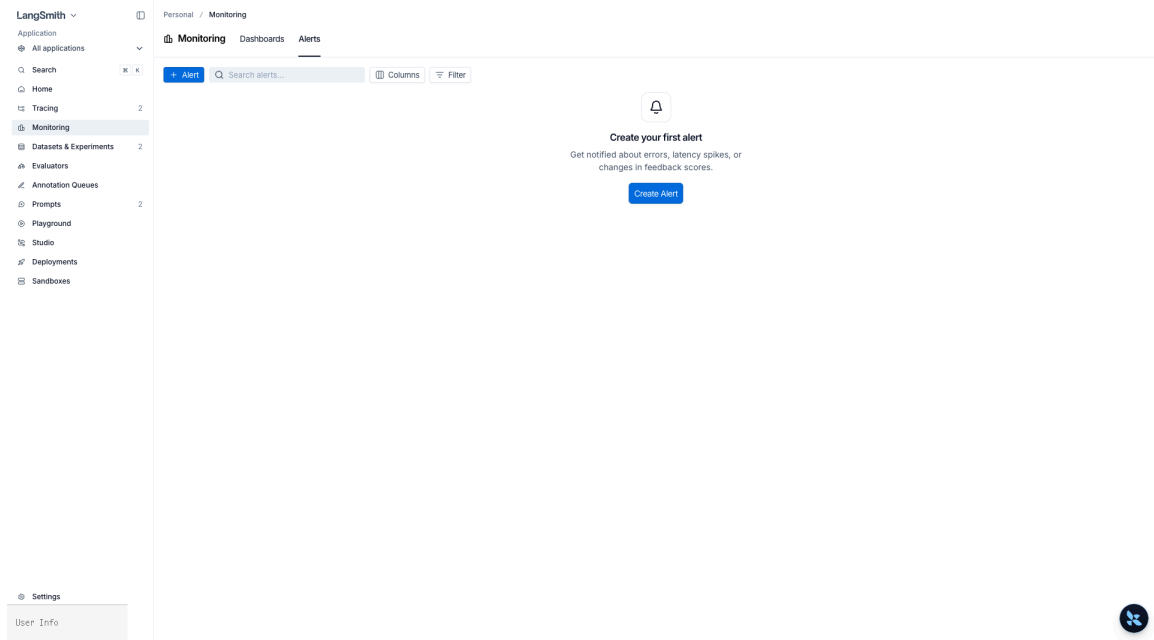


Figure 27: Monitoring > Alerts — the organization-wide alert hub. Clicking [Create Alert](#) opens a modal

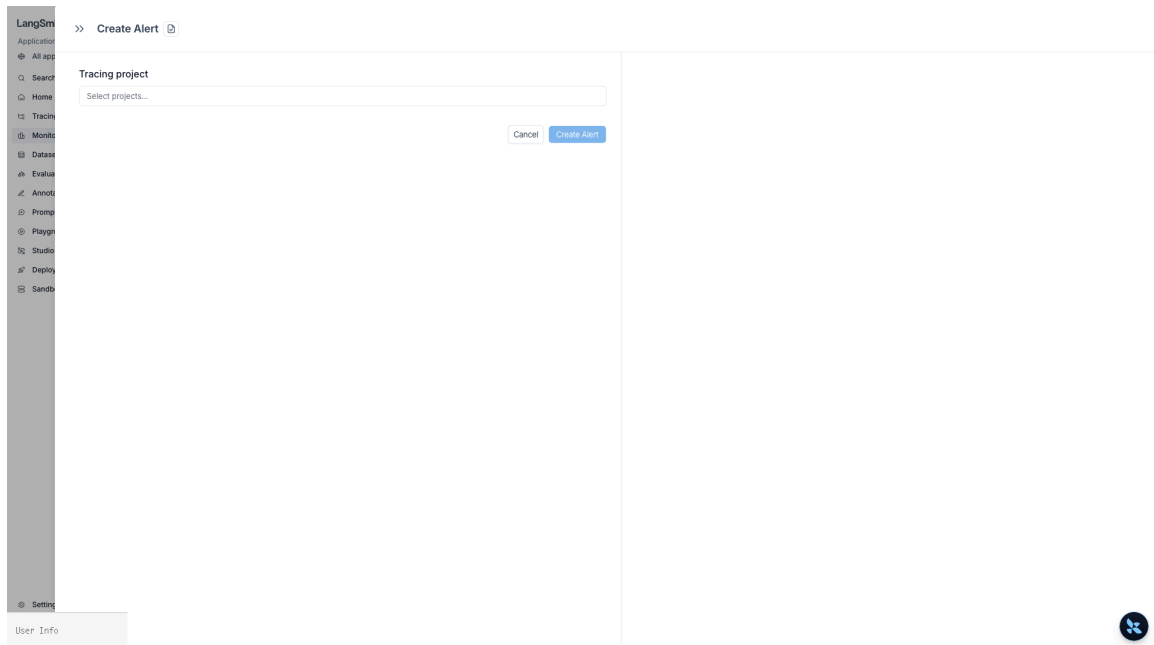


Figure 28: Create Alert modal — select a tracing project, then specify a condition (error rate / latency / feedback change) → webhook URL

5.5 High-volume sampling

At hundreds of requests per second, sending every trace wastes cost and bandwidth. Control it in two layers.

Layer	Mechanism	Characteristic
Process-wide	<code>LANGSMITH_TRACING_SAMPLING_RATE=0.1</code>	Applies to <code>@traceable</code> , <code>RunTree</code> , and all auto-instrumentation
Per-request	<code>Client(tracing_sampling_rate=...)</code> + <code>tracing_context</code>	Keep admin / payment requests at 100%

Combining them implements policies such as “10% sampling for normal traffic, 100% for payment traffic.”

5.6 PII scrubbing

Defend in two layers.

1. **Model-input stage:** `langchain.agents.middleware.PIIMiddleware` blocks or masks email / card numbers / API keys in messages before they reach the LLM — so the model itself never sees PII
2. **Trace-transmission stage:** the LangSmith client’s `hide_inputs` / `hide_outputs` or `anonymizer` scrubs inputs / outputs one more time before they ship to the server

You need both so there is no “made it into the model but not into the trace” and no “model never saw it but it ended up in the trace”.

```
# .env
LANGSMITH_HIDE_INPUTS=true
LANGSMITH_HIDE_OUTPUTS=true
```

5.7 Slack / PagerDuty webhook integration

The webhook action in Rules POSTs a payload to the URL you configure. The receiver converts it into Slack/ PagerDuty format and raises the alert.

```
from fastapi import FastAPI, Request
import httpx, os

app = FastAPI()
SLACK_URL = os.environ["SLACK_INCOMING_WEBHOOK"]
PAGERDUTY_URL = os.environ.get("PAGERDUTY_EVENTS_V2_URL")

@app.post("/langsmith/alert")
async def on_alert(req: Request):
    event = await req.json()
    run_id = event.get("run_id") or event.get("trace_id")
    status = event.get("status", "unknown")
    tags = event.get("tags", [])
    trace_url = f"https://smith.langchain.com/o/-/projects/-/r/{run_id}"

    async with httpx.AsyncClient() as hc:
        await hc.post(SLACK_URL, json={
            "text": f":rotating_light: LangSmith alert - status={status} tags={tags}\n<{trace_url}|open trace>",
```

```

    })
    if PAGERDUTY_URL and "critical" in tags:
        await hc.post(PAGERDUTY_URL, json={
            "routing_key": os.environ["PAGERDUTY_ROUTING_KEY"],
            "event_action": "trigger",
            "payload": {"summary": f"LangSmith {status}", "severity": "error", "source":
"langsmith"},
        })
    return {"ok": True}

```

Register this endpoint URL as a webhook target in the UI's Rules and set the filter to

`and(has(tags, "env:prod"), eq(status, "error"))` to Slack only production errors.

5.8 LangSmith Deployments — observability bundled with deployment

When you deploy a LangGraph graph on *LangGraph Platform*, a LangSmith project is created and linked automatically. With no extra environment variables, every graph invocation lands as a trace in that organization's LangSmith project.

Deployment screen	Linked LangSmith resource	Operational action
Deployments → Traces	Project of the same name	Confirm executions in real time
Deployments → Threads	The project's Threads view	Debug multi-turn conversations
Deployments → Monitoring	Same metrics as the project's Monitor tab	Alarms on latency · cost · error rate
Deployments → Settings	Per-environment secrets · <code>LANGSMITH_PROJECT</code> override	Separate staging vs prod projects

Typical operational pattern:

- LangGraph Platform owns deployment — `langgraph deploy` rolls out by commit with zero downtime
- LangSmith owns observability — traces from that deployment flow into the project automatically
- Alerts go through the Rules of section 5.4 — filter by `metadata.deployment_id == "..."` to scope alerts per deployment
- Evaluation attaches the chapter-3 online evaluator to prod traffic and surfaces the score trend on the dashboard

The `langsmith.Client` exposes deployment metadata through the same API key, so an external in-house dashboard (Grafana etc.) can track SLOs keyed by LangGraph deployment ID.

5.9 Insights Agent (paid)

Project > Insights tab — the LangSmith **Insights Agent** automatically extracts usage patterns / common failure modes from production traces. The free plan requires an upgrade.

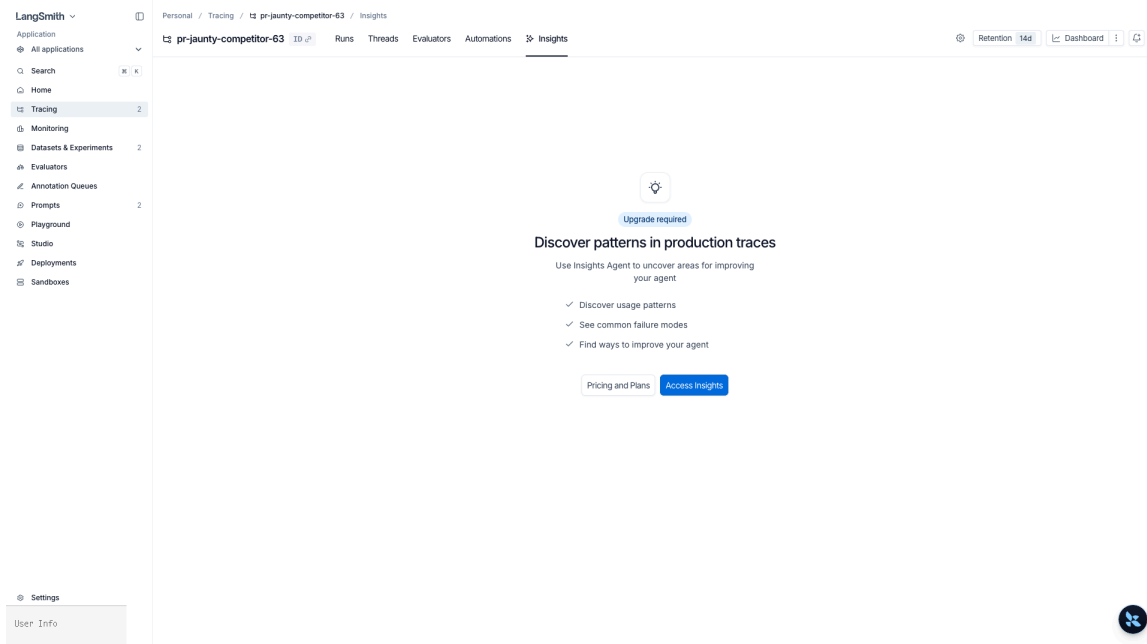


Figure 29: Insights tab — Upgrade required. Production-trace auto-analysis on paid plans

Key Takeaways

- Dashboards · Evaluators · Automations — the three tabs are the production-observability backbone
- Online evaluator + Rules webhook connect “real-time quality → alerts”
- Send user feedback in the background and expose score trends per `key` on the dashboard
- Sampling combines global (env var) + per-request (`tracing_context`) layers
- PII is protected in two layers: `PIIMiddleware` (model) + `anonymizer` / `hide_inputs` (trace)
- A webhook receiver routes to Slack / PagerDuty — the default filter is `env:prod` + `status:error`
- LangGraph Platform deployments auto-link to a same-named LangSmith project — separate SLOs per environment via deployment-ID metadata

P A R T

VII

Applied Examples

Real-World Applications

What You Will Learn in This Part

- 1. RAG Agent
- 2. SQL Agent
- 3. Data Analysis Agent
 - 4. ML Agent
- 5. Deep Research Agent

01 RAG Agent

5 Building Blocks + 3-Node Pattern

Learning Objectives

- Understand the *five building blocks* of LangChain RAG (document loaders, text splitters, embedding models, vector stores, retrievers)
- Compare the three RAG architectures: *2-Step*, *Agentic*, and *Hybrid*
- Separate query rewriting from retrieval with the Rewrite → Retrieve → Agent three-node pattern
- Define a retrieval tool with the content_and_artifact return pattern
- Build a RAG agent with create_deep_agent and apply v1 middleware (ModelCallLimitMiddleware , ToolRetryMiddleware)
- Use the Skills system to progressively disclose RAG domain knowledge

Overview

Item	Details
Framework	LangChain v1 + Deep Agents
5 building blocks	Document loaders · Text splitters · Embedding models · Vector stores · Retrievers
Architecture	2-Step (static RAG) · Agentic (tool calls) · Hybrid (Rewrite → Retrieve → Agent)
Agent pattern	content_and_artifact tool → create_deep_agent
Backend	FilesystemBackend(root_dir="", virtual_mode=True)
Skill	skills/rag-agent/SKILL.md – progressive disclosure of RAG domain knowledge

TIP

See docs/langchain/24-retrieval.md for a side-by-side comparison of the 2-Step, Agentic, and Hybrid RAG architectures.

```
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "Set OPENAI_API_KEY in .env"
```



```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")
```

The 5 RAG Building Blocks

LangChain RAG consists of five composable building blocks. Each one is replaceable, and the way you wire them together defines the architecture.

Block	Role
<i>Document loaders</i>	Load PDFs, web pages, or databases into <code>Document</code> objects
<i>Text splitters</i>	Split long documents into retrieval-friendly chunks
<i>Embedding models</i>	Convert text into vectors (e.g. <code>OpenAIEmbeddings</code>)
<i>Vector stores</i>	Persist embeddings and run similarity search (FAISS, Chroma, InMemory)
<i>Retrievers</i>	Return the top-N relevant documents for a query — usually wrapped as an agent tool

2-Step / Agentic / Hybrid Architectures

The same building blocks produce three different architectures depending on how they are composed.

Architecture	Flow	Strengths	Trade-offs
<i>2-Step RAG</i>	Query → retrieve → LLM response (static)	Simple, fast, predictable	No re-querying or multi-retrieval
<i>Agentic RAG</i>	Agent calls <code>retrieve</code> on demand	Handles multi-step and comparison queries	Higher token usage
<i>Hybrid</i>	Three nodes: Rewrite → Retrieve → Agent	Better recall through query rewriting	More graph wiring up front

The Rewrite → Retrieve → Agent 3-Node Pattern

The Hybrid layout pulls *query rewriting* and *retrieval* out into separate nodes and lets the agent synthesize the final answer. This separation keeps retrieval accuracy stable even when the user's question is vague or carries multiple intents.

```
# Conceptual graph — implement each node with LangGraph
# rewrite_node: original query → rewritten retrieval query
```

```
# retrieve_node: vectorstore.similarity_search(query, k=K)
# agent_node: create_deep_agent + tools=[retrieve_tool]
```

Step 1: Create Sample Documents

The first step in a RAG pipeline is preparing the documents you want to search. In a real system, you would load documents from PDFs, web pages, or databases. Here, for learning purposes, we create `Document` objects directly.

```
from langchain_core.documents import Document

docs = [
    Document(page_content="LangChain is a framework for building LLM applications. It supports tools, chains, and agents.", metadata={"source": "langchain"}),
    Document(page_content="LangGraph is a framework for building stateful workflows. It provides a Graph API and a Functional API.", metadata={"source": "langgraph"}),
    Document(page_content="Deep Agents is an all-in-one agent SDK. It creates agents with create_deep_agent and supports backends and subagents.", metadata={"source": "deepagents"}),
    Document(page_content="RAG stands for retrieval-augmented generation. It injects external knowledge into an LLM to produce more accurate answers.", metadata={"source": "rag"}),
    Document(page_content="A vector store is a database that stores embeddings and performs similarity search. FAISS and Chroma are common examples.", metadata={"source": "vectorstore"}),
    Document(page_content="An agent is a system in which an LLM uses tools to perform tasks autonomously. ReAct is a representative pattern.", metadata={"source": "agent"}),
]
print(f"Created {len(docs)} documents")
```

Step 2: Split the Text

Split larger documents into chunks that are easier to retrieve. `RecursiveCharacterTextSplitter` tries to split at natural boundaries such as paragraphs, sentences, and then words.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=200, chunk_overlap=50
)
splits = splitter.split_documents(docs)
print(f"Split result: {len(splits)} chunks")
```

Step 3: Build the Vector Store

Convert the text into vectors with an OpenAI embedding model and store them in `InMemoryVectorStore`. In production, you would typically use a persistent store such as FAISS or Chroma.

```
from langchain_openai import OpenAIEmbeddings
from langchain_core.vectorstores import InMemoryVectorStore

embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vectorstore = InMemoryVectorStore.from_documents(splits, embeddings)
print(f"Vector store ready — embedded {len(splits)} documents")
```

Step 4: Define a Retrieval Tool (`content_and_artifact`)

The `response_format="content_and_artifact"` pattern makes the tool return two things:

- *content*: a text summary shown to the agent
- *artifact*: the full `Document` objects for downstream processing

This pattern helps save context tokens while still preserving access to the original data.

```
from langchain.tools import tool

@tool(response_format="content_and_artifact")
def retrieve(query: str):
    """Search for relevant documents in the vector store."""
    results = vectorstore.similarity_search(query, k=3)
    content = "\n\n".join(d.page_content for d in results)
    return content, results
```

Step 5: Test the Retrieval Tool by Itself

Before connecting the tool to the agent, verify that it behaves correctly.

```
result = retrieve.invoke({"query": "What is an agent?"})
print(result)
```

Step 6: Create the RAG Agent (with v1 Middleware)

Load the prompt from the prompt module. The flow tries LangSmith Hub → Langfuse → a local default prompt.

Middleware	Role
<code>ModelCallLimitMiddleware</code>	Prevents infinite loops by limiting the number of model calls
<code>ToolRetryMiddleware</code>	Automatically retries failed retrieval tool calls

```
from deepagents import create_deep_agent
from deepagents.backends import FilesystemBackend
from langchain.agents.middleware import (
    ModelCallLimitMiddleware,
    ToolRetryMiddleware,
)
from prompts import RAG_AGENT_PROMPT

agent = create_deep_agent(
    model=model,
    tools=[retrieve],
    system_prompt=RAG_AGENT_PROMPT,
    backend=FilesystemBackend(root_dir=".", virtual_mode=True),
    skills=["/skills/"],
    middleware=[
        ModelCallLimitMiddleware(run_limit=10),
        ToolRetryMiddleware(max_retries=2),
    ]
)
```

```
    1,  
)
```

Step 7: Run a Simple Query and a Comparison Query

Use a simple query (single retrieval) and a comparison-style query (multiple retrieval steps) to verify that the RAG agent works as expected.

Summary

Item	Key Point
Vector store	<code>InMemoryVectorStore.from_documents()</code> — embedding-based similarity search
Retrieval tool	<code>\@tool(response_format="content_and_artifact")</code> — summary + original artifact separation
Agent	<code>create_deep_agent(model, tools=[retrieve], backend=..., skills=["*/skills/"])</code>
Skill	<code>skills/rag-agent/SKILL.md</code> — saves tokens through progressive disclosure

References:

- `docs/langchain/24-retrieval.md`
- [LangChain RAG Tutorial](#)
- `docs/deepagents/10-skills.md`

Next Step: → [02_sql_agent.ipynb](#): Build a SQL agent.

02 SQL Agent

Natural-Language Database Queries

Learning Objectives

- Generate SQL tools automatically with `SQLDatabaseToolkit`
- Apply AGENTS.md-based safety rules (READ-ONLY)
- Implement approval before query execution with a HITL interrupt
- Explicitly apply v1 middleware (`HumanInTheLoopMiddleware` , `ModelCallLimitMiddleware`)

Overview

Item	Details
Framework	LangChain + Deep Agents
Core components	<code>SQLDatabaseToolkit</code> , <code>SQLDatabase</code> , <code>InMemorySaver</code>
Agent pattern	AGENTS.md safety rules + skills-based workflow
HITL	<code>interrupt_on={"sql_db_query": {"allowed_decisions": [...]}}</code> + <code>Command(resume={"decisions": [...]})</code>
Database	Chinook (SQLite)
Skill	<code>skills/sql-agent/SKILL.md</code> – SQL safety rules + query workflow

```
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "Set OPENAI_API_KEY in .env"
```

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")
```

Step 1: Connect to the Database

Chinook is a sample database for a digital music store. It contains tables such as Artist, Album, Track, and Invoice.

```
from langchain_community.utilities import SQLDatabase

db = SQLDatabase.from_uri("sqlite:///../05_advanced/Chinook.db")
print(f"Tables: {db.get_usable_table_names()}")
```

Step 2: Create the `SQLDatabaseToolkit` Tools

`SQLDatabaseToolkit` automatically generates four tools from the database connection:

Tool	Description
<code>sql_db_list_tables</code>	List available tables
<code>sql_db_schema</code>	Inspect table schema (DDL)
<code>sql_db_query</code>	Execute a SQL query
<code>sql_db_query_checker</code>	Validate query syntax before execution

```
from langchain_community.agent_toolkits import SQLDatabaseToolkit

toolkit = SQLDatabaseToolkit(db=db, llm=model)
sql_tools = toolkit.get_tools()
for t in sql_tools:
    print(f"{t.name}: {t.description[:60]}")
```

Step 3: Load the Prompt (LangSmith / Langfuse / Default)

The prompt loader follows this order:

1. *LangSmith Hub* — if configured, pull the prompt from the Hub
2. *Langfuse* — if configured, load it from Langfuse
3. *Default* — otherwise, use the fallback prompt defined in code

The SQL agent prompt includes READ-ONLY safety rules and the expected query workflow.

```
from prompts import SQL_AGENT_PROMPT

print(SQL_AGENT_PROMPT)
```

Step 4: The Idea Behind Skills

Skills act as workflow guides for the agent. They document recurring task patterns so the agent works in a more consistent way.

Skill	Purpose
query-writing	Inspect tables → inspect schema → write SQL → execute

schema-exploration

List tables → inspect DDL → map relationships

Step 5: Create a Basic SQL Agent

Pass the SQL tools and AGENTS-style safety instructions into `create_deep_agent`. The `system_prompt` injects the SQL safety rules.

```
from deepagents import create_deep_agent
from deepagents.backends import FilesystemBackend

agent = create_deep_agent(
    model=model,
    tools=sql_tools,
    system_prompt=SQL_AGENT_PROMPT,
    backend=FilesystemBackend(root_dir=".", virtual_mode=True),
    skills=["/skills/"],
)
```

Step 6: A HITL Agent (`interrupt_on`)

Use the `interrupt_on` parameter of `create_deep_agent` to define approval policies for specific tools. Execution stops before `sql_db_query` runs, and then resumes with `Command(resume={"decisions": [...]})`. In v1 you can spell out the allowed decision types per tool with `allowed_decisions`.

Parameter	Role
<code>interrupt_on={"sql_db_query": {"allowed_decisions": [...]}}</code>	Pause before <code>sql_db_query</code> runs and declare which decision types are valid
<code>ModelCallLimitMiddleware</code>	Prevent infinite loops by limiting the run to 15 model calls
<code>InMemorySaver</code>	Enables checkpointing so interrupts and resumes work

```
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents.middleware import ModelCallLimitMiddleware

hitl_agent = create_deep_agent(
    model=model,
    tools=sql_tools,
    system_prompt=SQL_AGENT_PROMPT,
    backend=FilesystemBackend(root_dir=".", virtual_mode=True),
    skills=["/skills/"],
    checkpointer=InMemorySaver(),
    interrupt_on={
        "sql_db_query": {
            "allowed_decisions": ["approve", "edit", "reject", "respond"],
        },
    },
    middleware=[
        ModelCallLimitMiddleware(run_limit=15),
    ],
)
```

Step 7: Resume After Approval – Four Decision Types

Use `Command(resume={"decisions": [...]})` to continue a paused run. The v1 `HITLResponse` supports four decision types.

Decision Type	Description
<code>{"type": "approve"}</code>	Approve the tool call and run it unchanged
<code>{"type": "edit", "edited_action": {"name": "...", "args": {...}}}</code>	Modify the tool call before running it (e.g. add a <code>LIMIT</code> , tighten a <code>WHERE</code> clause)
<code>{"type": "reject", "message": "..."} </code>	Reject the tool call and return feedback to the agent
<code>{"type": "respond", "message": "..."} </code>	Skip tool execution and feed the human’s reply back as the tool result — the “ask user” pattern

```
from langgraph.types import Command

response = hitl_agent.invoke(
    Command(resume={"decisions": [{"type": "approve"}]}),
    config={**thread, **lf_config},
    version="v2",
)
```

Summary

Item	Key Point
<i>Tool generation</i>	<code>SQLDatabaseToolkit(db, llm).get_tools()</code> → automatically creates 4 SQL tools
<i>Safety rules</i>	Applies a READ-ONLY policy through the SQL agent instructions
<i>Skills</i>	Workflow guides for query writing and schema exploration
<i>HITL</i>	<code>interrupt_on={"sql_db_query": {"allowed_decisions": [...]}}</code> → 4-decision (<code>approve</code> / <code>edit</code> / <code>reject</code> / <code>respond</code>)

References:

- `docs/deepagents/examples/03-text-to-sql-agent.md`
- [LangChain SQL Agent Tutorial](#)
- `docs/deepagents/06-backends.md`

Next Step: → `03_data_analysis_agent.ipynb`: Build a data analysis agent.

03 Data Analysis Agent

Code Execution and Multi-Turn Analysis

Learning Objectives

- Set up a code execution environment with `LocalShellBackend`
- Combine custom tools with built-in tools such as `write_todos` and `execute`
- Perform iterative analysis with streaming and multi-turn conversations
- Apply v1 middleware (`SummarizationMiddleware` , `ModelCallLimitMiddleware`)

Overview

Item	Details
Framework	Deep Agents
Core components	<code>LocalShellBackend</code> , <code>InMemorySaver</code>
Built-in tools	<code>execute</code> (code execution), <code>write_todos</code> (planning)
Pattern	Streaming (<code>stream(subgraphs=True)</code>) + multi-turn conversation
Skill	<code>skills/data-analysis/SKILL.md</code> — analysis checklist + code execution rules

```
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "Set OPENAI_API_KEY in .env"
```

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")
```

Step 1: Compare Backends

Deep Agents supports multiple backends. For data analysis, code execution matters, so we use `LocalShellBackend` .

Backend	File Access	Code Execution	Best Use
---------	-------------	----------------	----------

StateBackend	Stored in state	✗	Scratchpad
FilesystemBackend	Local disk	✗	Read/write files
LocalShellBackend	Local disk	✓ execute	Data analysis, code execution
Sandboxes (Modal, etc.)	Isolated environment	✓	Production

 TIP

 LocalShellBackend runs commands on the host system. Always use `virtual_mode=True` .

```
from deepagents.backends import LocalShellBackend

backend = LocalShellBackend(root_dir=".", virtual_mode=True)
```

Step 2: Create a CSV File for Analysis

If the agent is going to run pandas code with `execute` , there needs to be a CSV file on disk. The agent could also write files itself with the built-in `write_file` tool, but here we prepare the file ahead of time.

```
import tempfile, os

# Create a CSV file in a temporary directory
tmp_dir = tempfile.mkdtemp()
csv_path = os.path.join(tmp_dir, "sales.csv")

CSV_DATA = """date,product,region,sales,quantity
2024-01-15,Widget A,Seoul,150000,30
2024-01-15,Widget B,Busan,89000,18
2024-02-10,Widget A,Seoul,175000,35
2024-02-10,Widget C,Daegu,62000,12
2024-03-05,Widget B,Seoul,134000,27
2024-03-05,Widget A,Busan,98000,20
2024-03-20,Widget C,Seoul,71000,14
2024-04-01,Widget A,Daegu,112000,22"""

with open(csv_path, "w", encoding="utf-8") as f:
    f.write(CSV_DATA.strip())
print(f"Saved CSV: {csv_path}")
```

Step 3: Define the Analysis Tools

We define two custom tools:

Tool	Role
<code>get_csv_path</code>	Returns the CSV file path

`run_pandas`

Executes pandas code directly and returns the result

· NOTE

`run_pandas` executes pandas code written by the agent with `exec()`. In many notebook environments, this is more stable than relying only on the built-in `execute` tool.

```
from langchain.tools import tool
import io, contextlib

@tool
def get_csv_path() -> str:
    """Return the path of the CSV file to analyze."""
    return csv_path

@tool
def run_pandas(code: str) -> str:
    """Execute pandas Python code. Use print() to display results."""
    import pandas as pd, numpy as np
    buf = io.StringIO()
    ns = {"pd": pd, "np": np, "csv_path": csv_path}
    try:
        with contextlib.redirect_stdout(buf):
            exec(code, ns)
        return buf.getvalue() or "Execution finished (no printed output)"
    except Exception as e:
        return f"Error: {e}"
```

Step 4: Create the Agent (with v1 Middleware)

Combine `LocalShellBackend` with the custom tools (`get_csv_path` , `run_pandas`).

Middleware	Role
<code>SummarizationMiddleware</code>	Automatically summarizes older messages when the conversation becomes long
<code>ModelCallLimitMiddleware</code>	Prevents infinite loops by limiting the run to 20 model calls

```
from deepagents import create_deep_agent
from deepagents.backends import LocalShellBackend
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents.middleware import (
    SummarizationMiddleware,
    ModelCallLimitMiddleware,
)
from prompts import DATA_ANALYSIS_PROMPT

agent = create_deep_agent(
    model=model,
    tools=[get_csv_path, run_pandas],
    system_prompt=DATA_ANALYSIS_PROMPT,
    backend=LocalShellBackend(root_dir=tmp_dir, virtual_mode=True),
    skills=["/skills/"],
    checkpointer=InMemorySaver(),
    middleware=[
        SummarizationMiddleware(model=model, trigger=("messages", 10)),
        ModelCallLimitMiddleware(run_limit=20),
    ],
)
```

```
    1,  
)
```

Step 5: Analyze with pandas Code Execution

When you ask the agent to analyze the data, it can first use `get_csv_path` to confirm the file path and then use `run_pandas` to write and execute pandas code.

Agent execution flow:

- 1. `get_csv_path()` → confirm the CSV file path
- 2. `run_pandas("import pandas as pd; ...")` → execute pandas code
- 3. interpret the result **and** produce an answer

Step 6: Ask a Follow-Up Question in the Same Thread

If you reuse the same `thread_id`, the agent keeps the conversation context and can continue the analysis from the earlier steps.

Built-In Tools Summary

Built-in Tool	Backend	Description
<code>read_file</code>	All backends	Read a file (including images)
<code>write_file</code>	All backends	Write a file
<code>edit_file</code>	All backends	Edit a file (find and replace)
<code>ls</code>	All backends	List directory contents
<code>glob</code>	All backends	Search for files by pattern
<code>grep</code>	All backends	Search file contents
<code>execute</code>	LocalShell, Sandbox	Run shell commands
<code>write_todos</code>	All backends	Create or update a task plan

Data Analysis Execution Flow

- 1. [Planning] `write_todos` – write the analysis plan
- 2. [Reading] `read_file` – inspect the CSV structure
- 3. [Execution] `execute` – run pandas code
- 4. [Iteration] `continue` analysis through follow-up questions
- 5. [Delivery] `summarize findings and report results`

@tool + pandas vs. CodeInterpreterMiddleware

There are two main ways a data analysis agent can run code. The `run_pandas` tool in this chapter is the `@tool + pandas` pattern; it can be swapped for the built-in `execute` of `LocalShellBackend` or for `CodeInterpreterMiddleware`.

Pattern	Isolation	State	Use Case
<code>@tool + inline Python</code>	None (same process)	New namespace per call	Notebooks, demos, small analyses
<code>LocalShellBackend</code> <code>execute</code>	None (host shell)	Filesystem only	Letting the agent shell out
<code>CodeInterpreterMiddleware</code> (QuickJS)	Inside the interpreter	Auto-restored between turns via snapshots	Safer code execution on Deep Agents 0.6+

TIP

Put your analysis checklist and code-execution rules in `skills/data-analysis/SKILL.md`. The agent loads them progressively when needed instead of cramming everything into the system prompt — that's the Skills pattern.

Summary

Item	Key Point
<i>Backend</i>	<code>LocalShellBackend(virtual_mode=True)</code> — supports code execution
<i>Built-in tools</i>	<code>execute</code> (code execution) + <code>write_todos</code> (planning)
<i>Streaming</i>	<code>stream(subgraphs=True)</code> — observe the process in real time
<i>Multi-turn</i>	<code>InMemorySaver</code> + same <code>thread_id</code> — preserve conversation context

References:

- `docs/deepagents/tutorials/data-analysis.md`
- `docs/deepagents/06-backends.md`

Next Step: → [04_ml_agent.ipynb](#): Build a machine learning agent.

04

Machine Learning Agent

A Flexible CSV-Based ML Workflow

Learning Objectives

- Set a data directory with `FilesystemBackend` and let the agent explore files freely
- Extend the `run_pandas` pattern from NB03 to create a `run_ml_code` tool with scikit-learn
- Let the agent explore data with built-in tools (`ls` , `read_file` , `glob`) and analyze it with `run_ml_code`
- Run EDA → preprocessing → model selection → training → evaluation through multi-turn conversation

Overview

Item	NB03 (Data Analysis)	NB04 (Machine Learning)
Backend	LocalShellBackend	FilesystemBackend
Data	Sales CSV (8 rows)	User-provided CSV (demo: breast cancer, 569 rows)
Custom tools	<code>get_csv_path</code> + <code>run_pandas</code>	<code>run_ml_code</code> (adds sklearn)
Built-in tools	—	<code>ls</code> , <code>read_file</code> , <code>glob</code> (file exploration)
Goal	Aggregation, statistics, trend analysis	EDA → preprocessing → model training → comparison

```
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "Set OPENAI_API_KEY in .env"
```

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")
```

NB03 vs. NB04: Extending the Backend and Tooling

In NB03, you used `LocalShellBackend` + `run_pandas` to execute pandas code. In NB04, you extend that setup in two ways:

1. **Backend:** `FilesystemBackend(root_dir=DATA_DIR)` — the agent can freely explore the data directory with built-in tools such as `ls`, `read_file`, and `glob`
2. **Tool:** `run_ml_code` — adds `sklearn` to the execution namespace so the agent can run ML pipelines

```
# NB03: LocalShellBackend + run_pandas
backend = LocalShellBackend(root_dir=tmp_dir, virtual_mode=True)
ns = {"pd": pd, "np": np, "csv_path": csv_path}

# NB04: FilesystemBackend + run_ml_code
backend = FilesystemBackend(root_dir=DATA_DIR, virtual_mode=True)
ns = {"pd": pd, "np": np, "sklearn": sklearn, "DATA_DIR": DATA_DIR}
```

TIP

Because `FilesystemBackend` does not expose `execute`, it is safer than `LocalShellBackend`.

Step 1: Set the Data Directory

If you change `DATA_DIR`, you can point the agent to *your own CSV data*. The agent will inspect the directory with built-in tools such as `ls`, `glob`, and `read_file` to understand which files exist.

```
# Example: point to your own data directory
DATA_DIR = "/path/to/your/data"
```

```
import tempfile
import pandas as pd
from sklearn.datasets import load_breast_cancer

# — Data directory setup —————
# Change this to a directory that contains your own CSV files.
# For this demo, we save the breast_cancer dataset as a CSV.
DATA_DIR = tempfile.mkdtemp()

# Demo data creation (remove this block if you already have your own CSV)
data = load_breast_cancer()
df = pd.DataFrame(data.data, columns=data.feature_names)
df["target"] = data.target
df.to_csv(os.path.join(DATA_DIR, "breast_cancer.csv"), index=False)

print(f"DATA_DIR: {DATA_DIR}")
print(f"Files: {os.listdir(DATA_DIR)}")
```

Step 2: Create the `FilesystemBackend`

`FilesystemBackend` provides built-in file tools below the `root_dir`:

Built-in Tool	Role
<code>ls</code>	List directory contents
<code>read_file</code>	Read file contents
<code>glob</code>	Find files by pattern
<code>write_file</code>	Write files (for saving results)

TIP

`virtual_mode=True` prevents path escape patterns such as `..` or `~`.

```
from deepagents.backends import FilesystemBackend

backend = FilesystemBackend(root_dir=DATA_DIR, virtual_mode=True)
```

Step 3: Define `run_ml_code`

Extend the `run_pandas` pattern from NB03 by adding `sklearn` to the execution namespace. Pass `DATA_DIR` into the namespace so the agent can load any CSV file inside the directory.

TIP

Use built-in `ls` / `read_file` for file exploration and `run_ml_code` for code execution — keep the responsibilities separate.

```
from langchain.tools import tool
import io, contextlib

@tool
def run_ml_code(code: str) -> str:
    """Execute sklearn/pandas Python code. Use print() to display results.
    Available modules: pd, np, sklearn, os. Access the data directory via DATA_DIR."""
    import pandas as pd, numpy as np, sklearn
    buf = io.StringIO()
    ns = {"pd": pd, "np": np, "sklearn": sklearn, "os": os, "DATA_DIR": DATA_DIR}
    try:
        with contextlib.redirect_stdout(buf):
            exec(code, ns)
        return buf.getvalue() or "Execution finished (no printed output)"
    except Exception as e:
        return f"Error: {e}"
```

Step 4: Create the Agent

The agent workflow is:

1. Explore CSV files inside `DATA_DIR` with built-in `ls` / `glob`
2. Preview data with built-in `read_file`
3. Run EDA → preprocessing → model training/comparison with `run_ml_code`

Middleware	Role
<code>ToolRetryMiddleware</code>	Automatically retries failed tool calls (up to 2 times)
<code>ModelCallLimitMiddleware</code>	Prevents infinite loops by limiting the run to 20 model calls

```

from deepagents import create_deep_agent
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents.middleware import (
    ToolRetryMiddleware,
    ModelCallLimitMiddleware,
)
from prompts import ML_AGENT_PROMPT

ml_agent = create_deep_agent(
    model=model,
    tools=[run_ml_code],
    system_prompt=ML_AGENT_PROMPT,
    backend=backend,
    skills=["/skills/"],
    checkpointer=InMemorySaver(),
    middleware=[
        ToolRetryMiddleware(max_retries=2),
        ModelCallLimitMiddleware(run_limit=20),
    ],
)

```

Step 5: File Exploration + EDA

Ask the agent to explore the data directory and analyze the dataset. The agent can inspect the file list with built-in `ls` and then perform EDA with `run_ml_code`.

Step 6: Train and Compare Models

Ask the agent to train at least three suitable models and compare them with cross-validation. The agent chooses the algorithms by itself.

Step 7: Multi-Turn Follow-Up — Feature Importance

Reuse the same `thread_id` so the follow-up analysis keeps the earlier conversation context.

Step 8: Streaming — Additional Analysis

Observe the execution process in real time with `stream(subgraphs=True)`.

@tool + scikit-learn Pattern

`run_ml_code` exposes pandas, numpy, and sklearn through a single `@tool`. The agent can run a continuous sklearn pipeline — from EDA through training and evaluation — in a single tool call.

```
@tool
def run_ml_code(code: str) -> str:
    """Run pandas + numpy + sklearn code."""
    import pandas as pd, numpy as np, sklearn
    ns = {"pd": pd, "np": np, "sklearn": sklearn, "os": os, "DATA_DIR": DATA_DIR}
    # ... capture print output and return the result
```

➡ TIP

- Built-in `ls` / `read_file` → *file exploration*
- `run_ml_code` → *code execution*
- Separating these responsibilities makes it obvious to the agent which tool to call at each step.

Summary

Item	Key Point
<i>Backend</i>	<code>FilesystemBackend(root_dir=DATA_DIR)</code> — point the agent at your data directory
<i>Built-in tools</i>	<code>ls</code> , <code>read_file</code> , <code>glob</code> — explore files
<i>Custom tool</i>	<code>run_ml_code</code> (pandas + numpy + sklearn) — execute ML code
<i>Workflow</i>	File exploration → EDA → preprocessing → model selection → cross-validation comparison
<i>Multi-turn</i>	<code>InMemorySaver</code> + <code>same thread_id</code> — preserve analysis context

Using Your Own Data

```
# Just change DATA_DIR in Step 1
DATA_DIR = "/path/to/your/data" # directory containing CSV files
```

The agent will explore files with `ls` and analyze them freely with `run_ml_code`.

References:

- `docs/deepagents/06-backends.md`
- `docs/deepagents/tutorials/data-analysis.md`
- [scikit-learn documentation](#)

Next Step: → [05_deep_research_agent.ipynb](#): Build a deep research agent.

05 Deep Research Agent

Parallel Subagents and a Five-Step Workflow

Learning Objectives

- Configure three parallel subagents (`researcher-1` , `researcher-2` , `fact-checker`)
- Implement strategic reflection with `think_tool`
- Design a five-step workflow (Plan → Delegate → Synthesize → Verify → Report)
- Apply v1 middleware (`SummarizationMiddleware` , `ModelCallLimitMiddleware` , `ModelFallbackMiddleware`)

Overview

Item	Details
<i>Framework</i>	Deep Agents
<i>Core components</i>	Three parallel subagents, <code>think_tool</code>
<i>Workflow</i>	5 steps: Plan → Delegate → Synthesize → Verify → Report
<i>Backend</i>	<code>FilesystemBackend(root_dir="", virtual_mode=True)</code>
<i>Built-in tools</i>	<code>write_todos</code> (planning), <code>task</code> (subagent call)
<i>Skill</i>	<code>skills/deep-research/SKILL.md</code> — research method + citation rules

```
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "Set OPENAI_API_KEY in .env"
```

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")
```

Step 1: `think_tool` — A Strategic Reflection Tool

`think_tool` lets the agent record “thoughts” before it acts. This pattern can improve the quality of agent decision-making:

- Analyze search results and plan the next action
- Evaluate whether the collected information is sufficient
- Make delegated tasks more specific before sending them to subagents

```
from langchain.tools import tool

@tool
def think_tool(thought: str) -> str:
    """Strategic reflection – analyze the current situation and plan the next action."""
    return f"Reflection recorded: {thought}"
```

Step 2: A Simplified `web_search` Tool

In a real deep research workflow, you would typically use the Tavily API. Here, for learning purposes, we define a simplified search tool.

```
@tool
def web_search(query: str) -> str:
    """Perform a web search (simulated)."""
    results = {
        "AI agent": "AI agents are systems that perform tasks autonomously. Their adoption has accelerated rapidly since 2024.",
        "LangGraph": "LangGraph is a stateful workflow framework. It supports both the Graph API and the Functional API.",
        "Deep Agents": "Deep Agents is an all-in-one agent SDK. It supports subagents, backends, and skills.",
    }
    for key, val in results.items():
        if key.lower() in query.lower():
            return val
    return f"Search result for '{query}': no relevant information found."
```

Step 3: Prompt for the Five-Step Research Workflow

The prompt loader pulls the prompt using the following order: LangSmith Hub → Langfuse → local default.

Step	Name	Description
1	<i>Plan</i>	Create a research plan with <code>write_todos</code>
2	<i>Delegate</i>	Parallelize investigation across subagents (up to 3 at once)
3	<i>Synthesize</i>	Combine the gathered information
4	<i>Verify</i>	Ask the fact-checker to verify the claims
5	<i>Report</i>	Produce the final report

```
from prompts import RESEARCH_AGENT_PROMPT

print(RESEARCH_AGENT_PROMPT)
```

Step 4: Define Three Subagents

The deep research agent uses three specialized subagents:

Subagent	Role	Tools
researcher-1	Primary topic research	web_search , think_tool
researcher-2	Complementary or contrasting research	web_search , think_tool
fact-checker	Verify factual accuracy	web_search

```
researcher_1 = {
    "name": "researcher-1",
    "description": "Performs in-depth research on the topic",
    "system_prompt": "You are a research specialist. Investigate the topic deeply and summarize the key findings. Reflect with think_tool after searching.",
    "tools": [web_search, think_tool],
}
```

```
researcher_2 = {
    "name": "researcher-2",
    "description": "Performs complementary research from another perspective",
    "system_prompt": "You are a complementary researcher. Collect additional information from a different angle. Reflect with think_tool after searching.",
    "tools": [web_search, think_tool],
}
```

```
fact_checker = {
    "name": "fact-checker",
    "description": "Verifies the factual correctness of collected information",
    "system_prompt": "You are a fact-checker. Verify the accuracy of the provided information and point out any errors.",
    "tools": [web_search],
}
```

Step 5: Create the Deep Research Agent (with v1 Middleware)

Combine the tools and subagents into the final agent. The v1 middleware improves stability and reliability:

Middleware	Role
SummarizationMiddleware	Automatically summarizes long research conversations to save context
ModelCallLimitMiddleware	Prevents research loops by limiting the run to 30 model calls
ModelFallbackMiddleware	Falls back to a backup model if the primary model fails

Enable checkpointing with `InMemorySaver` so interrupted research runs can resume.

```
from deepagents import create_deep_agent
from deepagents.backends import FilesystemBackend
```

```
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents.middleware import (
    SummarizationMiddleware,
    ModelCallLimitMiddleware,
    ModelFallbackMiddleware,
)

research_agent = create_deep_agent(
    model=model,
    tools=[web_search, think_tool],
    subagents=[researcher_1, researcher_2, fact_checker],
    system_prompt=RESEARCH_AGENT_PROMPT,
    backend=FilesystemBackend(root_dir=".", virtual_mode=True),
    skills=["/skills/"],
    checkpointer=InMemorySaver(),
    middleware=[
        SummarizationMiddleware(model=model, trigger=("messages", 15)),
        ModelCallLimitMiddleware(run_limit=30),
        ModelFallbackMiddleware("gpt-5.4-mini"),
    ],
)
```

DeepAgents Pattern — **ToDoList + dispatch + 5-Tool Async**

The deep research agent in this chapter combines the three core DeepAgents patterns:

Pattern	Description
<i>ToDoList</i>	The built-in <code>write_todos</code> makes the plan explicit during the Plan step — progress is anchored in the message history
<i>dispatch</i>	The built-in <code>task</code> delegates work to subagents — only the summary returns to the main agent, keeping its context clean
<i>5-tool async</i>	Five tools — <code>web_search</code> , <code>think_tool</code> , <code>write_todos</code> , <code>task</code> , <code>read/write_file</code> — drive the async workflow

 **TIP**

Pass an `AsyncSubAgent` spec to `subagents` and `create_deep_agent` automatically wires in `AsyncSubAgentMiddleware` , so the subagents run in parallel asynchronously (see `docs/deepagents/12-async-subagents.md`).

Step 6: Run the Research Workflow

Give the agent a research topic and let it execute the five-step workflow.

Step 7: Streaming — Track Namespaces

With `stream(subgraphs=True)` , you can follow the execution of the main agent and the subagents by namespace. This makes it easy to see when each subagent is called.

Subagent Design Best Practices

Principle	Description
<i>Clear descriptions</i>	Write specific <code>description</code> values so the main agent knows when to delegate
<i>Specialized prompts</i>	Put output format, constraints, and workflow expectations into <code>system_prompt</code>
<i>Minimal tools</i>	Give each subagent only the tools it actually needs
<i>Concise outputs</i>	Tell subagents to return summaries rather than raw data

Summary

Item	Key Point
<code>think_tool</code>	Strategic reflection — analyze search results and plan the next step
<i>Subagents</i>	Parallel execution with <code>researcher-1</code> , <code>researcher-2</code> , and <code>fact-checker</code>
<i>Workflow</i>	Plan → Delegate → Synthesize → Verify → Report
<i>Context management</i>	Only subagent results are passed back to the main agent; intermediate work stays isolated

References:

- `docs/deepagents/examples/02-deep-research.md`
- `docs/deepagents/07-subagents.md`
- `docs/deepagents/06-backends.md`

Previous Step: ← [04_ml_agent.ipynb](#): Machine learning agent

P A R T

VIII

Integrations

LangChain Ecosystem Integrations

What You Will Learn in This Part

- 1. Integration Category Overview
- 2. Anthropic Prompt Caching
 - 3. Claude Bash Tool
 - 4. Claude Text Editor
 - 5. Claude Memory
- 6. Anthropic File Search
- 7. Bedrock Prompt Caching
- 8. OpenAI Moderation

01 Integration Category Overview

A map of the LangChain ecosystem

LangChain · LangGraph · Deep Agents take the shape of a common agent interface beneath which *per-provider implementations* plug in. Parts II–IV focused on “how to use the agent”; this Part focuses on “*what* you connect the agent to”. We scan the thirteen integration categories and lay out the representative packages and selection criteria for each.

Learning Objectives

LEARNING OBJECTIVES

- Survey the thirteen integration categories of the LangChain ecosystem
- Understand why Provider Middleware is a separate category
- Distinguish vendor-sensitive areas (Chat Models, Vector Stores) from standardized ones (Middleware, Checkpointers)
- Map the seven Provider Middleware chapters this Part covers

1.1 Baseline version snapshot

The code in this Part assumes the following versions. Check the latest releases in [docs/skills/langchain-dependencies.md](#).

- `langchain` 1.3+ (event streaming v3)
- `langgraph` 1.2+ (per-node timeout · `DeltaChannel` · graceful shutdown)
- `deepagents` 0.6.1+ (`CodeInterpreterMiddleware` , async subagents, interpreter snapshots)

! WARNING

LangChain 1.2 `create_agent` rejects *duplicate instances* of the same middleware class. If you hit `AssertionError: Please remove duplicate middleware instances.`, split the two instances into separate types via subclassing.

1.2 The thirteen integration categories

Category	Representative packages	Selection criteria
Chat Models	<code>langchain-openai</code> , <code>langchain-anthropic</code> , <code>langchain-google-genai</code> , <code>langchain-aws</code>	Model performance · cost · regional requirements
Embeddings	<code>langchain-openai</code> , <code>langchain-cohere</code> , <code>langchain-voyageai</code>	Domain benchmarks · license · multilingual support
Vector Stores	<code>langchain-chroma</code> , <code>langchain-pinecone</code> , <code>langchain-pgvector</code>	Operational scale · metadata filtering · managed option
Document Loaders	<code>langchain-community.document_loaders</code>	Source format (PDF, Notion, Slack, etc.)
Retrievers	<code>langchain.retrievers</code> , vendor-specific retrievers	Hybrid search · MMR · reranker integration
Text Splitters	<code>langchain-text-splitters</code>	Language · structure (code vs prose) · overlap strategy
Tools	<code>langchain-community.tools</code> , MCP tools	External API integration · auth methods · call constraints
Checkpointers	<code>langgraph-checkpoint-postgres</code> , <code>-sqlite</code>	Durability needs · multi-tenancy · backup strategy
Stores	<code>langgraph-store-postgres</code> , vendor-managed	Persistent memory scale · lookup method (key / vector)
Sandboxes	<code>langchain-daytona</code> , <code>langchain-modal</code> , <code>langchain-runloop</code>	Isolation level · cold-start · cost
Provider Middleware	<code>langchain-anthropic</code> , <code>langchain-aws</code> , <code>langchain-openai</code>	Provider-specific features (caching · native tools · policy)
Observability	<code>langsmith</code> , <code>langfuse</code> , OpenTelemetry	SaaS vs self-hosted · PII policy
Providers (Hosted Runtime)	LangGraph Platform, Anthropic Skills, OpenAI Agents	Managed runtime vs self-hosted · SDK compatibility

Provider Middleware wraps features that are activated *on the provider's servers* (prompt caching, native tools, content policy) into the LangChain middleware format. Official docs often mention them in a single line, so there is a lot of value in organizing them as working code. Chapters 2–8 of this Part each cover one of the seven middleware.

1.3 Provider Middleware roadmap

Chapter	Middleware	Effect
---------	------------	--------

2	AnthropicPromptCachingMiddleware	Server-side cache for long system prompts / tool definitions (5m / 1h)
3	ClaudeBashToolMiddleware	Claude native bash tool + three execution policies
4	StateClaudeTextEditorMiddleware / FilesystemClaudeTextEditorMiddleware	Six operations of the native text editor
5	StateClaudeMemoryMiddleware / FilesystemClaudeMemoryMiddleware	Model self-memory via the <code>/memories/*</code> path contract
6	StateFileSearchMiddleware	glob / grep over virtual files in state
7	BedrockPromptCachingMiddleware	Cache for Claude / Nova through AWS Bedrock
8	OpenAIModerationMiddleware	Pre- / post-check via OpenAI Moderation API

1.4 Common pattern

Every chapter in this Part shares the same three-step loop.

1. **Environment setup** — put the provider key in `.env` and install the package
2. **Basic usage** — flip the feature on with one line, `create_agent(..., middleware=[Middleware()])`
3. **Validation** — read `usage_metadata`, `tool_calls`, or the Moderation response to confirm the feature actually ran

TIP

Provider-specific middleware needs *a matching model*. If you attach Anthropic middleware to an OpenAI model, behavior is governed by `unsupported_model_behavior` — either a warning or an exception. In multi-provider pipelines, watch the order when combining with `ModelFallbackMiddleware`.

1.5 Observability env-var standardization

From LangChain 1.1, tracing-related environment variables were standardized from `LANGCHAIN_*` to `LANGSMITH_*`. The old names continue to work for a while for compatibility, but *new code uses the new names exclusively*.

Variable	Role	Notes
<code>LANGSMITH_TRACING</code>	Enable tracing (<code>true</code> / <code>false</code>)	Auto-instruments every <code>create_agent</code> <code>run</code>
<code>LANGSMITH_API_KEY</code>	LangSmith account key	<code>ls_</code> prefix

LANGSMITH_PROJECT	Project name for trace grouping	Per-call overridable via <code>tracing_context</code>
LANGSMITH_ENDPOINT	Endpoint for self-hosted LangSmith	Optional – unneeded for SaaS

Key Takeaways

- Integrations are organized into twelve categories, each sharing the “provider plugin + common interface” structure
- Provider Middleware is the category that wraps provider *server-side* features as LangChain middleware
- Chapters 2–8 cover the seven Provider Middleware with runnable code, one per chapter
- Duplicate-instance constraints, model matching, and multi-provider fallback are the common concerns of Provider Middleware

02 Anthropic Prompt Caching

Cut input-token costs by 90% with server-side caching

`AnthropicPromptCachingMiddleware` is a Claude-only prompt cache middleware that caches long system prompts, tool definitions, and initial conversation context on Anthropic's servers for 5 minutes to 1 hour, cutting *input-token cost and latency substantially*. When the same agent is invoked many times with the same system prompt, the input-token price drops by roughly 90% from the second call onward.

Learning Objectives

LEARNING OBJECTIVES

- Understand the four parameters (`type` , `ttl` , `min_messages_to_cache` , `unsupported_model_behavior`)
- Verify cache hits by reading `cache_creation` / `cache_read` under `usage_metadata.input_token_details`
- Pick between the 1-hour TTL beta and the 5-minute default
- Distinguish `warn` / `ignore` / `raise` behaviors when applied to non-Anthropic models

2.1 When to use it

- Agents that send a long system prompt (multi-thousand-token corporate policy, style guide) every turn
- Setups that repeatedly send a large tool-definition list (20+ JSON schemas)
- RAG context reuse: the same document bundle across many queries
- Multi-turn conversations where the prefix context is stable

2.2 Environment setup

Required packages: `langchain` , `langchain-anthropic` . `ANTHROPIC_API_KEY` must be present in `.env` .

```
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_anthropic.middleware import AnthropicPromptCachingMiddleware

load_dotenv()
```

2.3 Basic usage

Passing the middleware to `create_agent`'s `middleware` list makes LangChain attach `cache_control: {"type": "ephemeral"}` markers automatically at the *system prompt · tool definitions · last message* positions.

```
long_system_prompt = (
    "You are a senior software engineer... " * 200 # ~2,000 tokens
)

agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    system_prompt=long_system_prompt,
    tools=[],
    middleware=[AnthropicPromptCachingMiddleware()],
)
```

2.4 Verifying cache hits

Call the same agent twice in a row and read `usage_metadata`. The first call shows a large `cache_creation_input_tokens`; subsequent calls show `cache_read_input_tokens`.

```
for i in range(2):
    result = agent.invoke(
        {"messages": [{"role": "user", "content": f"Request {i}"}]},
    )
    last = result["messages"][-1]
    print(i, last.usage_metadata.get("input_token_details"))
```

```
0 {'cache_creation': 2048, 'cache_read': 0}
1 {'cache_creation': 0, 'cache_read': 2048}
```

2.5 Full parameters

Parameter	Default	Description
<code>type</code>	"ephemeral"	The only type Anthropic currently supports
<code>ttr</code>	"5m"	Cache duration. "5m" or "1h". 1 hour is an Anthropic beta
<code>min_messages_to_cache</code>	0	Attach <code>cache_control</code> only when the message count is at least this value
<code>unsupported_model_behavior</code>	"warn"	For non-Anthropic models: "warn" / "ignore" / "raise"

1-hour TTL example

```
AnthropicPromptCachingMiddleware(
    ttl="1h",
```

```
        min_messages_to_cache=2,
    )
```

The 1-hour cache is a fit for scenarios where *the same system prompt is reused over many hours* — long advisory sessions, daily batch analytics. The 5-minute default targets continuous user input in a real-time chatbot.

2.6 Behavior on non-Anthropic models

Because this middleware injects `cache_control` blocks internally, other providers' models either ignore them or error. In pipelines where the model can be swapped, make `unsupported_model_behavior` explicit.

```
# Log a warning and continue
AnthropicPromptCachingMiddleware(unsupported_model_behavior="warn")

# Silently skip
AnthropicPromptCachingMiddleware(unsupported_model_behavior="ignore")

# Fail immediately (CI / tests)
AnthropicPromptCachingMiddleware(unsupported_model_behavior="raise")
```

2.7 Aggregating with `UsageMetadataCallbackHandler`

To aggregate cache-hit rate across a whole thread, attach `langchain.callbacks.UsageMetadataCallbackHandler`. Every LLM call's `usage_metadata` accumulates, with `cache_creation` / `cache_read` preserved as-is.

```
from langchain.callbacks import UsageMetadataCallbackHandler

handler = UsageMetadataCallbackHandler()
agent.invoke(
    {"messages": [{"role": "user", "content": "..."}]},
    config={"callbacks": [handler]},
)
print(handler.usage_metadata) # {model_name: {input/output/cache_creation/cache_read}}
```

TIP

Receiving Anthropic responses with `output_version="v1"` (`ChatAnthropic(..., output_version="v1")`) places `cache_creation` / `cache_read` inside the *standardized* `usage_metadata.input_token_details` rather than the message body — multi-provider aggregation code becomes much simpler.

2.8 Differences from Bedrock

When calling Claude via AWS Bedrock, use `BedrockPromptCachingMiddleware` (ch7). The two middleware share parameter names and semantics, but they target *different packages* and have different TTL constraints (Nova supports only 5m, no tool-definition cache).

Key Takeaways

- Agents that repeatedly send long system prompts / tool definitions cut input-token cost by 90%
- Verify cache hits by watching `cache_creation_input_tokens` move to `cache_read_input_tokens`
- TTL `"5m"` for real-time conversation, `"1h"` for long-running batches / advisory sessions
- Declare `unsupported_model_behavior="warn"` in multi-provider pipelines to avoid silent failures

03 Claude Bash Tool

Native bash_20250124 + execution policies

`ClaudeBashToolMiddleware` injects the Anthropic native `bash_20250124` tool into Claude models. Unlike a generic `ShellToolMiddleware`, *Anthropic's servers manage the bash call schema directly*, so the prompt stays short and tool-schema tokens approach zero. This chapter compares the three execution policies (Host / Docker / CodexSandbox) and covers `redaction_rules` for masking secrets.

Learning Objectives

LEARNING OBJECTIVES

- Understand the four main parameters of `ClaudeBashToolMiddleware`
- Distinguish the isolation levels of `HostExecutionPolicy` / `DockerExecutionPolicy` / `CodexSandboxExecutionPolicy`
- Mask tokens and keys in output via `RedactionRule`
- Know how Claude native bash differs from a generic shell tool

3.1 When to use it

- Letting Claude run real shell commands for code execution, file inspection, builds
- Deep-Agents-style long-running workspaces where you need to preserve session state (cwd, env vars)
- Running arbitrary code safely in an isolated Docker container
- Shell output may contain API keys / credentials, so redaction is required

3.2 Environment setup

Required packages: `langchain`, `langchain-anthropic`. The Docker policy example requires a local Docker daemon running.

```
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_anthropic.middleware import ClaudeBashToolMiddleware

load_dotenv()
```

3.3 Host execution policy (simplest)

`HostExecutionPolicy` runs commands in the local process. Fast with no setup, but *no isolation* — it has full access to network, filesystem, and environment variables, so reserve it for trusted scripts or local development.

```
from langchain.agents.middleware import HostExecutionPolicy

agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        ClaudeBashToolMiddleware(
            workspace_root="/tmp/work",
            startup_commands=["export PATH=$HOME/.local/bin:$PATH"],
            execution_policy=HostExecutionPolicy(),
        ),
    ],
)
```

- `workspace_root` : default directory for the shell session
- `startup_commands` : commands run automatically at session start (PATH configuration, venv activation, etc.)

3.4 Docker execution policy (recommended, isolated)

`DockerExecutionPolicy` runs commands *inside a container*, fully isolated from the host filesystem and network. Make it the effective default when executing model-generated code.

```
from langchain.agents.middleware import DockerExecutionPolicy

agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        ClaudeBashToolMiddleware(
            execution_policy=DockerExecutionPolicy(image="python:3.11"),
            startup_commands=["pip install requests numpy"],
        ),
    ],
)
```

The container is created on the first bash call and cleaned up at session end. For package requirements, add

`pip install ...` to `startup_commands` .

3.5 Codex sandbox policy

`CodexSandboxExecutionPolicy` runs commands on Anthropic’s managed sandbox runner. It is the alternative when you want isolation without local Docker. The network whitelist and resource limits come with strong defaults.

3.6 Output redaction (`redaction_rules`)

Shell output can leak sensitive values — API keys, tokens, emails. Pass a list of `RedactionRule` objects to mask tool responses *before they enter the agent context*. LangChain 1.2 unified the `RedactionRule` signature with `PIIMiddleware` — use `(pii_type, strategy, detector)` instead of the removed `pattern=` / `replacement=` kwargs.

```

from langchain.agents.middleware import RedactionRule

agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        ClaudeBashToolMiddleware(
            execution_policy=DockerExecutionPolicy(),
            redaction_rules=[
                RedactionRule(
                    pii_type="openai_api_key",
                    strategy="redact",
                    detector=r"sk-[a-zA-Z0-9]{32,}",
                ),
                RedactionRule(
                    pii_type="github_token",
                    strategy="redact",
                    detector=r"ghp_[a-zA-Z0-9]{36}",
                ),
            ],
        ),
    ],
)

```

3.7 Falling back to a generic `@tool`

To carry the same flow to non-Claude models, define the shell function directly with LangChain 1.3's `@tool` decorator. Injecting Anthropic programmatic tool-calling metadata via the `extras` field (introduced in LangChain 1.2) lets the same function be called with *the same shape* as the Claude-native version.

```

from langchain_core.tools import tool

@tool(extras={"anthropic_cache_control": {"type": "ephemeral"}})
def run_bash(command: str) -> str:
    """Run a bash command in an isolated container and return stdout/stderr."""
    return run_in_container(command) # your own implementation

```

TIP

`@tool(extras=...)` is handy for *partial caching* in multi-provider pipelines. Claude interprets `cache_control`; other providers ignore it — so the same codebase is exposed to both sides unchanged.

3.8 Native vs generic comparison

Item	ClaudeBashToolMiddleware	Generic ShellToolMiddleware
Tool type	Anthropic native <code>bash_20250124</code>	Ordinary <code>@tool</code> function
Supported models	Claude only	All models
Tool-schema tokens	Server-side → near zero	Sent every turn
Session state	cwd, env preserved and accumulated	Depends on implementation

Execution-policy API	Shared (<code>HostExecutionPolicy</code> , etc.)	Shared
----------------------	---	--------

Selection criteria: If you are Claude-only, the native version is cheaper and cleaner. For multi-provider pipelines, unify on the generic `ShellToolMiddleware` .

Key Takeaways

- Claude native bash brings tool-schema tokens close to zero
- Pick execution policy in three tiers: Host (no isolation) → Docker (recommended) → CodexSandbox (Anthropic-managed)
- `redaction_rules` masks sensitive values in tool responses before they enter agent context
- In multi-provider pipelines, unifying on the generic `ShellToolMiddleware` is the safer default

04 Claude Text Editor

State vs Filesystem variants

The Claude native `text_editor_20250728` tool supports six operations: `view` / `create` / `str_replace` / `insert` / `delete` / `rename`. LangChain ships middleware that wraps this tool in two variants — the *State variant* writes into virtual files inside LangGraph state; the *Filesystem variant* writes to a real directory. Choose based on whether the artifact’s lifetime ends with the thread or must persist on disk.

Learning Objectives

LEARNING OBJECTIVES

- Use the six operations of the Claude native text editor
- Distinguish the State and Filesystem variants by lifetime and sharing scope
- Restrict paths via `allowed_path_prefixes` / `allowed_prefixes`
- Bound the disk variant with `root_path` and `max_file_size_mb`

4.1 When to use it

- Model edits that are inherently *multi-step* (large changes that are hard to output as a single diff)
- The result needs to live *only inside the graph state*: use the State variant
- The result must *remain in a real repo / directory*: use the Filesystem variant
- When you would rather reuse Claude’s learned tool schema than build a generic `@tool` for `read_file` / `write_file`

4.2 Environment setup

Required packages: `langchain`, `langchain-anthropic`, `ANTHROPIC_API_KEY` in `.env`.

```
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_anthropic.middleware import (
    StateClaudeTextEditorMiddleware,
    FilesystemClaudeTextEditorMiddleware,
)

load_dotenv()
```

4.3 State variant — virtual files in graph state

`StateClaudeTextEditorMiddleware` stores file contents under the `text_editor_files` key in LangGraph state. It does not touch disk, so it is a fit for temporary work that *disappears when the thread ends*.

```
agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        StateClaudeTextEditorMiddleware(
            allowed_path_prefixes=["/src"], # virtual-path restriction
        ),
    ],
)

result = agent.invoke(
    {"messages": [{"role": "user", "content": "Write hello world to /src/hello.py"}]},
)
print(result.get("text_editor_files"))
```

- `allowed_path_prefixes` : allow-listed virtual-path prefixes. Unrestricted if omitted

TIP

If the model writes outside `allowed_path_prefixes`, the tool returns an error; the model reads that error and retries with a permitted path. Path restrictions are the compromise between *agent autonomy* and *safety*.

4.4 Filesystem variant — real disk

`FilesystemClaudeTextEditorMiddleware` uses a real directory as its root and reads/writes from there.

Parameter	Default	Description
<code>root_path</code>	(required)	Actual root directory for file operations
<code>allowed_prefixes</code>	<code>["/"]</code>	Allowed virtual-path prefixes (relative to <code>root_path</code>)
<code>max_file_size_mb</code>	10	Maximum file size allowed for read/write

```
agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        FilesystemClaudeTextEditorMiddleware(
            root_path="/tmp/editor-demo",
            allowed_prefixes=["/drafts", "/reports"],
            max_file_size_mb=5,
        ),
    ],
)
```

4.5 State vs Filesystem selection criteria

Axis	State variant	Filesystem variant
Storage	LangGraph state dict	Actual directory
Lifetime	Dies with the thread	Permanent (remains on disk)
Checkpoint compatibility	Included in state → auto-saved	Only the path is stored; files managed separately
Concurrency	Fully isolated per thread	Possible conflicts when sharing <code>root_path</code>
Typical uses	Temporary drafts, one-off analyses	Real codebase edits, report artifacts

Rule: If the artifact must survive after the thread, go Filesystem; otherwise, go State.

4.6 Relationship with generic file tools

Deep Agents' `FilesystemMiddleware` or a custom `@tool` implementing `read_file` / `write_file` can reach the same goal. The difference is that *Claude already saw this tool during training* — schema tokens drop and error rates are lower. Use this middleware first in Claude-only pipelines, and drop down to generic file tools for multi-provider setups.

Key Takeaways

- Claude native text editor brings tool-schema tokens close to zero while supporting six operations
- Choose State (dies with thread) vs Filesystem (persistent on disk) based on artifact lifetime
- Enforce access boundaries with `allowed_path_prefixes` / `allowed_prefixes`
- The Filesystem variant uses `max_file_size_mb` to keep memory safe

05 Claude Memory

The ``/memories/*`` path contract

The Claude native `memory_20250818` tool implements *long-term memory* the model writes and reads itself. The injected system prompt instructs the model each turn to “check `/memories` first”, so preferences and facts accumulate naturally across multiple sessions with the same user. Understanding the persistence difference between the State and Filesystem variants is the core of this chapter.

Learning Objectives

LEARNING OBJECTIVES

- Understand the Claude memory tool’s `/memories/` path contract
- Distinguish persistence scope between State and Filesystem variants
- Know how the injected `system_prompt` instructs the model to use memory
- Clarify the division of labor with Deep Agents’ `StoreBackend`

5.1 When to use it

- A chatbot that must remember preferences / facts across *multiple sessions* with the same user
- An agent working on a long task that needs to *write and later read its own intermediate notes*
- Unlike RAG, you want the *model itself to manage* memory contents (write, delete, edit)

5.2 Environment setup

Required packages: `langchain`, `langchain-anthropic`, `ANTHROPIC_API_KEY` in `.env`.

```
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_anthropic.middleware import (
    StateClaudeMemoryMiddleware,
    FilesystemClaudeMemoryMiddleware,
)
from langgraph.checkpoint.memory import MemorySaver

load_dotenv()
```


5.3 State variant — persisted within the thread

`StateClaudeMemoryMiddleware` stores memo content in LangGraph state (`memory_files`). With a checkpointer such as `MemorySaver` / `PostgresCheckpoint`, content is *restored automatically when resuming the same* `thread_id`.

Parameter	Default	Description
<code>allowed_path_prefixes</code>	<code>["/memories"]</code>	Allowed memo paths. Usually leave as is
<code>system_prompt</code>	Anthropic default	Instructs the model to “check /memories first” at the start of every turn

```
agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    checkpointer=MemorySaver(),
    middleware=[StateClaudeMemoryMiddleware()],
)

cfg = {"configurable": {"thread_id": "user-42"}}

agent.invoke(
    {"messages": [{"role": "user", "content": "My name is Jihun."}]},
    config=cfg,
)

# Re-invoke with the same thread_id → memo reused
result = agent.invoke(
    {"messages": [{"role": "user", "content": "Do you remember my name?"}]},
    config=cfg,
)

print(result["messages"][-1].content)
```

On the second call, the model does not ask for the name — it reads `/memories` and answers with “Jihun”.

5.4 Filesystem variant — across process boundaries

`FilesystemClaudeMemoryMiddleware` keeps memos on *an actual disk directory*. Even if the process exits or the checkpoint store changes, the files stay on disk and remain readable.

Parameter	Default	Description
<code>root_path</code>	(required)	Actual directory to store memories in
<code>allowed_prefixes</code>	<code>["/memories"]</code>	Allowed virtual paths for memo writes
<code>max_file_size_mb</code>	10	Maximum per-memo file size
<code>system_prompt</code>	Built-in	The prompt directing memory usage

```
agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        FilesystemClaudeMemoryMiddleware(
            root_path="/var/data/claude-memory",
            max_file_size_mb=2,
        ),
    ],
)
```

```
1,
)
```

5.5 Scaling to a durable checkpointer

`MemorySaver` is an in-process dict — it evaporates on restart. In production, use the context-manager pattern of `langgraph-checkpoint-postgres` or `-sqlite`: acquire a pool with `from_conn_string()`, then call `setup()` once to create the schema.

```
from langgraph.checkpoint.postgres import PostgresSaver

DB = "postgresql://user:pass@localhost/agent"

with PostgresSaver.from_conn_string(DB) as checkpointer:
    checkpointer.setup() # one-time - creates tables / indexes

    agent = create_agent(
        model="anthropic:claude-sonnet-4-6",
        checkpointer=checkpointer,
        middleware=[StateClaudeMemoryMiddleware()],
    )
```

TIP

For async graphs, wrap `AsyncPostgresSaver.from_conn_string(DB)` with `async with`. `setup()` also needs `await`. SQLite follows the same context-manager API (`SqliteSaver` / `AsyncSqliteSaver`).

5.6 Multi-tenant isolation via `runtime.context.user_id`

When many users share the same disk directory (or the same Store), isolate with a *namespace*. From LangGraph 1.2, the standard pattern reads `runtime.context.user_id` directly and uses it as the Store namespace key.

```
from langgraph.store.postgres import PostgresStore
from langgraph.runtime import get_runtime

with PostgresStore.from_conn_string(DB) as store:
    store.setup()

    def upsert_profile(profile: dict) -> str:
        rt = get_runtime()
        ns = ("profiles", rt.context.user_id) # per-user namespace
        store.put(ns, "self", profile)
        return "ok"
```

TIP

The Claude Memory Middleware’s `/memories/*` path contract and the LangGraph Store’s `(namespace, key)` contract live at *different layers*. The former lets Claude manage memos autonomously; the latter requires the code to assign keys explicitly — so multi-tenant isolation is a more natural fit for the Store.

5.7 Relationship with Deep Agents’ `StoreBackend`

Deep Agents 0.5 ships a separate `StoreBackend` for persistent memory. The two have *similar roles but different scope*.

Axis	Claude Memory Middleware	Deep Agents’ <code>StoreBackend</code>
Provider	Anthropic native tool	LangGraph Store API wrapper
Supported models	Claude only	All models
Storage shape	<code>/memories/*</code> files	<code>(namespace, key) → dict</code>
Retrieval	File list + content read	Embedding vector search supported
Typical uses	Claude freely managing memos	Structured profiles / facts

Combine tip: You can use both. Have Claude keep short-term work notes in `/memories`, and put long-term profiles in Deep Agents’ `StoreBackend` to share with agents from other providers — a pragmatic pattern.

Key Takeaways

- The Claude native memory tool enforces “check first, update when needed” via the `/memories/*` path contract and an auto-injected system prompt
- The State variant restores on thread resume via a checkpoint; the Filesystem variant persists on disk permanently
- `max_file_size_mb` prevents the model from dumping everything into a single memo
- Deep Agents’ `StoreBackend` complements via vector search and cross-provider sharing

06 Anthropic File Search

glob + grep over virtual files

`StateFileSearchMiddleware` offers a Claude native tool that runs *glob + grep* over virtual files sitting inside graph state (for example `text_editor_files`, `memory_files`). If the text editor / memory middleware “create and edit files”, this middleware “finds and reads” files on top of them.

Learning Objectives

LEARNING OBJECTIVES

- Select the target store via `StateFileSearchMiddleware(state_key=...)`
- Complete the “write → find → read” loop by pairing it with `StateClaudeTextEditorMiddleware`
- Switch the search target to memory files via `state_key="memory_files"`
- Know the duplicate-middleware constraint and the subclassing workaround

6.1 When to use it

- The agent has accumulated *tens to hundreds* of virtual files in state and needs to explore them
- You know only the name and want to find files by pattern (`**/*.md`) or filter by a keyword
- You want the model to `grep` its own accumulated past notes in memory to reference them

6.2 Environment setup

Required packages: `langchain`, `langchain-anthropic`, `ANTHROPIC_API_KEY` in `.env`.

```
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_anthropic.middleware import (
    StateClaudeTextEditorMiddleware,
    StateFileSearchMiddleware,
)

load_dotenv()
```

6.3 Text editor + file search combination

Search by itself is meaningless — *the files must exist in state first*. The most common pairing is to create files with the text editor and find them with file search.

Parameter	Default	Description
<code>state_key</code>	<code>"text_editor_files"</code>	State key containing the dict of files to search. Switch to <code>"memory_files"</code> to search memory

```
agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        StateClaudeTextEditorMiddleware(),
        StateFileSearchMiddleware(state_key="text_editor_files"),
    ],
)
```

6.4 Step 1 — seed state with multiple files

Ask the model to create a few notes under `/docs/`. These become the targets for the later search query.

```
cfg = {"configurable": {"thread_id": "search-demo"}}
agent.invoke(
    {
        "messages": [
            {
                "role": "user",
                "content": (
                    "Create three short drafts: "
                    "/docs/architecture.md, /docs/api.md, /docs/release-notes.md."
                ),
            }
        ],
    },
    config=cfg,
)
```

6.5 Step 2 — the same agent searches with glob/grep

Calling again on the same thread carries the previous files in state. The model uses `glob` and `grep` tools provided by `StateFileSearchMiddleware` to find files and read their content.

```
result = agent.invoke(
    {
        "messages": [
            {
                "role": "user",
                "content": "Among md files under /docs/, tell me which ones mention 'API'",
            }
        ],
    },
    config=cfg,
)
```

```
)
print(result["messages"][-1].content)
```

6.6 Targeting memory files too

Switch to `state_key="memory_files"` and you can search the memos accumulated by `StateClaudeMemoryMiddleware`.

! WARNING

Duplicate-middleware constraint — LangChain 1.2 `create_agent` rejects duplicate instances of the same middleware class (`AssertionError: Please remove duplicate middleware instances.`). To search both text-editor files and memory files at once, register only one at a time — or subclass `StateFileSearchMiddleware` into a separate class.

```
class MemoryFileSearchMiddleware(StateFileSearchMiddleware):
    """Search middleware dedicated to `memory_files`."""

agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        StateClaudeTextEditorMiddleware(),
        StateClaudeMemoryMiddleware(),
        StateFileSearchMiddleware(state_key="text_editor_files"),
        MemoryFileSearchMiddleware(state_key="memory_files"),
    ],
)
```

6.7 Position in the five-step retrieval building blocks

LangChain RAG is standardized into five stages — *Document Loaders* → *Text Splitters* → *Embeddings* → *Vector Stores* → *Retrievers* (categories 04–05 in 08_integration). `StateFileSearchMiddleware` is a *lite path* that bypasses these five stages, scanning *files already in state* with literal grep and zero embedding cost.

Axis	State File Search	Five-step RAG pipeline
Preprocessing	None — files as they arrive in state	Loader + Splitter + Embed + Index
Retrieval	literal glob + grep	vector similarity + filter
Latency	Milliseconds	Embedding call + ANN
Suitable scale	Tens to hundreds of files	Tens of thousands to millions of chunks

```
# Scaling out to a five-step RAG pipeline
from langchain.embeddings import init_embeddings
from langchain_chroma import Chroma

embeddings = init_embeddings("openai:text-embedding-3-small")
vectorstore = Chroma(collection_name="docs", embedding_function=embeddings)
retriever = vectorstore.as_retriever(search_kwargs={"k": 5})
```

 TIP

`init_embeddings("provider:model")` mirrors the `init_chat_model` pattern — swap the embedding backend in a single line using a provider-prefixed string. Moving from State File Search to a vector-store RAG keeps code changes minimal.

6.8 When to pick this over a vector store

- Exact filename/patterns matter, and *literal grep* is enough — no semantic search needed
- Document count is *tens to hundreds* and you would rather not pay to re-embed each turn
- The files are *just created in the current session* and no vector index exists yet

For large corpora or semantic search, move over to Part VIII’s Chat Models / Vector Stores area (future expansion).

Key Takeaways

- `StateFileSearchMiddleware` provides glob + grep over virtual files in state
- text editor + file search is the standard “write → find → read” loop
- Switch `state_key` to search memory files; work around the duplicate-middleware limit via subclassing
- When literal search suffices, this middleware is much cheaper than a vector store

07 Bedrock Prompt Caching

Claude / Nova caching through AWS

When calling Claude / Nova through AWS Bedrock, `BedrockPromptCachingMiddleware` plants cache checkpoints automatically at the *system prompt · tool definitions · last message* positions to reduce input-token cost. Parameter names and semantics are *almost identical* to `AnthropicPromptCachingMiddleware` (ch2), but the target package and per-model constraints differ.

Learning Objectives

LEARNING OBJECTIVES

- Understand the four parameters of `BedrockPromptCachingMiddleware`
- Verify hits via `cache_creation` / `cache_read` in `usage_metadata.input_token_details`
- Know how `type` is handled differently between `ChatBedrock` and `ChatBedrockConverse`
- Distinguish Nova constraints (5m TTL only, no tool-definition cache)

7.1 When to use it

- Enterprise setups that must keep LLM calls inside AWS account / VPC boundaries (via Bedrock)
- Using Claude through *Bedrock billing* rather than the direct Anthropic API
- Repeated use of long system prompts with Amazon Nova

7.2 Environment setup

Required packages: `langchain`, `langchain-aws`. AWS credentials (`AWS_ACCESS_KEY_ID` etc.) and a region must be configured via `.env` or environment. In the Bedrock console you also need to grant *model access* before the call works.

```
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_aws.middleware import BedrockPromptCachingMiddleware

load_dotenv()
```


7.3 ChatBedrockConverse + Claude (recommended)

The Converse API is Bedrock's *unified interface* and can call many models through the same endpoint. Adding `BedrockPromptCachingMiddleware` automatically inserts cache checkpoints at the system / tool / last-message positions.

! WARNING

For a checkpoint to actually hit, the cached block must be *roughly 1,024 tokens or more*. On short system prompts, `cache_creation` may not register.

```
long_prompt = (
    "You are an enterprise policy assistant. Always cite section numbers. "
    * 200 # ~2,000 tokens
)

agent = create_agent(
    model="bedrock_converse:anthropic.claude-sonnet-4-20250514-v2:0",
    system_prompt=long_prompt,
    tools=[],
    middleware=[BedrockPromptCachingMiddleware(ttl="1h")],
)
```

7.4 Verifying cache hits

Call twice in a row and read `usage_metadata.input_token_details` to confirm cache creation / hits.

```
for i in range(2):
    result = agent.invoke(
        {"messages": [{"role": "user", "content": f"Request {i}"}]},
    )
    last = result["messages"][-1]
    print(i, last.usage_metadata.get("input_token_details"))
```

`cache_creation` is large on the first call; `cache_read` fills in on subsequent calls.

7.5 Full parameters

Parameter	Default	Description
<code>type</code>	"ephemeral"	Only takes effect on <code>ChatBedrock</code> ; <code>ChatBedrockConverse</code> ignores it and uses "default"
<code>ttl</code>	"5m"	"5m" or "1h" . Nova-class models support "5m" only
<code>min_messages_to_cache</code>	0	Attach a cache checkpoint only when the message count is at least this value
<code>unsupported_model_behavior</code>	"warn"	For cache-unsupported models: "ignore" / "warn" / "raise"

7.6 ChatBedrock (Invoke model) example

`ChatBedrock` is the legacy invoke-model wrapper. `BedrockPromptCachingMiddleware` supports it too, and the `type="ephemeral"` parameter *does* take effect here. On the ChatBedrock side, for Anthropic models, both tool-definition caching and the extended TTL (1h) are supported.

```
from langchain_aws import ChatBedrock

model = ChatBedrock(model_id="anthropic.claude-sonnet-4-20250514-v2:0")

agent = create_agent(
    model=model,
    system_prompt=long_prompt,
    tools=[],
    middleware=[
        BedrockPromptCachingMiddleware(type="ephemeral", ttl="1h"),
    ],
)
```

7.7 Reasoning · profiling

Claude 4.6/4.7 models support *extended thinking* (reasoning) on Bedrock too. Caching still applies to reasoning-enabled calls, but the *reasoning block itself is always a cache miss and recomputed each time* — so token savings are smaller than the non-reasoning case.

```
from langchain_aws import ChatBedrockConverse

model = ChatBedrockConverse(
    model_id="anthropic.claude-sonnet-4-20250514-v2:0",
    additional_model_request_fields={
        "reasoning_config": {"type": "enabled", "budget_tokens": 4096},
    },
)
print(model.profile.reasoning_output) # True → response includes a reasoning block
```

TIP

`ChatBedrockConverse(...).profile` (LangChain 1.1+) exposes capabilities such as `tool_calling`, `reasoning_output`, and `max_input_tokens` — using it as a capability check in multi-model routing logic keeps the code clean.

7.8 Amazon Nova constraints

Item	ChatBedrockConverse + Anthropic	ChatBedrockConverse + Nova
System-prompt cache	O	O
Tool-definition cache	O	X
Message cache	O	O (excluding tool-result messages)

TTL <code>"1h"</code>	O	X (5m only)
-----------------------	---	-------------

When targeting Nova, pin `ttl="5m"` and set `unsupported_model_behavior="warn"` so an accidental `"1h"` is not silently ignored.

Key Takeaways

- When using Claude / Nova inside AWS boundaries, the Bedrock middleware reduces input-token cost
- Be aware of the `type` parameter difference between `ChatBedrock` and `ChatBedrockConverse`
- Nova supports 5m TTL only with no tool-definition cache — pin `ttl="5m"`
- Cache blocks must be roughly 1,024 tokens or more to actually hit

08 OpenAI Moderation

Pre- / post-block policy-violating content

`OpenAIModerationMiddleware` uses the OpenAI Moderation API to scan *user input · model output · tool results* automatically and, when a policy violation is detected, blocks, replaces, or raises an exception on the conversation. Use it to preemptively filter categories like hate, violence, or self-harm for a public chatbot, and to stop second-order contamination from external tool results.

Learning Objectives

LEARNING OBJECTIVES

- Understand the meaning and cost/latency tradeoff of the three scan points `check_input` / `check_output` / `check_tool_results`
- Distinguish the three `exit_behavior` modes: `"end"` / `"error"` / `"replace"`
- Polish user-facing responses via a custom `violation_message` template
- Inject a pre-configured OpenAI client via `client` / `async_client`

8.1 When to use it

- Public chatbots that need to preempt Moderation categories such as *hate* / *violence* / *self-harm*
- Preventing harmful content in tool outputs (web search, email bodies) from reaching the model
- Leaving *violation metadata* in the response flow for audit / reporting

8.2 Environment setup

Required packages: `langchain` , `langchain-openai` . `OPENAI_API_KEY` in `.env` .

```
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_openai.middleware import OpenAIModerationMiddleware

load_dotenv()
```

8.3 Basic usage — scan both input and output, end on violation

Defaults are `check_input=True` , `check_output=True` , `exit_behavior="end"` . If user input trips the Moderation API, the model call is skipped entirely and *the conversation ends with a violation message*.

Parameter	Default	Description
<code>model</code>	"omni-moderation-latest"	Moderation model to use
<code>check_input</code>	<code>True</code>	Scan user messages before passing to the model
<code>check_output</code>	<code>True</code>	Scan model-generated responses before returning to the user
<code>check_tool_results</code>	<code>False</code>	Scan tool-execution results before they enter the model
<code>exit_behavior</code>	"end"	"end" / "error" / "replace"
<code>violation_message</code>	default template	The text shown to the user on violation
<code>client</code> / <code>async_client</code>	None	Inject a pre-configured OpenAI client (optional)

```
agent = create_agent(
    model="openai:gpt-5.4",
    tools=[],
    middleware=[OpenAIModerationMiddleware()],
)
```

8.4 `exit_behavior="end"` — end immediately on violating input

When a harmful category is detected (for example self-harm encouragement), the flow ends with the violation message — no model call. The final message on the response contains the default violation text.

8.5 `exit_behavior="error"` — fail loudly

Use this in testing / batch environments where you would rather *not let the violation slip by silently*. It raises `OpenAIModerationError` so the upstream pipeline can catch it and write to the audit log.

```
from langchain_openai.middleware import OpenAIModerationError

agent = create_agent(
    model="openai:gpt-5.4",
    tools=[],
    middleware=[OpenAIModerationMiddleware(exit_behavior="error")],
)

try:
    agent.invoke({"messages": [{"role": "user", "content": "..."}]})
except OpenAIModerationError as e:
    # Record in the audit log
    print("Moderation violation:", e.categories)
```

8.6 `exit_behavior="replace"` — swap the message and keep going

When output scanning catches a violation, *only that response* is swapped out for the violation message and the graph keeps running. Useful in multi-turn conversations where you want to *tidy up a specific response* without ending the whole conversation.

```
agent = create_agent(
    model="openai:gpt-5.4",
    tools=[],
    middleware=[
        OpenAIModerationMiddleware(
            exit_behavior="replace",
            violation_message=(
                "This response cannot be shown under our safety policy "
                "(categories: {categories}). Please try a different question."
            ),
        ),
    ],
)
```

`violation_message` is a template string that supports the `{categories}`, `{category_scores}`, and `{original_content}` variables.

8.7 Scanning tool results (`check_tool_results=True`)

Prevents *harmful content authored by third parties* in web search, crawling, or DB query tool results from flowing unchanged into the model. Cost scales with the number of tool calls, so only turn this on in pipelines that touch *untrusted external sources*.

```
agent = create_agent(
    model="openai:gpt-5.4",
    tools=[web_search_tool, fetch_url_tool],
    middleware=[
        OpenAIModerationMiddleware(
            check_input=True,
            check_output=True,
            check_tool_results=True,
        ),
    ],
)
```

8.8 Observing violation flows with LangSmith

Moderation blocks are *rare but must always show up in the audit log*. With LangSmith auto-tracing on (`LANGSMITH_TRACING=true` + `LANGSMITH_API_KEY`), the nodes where `OpenAIModerationMiddleware` triggers land in the trace tree as usual.

```
import os
from langsmith.run_helpers import tracing_context

os.environ["LANGSMITH_TRACING"] = "true"
# os.environ["LANGSMITH_API_KEY"] = "ls_..."
# os.environ["LANGSMITH_PROJECT"] = "moderation-audit"

with tracing_context(
```

```

project_name="moderation-audit",
tags=["moderation", "input-only"],
metadata={"region": "kr"},
):
    agent.invoke({"messages": [{"role": "user", "content": "..."}]})

```

TIP

`tracing_context` routes *only a specific call range* to a different project / tag set — a natural pattern is to keep regular traces in `production` while sending only Moderation-blocked cases to `moderation-audit`. The `LANGCHAIN_*` environment variables were standardized to `LANGSMITH_*` from 1.1 onward — the old names still work for a while, but new code should use `LANGSMITH_*` exclusively.

8.9 Cost / latency tradeoff

Setting	Cost	Latency	Main effect
<code>check_input=True</code>	Mod 1 + Model 1	+1 RT	Block harmful input before it reaches the model
<code>check_output=True</code>	Model 1 + Mod 1	+1 RT	Protect the end user when the model produced a bad response
<code>check_tool_results=True</code>	# of tools × Mod	+1 per tool	Block externally contaminated data

TIP

Practical guidance: In production it is common to always keep `check_input` on, run `check_output` under a sampling strategy (e.g., 10%), and limit `check_tool_results` to untrusted external sources.

Key Takeaways

- Pick the three scan points on a cost/latency tradeoff: input always on, output sampled, tool_results conditional
- `exit_behavior` picks between ending the conversation (`end`), raising immediately (`error`), or swapping the message (`replace`)
- `violation_message` templating produces user-friendly guidance
- Reuse a pre-configured OpenAI client via `client` / `async_client` to save initialization cost

A Appendix A. Glossary

Core terms that appear throughout the handbook

This appendix collects the key terms that appear repeatedly throughout the handbook so you can review them in one place. You do not need to memorize everything at once; revisit this section whenever you encounter an unfamiliar term.

· NOTE

The goal of this glossary is not to replace the official API documentation, but to let you quickly confirm the concepts that appear often while reading the handbook. For implementation details, refer to the relevant chapter and reference documents.

Frameworks and Products

LangChain A high-level framework for quickly building agents and LLM applications by combining models, tools, prompts, and middleware.

LangGraph A runtime and framework for orchestrating complex workflows and long-running agents with state, nodes, edges, and checkpoints.

Deep Agents An all-in-one agent SDK built on top of LangGraph that adds planning, filesystem tooling, subagents, memory, and sandbox capabilities.

LangSmith An observability and quality platform for tracing, evaluation, dataset management, and debugging.

Langfuse An open-source-style observability tool for collecting traces and runtime telemetry.

MCP Short for Model Context Protocol, a protocol that connects external tools and resources to models and agents in a standard way.

ACP Short for Agent Client Protocol, a communication protocol for connecting agents to clients such as editors and IDEs.

Agents and Execution Model

Agent An execution unit in which an LLM uses tools, observes results, and keeps acting until the task is complete.

ReAct Short for Reasoning + Acting, a common agent pattern in which the model alternates between reasoning and tool use.

Workflow An execution flow whose steps and order are relatively well-defined. It is usually more deterministic than a free-form agent loop.

Orchestrator A higher-level controller that coordinates several steps or several workers and manages the overall flow.

Worker An execution unit that performs a specific task delegated by an orchestrator.

Subagent A helper agent called by a main agent to handle a more focused sub-task, often with its own context.

Handoff A pattern in which control is transferred to another role or agent depending on the current step or state.

Router A pattern that classifies the input or current state and sends it to the most appropriate path or specialist agent.

Human-in-the-Loop A safety mechanism that inserts human approval, edits, or rejection at sensitive execution steps or important branch points.

Interrupt A mechanism that pauses graph or agent execution in the middle and waits for external input.

Resume The action of continuing a paused execution from its prior state. This usually requires the same `thread_id` and a checkpoint.

Durable Execution An execution style that stores progress so the system can restart from the last successful point after an interruption or failure.

Pregel A message-passing computation model that influences LangGraph's internal runtime, where nodes run in supersteps.

Superstep A parallel computation round in the Pregel model during which several nodes run together.

State and Memory

State The collection of current task information that an agent or graph maintains and updates while it runs.

AgentState The default state schema used in LangChain agent execution. It includes reserved fields such as `messages`.

MessagesState A convenient LangGraph state type centered on a list of messages.

Checkpoint A component that stores and restores execution state. It is essential for multi-turn conversations, interrupt/resume flows, and durable execution.

Thread ID A unique identifier for a conversation or execution flow. You must reuse the same `thread_id` to continue from a previous state.

Runtime Context Context such as metadata, permissions, or user information that is injected only at execution time.

`context_schema` A schema that defines the structure of static context that does not change during execution.

`state_schema` A schema that defines the structure of dynamic state that can keep changing during execution.

Short-term memory Memory that is kept only within a single thread or conversation, usually the recent message history and work state.

Long-term memory Memory that persists after a conversation ends and can be reused in later threads, such as preferences or learned facts.

Store A storage layer that saves and retrieves data outside a single thread.

InMemoryStore A simple in-memory store implementation. It is convenient for development and testing, but data disappears on restart.

Semantic memory Long-term memory that stores meaning-based information such as facts, preferences, and concepts.

Episodic memory Memory whose time order matters, such as specific events or interaction history.

Procedural memory Memory that captures rules, procedures, or behavior patterns the agent should follow.

Tools and Interfaces

Tool A function or external action interface that an agent can call, such as search, calculation, file I/O, or an API request.

ToolRuntime A runtime object that gives tools access to the current state, context, store, and other execution-time resources.

`create_agent()` The main LangChain API for quickly creating a default agent.

`create_deep_agent()` The Deep Agents API for creating a harness-style agent with planning, files, and subagents.

`StateGraph` The LangGraph builder class used to define state-based graph workflows.

Graph API The LangGraph programming style in which you explicitly declare nodes and edges and assemble the graph structure.

Functional API The LangGraph programming style in which you describe workflows with Python functions, `@entrypoint`, and `@task`.

- @entrypoint** A decorator in the Functional API that defines the starting function of a workflow.
- @task** A Functional API decorator that wraps a unit of work that should have durability guarantees.
- Send** A LangGraph API used to fan out work dynamically to multiple workers.
- Command** A LangGraph control object that can express state updates, resume signals, and parent-graph transitions.

Structured output An output style that forces the model's answer into an explicit schema such as a Pydantic model or another structured format.

Backends and Execution Environments

Backend The storage and execution layer in Deep Agents that abstracts file access and execution environments.

StateBackend An ephemeral file backend whose contents are kept only within a conversation thread.

FilesystemBackend A backend that accesses the local disk. In practice it is often used with `virtual_mode=True` to restrict paths.

StoreBackend A backend that provides persistent cross-thread storage through a LangGraph store.

CompositeBackend A mixed backend that routes requests to different backends depending on the path.

LocalShellBackend A powerful but risky backend that adds shell command execution on top of local file access.

Sandbox An execution environment that isolates code execution and file operations from the host system.

Modal A sandbox and serverless execution provider that is especially strong for GPU and AI/ML workloads.

Daytona A sandbox provider well suited to fast devbox provisioning and development-environment workflows.

Runloop A sandbox provider designed for disposable devboxes and isolated execution.

Retrieval, Streaming, and Quality

RAG Short for Retrieval-Augmented Generation, a pattern in which external knowledge is retrieved and injected into the model's response process.

Retriever A component that searches for and returns documents or chunks relevant to a query.

Embedding A representation that turns text into vectors in semantic space so similarity search becomes possible.

Vector store A database or storage layer that saves embeddings and performs similarity search.

Chunking The process of splitting a long document into smaller units suited for retrieval and context injection.

SQL Agent An agent that translates natural-language questions into SQL queries and interprets the results.

Streaming A delivery style in which a model response or execution state is sent incrementally before the run is fully complete.

StreamEvent An event unit emitted during streaming, such as token output, tool start/end, or a state change.

TTFT Short for Time to First Token, the time it takes for the model to emit its first token.

TTFA Short for Time to First Audio, the time it takes for a voice agent to emit its first audio output.

Tracing An observability technique that records model calls, tool executions, state transitions, and errors so you can inspect the execution flow.

Evaluation The process of measuring the quality of an agent's output or execution trajectory.

LLM-as-Judge An evaluation method in which another LLM acts as a scorer for response quality or execution outcomes.

Trajectory The full path of tool calls, intermediate reasoning, and state changes that leads to the agent's final answer.

Guardrail A control mechanism that checks safety, policy compliance, or quality at the input, tool, or output stage.

PII Short for Personally Identifiable Information, sensitive data that can identify a person, such as an email address, phone number, or national ID number.